ADVANCED COMPUTER PROGRAMMING

Prof. Raffaele Della Corte – A.A. 2025/26

INDICE DEGLI ARGOMENTI

PROGRAMMAZIONE AVANZATA IN PYTHON

1. INTRODUZIONE AL LINGUAGGIO DI PROGRAMMAZIONE (p. 3)
2. MODULI E PACKAGE (p. 9)
3. TUPLE, LISTE E DIZIONARI (p. 11)
4. PASSAGGIO DEI PARAMETRI (p. 17)
5. GESTIONE DEI FILE E DELLE ECCEZIONI (p. 20)
6. PROGRAMMAZIONE ORIENTATA AGLI OGGETTI (p. 25)
7. PROGRAMMAZIONE CONCORRENTE (p. 34)
8. PROGRAMMAZIONE SU RETE IN PYTHON (p. 61)
9. SISTEMI MIDDLEWARE (p. 72)
10. REMOTE PROCEDURE CALL IN PYTHON (p. 81)
11. MESSAGE-ORIENTED MIDDLEWARE IN PYTHON (p. 92)
12. WEB SERVICES E PYTHON FLASK (p. 99)
13. NO-SQL DATABASE (p. 112)

PROGRAMMAZIONE AVANZATA IN JAVA

14. MULTITHREADING IN JAVA (p. 119)
15. MECCANISMI DI SINCRONIZZAZIONE IN JAVA (p. 125)
16. JAVA NETWORKING (p. 130)
17. REALIZZAZIONE DI OGGETTI REMOTI E PROXY-SKELETON (p. 134)
18. JAVA MESSAGE SERVICE (p. 137)
19. REMOTE PROCEDURE CALL IN JAVA (p. 149)
20. CONTAINER-BASED SERVICE DEPLOYMENT (p. 153)

PROGRAMMAZIONE AVANZATA IN PYTHON

INTRODUZIONE AL LINGUAGGIO DI PROGRAMMAZIONE

I linguaggi di programmazione di alto livello devono essere tradotti in linguaggio macchina per poter essere eseguiti; esistono principalmente due **metodologie di traduzione**:

- **Compilazione**, tramite compilatori che traducono i programmi in linguaggio macchina in un programma separato, che può essere eseguito in qualunque altro momento;
- **Interpretazione**, tramite interpreti che traducono ed eseguono le istruzioni di un programma scritto in linguaggio di alto livello.

Si può notare già una piccola ma essenziale differenza tra i linguaggi compilati e **interpretati**, che consiste nel fatto che per quest'ultimi **sono processate una alla volta le istruzioni**, **non l'intero programma**, non c'è un programma separato scritto in un altro linguaggio ed eseguibile arbitrariamente. L'implementazione di riferimento dell'interprete Python e CPython, scritto sia in C che in Python stesso, e non converte il codice in istruzioni in linguaggio macchina ma trasforma l'intero codice in qualcosa chiamato byte code; all'interno dell'interprete Python avviene comunque una compilazione ma non converte l'intero codice in linguaggio macchina come avviene per linguaggi compilati tipo C/C++, **byte code e assembly non sono la stessa cosa**, **il primo è generato all'interno di una virtual machine e per una virtual machine** (software di sistema intermedio), mentre **il linguaggio assembly viene creato per una specifica CPU**.

Il compilatore Python legge le istruzioni di un programma scritto in linguaggio di alto livello o il codice sorgente di Python, **verifica se le istruzioni sono formattate correttamente**, cioè verifica la struttura sintattica di ogni riga del programma e, nel caso in cui si riscontri un errore, la traduzione viene immediatamente interrotta e viene visualizzato un messaggio di errore, ma se non ci sono errori il compilatore **traduce le istruzioni di alto livello in linguaggio intermedio** (byte code); quest'ultimo, viene poi **fornito al vero e proprio interprete di Python**, la cosiddetta Python Virtual Machine (PVM): essa **converte il byte code Python in istruzioni a livello macchina o in codice binario equivalente** e, se si verifica un errore in questa fase di interpretazione, la conversione si arresta ed è, anche in questo caso, mostrato un messaggio di errore.

L'interprete Python può essere utilizzato in due modalità:

- **Interactive mode**, le istruzioni Python sono scritte ed eseguite una alla volta;
- **Script mode**, le istruzioni Python sono tutte scritte e salvate in uno script `.py` da eseguire in un secondo momento.

Seguendo la prima modalità, si utilizza un prompt dei comandi, con il quale l'interprete rimane in attesa di istruzioni Python da scrivere, riappare dopo l'esecuzione di un'istruzione e, nel caso in cui non fosse corretta, mostra eventuali messaggi di errore. Tuttavia, la interactive mode, sebbene utile in qualche applicativo (oltre la didattica), non è utile per l'esecuzione di interi programmi Python, in quel caso è necessario ricorrere alla script mode; l'esecuzione di un file `.py` già salvato con all'interno le istruzioni Python da eseguire avviene, da terminale, con il comando:

```
python filename.py
```

Con alcune semplici variazioni dipendenti dalla versione di Python in utilizzo. In modalità script vengono forniti anche i classici meccanismi per ottenere argomenti da linea di comando e/o

redirezione di I/O; **Python possiede anche un meccanismo per consentire ai programmi di agire sia da script che da modulo**, da importare ed essere usato da altri programmi Python.

I file `.pyc` sono file byte code compilati che vengono generati dall'interprete Python quando uno script viene importato o eseguito e possono essere anche direttamente eseguiti dall'interprete, senza la necessità di una seconda compilazione, riducendo i tempi di esecuzione degli script. Per ottenere il file `.pyc` a partire da un file `.py`:

```
python -m compileall filename.py
```

Un programma Python è una sequenza di definizioni e comandi, le prime sono valutate, mentre le seconde sono eseguite dall'interprete Python in una shelle (terminal). **I comandi istruiscono l'interprete a fare qualcosa** e possono essere sia digitati direttamente all'interno della shell che immagazzinati in un file che viene letto dalla shell stessa per essere valutato.

I programmi Python manipolano i cosiddetti data objects, oggetti dotati di un tipo che definisce il genere di cose che il programma può fare su di essi e possono essere:

- **Scalari**, non possono essere divisi:
- **Non-scalari**, hanno una struttura interna che può essere acceduta.

In Python, gli oggetti scalari sono i tipi di variabili primitivi:

- `int`, numeri interi;
- `float`, numeri reali;
- `bool`, valori booleani;
- `NoneType`, tipo speciale che può assumere solo il valore None;

Ai quali è possibile accedere tramite la funzione `type()`. È possibile convertire un oggetto di un tipo in un oggetto di un altro tipo specificando il tipo verso cui fare casting:

- `float(3)` converte l'intero 3 nel numero reale 3.0;
- `int(3.9)` tronca il numero reale 3.9 nel numero intero 3.

Per mostrare l'output al di fuori della shell di interpretazione è possibile utilizzare il comando `print()` e passargli all'interno la variabile da mostrare; può capitare spesso che si visualizzi il risultato di un'istruzione anche senza utilizzare `print()` ma, in quel caso, non è un output verso l'utente, è solo un'interazione interna alla shell che l'interprete porta al di fuori ed è evidenziata da `Out`:

```
In[11]: 3+2
Out[11]:5
In[12]: print(3+2)
5
```

Tra due interi e/o float è possibile fare le seguenti operazioni algebriche:

- Somma, `i+j`;
- Differenza, `i-j`;
- Prodotto, `i*j`;
- Divisione, `i/j`;

- Modulo, `i%j`;
- Elevamento a potenza di `i` a `j`, `i**j`.

Per i primi tre, se entrambi sono interi/float, il risultato sarà sempre un intero/float, per la divisione, invece, il risultato è per definizione un float. Per quanto riguarda le comparazioni, è possibile usare tutti i classici operatori di uguaglianza (==, `!=`) e disuguaglianza (>, <, >=, <=).

Le stringhe sono definite utilizzando sia un doppio apice che un singolo apice, è possibile concatenare più stringhe con l'operatore di addizione e, addirittura, effettuare operazioni pseudo-algebriche (come descritto da documentazione):

```
hello = "hello"
name = 'name'
greet = hello + " " + name
triple_greet = hello + " " + 3 * name
```

È possibile pensare alle stringhe come sequenza case-sensitive di caratteri; il che permette anche di comparare stringhe tramite gli operatori di uguaglianza (==, `!=`) e disuguaglianza (>, <, >=, <=). La funzione `len()` è utilizzata per ottenere la lunghezza di una stringa passata come argomento:

```
s = "abc"
len(s)    #Output: 3
```

Poiché sono sequenze di caratteri, **le stringhe possono essere indicizzate come un array**; Python, a differenza di altri linguaggi di programmazione, permette di **indicizzare anche con indici negativi**, andando a **prelevare dall'ultimo elemento del vettore verso il primo**:

```
s[0]      #a
s[1]      #b
s[2]      #c
s[-1]     #c
s[-2]     #b
s[-3]     #a
```

Python mette a disposizione anche l'operazione di **slice**, che permette di **prelevare un array di elementi di un array specificando indice di start, indice di stop e step** (di default 1) come `s[start:stop:step]`:

```
s = "abcdefgh"
s[3:6:2]       #df
s[::]          #abcdefgh, stesso di s[0:len(s):1]
s[::-1]        #hgfedcba, inversione di stringa
s[4:1:-2]      #ec
```

Le stringhe, tuttavia, **sono immutabili**; ciò significa che **questa operazione non è possibile** su di esse:

```
s = "hello"
s[0] = y       #non verrà ritornato yello
```

Ma è possibile comunque pensare ad una **modifica come uno slicing e una concatenazione**:

```
s = 'y' + s[1:len(s)]    #ritorna yello
```

Tuttavia, questa soluzione **non modifica la stringa originale, ne crea una nuova** che viene assegnata allo stesso puntatore della stringa precedente.

La stampa in output delle stringhe avviene, così come per gli oggetti scalari numerici, con la funzione `print()` e, per stampare più di una stringa, è possibile ricorrere o all'inserimento di più parametri che alla concatenazione:

```
s = "hello"
name = "Luca"
print(s, " my name is ", name)
print(s + " my name is " + name)
```

Il comando `input()` effettua la stampa di ciò che vi è inserito all'interno tra doppi apici e attende che l'utente inserisca qualcosa da tastiera; **una volta che è l'input da tastiera è inviato, il bind avviene tra il valore inserito e la variabile che viene assegnata ad `input()`**:

```
name = input("What's your name?")
```

`input()` restituisce una stringa, è necessario il casting per variabili di altro tipo:

```
num = int(input("How old are you?"))
```

Tra due variabili booleane non è possibile utilizzare gli operatori di comparazione finora introdotti, è necessario specificare le operazioni logiche di `and`, `or` e `not`:

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

Il **flusso di controllo tramite branching** avviene con i seguenti costrutti:

```
if <condition>:
    <expression>
    <expression>
    ...
```

```
if <condition>:
    <expression>
    <expression>
    ...
else:
    <expression>
    <expression>
    ...
```

```
if <condition>:
    <expression>
    <expression>
    ...
elif <condition>:
    <expression>
    <expression>
    ...
else:
    <expression>
    <expression>
    ...
```

Sebbene in linguaggi di programmazione classici o compilati non sia tanto rilevante se non per una questione grafico-organizzativa, **in Python l'indentazione delle varie istruzioni è essenziale per far comprendere all'interprete lo scope di un'istruzione**; infatti, **un livello di indentazione indica un blocco di istruzioni con scope locale proprio**:

```
x = float(input("Enter a number for x: "))
y = float(input("Enter a number for y: "))
if x == y:
    print("x and y are equal")
    if y != 0:
        print("therefore, x / y is", x/y)
elif x < y:
    print("x is smaller")
else:
    print("y is smaller")
print("thanks!")
```

Così come negli altri linguaggi di programmazione classici, c'è differenza tra l'operatore = e l'operatore ==: il primo è l'operatore di assegnazione di valore a variabile (o variabile a variabile), il secondo è l'operatore di uguaglianza con il quale si verifica l'uguaglianza di due valori dello stesso tipo.

Come altri flussi di controllo, Python prevede i classici **cicli iterativi**:

```
while <condition>:
    <expression>
    <expression>
    ...
```

```
for <variable> in range(<some_num>):
    <expression>
    <expression>
    ...
```

La funzione range ammette in ingresso tre parametri, `range(start, stop, step)`, di cui i primi due sono opzionali (di default 0 e 1) ed esegue inizializzando la variabile di conteggio `i` a `start`, aumentandola di `step` ogni iterazione e fermando l'intero ciclo quando raggiunge il valore `stop - 1`. **È possibile uscire immediatamente da qualsiasi sia il tipo e lo stato del ciclo utilizzato tramite l'istruzione `break`**, con la quale si salta la valutazione delle espressioni rimanenti nel blocco di codice; nell'esempio che segue, `expression_b` non è eseguita:

```
while <condition_2>:
    <expression_a>
    break
    <expression_b>
```

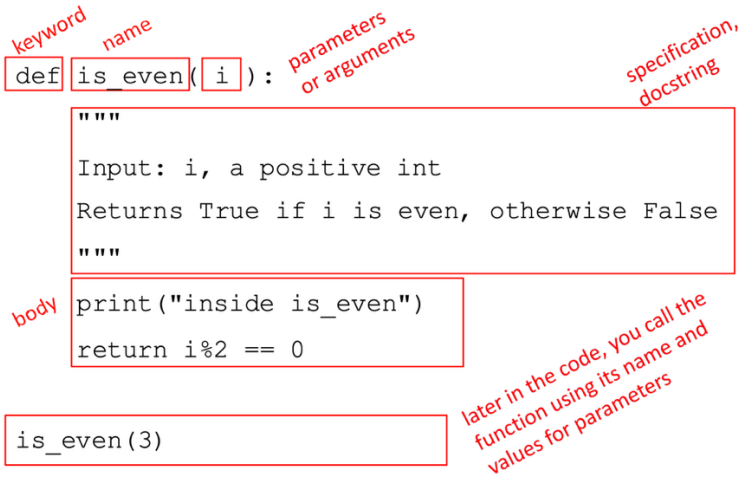
Chiaramente, i cicli for sono nella maggior parte dei casi implementati con la funzione `range()`; tuttavia, esistono alcuni casi in cui **la variabile o il set di variabili sui cui si deve iterare è un tipo di variabile iterabile**, cioè **predisposta alle iterazioni su di essa**. Un tipo di variabile iterabile è la stringa, sulla quale il ciclo for può essere scritto nel seguente modo, più Pythonico:

```
for char in s:
    if char == 'i' or char == 'u':
        print("There is an i or u")
```

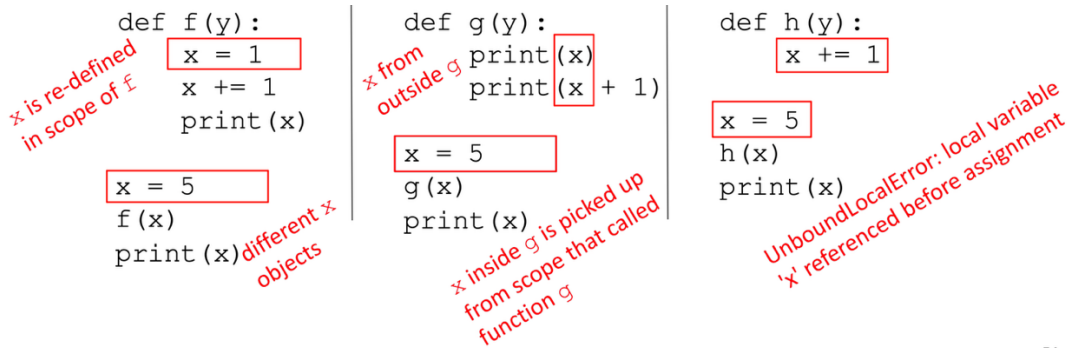
Chiaramente, è possibile lo stesso implementare la versione con `range()` ma al costo di leggibilità ed eleganza del codice inferiori.

Le funzioni permettono di scrivere codice riutilizzabile, non vengono eseguite nel programma finché non sono chiamate o invocate e, in Python, sono caratterizzate da un nome, dei parametri (0 o più),

una **docstring** (opzionale ma consigliata, è **il commento che si inserisce nella prima istruzione del corpo e funge da documentazione**, come in Matlab), un corpo e un ritorno:



Il parametro che viene inserito nella definizione della funzione è detto **parametro formale** e viene sempre legato al valore del **parametro effettivo**, quello che viene passato alla funzione in fase di chiamata; **una volta entrati in una funzione, viene creato un nuovo scope** (o frame o environment), **un mapping tra nomi ed oggetti con il quale è possibile utilizzare vantaggiosamente, o evitare, fenomeni di aliasing** tra variabili dello stesso nome o **limitare l'accesso a determinate locazioni di memoria da parte di blocchi di codice che non dovrebbero avere permessi di lettura e/o scrittura**. Dentro una funzione, è possibile accedere ad una variabile definita al di fuori della funzione ma non è possibile la sua modifica (sarebbe possibile con variabili globali, ma il loro uso non è una buona pratica):



Sebbene sia stato specificato come obbligatorio, **il ritorno di una funzione può essere omesso**; in tal caso, ad essere ritornato è il valore **`None` di tipo `NoneType`**. `None` rappresenta **l'assenza di valore** ed è paragonabile al tipo `void` in linguaggi come C/C++. Dal momento in cui Python non è rigido sul tipo delle variabili, **è possibile specificare una funzione** (in particolare, il suo valore di ritorno) **come parametro effettivo di una funzione**.

```
def func_c(z):
    print 'inside func_c'
    return z()
```

Generalmente, la funzione `main` è l'entry point di un programma; tuttavia, **l'interprete Python esegue il codice fin dalla prima riga di codice sorgente, a prescindere dal fatto che ci sia o meno la definizione di una funzione `main`**. In Python, non esiste una funzione `main`, quando viene dato

all'interprete il **comando di esecuzione di un programma Python**, **viene eseguito il codice che si trova al livello 0 di indentazione**; tuttavia, vengono in aiuto delle variabili speciali, come **`__name__`, una variabile built-in che valuta il nome del modulo corrente**: se il file `.py` viene eseguito come **programma principale**, **l'interprete imposta la variabile `__name__` in modo che abbia valore `__main__`**, se, invece, il file `.py` viene importato da un altro modulo, `__name__` assumerà il nome del modulo stesso. Ciò significa che con una semplice **direzione del flusso**, è possibile **determinare una "finta" funzione `main`**:

```
if __name__ == "__main__":
     funzione_main()
```

In questo modo, **`funzione_main()` viene eseguita come entry point solo se il file `.py` in cui è inserita è eseguito come programma principale**.

MODULI E PACKAGE

All'uscita dall'interprete le definizioni di funzioni e di variabili non sono salvate, è necessario ricompilare e rieseguire il codice; **i moduli sono file con estensione `.py` che consentono di salvare queste definizioni per accedervi in un secondo momento** e di eseguire le istruzioni come script eseguibili. L'importazione di un modulo avviene specificando il nome del file senza estensione preceduto dalla keyword `import`:

```
import example
```

Dal modulo `example.py`. Per rendere più facile l'accesso, è possibile importare singole definizioni all'interno di un modulo o di dare un alias al nome del modulo stesso:

```
import example as e
from example import func1, func2
```

Quando i moduli sono importati, tutte le dichiarazioni e le definizioni saranno eseguite. Per utilizzare le funzioni definite nel modulo, occorre digitare dapprima il nome del modulo (o il suo alias) seguito dal punto:

```
example.func1(3)
```

Ma se la funzione nel modulo è stata importata individualmente **con la keyword `from`, il nome del modulo e il punto possono essere omessi**:

```
func2(3)
```

Nel caso si volesse eseguire uno script Python in riga di comando:

```
python example.py <arguments>
```

Dove la keyword `python` va opportunamente modificata sulla base della versione di Python installata sulla macchina. I moduli importati e quelli eseguiti possono essere distinti accedendo alla variabile globale `__name__`, che assume il valore "`__main__`" solo quando i moduli sono eseguiti nel programma principale.

L'interprete cerca tutti i moduli importati in determinati luoghi disegnati, nell'ordine seguente, finché il modulo non viene trovato:

- Directory corrente;
- L'elenco di directory nella variabile d'ambiente `PYTHONPATH`;
- Percorso predefinito dipendente dall'installazione (default path).

Tutte queste informazioni sono memorizzate nella variabile `sys.path`.

Esiste una **libreria standard di moduli Python** con informazioni integrate che non fanno parte del nucleo di Python ma che sono comunque necessarie allo sviluppatore tipo (alcuni moduli vengono importati solo se si utilizza un determinato sistema operativo); un importante esempio di modulo di libreria standard è `sys`. In `sys` sono contenute due variabili, `ps1` e `ps2`, che racchiudono i valori delle stringhe nei prompt primario e secondario e possono essere arbitrariamente modificate:

```
>>> import sys
>>> sys.ps1 = 'plurt> '
plurt> print 'plurt'
plurt
plurt>
```

La funzione `dir()` restituisce un elenco di nomi (variabili e funzioni) definiti dal modulo ad essa passato o l'elenco di tutti i nomi attualmente definiti (ad eccezione di quelli incorporati) se non vi è passato nulla:

```
>>> dir(sqrt)
['num', 'result']
```

I package sono raccolte organizzate di moduli, **moduli all'interno di moduli**, i quali sono memorizzati in directory, con **ogni directory che contiene un file `__init__.py`**. Questo **istruisce l'interprete Python di trattare la directory come se fosse un package** e può essere semplicemente un file vuoto, eseguire codice di inizializzazione per il package o impostare la variabile `__all__` (della quale si discuterà a breve). **Ogni sottodirectory della directory principale del pacchetto è considerata un sotto pacchetto**. Ne segue che un package o uno qualsiasi dei suoi sotto-package può essere importato:

```
importo currency.conversion.euro
```

Questa istruzione permette di accedere a qualsiasi definizione o sotto-package di `euro`, utilizzando però la dotted notation:

```
euro.convertFromDollar(10)
```

A volte si vuole importare tutto ciò che si trova all'interno di un package o di un sotto-package e lo si può fare utilizzando la seguente notazione:

```
from example.subexample import *
```

Questo carica tutti i nomi dei moduli contenuti nella variabile `__all__`, a sua volta contenuta nel file `__init__.py` della cartella del package. Se la variabile `__all__` non è definita, per impostazione predefinita, l'import tramite `*` da un package non importerà tutti i sotto-moduli dei

package ma assicurerà di importare tutto il package, possibilmente eseguendo il codice di inizializzazione contenuto in `__init__.py` e importerà tutti i nomi definiti del package. Eventuali sotto-moduli non saranno importati, a meno che non siano esplicitamente importati da `__init__.py`.

Quando i package sono strutturati in sotto-package, si possono usare le importazioni assolute per fare riferimento ai sotto-moduli dei package "fratelli"; ad esempio, se il modulo `sound.filters.vocoder` deve usare il modulo `echo` del pacchetto `sound.effects`, può usare:

```
from sound.effect import echo
```

Ma è anche possibile scrivere importazioni relative, con la dichiarazione:

```
from module import name
```

Queste importazioni utilizzano dei punti iniziali per indicare il pacchetto corrente e quello padre coinvolti nell'importazione relativa; ad esempio:

```
from . import echo
from .. import formats
from ..filters import equalizer
```

La posizione in cui un package cerca il suo `__init__.py` può essere cambiata, modificando la variablie `__path__`, che memorizza il valore della cartella in cui si trova `__init__.py`.

TUPLE, LISTE E DIZIONARI

Una tupla è una sequenza ordinata e immutabile di elementi con, eventualmente, tipi diversi ed è rappresentata con delle parentesi. Sulle tuple è possibile fare le stesse operazioni di accesso, concatenazione e slicing degli array:

```
t = ()          empty tuple
t = (2,"mit",3)
t[0]                        → ritorna 2
(2,"mit",3) + (5,6)         → ritorna (2,"mit",3,5,6)
t[1:2]      → slice tuple, ritorna ("mit",)
t[1:3]      → slice tuple, ritorna ("mit",3)
len(t)      → ritorna 3
t[1] = 4    → gives error, can't modify object
```

extra comma means a tuple with one element

I principali impieghi delle tuple sono nello **swap di variabili** e nella possibilità di **ritornare più parametri in output da una funzione camuffandoli da unica entità**:

```
(x, y) = (y, x)
```

```
def quotient_and_remainder(x, y):
    q = x // y        integer division
    r = x % y
    return (q, r)
(quot, rem) = quotient_and_remainder(4,5)
```

Una lista è una sequenza ordinata di informazioni, accessibile tramite un indice e denominata da parentesi quadre; **è il tipo di collezione più vicina agli array classici**, permette la memorizzazione sequenziale di elementi dello stesso tipo ma, a differenza, **può contenere valori di tipi eterogenei** (non comune). **Gli elementi di una lista possono cambiare**, il che rende la lista intrinsecamente mutabile. **Essendo molto simile ad un array, segue le stesse regole di ordinamento ed indicizzazione:**

```
a_list = []   empty list
L = [2, 'a', 4, [1,2]]
len(L)   → ritorna 4
L[0]     → ritorna 2
L[2]+1   → ritorna 5
L[3]     → ritorna [1,2], un'altra lista!
L[4]     → ritorna un errore
i = 2
L[i-1]   → ritorna 'a' perchè L[1]='a'
```

Per operare su intere liste è necessario iterare su di esse; **così come le stringhe, è possibile sia iterare per indice** (quindi con `for i in range(len(L))`) **sia iterare per elemento** (più Pythonicamente, con `for item in L`). Per l'aggiunta in coda di un elemento alla lista `L` si utilizza il metodo `L.append(element)`, mutando la lista finale. **In Python ogni singola entità è un oggetto e, pertanto, l'accesso ad una proprietà avviene con la dotted notation**.

Per combinare più liste è necessario ricorrere all'operazione di concatenamento tramite l'operatore +, con il quale si ottiene una nuova lista (non mutando le precedenti) comprensiva di tutti gli elementi delle liste di ingresso:

```
L3 = L1 + L2
```

In alternativa, è possibile mutare la lista `L1` in modo che contenga anche gli elementi di `L2`, tramite il metodo extend:

```
L1.extend(L2)
```

Nel primo caso gli addendi non cambiano, viene creata una lista ex novo, mentre in quest'ultimo caso, sebbene si utilizzi una quantità di variabili inferiore, una delle due liste deve essere alterata. Per quanto riguarda l'operazione di rimozione, è possibile cancellare un elemento ad uno specifico indice:

```
del(L[index)
```

Oppure, per rimuovere l'elemento in coda alla lista:

`L.pop()`

Per rimuovere uno specifico elemento, si ricorre alla funzione `L.remove(element)`, con la quale:

- È ricercato l'elemento all'interno della lista;
- Alla prima occorrenza è eliminato (ciò significa che se ci sono più occorrenze, ne verrà eliminata solo una);
- Se l'elemento non è nella lista, ritorna un errore.

La conversione di liste coinvolge principalmente la loro mutazione in stringhe può avvenire in tre metodi diversi:

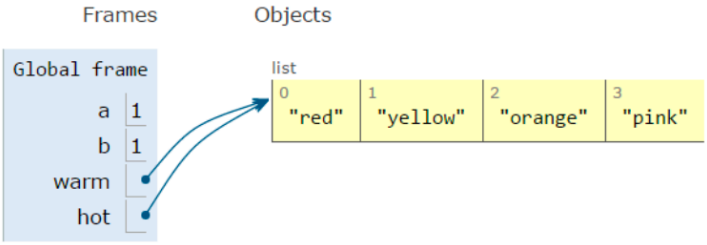
- `L=list(s)`, converte una stringa ritornando una lista che include ogni carattere da `s` in un elemento di `L`;
- `s.split()`, effettua lo split di una stringa su un carattere specificato come parametro, utilizzando lo spazio se la chiamata è senza parametri;
- `''.join(L)`, converte una lista di caratteri in una stringa, dando un parametro tra singoli apici per aggiungere un carattere tra ogni elemento.

Le operazioni sulle liste sono infinite, ci sono `sort()`, `sorted()` e `reverse()` ma anche molte altre consultabili nella documentazione di Python.

La lista, come già anticipato, è mutabile e, pertanto, **il comportamento in memoria è diverso da quello dei tipi immutabili** (come stringhe e tuple). La lista è, a tutti gli effetti, un oggetto in memoria e il nome di variabile non fa altro che puntare ad una locazione di memoria; **ogni variabile che punta all'oggetto lista è impattata da ogni modifica** e, pertanto, non sono rari effetti collaterali. Uno dei **più comuni effetti collaterali è l'alias**, che occorre quando **due nomi di variabili puntano la stessa locazione di memoria**; in questi casi, **la modifica dell'oggetto lista dal punto di vista di uno dei puntatori ha ripercussioni anche dal punto di vista dell'altro**:

```
1  a = 1
2  b = a
3  print(a)
4  print(b)
5
6  warm = ['red', 'yellow', 'orange']
7  hot = warm
8  hot.append('pink')
9  print(hot)
10 print(warm)
```

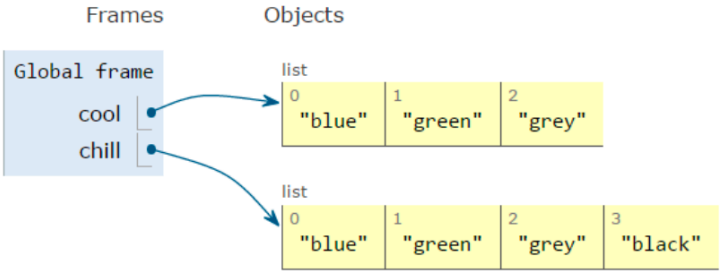
```
1
1
['red', 'yellow', 'orange', 'pink']
['red', 'yellow', 'orange', 'pink']
```



Per **clonare una lista** è possibile ricorrere all'operazione di **slicing su tutta la lista**, che **restituisce gli elementi della lista, non il puntatore**:

```
1 cool = ['blue', 'green', 'grey']
2 chill = cool[:]
3 chill.append('black')
4 print(chill)
5 print(cool)
```

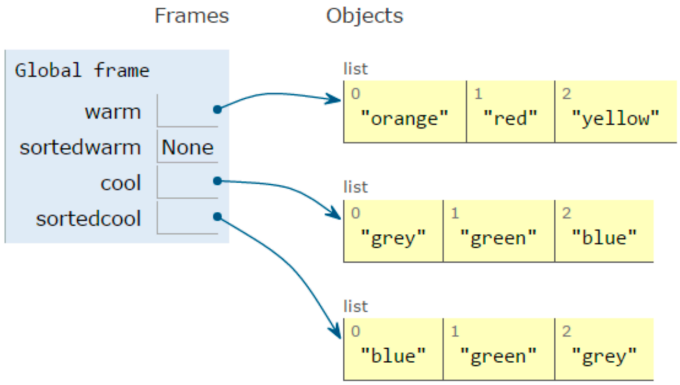
```
['blue', 'green', 'grey', 'black']
['blue', 'green', 'grey']
```



Si necessita adesso discutere delle due funzioni `sort()` e `sorted()`; la prima muta la lista nella sua versione ordinata, ritornando None, mentre la seconda non muta la lista, a meno che non è assegnata ad una seconda variabile:

```
1 warm = ['red', 'yellow', 'orange']
2 sortedwarm = warm.sort()
3 print(warm)
4 print(sortedwarm)
5
6 cool = ['grey', 'green', 'blue']
7 sortedcool = sorted(cool)
8 print(cool)
9 print(sortedcool)
```

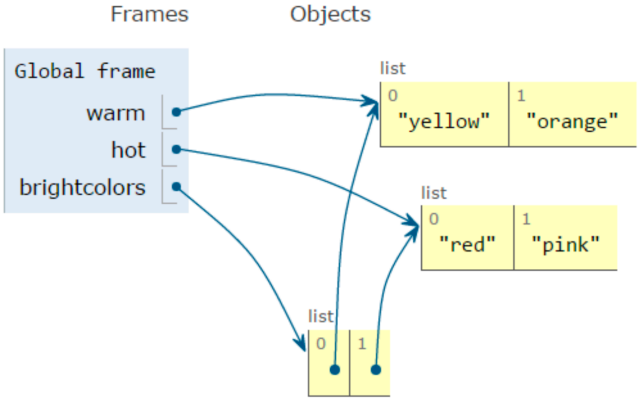
```
['orange', 'red', 'yellow']
None
['grey', 'green', 'blue']
['blue', 'green', 'grey']
```



Python permette di creare **liste innestate**, **similmente alle matrici** per linguaggi di programmazione come C/C++, sebbene sia possibile incorrere in effetti collaterali dopo la loro modifica:

```
1 warm = ['yellow', 'orange']
2 hot = ['red']
3 brightcolors = [warm]
4 brightcolors.append(hot)
5 print(brightcolors)
6 hot.append('pink')
7 print(hot)
8 print(brightcolors)
```

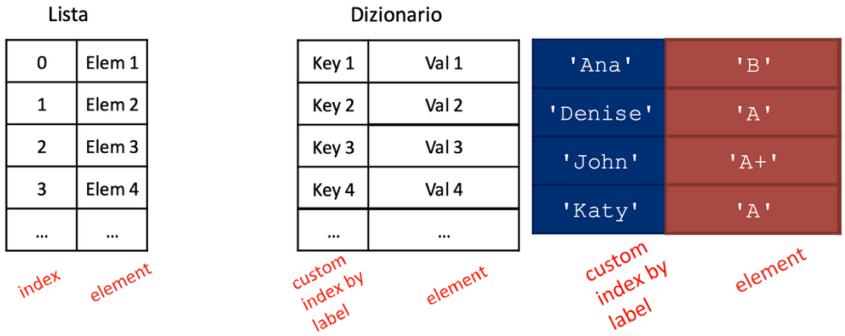
```
[['yellow', 'orange'], ['red']]
['red', 'pink']
[['yellow', 'orange'], ['red', 'pink']]
```



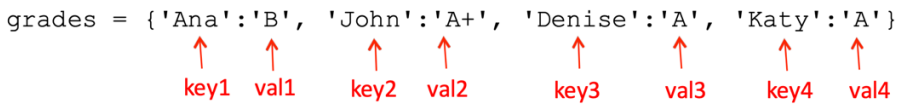
Tendenzialmente, **è bene evitare la modifica di una lista mentre si itera sui suoi elementi**; invece, è utile **creare una dummy copy su cui effettuare tutte le manipolazioni del caso** e, nel caso in cui sia poi necessario propagare all'esterno i risultati di queste manipolazioni, **effettuare la clonazione della dummy copy nella lista originale**. Python utilizza un contatore interno per tenere traccia dell'indice nel ciclo, **i cambiamenti mutano la lunghezza della stringa ma non il contatore**; ciò significa che se l'iterazione avviene su N elementi e, in itinere, ne si elimina 1 (per un totale di N-1 elementi), l'iterazione avverrà comunque N volte.

Si consideri la necessità di **memorizzare informazioni omologhe che fanno riferimento ad entità differenti** (ad esempio, diversi studenti, ognuno dei quali caratterizzato da un nome, un voto e un corso); una prima soluzione può essere l'impiego di una lista separata per ogni attributo dell'entità generale (una lista per nomi, una lista per voti e una lista per corsi), facendo attenzione a mantenere la stessa lunghezza per ogni lista e l'accoppiamento tra indici (il nome del primo studente sarà in indice 0 della lista nomi, il voto del primo studente sarà in indice 0 della lista dei voti e il corso del primo studente sarà in indice 0 della lista dei corsi). Questo approccio, sebbene sia molto intuitivo, è limitante a causa della grande confusione che si crea con tante informazioni di cui tenere traccia, è necessario mantenere molte liste, utilizzarle sempre come argomento e sempre indicizzarle tramite interi. **Il dizionario è una struttura dati tipica di Python che ha lo scopo di porre fine a questo tipo di problemi**.

I dizionari sono **strutture dati utilizzate per indicizzare elementi di interesse direttamente, cioè tramite chiavi non necessariamente intere** (anzi, sono le liste che possono essere viste come un dizionario indicizzato tramite intero), e per utilizzare una sola struttura dati per raccogliere diverse tipologie di informazioni. I dizionari sono costruiti per associazione chiave-valore:



Infatti, **è memorizzata sempre una coppia di valori, key-value, e l'accesso ad un valore è fatto sempre e solo tramite la chiave** (riprendendo l'esempio dello studente, esisteranno 3 dizionari, nomi, voti e corsi, ognuno dei quali rappresentazione di una proprietà dello studente, con N chiavi, ognuna relativa ad uno studente; l'accesso al valore di una proprietà di uno studente avviene specificando la chiave relativa allo studente di una particolare proprietà).



L'operazione di lookup è simile all'indicizzazione delle liste, solo che al posto di specificare l'indice numerico per accedere al i-esimo elemento, **si specifica la chiave per accedere al valore corrispondente**:

```
grades = {'Ana':'B', 'John':'A+', 'Denise':'A', 'Katy':'A'}
grades['John']      ➔ ritorna 'A+'
grades['Sylvan']    ➔ Ritorna un KeyError
```

È possibile effettuare le seguenti operazioni sui dizionari:

- Aggiungere una entry

```
grades['Sylvan'] = A
```

- Test se la chiave è nel dizionario

```
'John' in grades    #True
'Daniel' in grades  #False
```

- Cancellazione di una entry

```
del(grades['Ana'])
```

- Ottenere un iterabile che sia come una lista di tutte le chiavi

```
grades.keys()
```

- Ottenere un iterabile che sia come una lista di tutti i valori

```
grades.values()
```

Per quanto riguarda le chiavi e i valori in sé e per sé, è necessario specificare alcuni vincoli:

- Valori:
  - Possono essere di qualunque tipo (mutabile o immutabile);
  - Possono essere duplicati;
  - I valori di un dizionario possono essere liste o anche altri dizionari;
- Chiavi:
  - Devono essere uniche;
  - Devono essere di tipo immutabile (`int`, `float`, `string`, `tuple`, `bool`);
  - I tipi `float` come chiavi possono essere problematici se non maneggiati accuratamente.

Non esiste un ordinamento né per le chiavi né per i valori. Di seguito è proposta una comparazione tra liste e dizionari:

- **Lista**:
  - Sequenza ordinata di elementi;
  - La ricerca degli elementi è fatta attraverso un indice intero;
  - Gli indici hanno un ordinamento;
  - Gli indici sono sempre degli interi;
- **Dizionario**:
  - Associa chiavi a valori;
  - La ricerca di un elemento è fatta attraverso un altro elemento;
  - Nessun ordinamento è garantito;
  - Le chiavi possono essere di qualunque tipo immutabile.

Volendo chiudere la trattazione sui dizionari, **è possibile iterare lungo le chiavi** (ricorrendo al metodo `keys()`) e, a differenza delle liste, **non è disruttivo mutare direttamente il dizionario durante un ciclo iterativo**.

```
def lyrics_to_frequencies(lyrics):
    myDict = {}
    for word in lyrics:
        if word in myDict:
            myDict[word] += 1
        else:
            myDict[word] = 1
    return myDict

def most_common_words(freqs):
    values = freqs.values()
    best = max(values)
    words = []
    for k in freqs:
        if freqs[k] == best:
            words.append(k)
    return (words, best)

def words_often(freqs, minTimes):
    result = []
    done = False
    while not done and len(freqs)>0:
        temp = most_common_words(freqs)
        if temp[1] >= minTimes:
            result.append(temp)
            for w in temp[0]:
                del(freqs[w])
        else:
            done = True
    return result


print(words_often(beatles, 5))
```

Il codice qui riportato è parte di un analizzatore di testo di una canzone che, rispettivamente:

1. Crea una frequency dictionary che mappa `str:int`;
2. Trova la parola più frequente e quante volte occorre nel testo della canzone;
3. Trova la parola che occorre almeno X volte.

PASSAGGIO DEI PARAMETRI

Come già anticipato, **tutto in Python è un oggetto** e, pertanto, **è possibile interagire con essi solo tramite riferimenti**. Quando una variabile è dichiarata, essa non conterrà effettivamente il valore ma solo il puntatore all'area di memoria in cui tale valore è conservato; per conoscere questo riferimento è possibile utilizzare la funzione `id()`. **Ne consegue che tutti i parametri in Python sono passati,**

di default, per riferimento, cioè **è passata una copia del puntatore all'interno del parametro formale**; ciò fa si che una modifica al parametro all'interno della funzione risulti in una modifica al contenuto della variabile inserita come parametro (non come in linguaggi tipo C/C++ in cui è modificata una copia e non la locazione originale).

```
def fun(x):
    print("Inside fun: x=", x, "ref_x=", id(x))

x = 7
fun(x)
print("Outside fun: x=", x, "ref_x=", id(x))
```

Output:

```
Inside fun: x= 7 ref_x= 4308052464
Outside fun: x= 7 ref_x= 4308052464
```

Nel caso in cui il parametro sia un oggetto mutabile, **ogni modifica al parametro all'interno della funzione si riflette anche sul chiamante**; nel caso in cui il riferimento venga sovrascritto all'interno della funzione, il parametro diventa locale alla funzione e ogni modifica non si riflette sul chiamante.

```
def fun(mylist):
   mylist = [1,2,3,4]
   print ("Inside fun: val=", mylist, "ref=", id(mylist))


mylist = [10,20,30]
fun( mylist )
print ("Outside fun: val=", mylist, "ref=", id(mylist))
```

Output:

```
Inside fun: val= [1, 2, 3, 4] ref= 4353638464
Outside fun: val= [10, 20, 30] ref= 4352610752
```

Nel caso in cui il parametro sia un oggetto immutabile, anche se la funzione dispone del riferimento all'oggetto, **non gli sarà possibile apportare modifiche**; infatti, in tal caso, viene creato un nuovo oggetto in memoria e ogni operazione sul parametro non avrà effetto sul chiamante.

```
def fun(x):
   print ("Inside fun before: val=", x, "ref=", id(x))
   x**=2
   print ("Inside fun after: val=", x, "ref=", id(x))

x = 3
fun( x )
print ("Outside fun: val=", x, "ref=", id(x))
```

Output:

```
Inside fun before: val= 3 ref= 4308052336
Inside fun after: val= 9 ref= 4308052528
Outside fun: val= 3 ref= 4308052336
```

La sintassi **`*args` può essere usata nella definizione delle funzioni Python per passare un numero variabile di argomento**; la funzione riceverà una **tupla di argomenti** (`args` nell'esempio), la quale può essere iterata per recuperare tutti gli argomenti:

```
def fun(*args):
   for arg in args:
      print(arg)

fun("first", "second", "third")
```

Output:
```
first
second
third
```

```
def fun(x, *args):
   print(x)
   for arg in args:
      print(arg)

fun(100, "first", "second", "third")
```

Output:
```
100
first
second
third
```

I keyword arguments permettono al chiamante di una funzione di passare alla funzione gli argomenti basandosi sul nome e non sulla posizione; ad esempio:

```
def fun(name, surname, country):
   print(name, surname, country)

fun(surname="Rossi", country="Italy", name="Mario")
```

Output:
```
Mario Rossi Italy
```

La sintassi **`**kwargs`** **può essere usata nella definizione delle funzioni Python per passare un numero variabile di keywords arguments**; **la funzione riceverà un dizionario di argomenti** (`kwargs` nell'esempio), il quale può essere iterato per recuperare tutti gli argomenti:

```
def fun(**kwargs):
    for key, value in kwargs.items():
            print(key, "=", value)

fun(day=8, month="March", year=2024)
```

Output:
```
day = 8
month = March
year = 2024
```

```
def fun(x, **kwargs):
    print(x)
    for key, value in kwargs.items():
            print(key, "=", value)

fun("Date:", day=8, month="March", year=2024)
```

Output:
```
Date:
day = 8
month = March
year = 2024
```

Python permette di definire dei **valori di default per gli argomenti di una funzione** e sono utilizzati quando nella chiamata non è specificato alcun valore per quell'argomento; **ciò permette di invocare una funzione con meno parametri rispetto alla quantità presenta nella sua definizione**:

```
def fun(username, password="changeme", days=30):
   print(username, password, days)

fun("admin", "admin", 120)
fun("user", "password")
fun("operator")
```

Output:
```
admin admin 120
user password 30
operator changeme 30
```

Gli argomenti con parametri di default devono essere specificati sempre dopo quelli obbligatori:

```
def fun(username, password="changeme", days=30, email):
   print(username, password, days, email)
fun("admin", "admin", 120, "admin@email.com")
```

Output:
```
SyntaxError: non-default argument follows default argument
```

```
def fun(username, email, password="changeme", days=30):
   print(username, password, days, email)
fun("admin", "admin@email.com", "admin", 120)
```

Output:
```
admin admin 120 admin@email.com
```

La sintassi `*args` utilizzata per i variable-lenght arguments può essere usata anche per operazioni di unpacking degli argomenti; gli argomenti sono già inseriti in una lista o tupla ma devono essere utilizzati in una funzione che richiede argomenti utilizzati in maniera separata:

```
>>> list(range(3, 6))    #normal call with separate arguments
[3, 4, 5]
>>> args = [3, 6]
>>> list(range(*args))   #call with arguments unpacked from a list
[3, 4, 5]
```

GESTIONE DEI FILE E DELLE ECCEZIONI

Quando si esegue un programma Python sono creati automaticamente tre oggetti file standard:

- `sys.stdin`, standard input file object;
- `sys.stoud`, standard output file object;
- `sys.stderr`, standard error file object.

Le applicazioni acquisiscono risorse (come file, connessioni di rete, connessioni a database) e **dovrebbero rilasciare le risorse appena non sono più necessario**. Python fornisce l'istruzione `with` che acquisisce una risorsa (in questo caso, l'oggetto file per `account.txt`) e assegna il suo oggetto corrispondente ad una variabile (in questo caso, `accounnt`), consentendo all'applicazione di utilizzare la risorsa tramite quella variabile e chiamando il metodo `close` dell'oggetto risorsa per rilasciarla quando è raggiunta la fine del blocco di istruzioni `with`.

```
with open('accounts.txt', mode='w') as accounts:
        accounts.write('100 Jones 24.98\n')
        accounts.write('200 Doe 345.67\n')
        accounts.write('300 White 0.00\n')
        accounts.write('400 Stone -42.16\n')
        accounts.write('500 Rich 224.62\n')
```

La funzione `open` apre un file e lo associa ad un oggetto file, mentre l'argomento `mode` specifica la modalità di apertura del file, indicando se aprirlo per lettura, scrittura o per entrambi; ad esempio, al modalità `w` apre il file per la scrittura, creando il file se non esiste (se non è specificato un percorso

per il file, Python lo crea nella cartella corrente), eliminando tutti i dati esistenti nel file (se il file è aperto per scrittura) e usando il metodo `write` sull'oggetto file per scrivervi sopra.

Character	Meaning
`'r'`	open for reading (default)
`'w'`	open for writing, truncating the file first
`'x'`	open for exclusive creation, failing if the file already exists
`'a'`	open for writing, appending to the end of file if it exists
`'b'`	binary mode
`'t'`	text mode (default)
`'+'`	open for updating (reading and writing)

È possibile iterare per elemento su un oggetto file considerandolo come una collezione di righe:

```
with open('accounts.txt', mode='r') as accounts:
    print(f'{"Account":<10}{"Name":<10}{"Balance":>10}')
    for record in accounts:
        account, name, balance = record.split()
        print(f'{account:<10}{name:<10}{balance:>10}')
```

Leggendo, così, una riga alla volta dal file e restituendola come stringa. Esiste anche un metodo `readlines` che può essere utilizzato per leggere un intero file di testo e restituisce ogni riga del file come una stringa in una lista di stringhe; per file di grandi dimensioni, tuttavia, `readlines` può essere molto onerosa in termini di complessità temporale, rendendo anche difficile l'impiego della lista di stringhe. Il metodo `seek`, invece, permette di posizionare un puntatore alla riga corrente (o ad una riga arbitraria in caso di iterazione):

```
file_object.seek(0)
```

Il modulo `os` in Python consente di utilizzare funzionalità dipendenti dal sistema oeprativo in maniera semplice; per la gestione dei file, ad esempio, è possibile utilizzare funzioni di rimozione e rinomina:

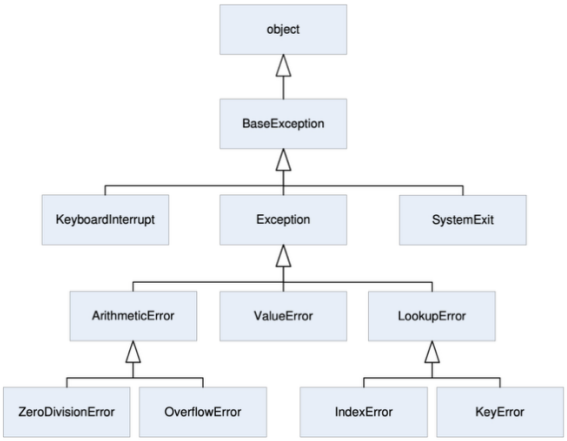
```
os.remove('accounts.txt')
os.rename('temp_file.txt', 'accounts.txt')
```

L'obiettivo di qualsiasi programmatore è avere programmi bug-free; ciò è ottenuto tramite strumenti di debugging come quelli built-in in un IDE, come Python Tutor, tramite l'utilizzo intenzionale di `print` o con una pipeline di lavoro sistematica. Il metodo che fa utilizzo di `print` è un buon modo per testare un'ipotesi, soprattutto quando la stampa è fatta in ingresso ad una funzione, per mostrare come sono passati i parametri, per visualizzare i risultati di una funzione, in modo da capire dove si trova l'errore nel flusso dei dati; un altro approccio prevede l'impiego del cosiddetto metodo di bisezione: viene messo un `print` in un punto mediano del codice e, sulla base dei valori ottenuti in stampa, rilevare la posizione del bug, se prima o dopo, se più internamente o più esternamente. **I principali passi per un debugging efficiente richiedono un metodo scientifico**: **si studiano i dati disponibili, si formulano ipotesi, si effettuano esperimenti ripetibili e si utilizza il più semplice degli input come testing**; il tutto è volto allo studio del codice del programma non per rispondere alla domanda "cosa c'è di sbagliato?" ma per capire come si è arrivati ad un output indesiderato.

Di seguito sono illustrati i messaggi di errore più comuni che si possono incontrare a valle di un errore:

- Accedere al di là dei limiti di una lista
  `test = [1,2,3]` then `test[4]` → `IndexError`
- Conversione in un tipo non appropriato
  `int(test)` → `TypeError`
- Riferirsi ad una variabile non esistente
  `a` → `NameError`
- Mischiare tipi di dato inappropriatamente
  `'3'/4` → `TypeError`
- Dimenticare di chiudere parentesi, apici (singoli e doppi), etc.
  ```
  a = len([1,2,3]
  print(a)
  ```
  → `SyntaxError`

Quando una funzione esegue del codice che produce condizioni non attese, come queste appena illustrate, **viene generata un'eccezione** (exception). La tassonomia delle eccezioni in Python (almeno di parte di esse) è la seguente:



Tipi di errori comuni sono:

- `SyntaxError`, Python non riesce ad analizzare il programma;
- `NameError`, non è stato trovato un nome, localmente o globalmente;
- `AttributeError`, un'attribute reference è fallita;
- `TypeError`, l'operando non è del tipo corretto o atteso;
- `ValueError`, l'operando è del tipo corretto o atteso ma assume un valore illegale;
- `IOError`, sono riportati malfunzionamenti al sistema di Input/Output.

Il codice Python può fornire handlers per gestire le eccezioni; in particolare, un'eccezione è:

- Sollevata da qualunque istruzione nel body del blocco `try`;
- Gestita dalle istruzioni nel blocco `except` e l'esecuzione continua nel body dell'omonimo blocco.

```
try:
    a = int(input("Tell me one number:"))
    b = int(input("Tell me another number:"))
    print(a/b)

except:
    print("Bug in user input.")
```

La pratica della gestione delle eccezioni ha come principale scopo l'evitare che il programma venga interrotto da una computazione non corretta, evitando che sistemi critici smettano di funzionare di punto in bianco. È raccomandato separare e personalizzare le clausole `except` per gestire un tipo particolare di eccezione; ognuno di questi blocchi deve essere posizionato dal più specifico al più generico (essendo le eccezioni valutate, come ogni istruzione, dall'alto verso il basso), in base alla gerarchia delle eccezioni; inoltre, **è buona pratica specificare alla fine di tutti i blocchi `except` che gestiscono particolari eccezioni un blocco `except` che gestisce il tipo di eccezione più generica**, così che se viene sollevata un'eccezione di un tipo non previsto è comunque possibile coglierla ed evitare un crash del sistema.

```
try:
    a = int(input("Tell me one number: "))
    b = int(input("Tell me another number: "))
    print("a/b = ", a/b)
    print("a+b = ", a+b)
except ValueError:
    print("Could not convert to a number.")
except ZeroDivisionError:
    print("Can't divide by zero")
except:
    print("Something went very wrong.")
```

only execute if these errors come up

for all other errors

Blocchi opzionali per la gestione delle eccezioni sono:

- `else`, il body di questo ramo è eseguito quando l'esecuzione del blocco try associato completa senza eccezioni;
- `finally`, il body di questo ramo è sempre eseguito dopo il blocco `try`, `else` ed `except`, anche se si sollevano altri errori o se si esegue un `break`, `continue` o `return`, ed è utile per invocare codice di clean-up che dovrebbe essere eseguito a prescindere di ciò che è accaduto (ad esempio, la chiusura di un file).

Quando si incontrano degli errori, **gli approcci sbagliati sono il fallimento silente** (si sostituiscono i valori di default o si continua e basta ma l'utente finale non sarà avvisato con alcun warning), **ritornare un valore di errore** (ma il codice si potrebbe complicare e non sarebbe scontata la scelta dell'errore); la scelta giusta è l'interruzione dell'esecuzione e la segnalazione con una condizione di errore, in Python: **raise an exception**.

```
raise Exception("Descriptive string")
```

Con l'istruzione `raise` non sono ritornati particolari valori quando si incontra un errore e non si controlla se questo "valore di errore" viene ritornato, **viene, invece, sollevata un'eccezione quando non si è in grado di fornire un risultato coerente con la specifica di una funzione** (poi è o procrastinata la gestione dell'errore o rimandata al programmatore che si occuperà di usare il codice e i programmi scritti):

```
raise <exceptionName>(<arguments>)
raise ValueError("something is wrong")
```

keyword

name of error you want to raise

optional, but typically a string with a message

In un programma è importante che certe assunzioni su un particolare stato di esecuzione siano come atteso; `assert` è il comando che si occupa di sollevare un'eccezione di tipo `AssertionError` se le assunzioni falliscono e il suo utilizzo è una buona pratica di defensive programming.

```
def avg(grades):
    assert len(grades) != 0, 'no grades data'
    return sum(grades)/len(grades)
```

function ends immediately if assertion not met

È un tipo di gestione delle eccezioni preventivo, **non correttivo**, ci si assicura che un determinato comportamento accade e, in caso contrario viene sollevata un'eccezione, altrimenti l'esecuzione procede normalmente; a differenza della classica modalità di gestione delle eccezioni, **l'errore non è gestito quando accade ma lo si cerca di prevenire a monte**. **Le asserzioni non consentono al programmatore di controllare la risposta a condizioni inattese, assicurano di terminare l'esecuzione non appena la condizione nell'asserzione fallisce**. Tipicamente sono usate per controllare gli input di una funzione (verifica di pre-condizionamento) ma possono essere usate anche per altri motivi, come il controllo dell'output (verifica di post-condizionamento) o per facilitare la ricerca di bug.

L'obiettivo di un'asserzione è trovare il bug il prima possibile ed eliminarli quando si palesano; pertanto, si rendono **necessarie in fasi di testing**, sollevano eccezioni se l'utente fornisce valori errati in input. Quindi, le asserzioni possono essere usate per controllare i tipi dei parametri/valori, controllare se esistono invarianti su strutture dati (condizioni logiche di una struttura dati che devono risultare sempre vere durante l'intero ciclo di vita del programma, tranne al massimo per brevi istanti in cui la struttura dati viene aggiornata internamente), controllare il rispetto di vincoli sui valori di ritorno o controllare violazioni di vincoli su funzioni (ad esempio, nessun duplicato in una lista).

Un ultimo tipo di gestione delle eccezioni coinvolge il cambio di policy in caso di ritorno da una funzione di un dato che solleverebbe eccezione; ad esempio, di seguito è mostrata una funzione che, se il rapporto da ritornare solleva un'eccezione `ZeroDivisionError`, cambia policy e ritorna `0.0`:

```
def avg(grades): try:
        return sum(grades)/len(grades) except
    ZeroDivisionError:
        print('warning: no grades data')
        return 0.0
```

PROGRAMMAZIONE ORIENTATA AGLI OGGETTI

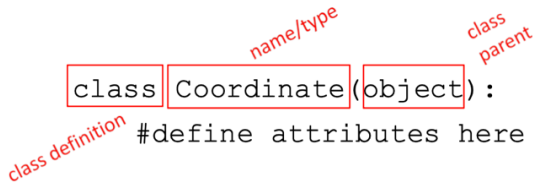
Come si è già potuto osservare, **Python supporta diversi tipi di dato e ognuno rappresenta un oggetto istanza di una classe che ha un tipo** (la classe di riferimento), una rappresentazione dei dati interna (tipo di dato primitivo o strutturato) e un insieme di procedure per l'interazione con gli oggetti. **È chiaro come in Python sia possibile non solo creare un oggetto di qualche tipo ma anche manipolare gli oggetti creati e distruggerli**, esplicitamente con `del` o "dimenticandosi" di loro (Python effettua un reclaim degli oggetti distrutti o inaccessibili attraverso un opportuno sistema di garbage collection).

Un oggetto è un'astrazione di dati che include, proprio come per i tipi già specificati, **una rappresentazione interna** (attraverso attributi membro) **e un'interfaccia per l'interazione con l'oggetto** (attraverso i metodi membro) **rispettando i criteri di information hiding** (cioè definendo i comportamenti ma nascondendo l'implementazione). Un esempio che chiarisce questa struttura è la linked list: un insieme di nodi, ognuno dei quali dotato di una propria struttura interna (valore del nodo-puntatore a prossimo nodo), su cui è possibile effettuare delle operazioni delle quali, al programmatore che utilizza la linked list, non è dato sapere l'implementazione.

```
L[i], L[i:j], +
len(), min(), max(), del(L[i])
L.append(),L.extend(),L.count(),L.index(),
L.insert(),L.pop(),L.remove(),L.reverse(), L.sort()
```

La rappresentazione interna dovrebbe essere privata, **il corretto comportamento di una lista può essere compromesso se questa viene direttamente manipolata**.

È necessario operare una distinzione tra **creare una classe e utilizzare un'istanza di una classe**; creare una classe significa definirne il nome, definirne gli attributi e i metodi, mentre l'utilizzo di un'istanza di una classe, ovvero di un oggetto, coinvolge la creazione stessa dell'oggetto e l'esecuzione arbitraria delle operazioni possibili con quell'istanza. In Python, **la creazione delle classi avviene tramite la keyword `class`, con la quale si definisce automaticamente un nuovo tipo di dato**:

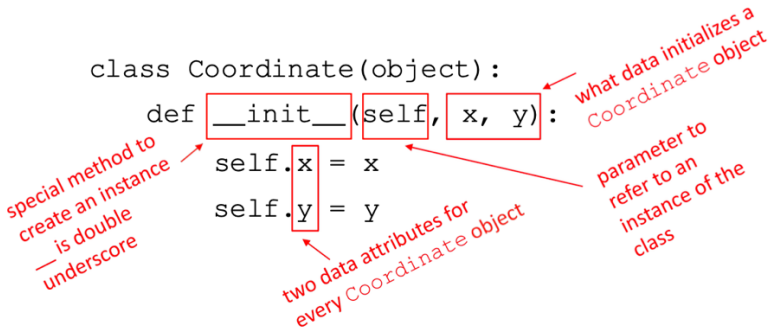


Similmente a come accade per le funzioni, **l'indentazione è importante a definire quali blocchi di codice**, quali attributi e quali metodi sono definiti all'interno della definizione di classe. **La parola `object` indica che `Coordinate` è un oggetto Python ed eredita tutti i suoi attributi**; in particolare, `object` è una superclasse (detta classe madre) di `Coordinate` e quest'ultima è una sottoclasse della prima. Per attributi si intendono i dati e le procedure che appartengono alla classe; in particolare, si fa distinzione tra:

- **Attributi**, altri oggetti che definiscono le proprietà costituenti della classe (nell'esempio della linked list, i nodi, il valore del nodo e il puntatore a prossimo nodo);

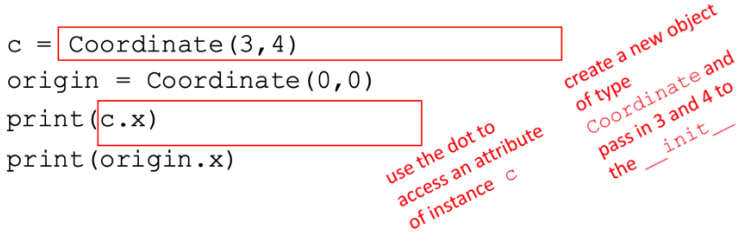
- **Metodi**, o attributi procedurali, sono le funzioni che descrivono i comportamenti dell'astrazione e che possono essere utilizzati solo nel contesto della classe, permettendo di interagire con l'oggetto istanza della classe definita (nell'esempio della linked list, l'operatore di concatenazione, la funzione di sorting, di append, …).

Prima di procedere a creare l'istanza di una classe è necessario istruire l'interprete sul modo in cui tale classe viene istanziata in memoria; per fare ciò è necessario implementare un metodo particolare, i costruttore (o **inizializzatore** per Python) che ha il compito di **inizializzare alcuni attributi e definire il modo in cui si troverà l'oggetto in memoria dopo l'allocazione**; il costruttore Python viene sempre definito tramite il metodo `__init__` ma, come sarà più chiaro a breve, viene invocato tramite il nome della classe:

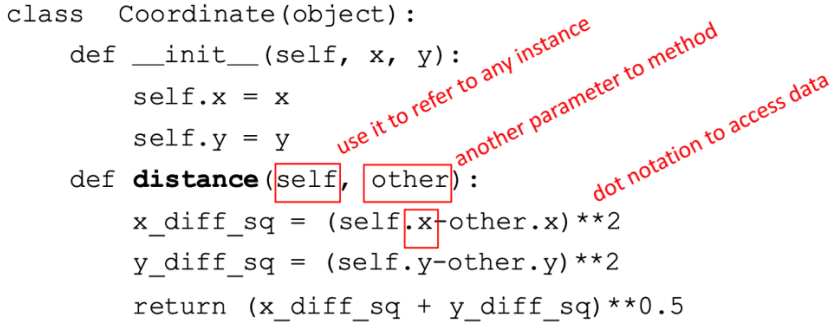


La keyword `self` rappresenta l'istanza della classe (che non c'è ma che in futuro ci sarà, è una sorta di finto puntatore che, quando l'oggetto sarà istanziato, verrà sostituito con il puntatore alla testa dell'oggetto) **e lega gli attributi con gli argomenti** (è l'analogo di `this` in C++ e Java).

L'istanza di una nuova classe avviene come segue:



Gli attributi relativi ad un'istanza di una classe sono chiamati variabili d'istanza. I metodi sono definiti come attributi procedurali, come se fossero funzioni che possono essere utilizzate solo nel contesto di tale classe; **Python passa di default sempre un oggetto come primo parametro alla chiamata di un metodo, per convenzione si utilizza `self`**. L'operatore `.` è utilizzato per accedere agli attributi di una classe (che siano attributi o metodi). Ad eccezione di self e della dotted notation, i metodi di una classe si comportano come semplici funzioni.



Dal momento in cui è specificato `self` all'interno della definizione della funzione, **è possibile chiamare un metodo in due modi**: il primo è la classica chiamata da puntatore ad oggetto, il secondo è una chiamata da classe nella quale si passa il puntatore ad oggetto (più simile ad una chiamata di metodo statico in C++ e Java).

```
c.distance(zero)
Coordinate.distance (c, zero)
```

Solitamente, se si fa una `print` di un riferimento ad un oggetto, si stampa a schermo una rappresentazione non informativa dell'oggetto:

```
>>> c = Coordinate(3,4)
>>> print(c)
<__main__.Coordinate object at 0x7fa918510488>
```

Questo perché **`c` è un puntatore all'area di memoria contenente l'oggetto**; per aggirare questo problema, **Python mette a disposizione la possibilità di implementare il metodo `__str__`**, che verrà **chiamato quando verrà fatta la `print` sul riferimento all'oggetto**, stampando ciò che è indicato nel metodo:

```
class Coordinate(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def distance(self, other):
        x_diff_sq = (self.x-other.x)**2
        y_diff_sq = (self.y-other.y)**2
        return (x_diff_sq + y_diff_sq)**0.5
    def __str__(self):
        return "<"+str(self.x)+","+str(self.y)+">"
```

name of special method

must return a string

```
>>> print(c)
<3,4>
```

È possibile verificare che un oggetto è istanza di una particolare classe utilizzando il metodo `instance(object, Class):`

```
>>>print(isinstance(c,Coordinate))
True
```

In realtà quello della stampa con `print()` non è l'unica operazione che può essere resa custom implementando un metodo, l'implementazione dei seguenti metodi permette l'utilizzo dei seguenti operatori (tra cui addizione, uguaglianza, comparazione, …):

```
__add__(self, other)    →    self + other
__sub__(self, other)    →    self - other
__eq__(self, other)     →    self == other
__lt__(self, other)     →    self < other
__len__(self)           →    len(self)
__str__(self)           →    print self
```
... e altri

Quest'operazione è detta **basic customization** e nell'opportuna documentazione di Python è possibile consultare la lista di tutti i comportamenti su cui è possibile fare quest'operazione di override nel contesto della classe.

Sempre in seno al principio dell'information hiding, **quando un attributo è reso** (per buona pratica, soprattutto) **privato ma è comunque necessario fornire all'esterno degli handler per la modifica e l'accesso, è utile implementare i metodi getter e setter**, metodi che permettono l'accesso e la modifica, eventualmente condizionata, alle informazioni nascoste della classe:

```
class Animal(object):
    def __init__(self, age):
        self.age = age
        self.name = None
    def get_age(self):
        return self.age
    def get_name(self):
        return self.name
    def set_age(self, newage):
        self.age = newage
    def set_name(self, newname=""):
        self.name = newname
    def __str__(self):
        return "animal:"+str(self.name)+":"+str(self.age)
```

getter

setter

L'utilizzo di getters e setters è un **buono stile di programmazione** che facilita la manutenzione del codice e previene la creazione di bug.

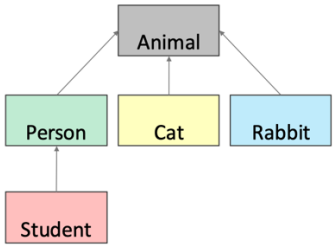
Python non è un buon linguaggio di programmazione per il supporto all'information hiding: **non esiste una vera privatizzazione degli attributi** (rimangono comunque accessibili all'esterno), è comunque possibile sovrascrivere i dati e creare nuovi dati membro all'esterno della definizione di classe; nessuna di queste proprietà rappresenta un buon modo di programmare considerando l'information hiding. **Un'altra bad practice è il cambiamento del nome dei dati membro**, crea inconsistenza e rischia di incorrere in errori nel caso in cui all'esterno si cercasse di accedere ai dati con il nome precedente.

Un'altra buona pratica è l'utilizzo dei parametri di default per i metodi di una classe, parametro formali che sono utilizzati quando nessun parametro attuale viene fornito, evitando l'interruzione del programma per la mancata presenza di argomenti:

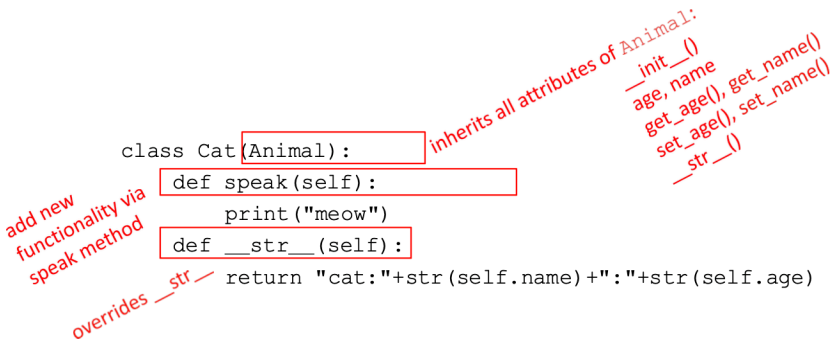
```
def set_name(self, newname=""):
    self.name = newname
```

L'ereditarietà è quella proprietà che mette in relazione due classi, una classe padre (o superclasse) e una classe figlia (o sottoclasse), **in modo che quella più in basso nella gerarchia erediti tutti i dati e tutti i metodi membro della classe padre, con la possibilità di aggiungerne altri o**

sovrascriverli (per aggiungere informazioni e specializzare la classe). Python, a differenza di linguaggi come Java, **supporta l'ereditarietà multipla**, per la quale una stessa sottoclasse può ereditare da più classi padri.



Nelle definizioni viste finora, è stata già utilizzata l'ereditarietà: **qualsiasi classe eredita dalla classe `object`, una classe che implementa tutte le operazioni di base in Python e che funge da classe base per qualsiasi altra classe**. Per mettere due classi in una relazione di ereditarietà è possibile ricorrere alla seguente sintassi:



Rispetto alla classe Animal, è aggiunta la funzionalità `speak` ed è sovrascritto il metodo `__str__`; su oggetti `Cat` è possibile invocare sia i nuovi metodi di `Cat` che i metodi di `Animal` ma su un oggetto `Animal` non è possibile invocare le proprietà di `Cat`. **Il costruttore, se non specificato, non è mancante ma viene riutilizzato quello della classe padre**. Una sottoclasse può avere dei metodi con lo stesso nome di quelli forniti dalla superclasse; in genere, per un'istanza di una classe si cercano prima i metodi tra quelli della definizione di classe corrente e, se il metodo non viene trovato, si cerca risalendo la gerarchia, usando il primo metodo trovato.

Come già anticipato, è possibile, in Python, usare meccanismi di ereditarietà multipla:

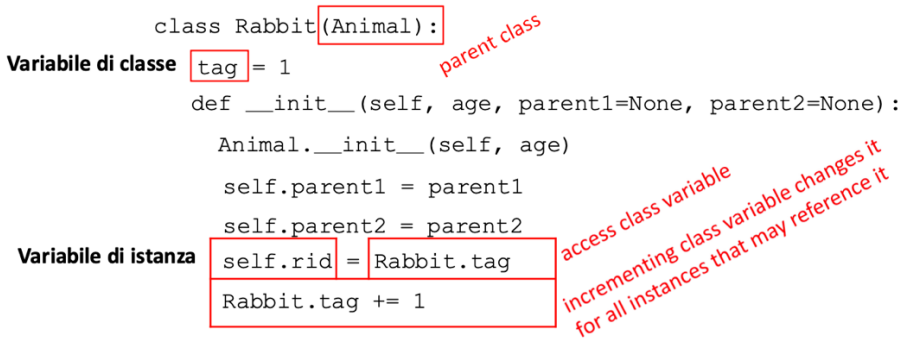
```
class Base1:
       Body of the class

class Base2:
     Body of the class

class Derived(Base1, Base2):
     Body of the class
```

L'ereditarietà multipla è una caratteristica in cui una classe può ereditare attributi e metodi da più di una classe genitore, comportando un elevato livello di complessità e ambiguità in situazioni come il problema del diamante; il problema del diamante è il termine generalmente usato facendo riferimento all'ambiguità che si presenta quando due classi, B e C, ereditano da una superclasse, A, e un'altra classe D eredita sia da B che da C. **Se c'è un metodo m in A per il quale B o C** (o addirittura

entrambi) **hanno fatto un override, se D non fa l'override di questo metodo, quale versione è ereditata**?

```
class A:
      def m(self):
         print("m of A called")

class B(A):
      def m(self):
         print("m of B called")

class C(A):
      def m(self):
         print("m of C called")

class D(B,C):
      pass
```

Una variabile statica (o variabile di classe) **è una variabile il cui valore è condiviso tra tutte le istanze di una classe e possono essere modificate dalla classe stessa invece che da un'istanza**. Una variabile statica è caratterizzata da una **singola allocazione in memoria** (infatti, si dice che sono variabili associate alla classe più che alle istanze degli oggetti), quando l'oggetto della classe viene creato per la prima volta, sono inizializzate al di fuori dei metodi ma all'interno di una classe, **possono essere accedute attraverso una classe ma non attraverso un'istanza e il loro comportamento non cambia al cambiare dell'oggetto**.



I metodi statici in Python sono metodi di una classe che possono essere invocati anche senza aver mai istanziato un oggetto di quella classe e non ricevono `self` come primo argomento (perché non essendo legati in memoria a nessun oggetto non hanno bisogno di un puntatore a cui fare riferimento). I metodi statici sono caratterizzati dalla definizione interna alla classe (come gli altri metodi non statici) ma privata del `self`, come già anticipato, dal fatto che la definizione deve essere preceduta da `@staticmethod`, dal fatto che possono essere invocati direttamente sulla classe o su un oggetto della classe e dal fatto che sono solitamente utilizzati per gestire aspetti non legati alle singole istanze di una classe (non avendo legami con esse, sono metodi che gestiscono l'astrazione, non la sua particolarizzazione).

I decoratori sono uno strumento molto potente e utile in Python, **consentono ai programmatori di modificare il comportamento di una funzione o di una classe**, **facendo il wrap di un'altra funzione per estendere il comportamento della funzione wrapped, senza modificarla in modo permanente**. Nei decoratori, le funzioni vengono prese come argomento in un'altra funzione e poi richiamate all'interno della funzione wrapper. `@staticmethod` è un particolare tipo di decoratore:

```
class Rabbit(Animal):
    tag = 1
    def __init__(self, age, parent1=None, parent2=None):
        Animal.__init__(self, age)
        self.parent1 = parent1
        self.parent2 = parent2
        self.rid = Rabbit.tag
        Rabbit.tag += 1
    def get_rid(self):
        return str(self.rid).zfill(3)
    def get_parent1(self):
        return self.parent1
    def get_parent2(self):
        return self.parent2
    @staticmethod
    def get_tag():
        return Rabbit.tag
```

Static method with no `self` parameter
Return the current tag value, which is a static variable

Ogni eccezione è un oggetto della classe `BaseException` in Python o un oggetto di una classe che eredita da questa classe base; le classi di eccezioni che ereditano direttamente o indirettamente dalla classe base `BaseException`, sono definite nel modulo `exception`. Python definisce quattro sottoclassi dalla classe base:

- `SystemExit`, termina l'esecuzione del programma (o una sessione interattiva) e non produce un traceback come gli altri tipi di eccezione;
- `KeyboardInterrupt`, si verifica quando l'utente digita il comando Ctrl+C sulla maggior parte dei sistemi;
- `GeneratorExit`, si verifica quando un generatore si chiude, normalmente quando termina la produzione di valori o quando il suo metodo `close` viene chiamato esplicitamente;
- `Exception`, è la classe base per le eccezioni più comuni (alcune sue sottoclassi sono `ZeroDivisionError`, `NameError`, `ValueError`, `StatisticsError`, `TypeError`, ...);

Un'eccezione custom può essere definita andando ad ereditare dall'opportuna classe di eccezioni:

```
class Error(Exception):
    """Base class for other exceptions"""
    pass

class ValueTooSmallError(Error):
    """Raised when the input value is too small"""
    pass

class ValueTooLargeError(Error):
    """Raised when the input value is too large"""
    pass
```

Il polimorfismo consente alle sottoclassi di avere metodi con gli stessi nomi dei metodi delle loro superclassi ed invocare il metodo opportuno a run-time; è Python che si occupa di eseguire, a tempo di esecuzione, il controllo dei tipi, a differenza dei linguaggi tipizzati statisticamente (come Java) che lo eseguono a tempo di compilazione. **Un caso particolare di tipizzazione dinamica in**

cui si utilizzano particolari caratteristiche del polimorfismo, tra cui il late binding e il dinamic dispatch, è il duck typing. Nella tipizzazione normale, l'idoneità è determinata solo dal tipo di oggetto, mentre con il duck typing si utilizza la presenza di metodi e proprietà per determinare l'idoneità piuttosto che il tipo effettivo dell'oggetto in questione. Ad esempio:

```
animals = [a, c, p1, p2, s1, s2, r1, r2, r3, r4, r5, r6]
for animal in animals:
        print(animal)
...
animal::4
cat:fluffy:5
person:jack:30
person:jill:25
student:alice:20:CS
student:beth:18:None
rabbit:001
rabbit:002
rabbit:003
rabbit:004
rabbit:005
rabbit:006
```

Una classe che contiene uno o più metodi astratti è detta classe astratta, **mentre un metodo astratto è un metodo che ha una dichiarazione ma non un'implementazione**. Una classe astratta può essere considerata come un **modello per altre classi**, consentendo di creare un insieme di metodi che devono essere creati all'interno di ogni classe figlia costruita dalla classe astratte. **Le classi astratte non possono essere istanziate in oggetti**, è necessario dapprima che una classe figlia erediti e implementi i metodi astratti per poter creare un'istanza di quel comportamento. **Una classe astratta che ha solo metodi astratti è detta interfaccia**.

Il modulo `abc` fornisce l'infrastruttura per la definizione di Abstract Base Class (ABC) in Python; in aggiunta, è fornita la metaclasse `ABCMeta` per definire questo particolare tipo di classi ed una classe ausiliaria `ABC` per definire alternativamente le ABC attraverso il meccanismo di ereditarietà.

```
from abc import ABC, abstractmethod

class Animal(ABC):
        @abstractmethod
        def doAction(self):
                pass

        def status(self):
                print("I am alive")
class Human(Animal):
        def doAction(self):
                print("I can walk and run")

class Snake(Animal):
        def doAction(self):
                print("I can crawl")

class Dog(Animal):
        def doAction(self):
                print("I can bark")

class Lion(Animal):
        def doAction(self):
                print("I can roar")
```

```
# A = Animal()
# a.doAction()
"""
questo scatena un
TypeError: Can't instantiate abstract class Animal with abstract method doAction

perchè non posso istanziare una classe astratta
"""

R = Human()
R.doAction()
R.status()

K = Snake()
K.doAction()
K.status()

R = Dog()
R.doAction()
R.status()

K = Lion()
K.doAction()
K.status()
```

`@abc.abstractmethod` è il decoratore che permette di specificare che un metodo è astratto e il suo uso richiede che la metaclasse della classe sia `ABCMeta` o che derivi da essa. Una classe che ha una metaclasse derivata da `ABCMeta` non può essere istanziata, a meno che tutti i suoi metodi astratti e le sue proprietà non siano state sovrascritte.

NON GENERA ECCEZIONE

```
class Animal(ABC):
        def doAction(self):
                pass

A = Animal()
A.doAction()
```

GENERA ECCEZIONE

```
class Animal(ABC):
        @abstractmethod
        def doAction(self):
                pass

A = Animal()
A.doAction()
```

Un'interfaccia funge da modello per la progettazione delle classi e contiene tutti i metodi astratti rispetto ad una classe astratta; l'approccio di Python alla definizione di interfacce è un po' diverso da come accade in linguaggi come C++ e Java: questi ultimi hanno una parola chiave per le interfacce, Python no. **Python fornisce due approcci per la specifica di interfacce: interfacce formali** (con l'uso di ABC) **e informali**. Le interfacce informali sono le più semplici da definire, **la natura dinamica di Python consente di implementare un'interfaccia informale come una classe che definisce metodi che possono essere overridden senza un'applicazione rigorosa**; ad esempio:

```
class InformalInterface:
      def method1(self, arg1, arg2):
            pass
      def method2(self, arg1)
            pass
```

`InformalInterface` definisce due metodi che non sono implementati, obbligando l'implementazione alle classi concrete che erediteranno da essa, e si presenta come una normale classe Python, sfruttando però il duck typing per informare gli utilizzatori che si tratta di un'interfaccia e che deve essere usata di conseguenza. Quindi, per utilizzare l'interfaccia, è necessario creare una classe concreta, ovvero una sottoclasse dell'interfaccia che fornisce un'implementazione dei metodi dell'interfaccia:

```
class InformalInterfaceImpl(InformalInterface):
      def method1(self, arg1, arg2):
            """ override method1 and implement it"""
      def method2(self, arg1)
            """ override method2 and implement it"""
```

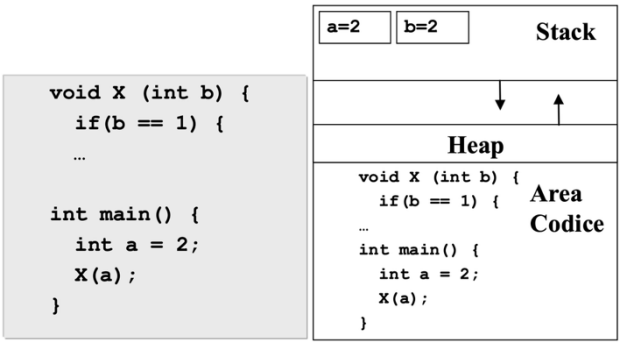
Per definire delle interfacce formali, invece, **in Python si utilizza il modulo `abc` e si specificano tutti i metodi come astratti**. Normalmente, si prevede di **sollevare un'eccezione di non implementazione (`NotImplementedError`) nel caso in cui la classe che implementa l'interfaccia non definisce alcun metodo astratto**.

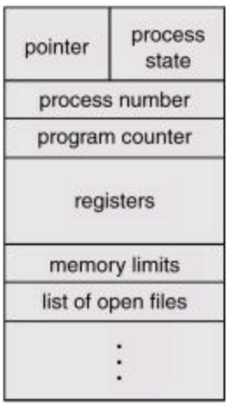
```
### interfaccia formale

import abc

class MyInterface(abc.ABC):

        @abc.abstractmethod
        def metodo1(self):
                raise NotImplementedError

        @abc.abstractmethod
        def metodo2(self):
                raise NotImplementedError

        @abc.abstractmethod
        def metodo3(self):
                raise NotImplementedError
```

```
class MyInterfaceImpl(MyInterface):

        def metodo1(self):
                print("metodo1")

        def metodo2(self):
                print("metodo2")

        def metodo3(self):
                print("metodo3")

#c = MyInterface() # => TypeError: Can't instantiate abstract
class MyInterface with abstract methods metodo1, metodo2,
metodo3

c = MyInterfaceImpl() # => OK! se implemento tutti i metodi
astratti

c.metodo1()
c.metodo2()
c.metodo3()
```

PROGRAMMAZIONE CONCORRENTE

Un programma è la codifica di un algoritmo in un linguaggio di programmazione, una descrizione statica delle operazioni da eseguire. **Il processo**, di contro, è un programma in esecuzione, **è l'unità di esecuzione del sistema operativo che identifica le attività dell'elaboratore relative all'esecuzione del programma**.





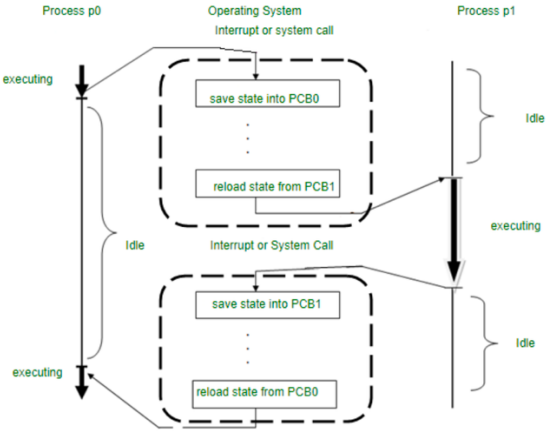
Il SO gestisce i processi associando ad ognuno di essi un descrittore (Process Control Block, o PCB) con il quale si effettua:

- L'identificazione del processo (PID, proprietario, …);

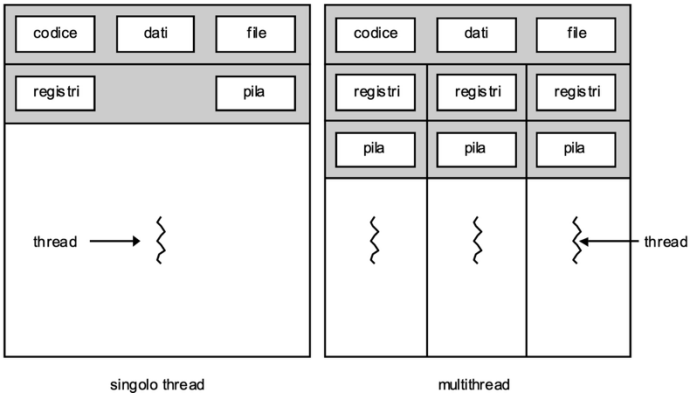
- Contesto del processore (registri della CPU, stack pointer, PC, …);
- Informazioni di controllo del processo:
  - Risorse possedute (memoria centrale, file aperti, …);
  - Parametri di schedulizzazione;
  - Stato del processo.

In generale, **il numero di processori è inferiore al numero dei processi**, ciononostante **si ha la percezione che più processi evolvano contemporaneamente**. Nel caso semplificativo di sistema monoprocessore, un solo processo alla volta può utilizzare l'unità di elaborazione e, pertanto, **è necessario che il SO alterni l'utilizzo della CPU tra i vari processi**, al fine di **sfruttare i tempi morti del processore** o attribuire le risorse di calcolo in base a determinati criteri. L'insieme delle operazioni eseguite per il pre-rilascio di un processo in esecuzione a favore di un nuovo processo è detto **context switch** e può avvenire per svariati motivi, tra cui il timeout (il tempo assegnato al processo è scaduto), le interruzioni di I/O o system call (il processo richiede un servizio al SO).

```
Procedure Context_Switch
begin
        Salvataggio_stato
        Scheduling_CPU
        Ripristino_stato
end
```



Un processo incapsula due concetti importanti: **esecuzione** (ad esempio, il flusso di controllo e lo stato di esecuzione) e il **possesso di risorse** (ad esempio, lo spazio di indirizzamento o le risorse di I/O). Il concetto di **thread**, invece, nasce con l'obiettivo di **separare l'esecuzione e il possesso di risorse**; un thread è un **flusso di controllo sequenziale in un processo**, laddove quest'ultimo definisce lo spazio di indirizzamento e le risorse che sono condivise fra i thread. Essendo un flusso di controllo sequenziale di un processo, **un thread ha un proprio insieme di registri**, **un proprio stack ma non ha una propria area heap e/o area dati statica** (a differenza dei processi).



Un programma multithread si configura come un unico processo di elaborazione in grado di eseguire più task concorrentemente (ha più punti di esecuzione).

È possibile dire che **un processo è detto anche "processo pesante"** in riferimento al contesto che si porta dietro, che **un thread è detto anche "processo leggero"** perché ha un contesto più semplice, che un thread è contenuto all'interno di un processo, che diversi thread contenuti nello stesso processo condividono lo stesso spazio di indirizzamento e che processi differenti non condividono le proprie risorse.

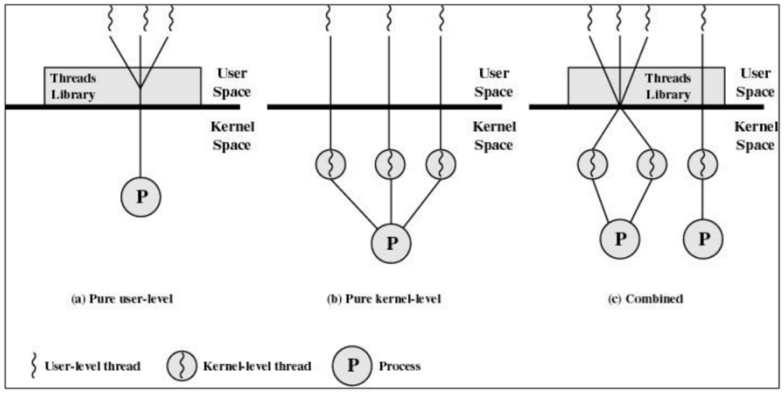
Sono diversi i vantaggi nell'uso dei thread:

- La creazione/terminazione di un thread è molto più efficiente della creazione di un processo;
- La comunicazione tra thread è molto più semplice ed efficiente di quella tra processi, non coinvolge il kernel;
- Il context switch tra thread ha decisamente un minor overhead di quello tra processi.

Ma sono molti di più i motivi per cui è conveniente la realizzazione di programmi multithread:

- Guadagno in performance;
- Bloccare parte del processo non implica bloccare tutto il processo;
- Rispecchia la natura intrinseca di molti programmi (task indipendenti, applicazioni server, calcoli numerici, …);
- E molto altro.

Esistono due tipi di thread, **il thread a livello utente** (ULT) **e a livello kernel** (KLT); il primo è implementato quando il kernel non supporta la programmazione in thread ma l'applicativo utente si, facendo in modo che nel contesto di quest'ultimo sia gestito tutto da più thread ma che nel contesto del kernel ci sia sempre un solo processo attivo, mentre i secondi prevedono l'implementazione dei thread a livello kernel ma non a livello applicativo, simulando in quest'ultimo una programmazione single-process. **Ovviamente, è possibile combinare, laddove possibile, i due approcci per una maggior efficienza**:

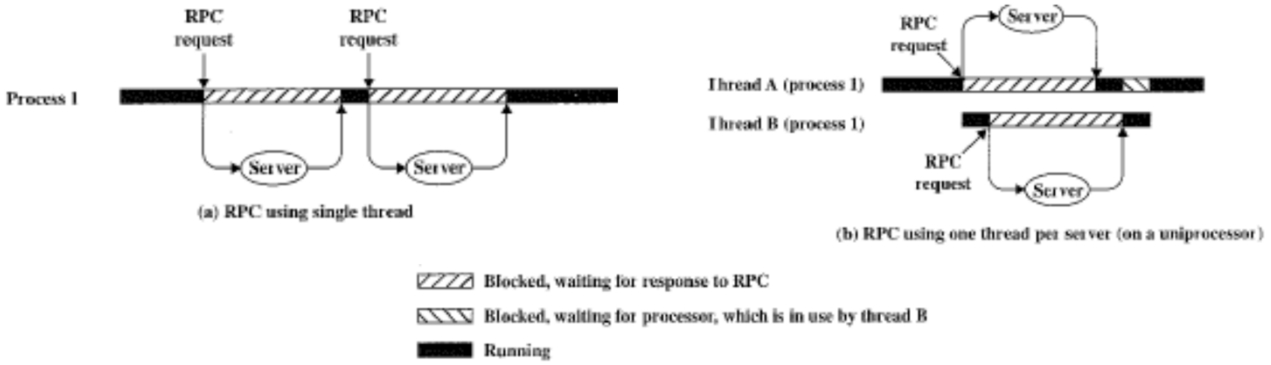


La concorrenza sfrutta i tempi morti del processore, **il multitasking aumenta la capacità di eseguire più processi contemporaneamente** (magari far eseguire un processo che richiede CPU quando in esecuzione ci sarebbe uno che è in attesa di eventi di I/O) **e il multithreading aumenta i flussi di controllo nello stesso processo**. Il parallelismo può essere implicito (Instruction Level Parallelism, ILP) se sfrutta il parallelismo intrinseco delle istruzioni, o esplicito se vengono impiegate architetture multiprocessore e multicomputer; nelle architetture parallele, più thread possono eseguire su processi diversi, ognuno gestito in concorrenza.

Concorrenza e parallelismo sono introdotte al fine di velocizzare l'esecuzione dei programmi; il parametro che permette la valutazione di questo incremento è lo **speed rate**, definito come il tempo di esecuzione sequenziale del processo in rapporto al tempo di esecuzione parallela o concorrente:

$$S = \frac{T_S}{T_P}$$

Il tutto è volto all'ottimizzazione dei tempi di attesa; ad esempio, nel caso seguente l'attesa di un thread non deve precludere l'esecuzione dell'altro thread (in caso di KLT, non ULT):



Tuttavia, **anche il multithreading ha i propri limiti**: l'uso indiscriminato di thread può **aumentare l'overhead dovuto allo scheduling e al context switch** (più thread = più context switch = meno tempo di elaborazione) e **il tempo di esecuzione concorrente è compromesso dalla presenza di punti di sincronizzazione tra i thread** (accesso in mutua esclusione a risorse condivise). Quando il tempo di esecuzione di ciascun thread è largamente superiore al tempo di context-switch tra i threads e quando l'esecuzione concorrente non è troppo vincolata dalla presenza di punti di sincronizzazione (codice non parallelizzabile), allora i thread apportano un buon vantaggio.

Molti problemi hanno una soluzione che è naturale ottenere con un insieme di programmi paralleli; tuttavia, eccetto che in casi molto particolari, lo speed-up che si può ottenere è meno che lineare rispetto al numero di CPU disponibili. Analogamente ai thread, **i programmi che girano in parallelo dovranno prima o poi sincronizzarsi per mettere in comune i dati elaborati da ciascuno**, sarà necessaria una fase comune di inizializzazione prima di far partire i vari programmi in parallelo. **C'è comunque sempre una parte di lavoro che non può essere svolta in parallelo a tutte le altre operazioni**, dopotutto le CPU lavorano sempre un'istruzione alla volta. Ad esempio, si consideri il seguente comando:

```
gcc main.c function1.c function2.c -o output
```

Si supponga che una macchina monoprocessore necessiti:

- 3 secondi per compilare `main.c`;
- 2 secondi per compilare `function1.c`;
- 1 secondo per compilare `function2.c`;
- 1 secondo per linkare gli oggetti.

Avendo a disposizione 3 CPU, i tre sorgenti potrebbero essere compilati in parallelo e poi linkati assieme utilizzando una delle tre CPU; tuttavia, l'operazione di linking può essere eseguita solo dopo che tutti e tre i file oggetto sono stati prodotti, ossia dopo 3 secondi (il tempo che al più ci mette una CPU ad eseguire una compilazione). Il tempo necessario per generare output sarà, quindi, 3 + 1 secondi, 4 secondi, per uno speed-up di 1.75 (7/4), pur avendo utilizzato il triplo dei processori.

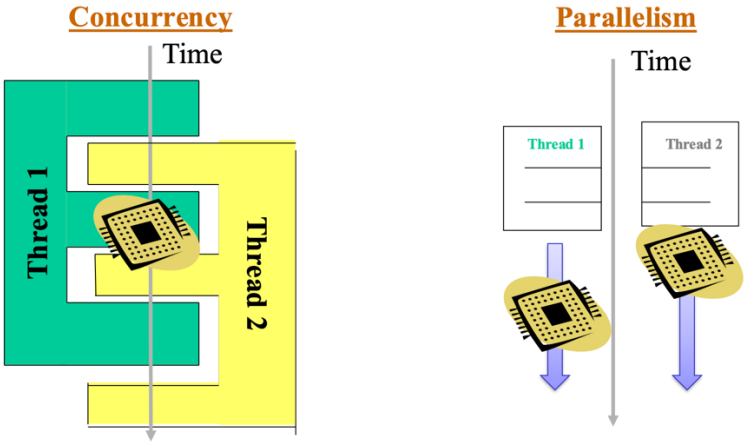
Sia P un programma che gira in un tempo T su un processore, con f la frazione di T dovuta a codice sequenziale e $(1 - f)$ la frazione di T dovuta a codice parallelizzabile; allora, il tempo di esecuzione

dovuto alla parte parallelizzabile passa da $(1-f)T$ a $(1-f)T/n$ se sono disponibili n processori e lo **speed-up** che si ottiene è:

$$S = \frac{fT + (1-f)T}{ft + \frac{(1-f)T}{n}} = \frac{1}{f + \frac{1}{n} - \frac{f}{n}} = \frac{n}{1 + (n-1)f}$$

Cioè, $S = n$ se e solo se $f = 0$, ovvero c'è la totale assenza di codice sequenziale (impossibile); inoltre, si noti che **al tendere all'infinito del numero di processori n, il vantaggio della parallelizzazione tende a non essere così tanto influente**.

Per **programmazione concorrente** si intende l'insieme di tecniche, metodologie e strumenti necessari a fornire il supporto **all'esecuzione di applicazioni software come un insieme di attività svolte simultaneamente**; in particolare, sono di rilievo le **primitive per definire attività indipendenti** (come processi e thread) e per la comunicazione e la sincronizzazione tra attività eseguite in modo concorrente (si ricordi che concorrenza non significa parallelismo). **I processi concorrenti sono l'insieme di processi la cui esecuzione si sovrappone nel tempo** o, più in generale, in un sistema monoprocessore **due processi si dicono concorrenti se la prima operazione di uno comincia prima dell'ultima dell'altro**; le problematiche che induce un'architettura di questo tipo sono l'assegnazione del tempo di CPU, la presenza di risorse condivise, la comunicazione tra i processi e la sincronizzazione.



I due punti critici nello sviluppo di programmi concorrenti sono:

- **Safety**, che consiste nell'assicurare la consistenza, cioè la mutua esclusione (le risorse condivise sono aggiornate in maniera atomica) e la condition synchronization (alcune operazioni potrebbero richiedere di essere differite se le risorse condivise non sono in uno stato appropriato, come la lettura di un buffer vuoto);
- **Liveness**, che consiste nell'assicurare l'avanzamento dell'elaborazione, cioè no deadlock (alcuni processi possono sempre accedere ad una risorsa condivisa) e no starvation (tutti i processi, prima o poi, possono accedere alle risorse condivise).

Due processi concorrenti possono essere indipendenti, se l'esecuzione di uno non è influenzata dall'esecuzione dell'altro (proprietà della riproducibilità), **o interagenti**, **se l'esecuzione di uno è influenzata dall'altro e/o viceversa**; in quest'ultimo caso, l'effetto dell'interazione dipende dalla velocità relativa dei processi, configurando un comportamento non riproducibile. Le tipologie di interazioni possibili sono:

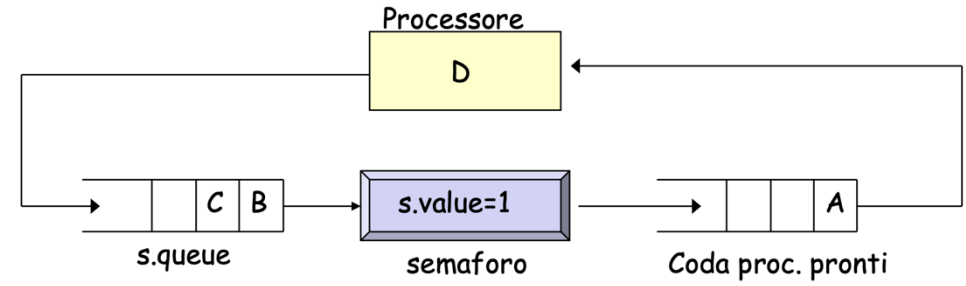
- **Mutua esclusione**, competizione per l'uso di risorse comuni che possono essere utilizzate contemporaneamente;
- **Comunicazione**, cooperazione nell'eseguire un'attività comune mediante scambio di informazioni;
- **Interferenza**, dovuta a competizione tra processi per uso non autorizzato di risorse comuni oppure ad un'erronea soluzione di problemi di competizione e di cooperazione e si manifesta spesso in modo non deterministico, in quanto dovuta alla differente velocità di esecuzione dei processi.

Nei casi citati, per un corretto funzionamento, **è necessario imporre vincoli nell'esecuzione delle operazioni dei processi**:

- **Competizione**, un solo processo alla volta deve avere accesso alla risorsa comune (sincronizzazione indiretta o implicita), con il rischio di mutua esclusione, deadlock e starvation;
- **Cooperazione**, le operazioni eseguite dai processi concorrenti devono seguire una sequenza prefissata (sincronizzazione diretta o esplicita), come il problema del Produttore/Consumatore.

Per **semaforo** si intende un **tipo di dato astratto che incapsula una variabile di tipo intero** (`s.value`) **una coda** (`s.queue`) **per tenere traccia dei processi che si sono sospesi nell'attesa di una signal**. Su un semaforo sono definite le seguenti operazioni:

- Inizializzazione della variabile ad un valore non negativo;
- Operazione di wait, che ha l'effetto di decrementare il valore del semaforo e, se questo risulta negativo, il processo viene bloccato;
- Operazione di signal, che ha l'effetto di incrementare il valore del semaforo e, se questo diventa minore o uguale a zero viene "sbloccato" un processo che si era sospeso durante l'esecuzione della wait.



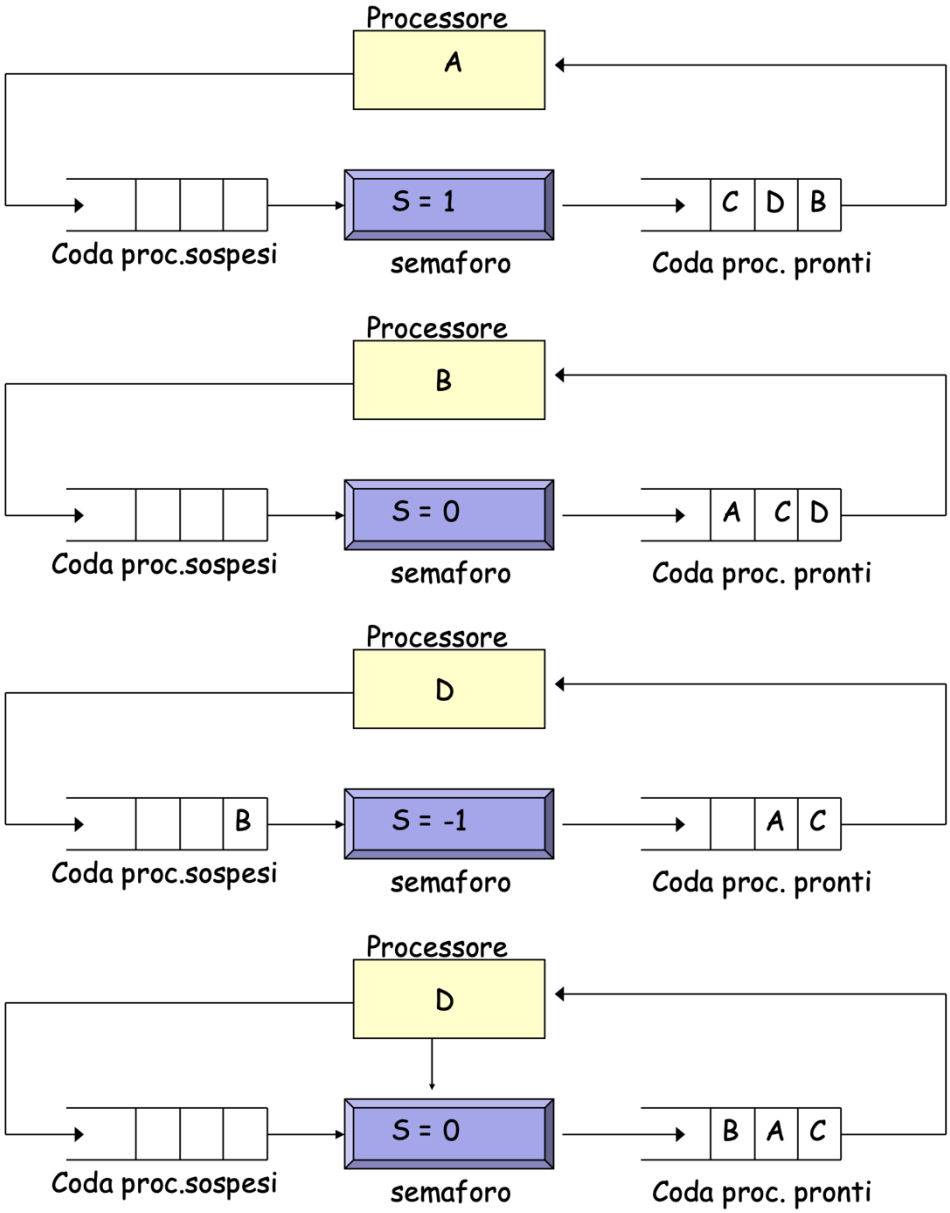

```
void wait(sempahore s) {
s.value--;
if (s.value<0){
        s.queue.insert(Process);
        suspend(Process);
        }
}
```

```
void signal(sempahore s) {
s.value++;
if (s.value<=0){
        s.queue.remove(Process);
        wake-up(Process);
        }
}
```

Si consideri la seguente simulazione di scheduling:

1. Inizialmente, A è in esecuzione, B, C e D sono ready e il valore del semaforo è 1, si esegue una wait;
2. A esegue una wait, il valore di S viene decrementato, A non viene sospeso;
3. B esegue una wait, il valore del semaforo è decrementato e B viene sospeso;

4. D esegue una signal, che permette a B di essere inserito nella coda dei processi pronti (si noti che in questo caso D non viene sospeso);



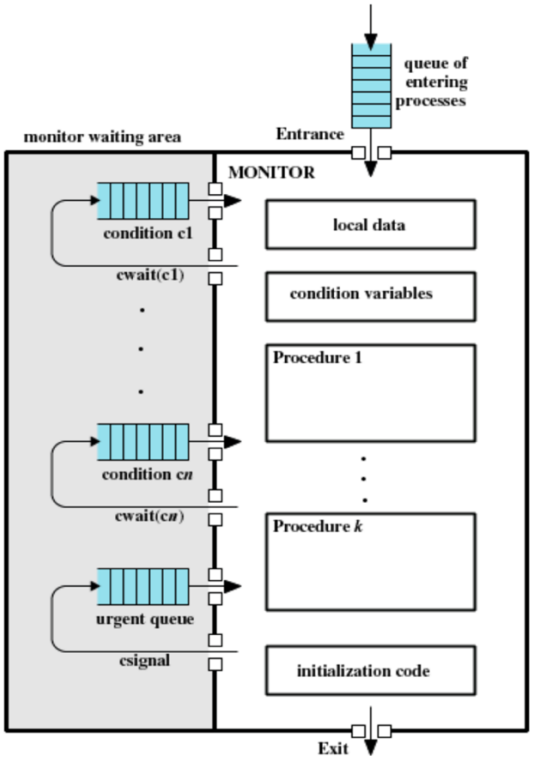
Una **Risorsa Critica** è una **risorsa ad uso esclusivo che due o più processi** (thread) **vogliono utilizzare**, mentre la **porzione di codice che utilizza la risorsa** è detta **Sezione Critica**. **Un solo processo alla volta può accedere alla sezione critica**; per garantire questo vincolo è utilizzato un particolare semaforo, detto **mutex** (mutual exclusion), inizializzato ad 1:

```
const int n = /* numero di processi */
semaphore mutex;
…
void P(int i) { /* il codice del processo i-mo*/
        …
        wait(mutex);
        /* sezione critica */
        signal(mutex);
        …
}
…
int main() {
        mutex.value = 1;
        /* esecuzione concorrente di P(1), …, P(n) */
}
```

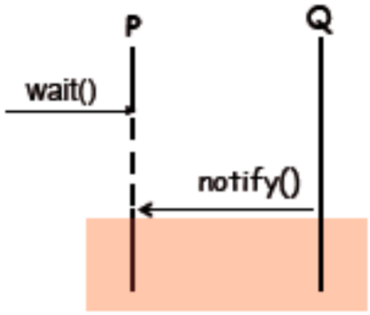
Un monitor è un costrutto sintattico che associa un insieme di operazioni ad una risorsa condivisa tra più processi (thread) **e garantisce la mutua esclusione**: un solo thread per volta può eseguire una procedura definita nel monitor ("il thread è nel monitor"), un thread che richiama una procedura di monitor, quando un altro thread è nel monitor, viene bloccato su una coda associata al monitor stesso e quando un thread all'interno del monitor si blocca, un altro thread deve potervi accedere. **La sospensione di un thread, nel caso in cui la condizione logica non sia verificata, avviene utilizzando una variabile di tipo condition**; le principali operazioni su variabili condition sono:

- **Wait**, il thread che la esegue rilascia il lock sul monitor e si mette in attesa di essere risvegliato da una signal sulla stessa condizione (per ogni variabile condition è implementata una coda);
- **Signal**, risveglia un thread in attesa di una determinata condizione.

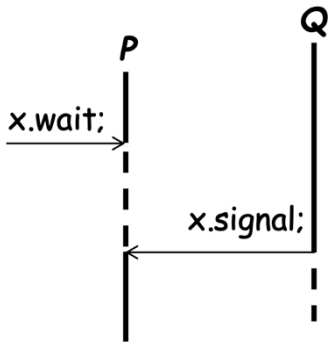
Per la variabile condition `var_cond x`, `x.wait` provoca la sospensione del processo fino a che un altro processo non esegue una `x.signal`.



A seguito di un'operazione di notifica (`notify()`), entrambi i thread possono, concettualmente, proseguire e violare le proprietà di un monitor, è necessario che uno dei due sia sospeso.

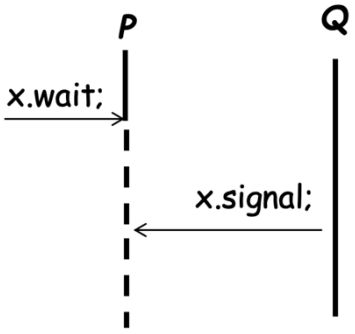


Una soluzione prevede l'implementazione di una "**signal and wait**", con la quale si prevede che il processo P risvegliato riprenda immediatamente l'esecuzione mentre Q viene sospeso:



Q viene sospeso per evitare che possa modificare nuovamente la condizione di sincronizzazione mentre P esegue. Un caso particolare di questo schema è la "**signal and urgent wait**".

Una seconda soluzione è la "**signal and continue**" (o "**wait e notify**"), con la quale è privilegiato il processo segnalante rispetto al segnalato: il processo Q segnalante prosegue la propria esecuzione mantenendo l'accesso esclusivo al monitor, dopo aver risvegliato P, il quale viene trasferito alla coda associata all'ingresso del monitor; poiché in questa coda possono esistere altri processi e poiché questi possono precedere l'esecuzione di P, è possibile che sia modificato il valore della condizione di sincronizzazione, bloccando nuovamente P quando sarà il suo turno.



Volendo approfondire il problema del **Produttore/Consumatore**, si consideri l'esistenza di due categorie di processi: i processi produttore, che depositano un messaggio su una risorsa condivisa, e i processi consumatore, che prelevano il messaggio dalla risorsa condivisa. I vincoli di questa categorizzazione sono i seguenti:

- Il produttore non può produrre un messaggio prima che qualche consumatore abbia prelevato il messaggio precedente;
- Il consumatore non può prelevare alcun messaggio fino a che un produttore non l'abbia depositato.

Pur esistendo (potenzialmente) un problema di mutua esclusione nell'utilizzo del buffer comune, la soluzione impone un ordinamento nelle operazioni dei due processi, rendendo necessario che produttori e consumatori si coordinino per indicare, rispettivamente, l'avvenuto deposito e l'avvenuto prelievo.

Un problema analogo, ma con diversi vincoli, detto **Lettore/Scrittore** prevede l'esistenza di due categorie di processi: i processi lettore, che leggono un messaggio su di una risorsa condivisa, e i processi scrittore, che scrivono un messaggio dalla risorsa condivisa. I vincoli su di questa categorizzazione sono i seguenti:

- I processi lettori possono accedere contemporaneamente alla risorsa;
- I processi scrittori hanno accesso esclusivo alla risorsa;
- I lettori e gli scrittori si escludono mutuamente dall'uso della risorsa.

Per la programmazione concorrente, Python mette a disposizione due moduli: **`threading`, per la creazione di software multi-thread, e `multiprocessing`, per la creazione di software multi-process**; tuttavia, entrambi i moduli mettono a disposizione i meccanismi necessari per supportare l'esecuzione di codice concorrente.

Il modulo `threading` dispone di un'interfaccia di alto livello per il multithreading, costruita a partire dal modulo `_thread`, il quale mette a disposizione primitive di basso livello per l'avvio e la sincronizzazione di thread; tuttavia, il suo utilizzo è deprecato. Complessivamente, l'insieme di classi messe a disposizione include:

- `Thread`;
- `Thread-local Data`;
- `Lock`/`RLock`;
- `Condition`;
- `Semaphore`;
- `Event`.

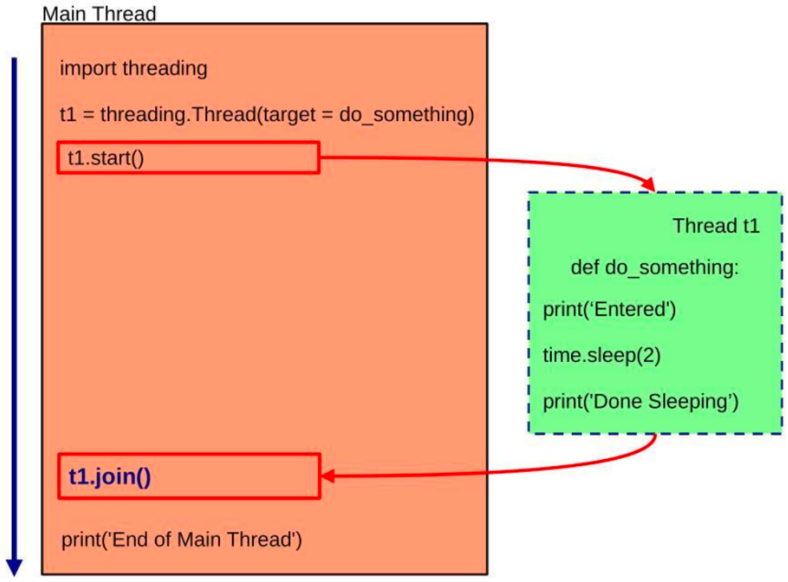
La classe `Thread` rappresenta un'attività che esegue in un flusso di controllo separato, un thread; esiste un oggetto, il main thread istanza della classe `_MainThread`, che corrisponde al flusso di controllo associato all'intero programma Python. **Tutti i thread in Python condividono lo spazio di memoria del loro processo padre**. Esistono vari modi per creare thread in Python, due di questi sono:

Istanziare un oggetto Thread, passando un "callable object" al costruttore della classe

```
import threading

def func():
    print("Thread running")


if __name__ == "__main__":

    # creating thread
    t1 = threading.Thread(target=func)

    # starting thread
    t1.start()

    # wait until the thread finishes
    t1.join()
```

Creare una classe che estende quella Thread, ridefinendo il metodo run()

```
import threading

class myThread(threading.Thread):

    def run(self):
        print("Thread running")


if __name__ == "__main__":

    # creating thread
    t1 = myThread()

    # starting thread
    t1.start()

    # wait until the thread finishes
    t1.join()
```



Alcuni dei metodi messi a disposizione dalla classe Thread sono:

- `Thread(group=None, target=None, name=None, args=(), kwargs={}, *, daemon=None)`, è il costruttore della classe:
  - `group`, riservato per future implementazioni di ThreadGroup, deve essere `None`;
  - `target`, rappresenta il callable object che sarà invocato dal metodo `run`;
  - `name`, il nome del thread;
  - `args`, tupla contenente gli argomenti da passare all'invocazione del target;
  - `kwargs`, dizionario di keyword da passare all'invocazione del target;
  - `daemon`, boolean che specifica se il thread deve essere un demone o meno;
- `start()`, avvia il thread portando all'esecuzione del metodo run non appena il thread sarà schedulato;
- `run()`, metodo che contiene il corpo del thread e che ne rappresenta il comportamento;
- `join(timeout=None)`, metodo blocante che attende fino alla terminazione (naturale o con eccezioni) del thread:
  - `timeout`, numero floating point utilizzato per definire il massimo tempo di attesa in secondi;
- `is_alive()`, ritorna se il thread è alive o meno e dovrebbe essere utilizzato in combinazione con il metodo `join` quando viene specificato un timeout (in questo caso, infatti, il `join` ritorna sempre `None`, anche nel caso di timeout scaduto).

```
from threading import Thread

class MyThread(Thread):

  def run(self):
    print("Running MyThread...\n")
    print("is_alive?: ", self.is_alive())
  def method1(self):
    print("method1 invocato...\n")

m = MyThread()
m.start()
m.join()
m.method1()
print("is_alive?: ", m.is_alive())
```

```
Running MyThread...

is_alive?: True

method1 invocato...

is_alive?: False
```

Un thread è ancora un normale oggetto Python; pertanto, è possibile invocare qualsiasi metodo o accedere a qualsiasi attributo del thread in qualsiasi momento, fintantoché l'oggetto esiste. La chiamata di join permette di attendere che il thread finisce l'esecuzione ma non distrugge l'oggetto in sé e per sé.

Un thread, come si è potuto intuire, **può essere contrassegnato come "demone",** volendo significare che **l'intero prorgamma Python termina quando rimangono solo thread dello stesso tipo**; in altre parole, **il programma non può terminare se c'è almeno un thread non-daemon ancora vivo**. Il valore iniziale, per la proprietà daemon, è ereditato dal thread da cui si sta creando il nuovo; inoltre, il flag daemon può essere triggerato sia in fase di costruzione che accedendo all'omonima proprietà. **I thread daemon vengono interrotti alla terminazione del processo e le loro risorse potrebbero non essere rilasciate correttamente**; se si vuole che i thread si arrestino in maniera graduale, bisogna renderli non-daemon e utilizzare meccanismi di segnalazione adeguati (come, ad esempio, un evento).

Esempi di non-daemon threads sono:

- Elaborazione di file, si supponga che il programma avvii un thread che scrive dati su un file o effettua operazioni di backup, non si vuole che il programma termini prima che l'operazione sia completata, altrimenti si potrebbe incorrere in file incompleti o corrotti;
- Richieste di rete, un thread normale può essere utilizzato per gestire attività importanti come l'invio o la ricezione di dati in rete, rendendo necessario che il thread termini la comunicazione anche se il programma principale viene completato;
- Elaborazione di dati in background, se i calcoli richiedono tempo è necessario che il thread che li gestisce completi prima che il programma esca, in modo da garantire la correttezza dei risultati.

```
# import module
from threading import *
import time

# creating a function
def thread_1():
        for i in range(5):
                print('this is non-daemon thread')
                time.sleep(2)

# creating a thread T
T = Thread(target=thread_1)

# starting of thread T
T.start()

# main thread stop execution till 5 sec.
time.sleep(5)
print('main Thread execution')
```

Output:
this is non-daemon thread
this is non-daemon thread
this is non-daemon thread
main Thread execution
this is non-daemon thread
this is non-daemon thread

Esempi di daemon threads sono:

- Logging, un thread demone può essere usato per eseguire logging continuamente di dati durante l'esecuzione del programma ma, una volta terminato il programma, il logging si interrompe immediatamente e non è necessario conservare i log dopo la fine del programma;
- Attività di manutenzione, compiti come la pulizia di file temporanei o il refresh delle cache non dovrebbero bloccare la terminazione del programma principale;
- Monitoraggio dei servizi in background, il monitoraggio delle risorse o una connessione di rete durante l'esecuzione del programma principale non deve dipendere dal programma principale, nella misura in cui se questo viene terminato il monitoraggio diventa irrilevante e il daemon thread termina.

```
# import modules
from threading import *
import time

# creating a function
def thread_1():
  for i in range(5):
    print('this is thread T')
    time.sleep(3)

# creating a thread
T = Thread(target = thread_1, daemon=True)

# starting of Thread T
T.start()
time.sleep(5)
print('this is Main Thread')
```

Output:
this is thread T
this is thread T
this is Main Thread

Poiché i thread in Python condividono lo spazio di memoria del loro processo padre, potrebbe essere necessario definire variabili specifiche per i casi d'uso specifici, in modo da evitare effetti collaterali indesiderati; in particolare, se è necessario memorizzare dati unici per ogni thread, Python fornisce un oggetto thread-local da utilizzare. La classe `local` consente di poter gestire dati locali ad un thread: creare un'istanza della classe `local` attraverso `threading` e salvare gli attributi che si vuole rendere locali al thread su tale istanza. Tutti i dati storicizzati nell'istanza `local` saranno differenti per ogni thread.

```
(MainThread) No value yet
(MainThread) value=999
(Thread-1) No value yet
(Thread-1) value=51
(Thread-2) No value yet
(Thread-2) value=19
```

```
import threading
import logging
import random

logging.basicConfig(level=logging.DEBUG,
                    format='(%(threadName)-0s)
                    %(message)s',)

def show(d):
    try:
        val = d.val
    except AttributeError:
        logging.debug('No value yet')
    else:
        logging.debug('value=%s', val)

def f(d):
    show(d)
    d.val = random.randint(1, 100)
    show(d)

if __name__ == '__main__':
    d = threading.local()
    show(d)
    d.val = 999
    show(d)

    for i in range(2):
        t = threading.Thread(target=f, args=(d,))
        t.start()
```

La classe `Lock` implementa la rispettiva primitiva di sincronizzazione di basso livello fornita dal modulo `_thread`; un lock può trovarsi in due stati: **locked o unlocked**. **Un lock è creato sempre nello stato unlocked ma passa allo stato locked quando un thread lo acquisisce** (successive acquisizioni dello stesso lock da altri thread porteranno questi in stato di attesa) **e torna allo stato unlocked quando un thread lo rilascia** (uno degli eventuali thread in attesa sui lock sarà risvegliato ed eseguito, la scelta dei thread varia a seconda delle implementazioni Python). La classe `Lock` prevede due metodi principali per l'acquisizione e il rilascio di lock:

- `acquire(blocking=True, timeout=-1)`:
  - quando lo stato del lock è unlocked, il metodo cambia lo stato in locked e ritorna immediatamente, altrimenti il metodo blocca il thread chiamante finché un altro thread non invoca `release()` per cambiare lo stato del lock in unlocked;
  - Ritorna `True` se il lock è stato acquisito e `False` in caso contrario;
  - `blocking`, definisce se la chiamata è bloccante:
    - `True`, la chiamata è bloccante se il lock è già stato precedentemente acquisito;
    - `False`, la chiamata torna immediatamente restituendo `False`;
  - `timeout`, imposta un tempo massimo di attesa:
    - Se è positivo, il chiamante resterà bloccato per al massimo `timeout` secondi;
    - Allo scadere del timeout, il metodo restituirà `False`.
- `release()`, rilascia il lock e ritorna (non prevede un particolare valore di ritorno) e può essere invocata solo quando lo stato del lock è locked e porta al passaggio del lock nello stato unlocked;
- `locked()`, verifica lo stato di un lock e, se è locked, ritorna `True`.

```
from threading import Thread, Lock

counter = 0

def increase(by, lock):
    global counter
    lock.acquire()

    local_counter = counter
    local_counter += by
    counter = local_counter
    print(f'counter={counter}')

    lock.release()

if __name__ == '__main__':

    lock = Lock()

    # create threads
    t1 = Thread(target=increase, args=(10, lock))
    t2 = Thread(target=increase, args=(20, lock))

    # start the threads
    t1.start()
    t2.start()

    # wait for the threads to complete
    t1.join()
    t2.join()

    print(f'The final counter is {counter}')
```

```
counter=10
counter=30
The final counter is 30
```

Lo statement `with` è utilizzato per semplificare l'accesso ad una risorsa (è stato mostrato anche in merito alla lettura e alla scrittura di file), **acquisendola e rilasciandola al termine del blocco di istruzioni specificato**; per la primitiva di locking, è possibile acquisire il lock con `with` e rilasciarlo automaticamente al termine dell'ultima istruzione del blocco di codice in esecuzione. I seguenti esempi codificano per lo stesso comportamento:

```
lock = Lock()
with lock:
   # critical section
```

```
lock = Lock()
lock.acquire()
#critical section
lock.release()
```

Si noti come l'utilizzo di `with` non richieda l'impiego dei metodi di acquisizione e rilascio della classe `Lock`.

`RLock` implementa i reentrant lock, lock che **possono essere acquisiti più volte dallo stesso thread**, ed è utile negli scenari in cui **lo stesso thread potrebbe aver bisogno di acquisire un blocco già in suo possesso** (come per funzioni ricorsive, metodi che chiamano altri metodi bloccanti o sezioni critiche innestate). Oltre allo stato di locked/unlocked, **i reentrant lock prevedono due ulteriori stati**:

- **owning thread**, quale thread ha acquisito il lock;
- **recursion level**, per tenere traccia del numero di acquisizioni fatte dallo stesso thread.

Come la classe `Lock`, sono previsti i metodi `acquire()` e `release()` per modificare lo stato, sebbene presentino delle leggere differenze:

- Se la `acquire()` è invocata da un thread che già ha acquisito il lock (owning thread), la chiamata non è bloccante e viene incrementato il recursion level;
- La `release()` rilascia il lock e decrementa il recursion level ma solo quando quest'ultimo diventa pari a zero il lock passa allo stato unlocked, con la conseguente possibilità di acquisizione da parte di un altro thread.

La classe `Condition` implementa una condition variable, variabili che permettono ad uno o più thread di rimanere in attesa (che una determinata condizione si verifichi) finché non saranno notificati da un altro thread (quando la condizione è verificata). **Una condition variable è sempre caratterizzata da un lock** che, se creato automaticamente con la condition variable (condizione di default), sarà un `RLock`; tuttavia, è sempre possibile utilizzare un lock già esistente, che esso sia `Lock` o `RLock`, passandolo al costruttore di classe e sono forniti i metodi di `acquire()` e `release()` che agiscono sul lock a cui la condition variable fa riferimento. Più nel dettaglio:

- `Condition(lock=None):`
  - Costruttore della classe;
  - `lock`, rappresenta il lock da associare alla condition variable e, se impostato a `None`, un `RLock` sarà creato automaticamente;
- `acquire();`
- `release();`
- `wait(timeout=None):`
  - Attende fino alla notifica o fino alla scadenza di un timeout;
  - Il metodo rilascia il lock e resta bloccato fino alla chiamata della `notify()` o `notify_all()` da parte di un altro thread sulla stessa condition variable, o fino alla scadenza del timeout;
  - Il lock viene riacquisito non appena il thread è risvegliato ed il metodo ritorna restituendo un booleano:
    - `False`, nel caso di timeout scaduto;
    - `True`, nel resto dei casi;
- `wait_for(predicate, timeout=None):`
  - Attende fino a quando una condizione diventa `True`;
  - `predicate`, deve essere una condizione o un callable object che ritorni un booleano;
  - `timeout`, massimo tempo di attesa in secondi;
- `notify(n=1):`

  - Risveglia il numero di thread indicati dal parametro n in attesa sulla condition variable;
  - Si noti che il thread risvegliato non potrà riprendere la sua esecuzione se non riuscirà a riacquisire il lock;
- `notify_all()`:
  - Risveglia tutti i thread sulla stessa condition variable.

Un esempio di uso della classe `Condition` è il seguente, notando che il ciclo `while` che verifica la condizione è necessario; inoltre, l'utilizzo di `with cv` evita la necessità di dover chiamare i metodi `acquire()` e `release()` e quando il thread viene risvegliato dalla `notify`, la condizione su cui era in attesa potrebbe non essere più verificata (un'alternativa sarebbe l'uso di `wait_for()`):

```
. . .

cv_lock = threading.Lock()
cv = Condition(lock=cv_lock)

. . .

# Consume one item

with cv:

    while not an_item_is_available(queue):

        cv.wait()

    get_an_available_item(queue)


# Produce one item

with cv:

    make_an_item_available(queue)

    cv.notify()
```

```
# Consume one item

with cv:

    cv.wait_for(an_item_is_available)

    get_an_available_item()
```

La classe `Semaphore` permette di istanziare un oggetto semaforo, il quale gestisce un contatore interno inizializzato alla creazione del semaforo, decrementato ad ogni operazione di `acquire()` e incrementato ad ogni relazione di `release()` e che, quando arriva a zero come conseguenza di una `acquire()`, comporta il bloccaggio del thread chiamante finché non viene effettuata un'operazione di `release()` da un altro thread sullo stesso oggetto semaforo. Le primitive fornite da questa classe sono:

- `Semaphore(value=1)`:
  - Costruttore della classe;
  - `value`, rappresenta il valore a cui è inizializzato il semaforo;
- `acquire(blocking=True, timeout=None)`:
  - Acquisice un semaforo e, se invocato senza parametri:
    - Se il valore del contatore è maggiore di zero, decrementa il valore di 1 e ritorna `True`;
    - Se il valore del contatore è zero, la chiamata è bloccante per il thread che l'ha invocata, il quale sarà risvegliato solo in seguito ad una `release()`, decrementando il contatore e ritornando `True`;
  - `blocking`, se impostato a `False`, la chiamata non è bloccante, ritornando `False` immediatamente invece di bloccare il chiamante;
  - `timeout`, rappresenta il massimo tempo (in secondi) per cui il chiamante resta bloccato sulla `acquire()` e, se scade, il metodo ritorna `False`;
- `release(n=1)`:
  - Rilascia un semaforo incrementando il valore del contatore a `n` e risvegliando un numero `n` di thread.

```
from threading import *
from time import sleep
from random import random

# creating thread instance where count = 3
obj = Semaphore(3)

def display(name):

    obj.acquire()

    value = random()
    sleep(value)
    print(f'Thread {name} got {value}')

    obj.release()

if __name__ == '__main__':

    threads = []

    # creating and starting multiple threads
    for i in range(10):
        t = Thread(target = display , args = ('Thread-' + str(i),))
        thread.append(t)
        t.start()

    # wait for the threads to complete
    for thread in threads:
        thread.join()
```

```
Thread 1 got 0.4468840323081351
Thread 0 got 0.7288038062917327
Thread 2 got 0.4497887327563145
Thread 4 got 0.019601471581036645
Thread 3 got 0.5114751539092154
Thread 6 got 0.6191428550062478
Thread 5 got 0.9893921843198458
Thread 8 got 0.022640379341017924
Thread 7 got 0.20649643092073067
Thread 9 got 0.18636311846540998
```

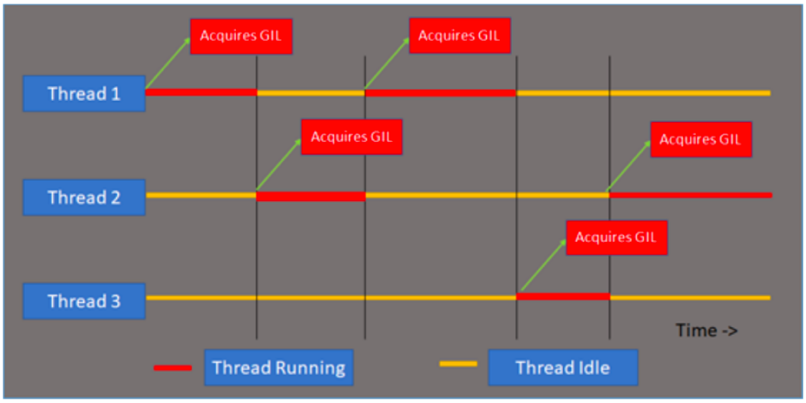
La classe `Event` fornisce un semplice meccanismo di comunicazione tra thread; infatti, attraverso questa classe, un thread può segnalare un evento per il quale altri thread sono in attesa. Ogni `Event` è caratterizzato da un flag interno (booleano) che può essere impostato a `True` con il metodo `set()` e resettato a `False` con il metodo `clear()`; un thread che è in attesa di un determinato evento può utilizzare il metodo `wait()` sull'evento e rimarrà bloccato finché il flag dell'evento stesso non tornerà `True`. I metodi della classe Event sono i seguenti:

- `is_set()`, ritorna `True` se il flag interno è settato a tale valore e `False` in caso contrario;
- `set()`, imposta il flag interno al valore `True` e tutti i thread in attesa dell'evento vengono risvegliati;
- `clear()`, imposta il flag interno al valore `False` e tutti i thread che invocheranno una `wait()` sull'evento saranno bloccati;
- `wait(timeout=None)`:
    - Pone il thread in attesa che il flag interno venga impostato a `True` (se già lo è ritorna immediatamente);
    - `timeout`, l'attesa massima in secondi sul metodo wait alla cui scadenza il metodo ritorna `False`.

```
import threading as thd
import time

def task(event, timeout):
   print("Started thread but waiting for event...")

   # make the thread wait for event with timeout set
   event_set = event.wait(timeout)

   if event_set:
     print("Event received, releasing thread...")
   else:
     print("Time out, moving ahead without event...")

if __name__ == '__main__':

    # initializing the event object
    e = thd.Event()

    # starting the thread
    thread = thd.Thread(name='Event-Thread', target=task, args=(e,4))
    thread.start()

    # sleeping the main thread for 3 seconds
    time.sleep(3)

    # generating the event
    e.set()
    print("Event is set.")

    # wait for the thread to complete
    thread.join()
```

```
Started thread but waiting for event...
Event is set.
Event received, releasing thread...
```

Per multi-threading con Global Interpreter Lock (GIL) si intende un meccanismo utilizzato dall'interprete Python per **assicurare che solo un thread alla volta possa eseguire bytecode**, semplificando l'implementazione dell'interprete stesso e rendendo l'object model Python implicitamente "safe" da accessi concorrenti. **GIL nasce per realizzare thread safety memory management ed è richiesto dalle librerie C utilizzate da Python**.



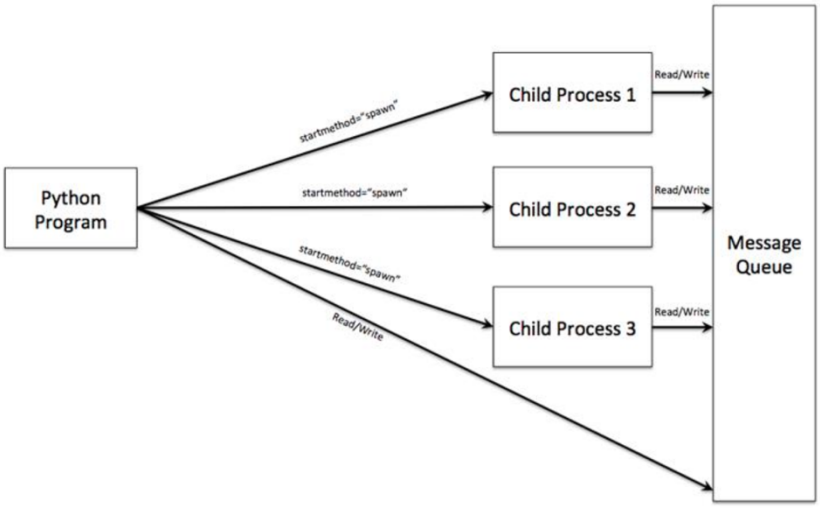
I vantaggi di GIL sono la facile implementazione del multithreading nell'interprete di Python, a scapito del parallelismo, le performance migliorate degli applicativi single-threaded, dovendo gestire solo un lock, e la facile integrazione delle librerie C non thread safe; di contro, gli svantaggi sono la **riduzione del livello di parallelismo ottenibile su macchine multi-processore e l'impatto elevato**, in termini di performances, **sui task CPU-bound**, dal momento in cui è un approccio pensato per task I/O-bound.

Alcuni moduli, sia standard che di terze parti, sono progettati per rilasciare il GIL quando vengono eseguiti task che includono operazioni di I/O bloccanti (come `timer.sleep()`); in passato, sono stati fatti tentativi di rimozione di GIL dall'interprete Python ma non hanno avuto successo: le performances degli applicativi single-thread venivano ridotte troppo, la complessità dell'interprete aumentava e c'era la necessità di modificare tutte le estensioni delle librerie C che sfruttano il GIL. Nonostante ciò, dalla versione 3.14 è possibile installare Python GIL-free utilizzando opportuni installer o facendo una build particolare; tuttavia, la versione 3.14 ha, di default, il GIL attivo.

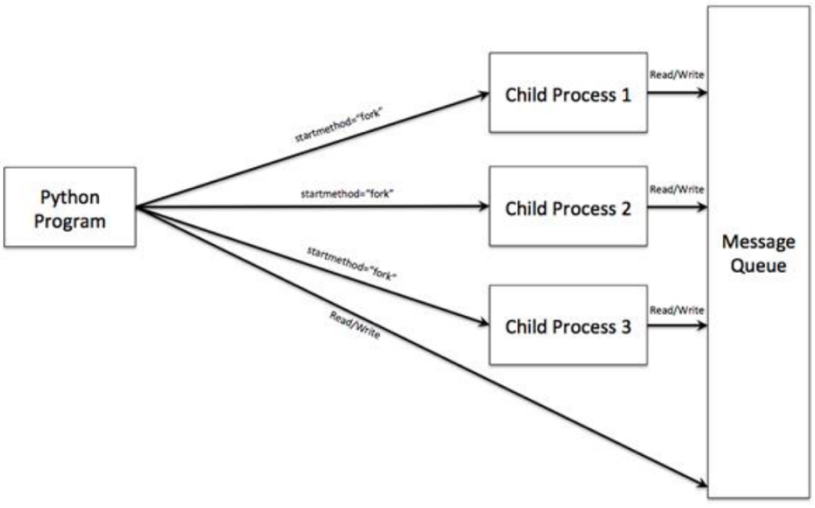
Il modulo `multiprocessing` supporta lo spawning di processi attraverso una API simile a quella messa a disposizione dal modulo `threading`, superando il limite dovuto al GIL attraverso l'impiego di processi multipli; in particolare, permette la creazione di più Python interpreter processes, ognuno dei quali con il proprio GIL per evitare i problemi del modulo `threading`, e permettendo al programmatore di sfruttare totalmente le architetture multicore. **È consigliabile questo modulo, piuttosto che `threading`, quando si hanno a disposizione core**

multipli e quando i task da eseguire sono principalmente CPU bound. A seconda della piattaforma, il modulo multiprocessing supporta tre modi per avviare un processo attraverso il set di `start_method`, il quale è impostato tramite il metodo `set_start_method(string)`, a disposizione nel modulo, e passandovi una stringa con il modo di avvio; questi ultimi possono essere:

- `spawn`, disponibile su Unix e Windows e di default per Windows e macOS ma anche il più lento, prevede che il processo padre avvii un nuovo Python interpreter process, mentre il processo figlio eredita solo le risorse necessarie ed esegue il metodo `run()`, escludendo le informazioni non necessarie (come i file descriptor);

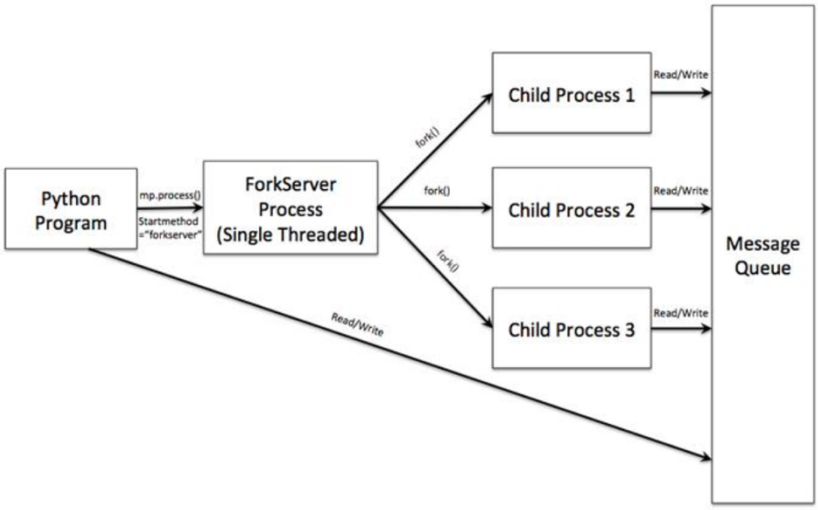


- `fork`, disponibile e di default solo su Unix, il processo padre utilizza la system call `fork` per avviare un nuovo Python interpreter process (processo figlio), il quale all'avvio eredita tutte le risorse del processo padre, sebbene la `fork()` sia, in questo caso, considerata unsafe perché i thread e le risorse del processo padre potrebbero non funzionare bene nel processo figlio (solo il thread running nel processo padre è attivo nel processo figlio e tutti i lock acquisiti da altri thread al momento della `fork()` rimangono acquisiti, portando a potenziali deadlock);



- `forksever`, disponibile su sistemi Unix che supportano il passaggio di file descriptor attraverso Unix pipe, viene creato un server process e, quando un nuovo processo è necessario, il processo padre si connette al server richiedendo ad esso la `fork` (che, dal momento in cui

il server process è singel threaded, è safe) di un nuovo processo, non ereditando le risorse necessarie.



Il modulo `multiprocess` è un'estensione del modulo `multiprocessing` per fornire un meccanismo di serializzazione migliorata, utilizzando `dill`; del modulo `multiprocessing` è sfruttato il supporto allo spawning dei processi utilizzando le API del modulo `threading` della libreria standard di Python, permettendo il trasferimento di oggetti tra processi tramite pipe o code multi-produttore/multi-consumatore e la condivisione di oggetti tra processi utilizzando un processo server o, per dati semplici, una shared memory.

Un oggetto della classe `Process` rappresenta un'attività da eseguire in un processo separato e possiede molti metodi equivalenti a quelli della classe `threading.Thread`:

- `Process(group=None, target=None, name=None, args=(), kwargs={}, *, daemon=None)`, è il costruttore della classe;
- `start()`, avvia il processo portando all'esecuzione del metodo `run()` non appena il processo sarà schedulato;
- `run()`, metodo che contiene il corpo del processo e che ne rappresenta il comportamento;
- `join(timeout=None)`, metodo bloccante che attende fino alla terminazione (naturale o con eccezioni) del processo;
- `is_alive()`, metodo che ritorna se il processo è alive o meno.

Come per la classe `Thread`, anche la classe `Process` permette la creazione di un processo in vari modi, usando callable object o estendendo la classe `Process` stessa:

Istanziare un oggetto Process, passando un "callable object" al costruttore della classe

```
import multiprocess as mp

def func():
   print ('Process running')
   return


if __name__ == '__main__':

    # creating process
    p = mp.Process(target = func)

    # starting process
    p.start()

    # wait until the process finishes
    p.join()
```

Creare una classe che estende quella Process, ridefinendo il metodo run()

```
import multiprocess as mp

class MyProcess(mp.Process):

    def run(self):
       print ('Process running')
       return

if __name__ == '__main__':

    # creating process
    p = MyProcess()

    # starting process
    p.start()

    # wait until the process finishes
    p.join()
```

Il modulo `multiprocessing` contiene primitive di sincronizzazione equivalenti a quelle fornite nel modulo `threading`; pertanto, anche con i processi, è possibile usare `Lock`, `RLock`, `Semaphore`, `Condition` ed `Event`.

Quando sono utilizzati processi multipli, è spesso necessario prevedere dei canali di comunicazione tra gli stessi che permettano di evitare l'uso di primitive di sincronizzazione. Il modulo multiprocessing supporta due canali di comunicazione tra processi:

- **`Pipe`**, che consente la connessione fra più processi;
- **`Queue`**, che supporta (di fatto implementa) il problema produttori-consumatori multipli.

Una `Pipe` è caratterizzata da una coppia di connection objects, che rappresentano gli endpoint della pipe, mentre **una `Queue` è una process shared queue**, implementata attraverso una pipe e locks/semaphores. Sia `Queue` che `Pipe` sono thread/process-safe.

La funzione `Pipe([duplex])` messa a disposizione da `multiprocessing` **ritorna una coppia di oggetti `Connection(conn1, conn2)`, interconnessi attraverso una pipe**, con `duplex` pari a `True` (di default), se la pipe è bidirezionale, o pari a `False`, se la pipe sarà unidirezionale (`cond1` può ricevere solo messaggi e `cond2` può solo inviare messaggi). I due oggetti `Connection` rappresentano i due endpoint della pipe, sui quali è possibile utilizzare metodi come:

- `send(obj)`, per inviare un oggetto verso l'altro endpoint della connessione, dove sarà utilizzata una `recv()`;
- `recv()`, ritorna un oggetto inviato dall'altro endpoint della connessione ed è bloccante nel caso in cui non ci siano ancora oggetti da ricevere.

Si noti che i dati nella pipe possono essere corrotti se due processi utilizzano lo stesso endpoint per leggere o scrivere dati e che nessun rischio, invece, **sussiste nel caso di processi che utilizzano endpoint diversi di una pipe**.

```
from multiprocess import Process, Pipe

def parentData(parent):

    parent.send(['Hello'])
    parent.close()

def childData(child):

    child.send(['Bye'])
    child.close()

if __name__ == '__main__':

    parent_conn, child_conn = Pipe()

    p1 = Process(target=parentData, args=(parent_conn,))
    p1.start()

    p2 = Process(target=childData, args=(child_conn,))
    p2.start()

    print(parent_conn.recv())
    print(child_conn.recv())

    p1.join()
    p2.join()
```

La classe `Queue` fornisce una process shared queue implementata attraverso una pipe e locks/semaphores; quando un processo immette un elemento nella `Queue`, un thread è avviato e trasferisce gli oggetti da un buffer verso la pipe che implementa la coda. Alcuni metodi della classe sono:

- `Queue([maxsize])`, costruttore della classe che ritorna una queue di tipo FIFO:
  - `maxsize`, intero che definisce il limite superiore sul numero di elementi che la coda può mantenere e che, quando raggiunto, fa si che l'inserimento di un nuovo elemento è bloccato fino a quando non avviene un prelievo, ma se `maxsize` è impostato a zero o ad un valore negativo, non è definito un limite superiore;
- `qsize()`, ritorna la dimensione della coda;
- `empty()`, ritorna `True` se la coda è vuota, `False` altrimenti;
- `full()`, ritorna `True` se la coda è piena, `False` altrimenti;
- `put(item, block=True, timeout=None)`, immette l'elemento nella coda:
  - `block`, valore booleano che definisce se la chiamata è bloccante o meno, nel caso in cui non ci sia spazio disponibile;
  - `timeout`, identifica il tempo massimo di attesa in secondi sulla `put`, allo scadere del quale viene sollevata una Full exception;
- `put_nowait(item)`, equivalente a `put(item, block=False)`;
- `get(block=True, timeout=None)`, rimuove un elemento dalla coda:
  - `block`, valore booleano che definisce se la chiamata è bloccante o meno, nel caso in cui non ci sia spazio disponibile;
  - `timeout`, identifica il tempo massimo di attesa in secondi sulla `get`, allo scadere del quale viene sollevata una Empty exception;
- `get_nowait()`, equivalente di `get(False)`.

```
from multiprocess import Process, Queue


def f(q):
    q.put([42, None, 'hello'])


if __name__ == '__main__':
    q = Queue()
    p = Process(target=f, args=(q,))
    p.start()
    print(q.get())
    p.join()
```

Le shared memory sono uno degli strumenti offerti dal modulo multiprocessing per la condivisione di dati tra processi. In Python, una shared memory è rappresentata da **oggetti di tipo `Value` o `Array`, che sono process e thread safe**, e sfruttano il modulo `ctypes`, una libreria di funzioni per Python che fornisce tipi di dato compatibili con C; in particolare, **è consentita la chiamata di funzioni DDL** (Windows) **o shared library** (Unix) **e può essere usata per fare il wrap di queste librerie scrivendo codice Python puro**.

ctypes type	C type	Python type
c_bool	_Bool	bool (1)
c_char	char	1-character bytes object
c_wchar	wchar_t	1-character string
c_byte	char	int
c_ubyte	unsigned char	int
c_short	short	int
c_ushort	unsigned short	int
c_int	int	int
c_uint	unsigned int	int
c_long	long	int
c_ulong	unsigned long	int
c_longlong	__int64 or long long	int
c_ulonglong	unsigned __int64 or unsigned long long	int
c_size_t	size_t	int
c_ssize_t	ssize_t or Py_ssize_t	int
c_time_t	time_t	int
c_float	float	float
c_double	double	float
c_longdouble	long double	float
c_char_p	char* (NUL terminated)	bytes object or None
c_wchar_p	wchar_t* (NUL terminated)	string or None
c_void_p	void*	int or None

`Value(typecode_or_type, *args, lock=True)` ritorna un ctypes object allocato in shared memory che, di default, è un wrapper synchronized dell'oggetto; gli argomenti:

- `typecode_or_type`, determina il tipo di oggetto ritornato;
- `*args`, rappresenta gli argomenti che saranno passati al costruttore del tipo;
- `lock`:

- Se impostato a `True`, verrà creato un `RLock` per sincronizzare gli accessi all'oggetto;
- Se popolato con un oggetto `Lock` o `RLock`, l'oggetto sarà utilizzato per la sincronizzazione degli accessi;
- Se impostato a `False`, gli accessi all'oggetto non saranno protetti da un lock.

`Array(typecode_or_type, size_or_initializer, *, lock=True)` ritorna un ctypes object allocato dalla shared memory che, di default, è un wrapper synchronized dell'oggetto; gli argomenti:

- `typecode_or_type`, determina il tipo degli elementi dell'array che viene ritornato, può essere un ctype o un character typecode (`f` per indicare `float`, ad esempio) del tipo utilizzato dal modulo array;
- `size_or_initializer`, un intero che determina la lunghezza dell'array, che sarà inizializzato con tutti 0, o una sequenza di valori utilizzata per inizializzare l'array la cui lunghezza sarà determinata dal numero di valori previsti;
- `lock`:
  - Se impostato a `True`, verrà creato un `RLock` per sincronizzare gli accessi all'oggetto;
  - Se popolato con un oggetto `Lock` o `RLock`, l'oggetto sarà utilizzato per la sincronizzazione degli accessi;
  - Se impostato a `False`, gli accessi all'oggetto non saranno protetti da un lock.

```
from multiprocess import Process, Value, Array

def f(n, a):
    n.value = 3.1415927

    for i in range(len(a)):
        a[i] = -a[i]

if __name__ == '__main__':

    num = Value('d', 0.0)
    arr = Array('i', range(10))

    p = Process(target=f, args=(num, arr))
    p.start()

    p.join()

    print(num.value)
    print(arr[:])
```

L'esempio stamperà:

```
3.1415927

[0, -1, -2, -3, -4, -5, -6, -7, -8, -9]
```

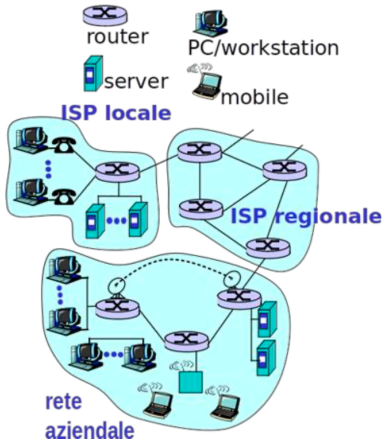
Da notare che gli argomenti `'d'` e `'i'` usati per la creazione degli oggetti `Value` ed `Array`, rispettivamente, rappresentano double precision float e singed integer.

PROGRAMMAZIONE SU RETE IN PYTHON

Per Internet si intende una rete di estensione mondiale basata sul protocollo TCP/IP, nata sotto il nome del progenitore Arpanet del 1969 come progetto finanziato da ARPA (l'agenzia del Dipartimento della Difesa USA) per esigenze militari e di ricerca scientifica. Ad oggi, Internet costituisce uno strumento per lo scambio e la ricerca di informazioni, per lo sviluppo di sistemi distribuiti e per connettere centinaia di milioni di dispositivi.

Nella sua accezione generale, **Internet può essere definita una rete di reti che coinvolge diversi tipi di componenti**:

- Dispositivi connessi, detti nodi o host (PC, server, smartphone, tablet, …);
- Link di comunicazione (cavi, fibra ottica, radio, satelliti, …);
- Router, dispositivi che instradano messaggi attraverso la rete.



Ad ogni nodo è assegnato un indirizzo Internet, detto **indirizzo IP**, secondo diversi formati:

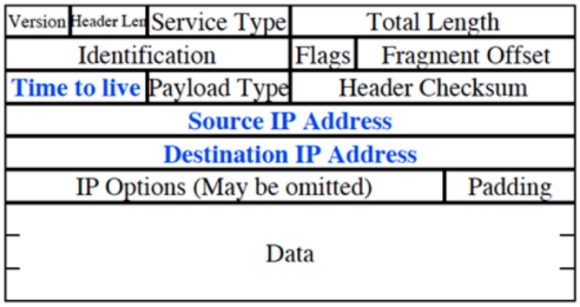
- **IPv4**, formato da 32 bit per un totale di circa 4,3 miliardi di indirizzi diversi possibili;
- **IPv6**, indirizzi a 128 bit per un totale di circa 65.571 miliardi di dispositivi.

Un indirizzo IP è sempre suddiviso in due porzioni, in modo da poterlo definire sia come standard IPv4 che IPv6:

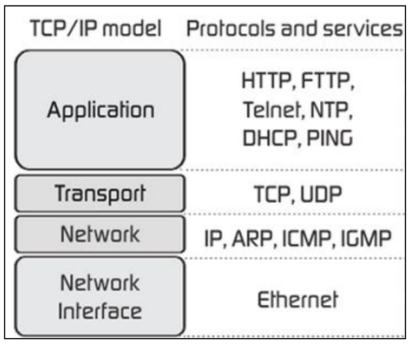
- 4 campi da 8 bit separati da un punto (per la rappresentazione IPv4), come 128.168.0.4;
- 8 campi da 16 bit separati da due punti (per la rappresentazione IPv6), come 2001:0db8:85a3:0000:1319:8a2e:0370:7344.

Grazie agli indirizzi IP è possibile identificare un terminale sulla rete ed inviargli dati; tuttavia, all'utente del terminale stesso può essere più familiare un nome come www.unina.it, la cui traduzione in indirizzo IP avviene tramite DNS, Domain Name Service.

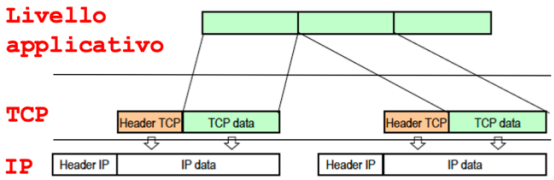
I dati end-to-end sono trasmessi in pacchetti, chiamati IP datagram, di dimensione massima 65.545 byte; il servizio che questo sistema offre è detto **best effort**: i pacchetti potrebbero giungere a destinazione corrotti, duplicati, non in ordine o, addirittura, potrebbero non arrivare, però sempre con la garanzia che **il protocollo e i router intermedi abbiano fatto tutto ciò in loro potere per mandare il pacchetto**.



TCP, Transmission Control Protocol, implementa un trasporto affidabile da sorgente a destinazione basato su IP, fornendo un flusso di byte full-duplex ed essendo orientato al concetto di connessione (quindi, si instaura una connessione, la si usa e la si chiude); in questo modo, **non solo i dati sono preservati ma ne è preservato anche l'ordine e ne sono prevenuti duplicati**. Inoltre, sono implementati meccanismi di **ritrasmissione dei pacchetti in caso di perdita o smarrimento** (si debba pur considerare che l'infrastruttura sottostante è IP, il cui servizio è best effort).

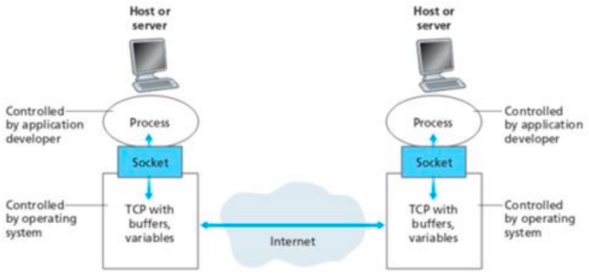


Infatti, **un protocollo o un'infrastruttura fa sempre riferimento ad una pila di modelli che descrive in maniera astratta l'iter di un'informazione all'interno di un dispositivo, in modo da poter essere trasmesso in rete**; la pila prende il nome di **ISO/OSI** ed è schematizzabile come sopra. Ogni singolo livello riceve e invia pacchetti secondo una particolare struttura, che è diversa da livello a livello; ciò fa si che un pacchetto generico sia sempre l'innesto di un dato interno e di un insieme di header, ognuno di essi aggiunti livello per livello:



UDP, User Datagram Protocol, è un protocollo del livello di trasporto analogo a TCP ma con una differenza sostanziale: **il paradigma non è basato sul concetto di connessione, bensì su quello di datagramma, il trasporto non è affidabile, è molto più simile al protocollo IP** (sebbene sia possibile distinguere tra porte differenti di uno stesso host).

La socket è un meccanismo di comunicazione tra processi, eventualmente distribuiti su macchine divrse, basato sulla rete:



Una socket è caratterizzata da due elementi: **un indirizzo IP del nodo e un numero di porta**; essendo un meccanismo di comunicazione tra livello trasporto e livello applicazione e agendo su quest'ultimo, TCP e UDP sono entrambi adatti all'uso delle socket. Alcune delle porte note che possono essere usate da una socket sono:

- 20/tcp **FTP** - Il file transfer protocol - data
- 21/tcp **FTP** - Il file transfer protocol - control
- 22/tcp **SSH** - Secure login
- 23/tcp **Telnet** insecure text communications
- 25/tcp **SMTP** - Simple Mail Transfer Protocol (E-mail)
- 53/tcp **DNS** - Domain Name Server
- 53/udp **DNS** - Domain Name Server
- 69/udp **TFTP** Trivial File Transfer Protocol
- 80/tcp **HTTP** HyperText Transfer Protocol (WWW)

In Python, per creare una socket si utilizza la funzione `socket.socket()` del modulo `socket`:

```
import socket
s = socket.socket(socket_family, socket_type)
```

Dove `socket_family` è la tipologia di indirizzo con cui si intende lavorare e `socket_type` la tipologia di socket che si vuole creare (sulla base del protocollo trasporto scelto):

```
socket_family = socket.AF_INET     #IPv4
socket_type = socket.SOCK_STREAM   #TCP
socket_type = socket.SOCK_DGRAM    #UDP
```

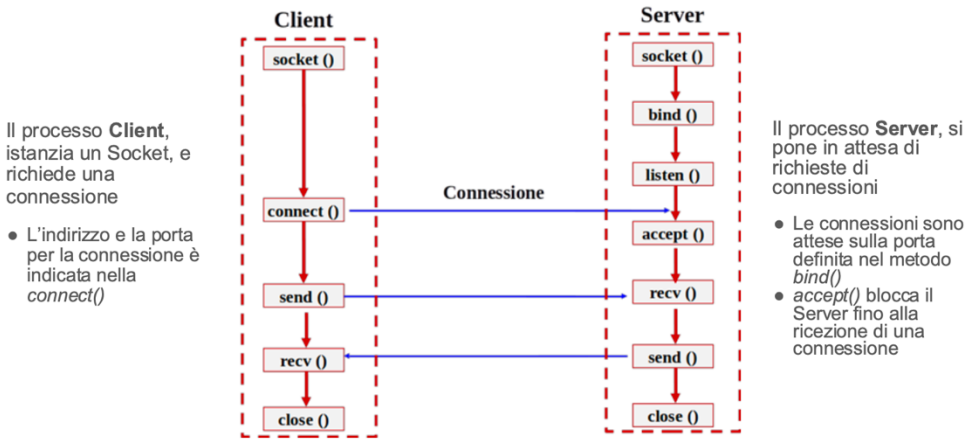
Le principali funzioni del modulo `socket` sono:

- `socket()`, crea una nuova socket;
- `bind(address)`, effettua il binding della socket con l'indirizzo specificato;
- `listen(backlog)`, si mette in ascolto di connessioni verso la socket (usata per TCP), con `backlog` che specifica il numero massimo di connessioni ammesse in coda;
- `accept()`, accetta una connessione, seguita da `bind()` e `listen()`, e restituisce la coppia `(conn, add)`, che specifica, rispettivamente, la nuova socket utilizzabile per inviare/ricevere dati e l'indirizo associato alla socket dall'altro capo della connessione;
- `connect(address)`, apre una connessione verso l'indirizzo specificato, con `address` coppia (IP, porta);
- `close()`, chiude la socket.

Per quanto riguarda i metodi specifici per il tipo di connessione (se TCP o UDP):

- TCP
  - `send(string)`, invia dati tramite una socket, con quest'ultima che deve essere connessa ad una socket remota;
  - `recv(bufsize)`, riceve i dati dalla socket, con valore di ritorno stringa e `bufsize` la massima quantità di dati che può essere ricevuta;

L'implementazione di un client-server TCP segue il seguente schema:



Un'implementazione possibile di un server TCP:

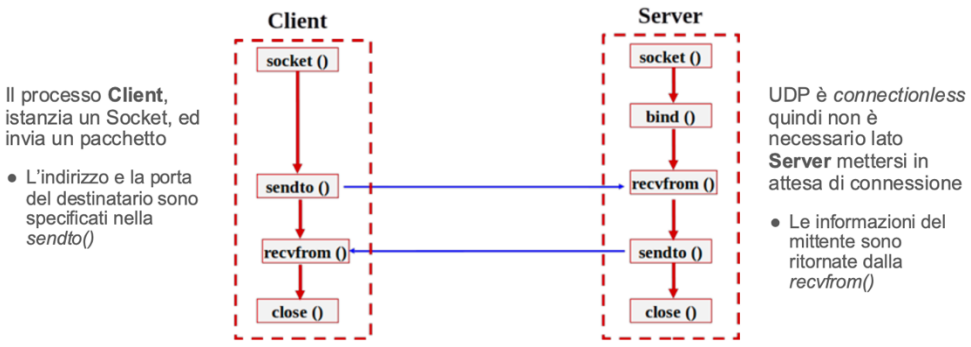
```
import socket

IP = 'localhost'
PORT = 0 # get first port available
BUFFER_SIZE = 1024

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind((IP, PORT))
s.listen(1)

cur_port = s.getsockname()[1] # get used port

print("server on: ", IP, "port: ", cur_port)

conn, addr = s.accept()
print  ('Connection address: {}' .format(addr))
toClient= "The world never says hello back!\n"

data = conn.recv(BUFFER_SIZE)
print ("received data: " + data.decode("utf-8"))

conn.send(toClient.encode("utf-8"))

conn.close()
s.close()
```

Un'implementazione possibile di un client TCP:

```
import socket, sys

def client(PORT):

    IP = 'localhost'
    BUFFER_SIZE = 1024
    MESSAGE = "Hello, World!\n"

    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect((IP, PORT))
    s.send(MESSAGE.encode("utf-8"))

    data = s.recv(BUFFER_SIZE)
    print ("received data: " + data.decode("utf-8"))

    s.close()

if __name__ == "__main__":
    try:
        PORT = sys.argv[1]
    except IndexError:
        print("Please, specify PORT arg...")

    assert PORT != "", 'specify port'
    client(int(PORT))
```

- UDP:
  - `sendto(string, address)`, invia dati alla socket, che non deve necessariamente essere connessa ad un'altra socket remota (dato che la destinazione è specificata da `address`);
  - `recvfrom(bufsize)`, riceve i dati dalla socket ritornando la coppia `(string, address)`, con il primo una stringa contenente i dati ricevuti e la seconda l'indirizzo della socket che invia i dati, mentre `bufsize` è la massima quantità di dati che può essere ricevuta.

L'implementazione di un client-server UDP segue il seguente schema:



Un'implementazione possibile di un server UDP:

```
import socket

msgServer = "Ciao client!"
serverAddressPort = ("localhost".encode("utf-8"), 7000)
bufferSize = 1024

s = socket.socket(family=socket.AF_INET, type=socket.SOCK_DGRAM)
s.bind(serverAddressPort)

msgClient, addr = s.recvfrom(bufferSize)
print("[Server]: Messaggio ricevuto: " + msgClient.decode("utf-8"))
print("[Server]: Indirizzo client: {}" .format(addr))

print ("[Server]: Invio dati al client")
s.sendto(msgServer.encode("utf-8"), addr)

s.close()
```

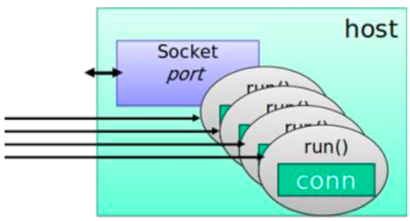
Un'implementazione possibile di un client UDP:

```
import socket, sys

def client(port):

    msgClient = "Ciao server!"
    serverAddressPort = ("localhost".encode("utf-8"), port)
    bufferSize = 1024

    s = socket.socket(family=socket.AF_INET, type=socket.SOCK_DGRAM)

    print ("[Client]: Invio dati al server. msg: ", msgClient)
    s.sendto(msgClient.encode("utf-8"), serverAddressPort)

    msgServer, addr = s.recvfrom(bufferSize)
    print("[Client]: Risposta server: " + msgServer.decode("utf-8"))

    s.close()

if __name__ == "__main__":
    try:
        port = sys.argv[1]
    except IndexError:
        print("Please, specify PORT arg...")

    assert port != "", 'specify port'
    client(int(port))
```

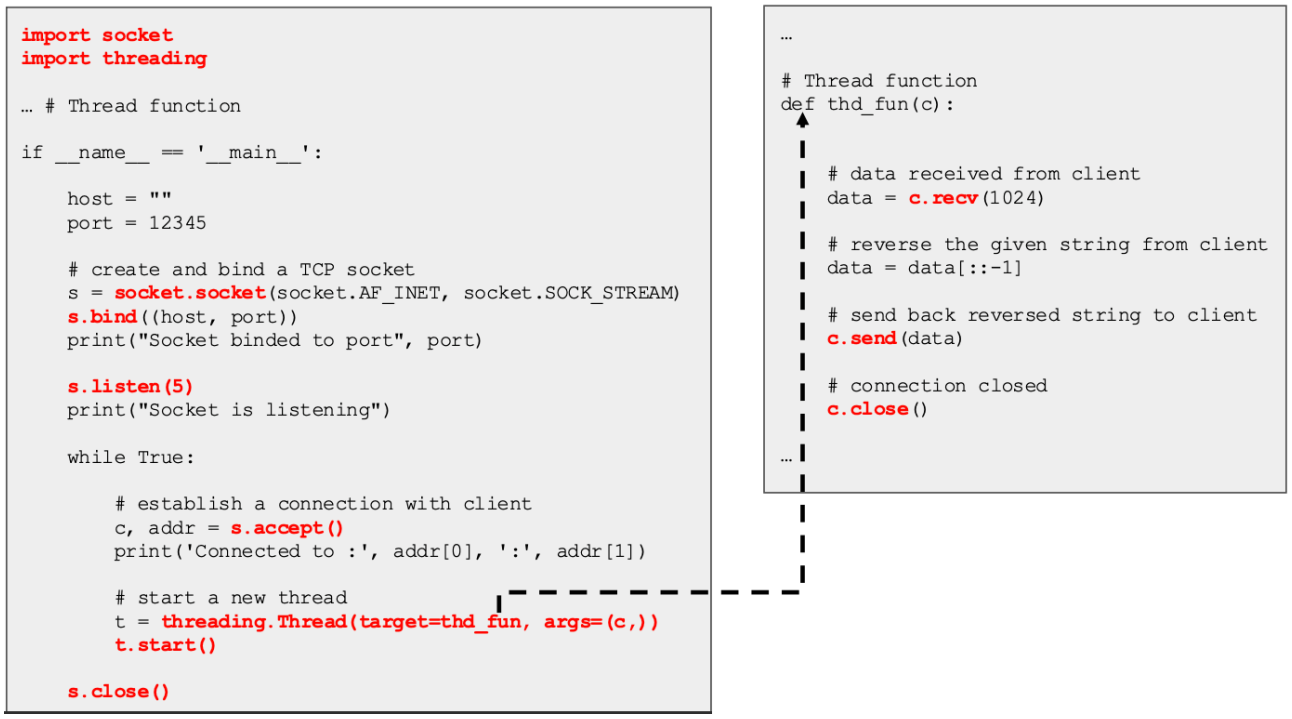
Sistemi multi-thread/multi-process migliorano l'efficienza di un server creando più thread/processi per gestire le richieste; questi, devono prevedere, tuttavia, un costruttore che accetti come parametri:

- Nel caso TCP una connessione (restituita dal metodo `accept`), in modo che tutte le connessioni siano gestite da un thread/processo differente;
- Nel caso UCP l'indirizzo/porta del mittente ed il messaggio ricevuto (restituito dal metodo `recvfrom`), in modo che ogni richiesta sia gestita da un thread/processo differente.

Nel metodo `run()` dei thread/processi deve esserci, poi, tutto il codice necessario per gestire una richiesta.



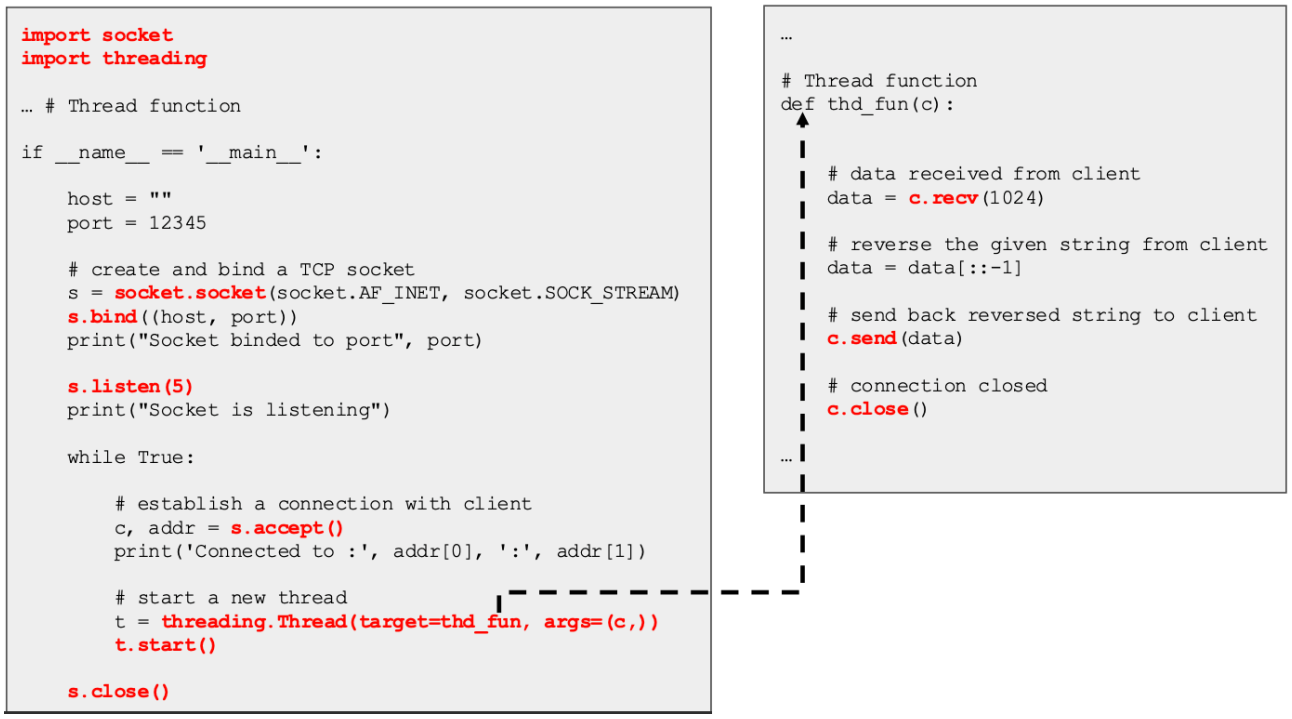
Un esempio di server TCP multi-thread:



```
import socket
import threading

… # Thread function

if __name__ == '__main__':

    host = ""
    port = 12345

    # create and bind a TCP socket
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.bind((host, port))
    print("Socket binded to port", port)

    s.listen(5)
    print("Socket is listening")

    while True:

        # establish a connection with client
        c, addr = s.accept()
        print('Connected to :', addr[0], ':', addr[1])

        # start a new thread
        t = threading.Thread(target=thd_fun, args=(c,))
        t.start()

    s.close()
```

```
…

# Thread function
def thd_fun(c):

    # data received from client
    data = c.recv(1024)

    # reverse the given string from client
    data = data[::-1]

    # send back reversed string to client
    c.send(data)

    # connection closed
    c.close()

…
```

Un esempio di server TCP multi-process:



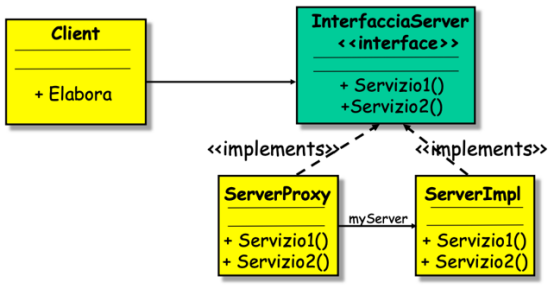
```
import socket
import threading

… # Thread function

if __name__ == '__main__':

    host = ""
    port = 12345

    # create and bind a TCP socket
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.bind((host, port))
    print("Socket binded to port", port)

    s.listen(5)
    print("Socket is listening")

    while True:

        # establish a connection with client
        c, addr = s.accept()
        print('Connected to :', addr[0], ':', addr[1])

        # start a new thread
        t = threading.Thread(target=thd_fun, args=(c,))
        t.start()

    s.close()
```

```
…

# Thread function
def thd_fun(c):

    # data received from client
    data = c.recv(1024)

    # reverse the given string from client
    data = data[::-1]

    # send back reversed string to client
    c.send(data)

    # connection closed
    c.close()

…
```

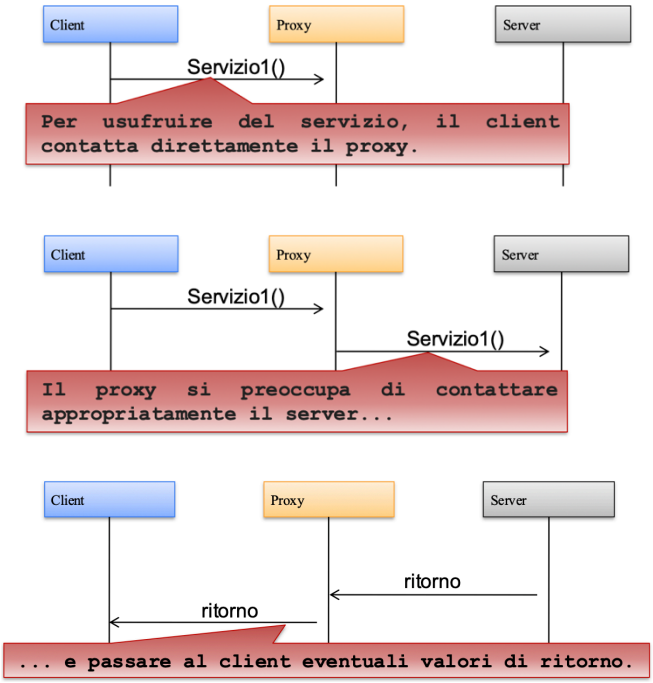
In un'applicazione con comunicazione basata su socket, **client e server prevedono meccanismi di connessione, esternalizzazione dei dati, invio-ricezione delle richieste e ricostruzione dei valori ricevuti**; i potenziali rischi di questo approccio sono due: l'implementazione dei meccanismi di comunicazione "distrae" dalla realizzazione delle funzionalità effettive dell'applicazione e la possibile sovrapposizione della logica applicativa con quella di comunicazione client-server. **Dal lato client, il programmatore dovrebbe concentrarsi sulla logica applicativa**, invocando i servizi del server specificati da un'interfaccia, e separare la logica applicativa dai meccanismi di

comunicazione/interazione con il server; analogamente, **il programmatore lato server dovrebbe concentrarsi sulla codifica dei servizi offerti**, avendo cura di comunicare al client una sua interfaccia e separando questa implementazione dai meccanismi di comunicazione/interazione con il client.

Per **pattern Proxy-Skeleton** si intende un pattern di programmazione su rete nel modello di interazione client-server che introduce il concetto di Surrogato (o Proxy): **il proxy si presenta come un'implementazione del servizio remoto**, **locale al client**, **i meccanismi di "basso livello" necessari per l'instaurazione della comunicazione**, e per l'invio-ricezione dei parametri delle invocazioni tramite TCP o UDP, **sono implementati ed incapsulati all'interno del Proxy**.

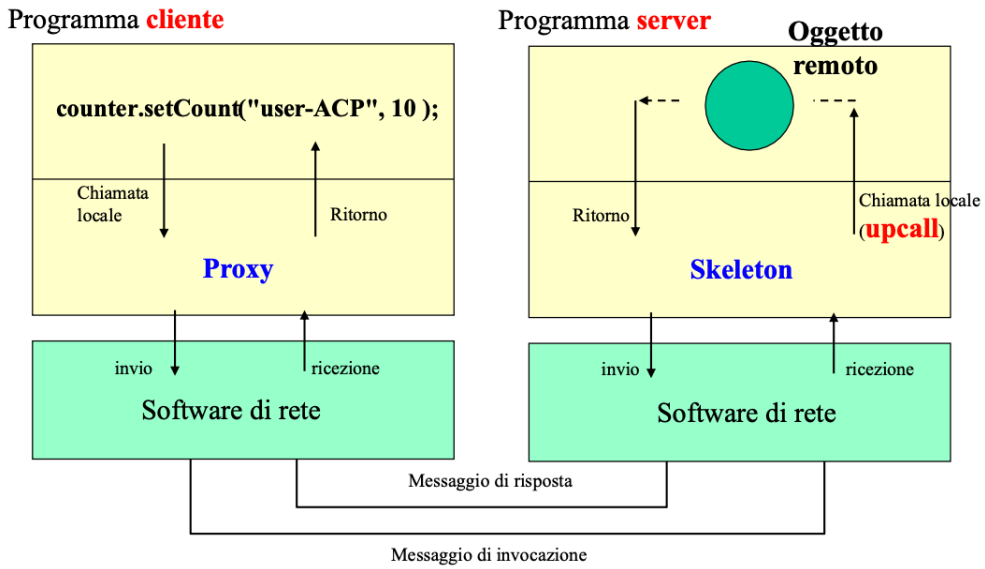


Il servizio fornito è specificato da `InterfacciaServer`, con una duplice implementazione: `ServerImpl`, che implementa effettivamente i servizi dichiarati nell'interfaccia, e `ServerProxy`, che si fa carico dell'inoltro delle richieste effettuate dal client verso il lato server. L'associazione direzionale tra `ServerProxy` e `ServerImpl` sintetizza tutti i meccanismi necessari al proxy per riferirsi al server. L'utilizzo di questo pattern prevede che il client possieda un riferimento ad un oggetto del tipo `InterfacciaServer` (ossia `ServerProxy`) e che utilizzi il proxy in sostituzione del servizio reale; quest'ultimo, poi, si fa carico dell'interazione effettiva con il lato server.

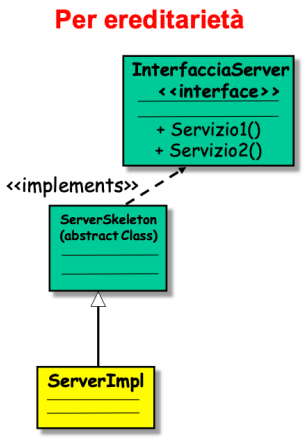


Con l'aggiunta del Proxy, il client è sollevato dalle problematiche di comunicazione. In maniera del tutto analoga, **un Proxy lato server, detto Skeleton, si fa carico della comunicazione con il Proxy lato client**; in altre parole, lo skeleton:

1. Riceve le richieste di servizio;
2. Struttura l'informazione fornita in ingresso tramite socket TCP o UDP;
3. Effettua l'up-call al servizio reale;
4. Riceve da questi eventuali risultati;
5. Invia una risposta al proxy lato client.



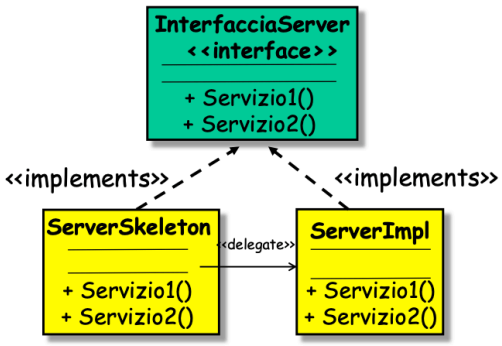
Lo skeleton può essere implementato per ereditarietà: la classe astratta `ServerSkeleton` implementa solo gli opportuni schemi di comunicazione ma lascia senza implementazione i metodi dell'interfaccia; sarà, poi, `ServerImpl`, sottoclasse di `ServerSkeleton`, a fornire implementazione ai metodi astratti.



Tuttavia, lo skeleton può essere implementato anche per delega: la classe `ServerSkeleton` presenta al suo interno un riferimento all'interfaccia `InterfacciaSkeleton`, `delegate`, e i metodi sono così realizzati:

```
Servizio1(){
     delegate.Servizio1();
}
```

Per delega



Volendo confrontare i due approcci, **lo skeleton per ereditarietà ha un'implementazione più semplice**, sebbene sia concettualmente poco attraente e sebbene possa non funzionare con classi già esistenti (in particolare in linguaggi con ereditarietà singola); di contro, **lo skeleton per delega ha una struttura molto più pulita**, permette di lavorare con classi di implementazione esistenti che fanno già parte di una gerarchia di ereditarietà ma è più complesso della sua variante alternativa.

In Python, con interfacce formali:

```
#interface.py
from abc import ABC, abstractmethod

class Subject(ABC):

        @abstractmethod
        def request(self, data):
                pass
```

Il Proxy:

```
#client.py

import socket, sys
from interface import Subject

class Proxy(Subject):

    def __init__(self, port):
        self.port = port

    def request(self, message):
        IP = 'localhost'
        BUFFER_SIZE = 1024
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect((IP, self.port))
        s.send(message.encode("utf-8"))
        data = s.recv(BUFFER_SIZE)
        print ("received data: " + data.decode("utf-8"))
        s.close()

...
```

```
...

if __name__ == "__main__":

    try:
        PORT = sys.argv[1]
        MESSAGE = sys.argv[2]
    except IndexError:
        print("Specify args!")

    print("Client: Generating request)
    proxy = Proxy(int(PORT))
    proxy.request(MESSAGE)
```

Lo Skeleton (per ereditarietà):

```
#server.py - skelethon per ereditarietà
import socket, sys, threading
from abc import ABC, abstractmethod
from interface import Subject
class Skeleton(Subject, ABC):

    def __init__(self, port):
        self.port = port

    @abstractmethod
    def request(self, data):
        pass

    def run_skeleton(self):
        host = 'localhost'
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.bind((host, self.port))
        self.port = s.getsockname()[1] # get used port
        s.listen(5)
        print("Socket is listening")
        while True:
            c, addr = s.accept()
            print('Connected to :', addr[0], ':', addr[1])
            t = threading.Thread(target=thd_fun, args=(c, self))
            t.start()
        s.close()
...
```

```
...

#thread function
def thd_fun(c, self):

    # data received from client
    data = c.recv(1024)

    # upcall
    result = self.request(data)

    # send reversed string to client
    c.send(result)

    # connection closed
    c.close()


...

class RealSubject(Skeleton):

    def request(self, data):

        # reverse the given string from client
        data = data[::-1]

        return data


if __name__ == "__main__":

    try:
        PORT = sys.argv[1]
    except IndexError:
        print("Please, specify PORT arg")

    print("Server running")

    realSubject = RealSubject(int(PORT))
    realSubject.run_skeleton()

...
```

SISTEMI MIDDLEWARE

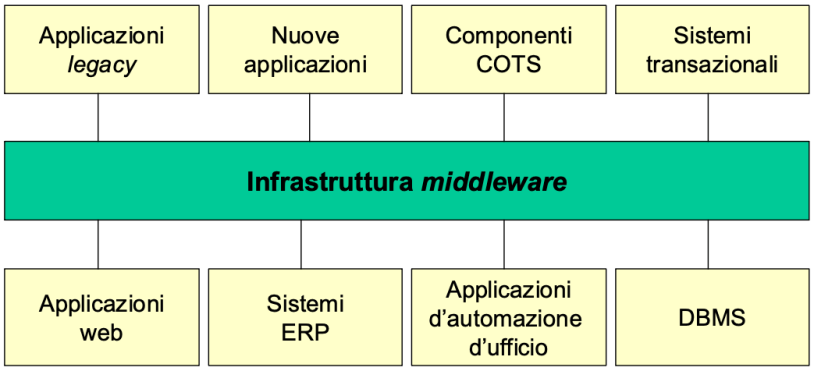
Un sistema distribuito è un sistema i cui componenti, localizzati in computer connessi in rete, comunicano e coordinano le loro azioni solo attraverso lo scambio di messaggi; in altre parole,

un programma gira su un sistema distribuito quando **"non funziona per colpa di una macchina della quale non si è mai sentiti parlare"**. Le principali implicazioni nell'impiego di un sistema distribuito sono la necessità di programmare con tecniche concorrenti, convivere con l'assenza di un clock globale e con malfunzionamenti indipendenti.

In genere, **i sistemi distribuiti sono eterogenei**, l'eterogeneità dei computer e delle tecnologie (hardware così come software) è da considerarsi la norma, non l'eccezione, anche all'interno di una grossa azienda o organizzazione, pubblica o privata che sia. **Il paradigma di elaborazione più generale prevede componenti interoperanti in ambiente distribuito ed eterogeneo**, detto heterogeneous distributed computing. In ambiente di sviluppo, **diversi fattori contribuiscono alla complessità dello sviluppo di software di qualità**:

- Le applicazioni sono distribuite su una rete di calcolatori di caratteristiche differenti (mainframes, servers, workstations, PCs, …), di hardware differenti e di sistemi operativi differenti (e spesso incompatibili);
- Occorre adoperare linguaggi di programmazione diversi, linguaggi di alto livello per lo sviluppo rapido della parte di presentazione e di funzionalità di un'applicazione (dette, rispettivamente, presentation logic e business logic), mentre linguaggi tradizionali e di basso livello (come COBOL, C, C++) sono usati per funzionalità legacy o di basso livello (che richiedono maggiore interazione con il sistema operativo) e linguaggi di scripting per lo sviluppo di funzionalità server-side (come Python, Php, …);
- I dati sono distribuiti su più nodi di elaborazione, memorizzati in archivi o con DBMS differenti e, in generale, condivisi da applicazioni locali e remote;
- Nello sviluppo software su rete, anche in ambiente omogeneo, occorre fronteggiare problematiche come la sicurezza, i guasti o la contesa e la condivisione delle risorse, che si presentano ben più complessi che per i sistemi centralizzati (poi, l'eterogeneità aggiunge a questi problemi ulteriore complessità).

Sistemi complessi funzionanti raramente vengono sviluppati integralmente ex novo, **tipicamente si procede con uno sviluppo evolutivo a partire da sistemi già funzionanti**; l'integrazione di sistemi informativi sviluppati in momenti, con linguaggi e tecniche diversi ed operanti su piattaforme eterogenee, è un problema centrale delle tecnologie software. In particolare, si richiede sempre di più di usare applicazioni di terze parti (sistemi COTS, Commercial Off-The-Shelf) o di riutilizzare applicazioni esistenti (sistemi ereditati o legacy). **L'integrazione di applicazioni**, siano esse sistemi di nuovo sviluppo, ereditati o COTS, **è da considerarsi lo scenario tipico di sviluppo software su larga scala**:

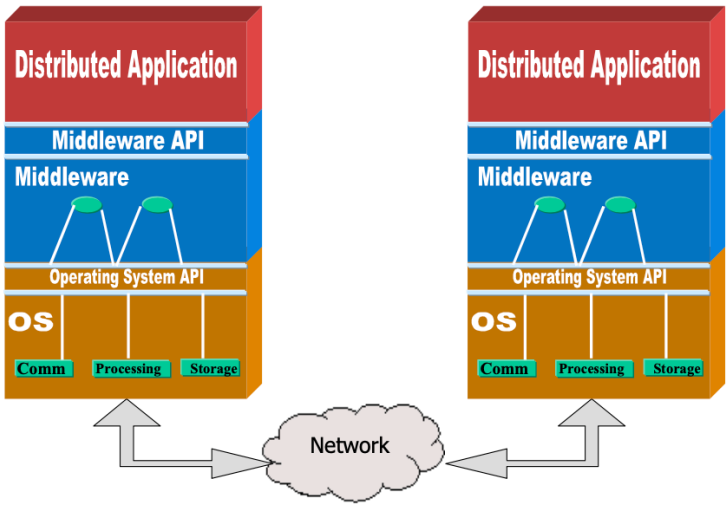


L'integrazione di applicazioni (EAI) è un processo spesso più complesso dello sviluppo ex novo; volendo confrontare i due:

SVILUPPO DI APPLICAZIONI	INTEGRAZIONE DI APPLICAZIONI
Nuove applicazioni	Riuso di applicazioni esistenti, spesso non ben documentate, e uso di componenti COTS
Possibilità di scelta di tecnologie e prodotti	Vincoli su sistemi operativi, protocolli, linguaggi, ambienti di sviluppo, ecc.; tecnologie dei sistemi esistenti spesso non interoperanti
Richiede competenze di analisi, progettazione, programmazione	Richiede competenze di architetture software, reingegnerizzazione, riuso, capacità di buona progettazione delle interfacce, adattamento di componenti, predisposizione all'evoluzione
Può essere svolto da giovani sviluppatori	Richiede familiarità con i sistemi esistenti; svolto da specialisti con anni di esperienza

In questa prospettiva, **la programmazione** (intesa come codifica di moduli software in un linguaggio di programmazione) **è**, ancor meno che in passato, **una fase critica del ciclo di vita del software**, sia in termini di risorse che in termini di competenze tecnologiche richieste; **assume maggiore importanza la capacità di adattare componenti esistenti e non originariamente pensati per interoperare, mediante l'attenta progettazione delle interfacce**. Molte tecnologie middleware sono nate con il preciso obiettivo di fornire una risposta al problema dell'EAI.

Con il termine **middleware** si intende uno **strato di software interposto tra il sistema operativo e le applicazioni**, in grado di fornire le astrazioni ed i servizi utili per lo sviluppo di applicazioni distribuite.



Lo strato middleware offre ai programmatori di applicazioni distribuite librerie di funzioni, o middleware API, in grado di mascherare l'eterogeneità dei sistemi su rete. Le piattaforme middleware vengono anche definire **software di connettività tra applicazioni**, o **glue technologies**, evidenziando la loro caratteristica di tecnologie integrative di applicazioni. **Il livello middleware può mascherare l'eterogeneità** mediante meccanismi di:

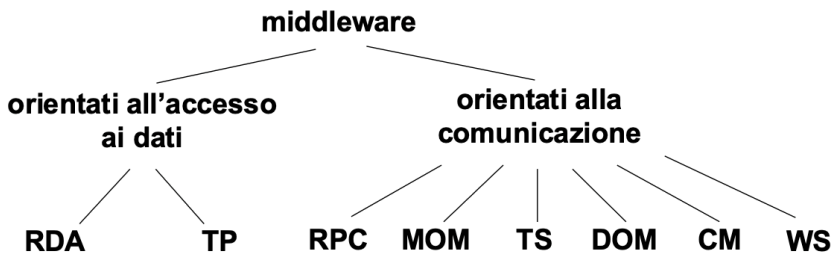
- Trasparenza del sistema operativo (operandone al di sopra, i servizi forniti dalle API middleware possono essere definiti in maniera indipendente da esso, consentendo la portabilità delle applicazioni tra diverse piattaforme);
- Trasparenza del linguaggio di programmazione (la comunicazione tra componenti sviluppati in linguaggi diversi pone problemi connessi alle differenze tra i tipi di dato supportati ed alla diversità tra i meccanismi di scambio di parametri nell'invocazione di sottoprogrammi;

inoltre, il livello middleware può consentire l'interoperabilità, definendo un sistema di tipi intermedio e regole non ambigue di corrispondenza con i tipi dei linguaggi più diffusi);
- Trasparenza della locazione (le risorse, dati e servizi, dovrebbero essere accessibili a livello logico, senza conoscerne l'effettiva locazione fisica; è lo strato middleware che deve farsi carico della localizzazione su rete del processo partner in una comunicazione e del trasporto dati);
- Trasparenza della migrazione (dati e servizi possono venir rilocati durante il loro ciclo di vita: se un componente migra, il middleware può consentire l'accesso a componenti mobili sulla rete, in maniera trasparente ai moduli clienti);
- Trasparenza ai guasti (un'elaborazione distribuita può fallire anche solo in maniera parziale per guasti che si verificano nei singoli nodi o nel sottosistema di comunicazione, occorrono tecniche complesse per poter gestire uno stato globale consistente della computazione; lo strato middleware può offrire al programmatore meccanismi ad alto livello per mascherare i guasti);
- Trasparenza della replicazione (la replicazione di componenti è una delle tecniche più comuni per realizzare politiche di tolleranza ai guasti ma può essere utilizzata anche per migliorare le prestazioni, attraverso un miglior bilanciamento del carico di elaborazione; l'esistenza di più copie di un componente, però, deve essere trasparente ai suoi clienti);
- Trasparenza delle implementazioni commerciali (alcune tecnologie middleware si presentano, in realtà, come specifiche di riferimento promosse da organizzazioni di produttori, spesso poi recepite dagli enti di standardizzazione; le varie implementazioni commerciali sono, quindi, realizzate in maniera conforme allo standard, eventualmente differenziandosi tra loro per aspetti secondari o per servizi aggiuntivi fuori standard. In questo modo, è resa possibile l'interoperabilità tra applicazioni basate su realizzazioni commerciali diverse dello stesso tipo di middleware).

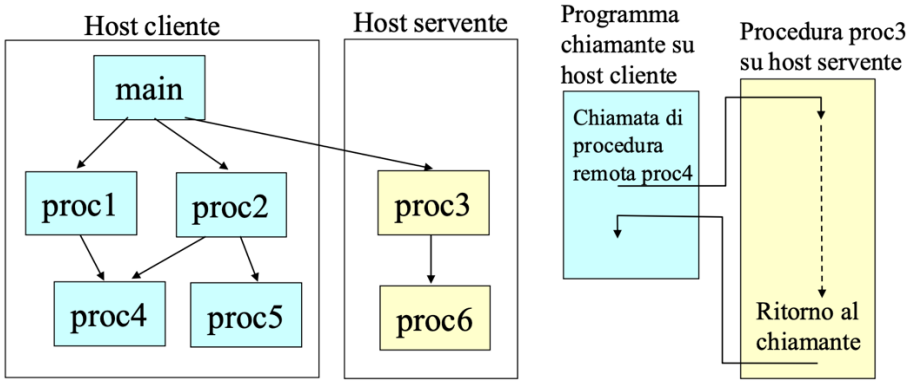
Esistono numerose tipologie di piattaforma middleware, sia in forma prototipale in ambito di ricerca che in forma commerciale; i principali modelli di programmazione impiegati in questo tipo di sviluppo software sono:

- Chiamata di procedura remota (RPC, Remote Call Procedure);
- Scambio di messaggi (MOM, Message Oriented Middleware);
- Oggetti distribuiti (DOM, Distributed Objects Middleware);
- Servizi Web (WS, Web Services);
- Transazionale (TP, Transactional Processing);
- Spazio delle tuple (TS, Tuple Space);
- Accesso remoto ai dati (RDA, Remote Data Access);
- Componenti (CM, Component Model).

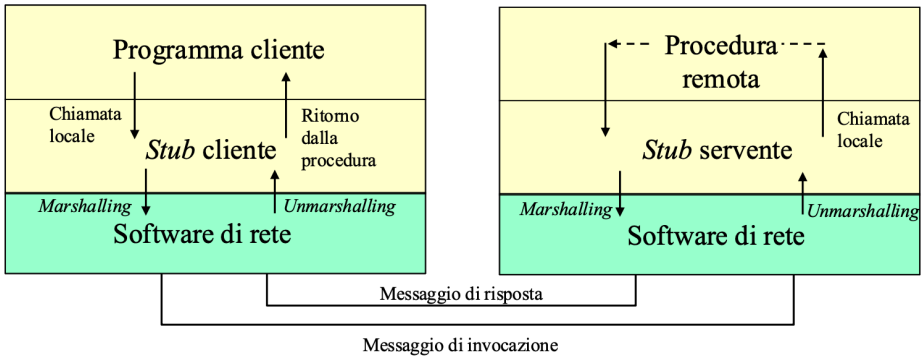
In particolare, **alcuni di questi sono modelli orientati all'accesso ai dati** (RDA e TP) e altri **orientati alla comunicazione** (RPC, MOM, TS, DOM, CM, WS):



In particolare, si voglia approfondire il **modello di chiamata a procedura remota**, **RCP**. Si tratta di un'estensione ai sistemi distribuiti del modello procedurale dei linguaggi di programmazione tradizionali; infatti, **la comunicazione tra processi è intrinsecamente di tipo sincrono e bloccante**: il processo chiamante resta sospeso fino al completamento della procedura remota e la disponibilità di meccanismi asincroni (non bloccanti) è da considerarsi un'eccezione.



In fase di compilazione vengono automaticamente generate speciali funzioni di interfacica, dette **stub**, **una dal lato cliente ed una dal lato servente**, **che vengono poi collegate con le due parti dell'applicazione**. Gli stub implementano, in maniera gran parte trasparente al programmatore, lo scambio di parametri e la comunicazione attraverso la rete tra il programma chiamante (client) e la procedura remota chiamata (server). A tal fine, **gli stub accedono direttamente ai servizi di trasporto dei protocolli di rete**.



In particolare **lo stub client preleva i parametri di scambio dal chiamante**, **li impacchetta in un apposito messaggio** (che affida al software di rete affinché sia trasmesso alla macchina dove risiede il server), **attende il messaggio di risposta** (che indica il completamento della procedura remota), **spacchetta il messaggio prelevando i valori dei parametri di uscita e restituisce i parametri al programma chiamante**, che riprende il controllo; **lo stub server**, invece, **attende** (dal software di rete) **un messaggio di invocazione della procedura**, **spacchetta il messaggio** (prelevando i parametri di scambio), **trasmette i parametri alla procedura chiamata** (cui cede il controllo), **riprende il controllo al termine della procedura**, **impacchetta i parametri di uscita in un messaggio di risposta al client e invia tale messaggio**, mettendosi nuovamente in attesa di una richiesta. Il programma chiamante invoca una procedura a tutti gli effetti locale (stub cliente) e, analogamente, la procedura remota viene invocata come una procedura locale da parte dello stub server.

L'operazione di **impacchettamento** (e, rispettivamente, di spacchettamento) dei parametri della chiamata di procedura, svolta dagli stub, prende il nome di **marshalling** (e, rispettivamente, di unmarshalling). Il marshalling dei parametri consiste di una **conversione di formato dei dati**, per

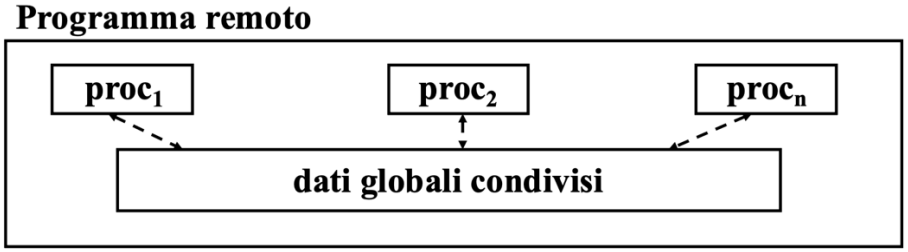
tenere conto delle differenze di rappresentazione tra cliente e servente (si ricordi che, generalmente, le due piattaforme sono eterogenee, ed è possibile che adottino rappresentazioni interne diverse), e in una **serializzazione dei dati**, che vengono trasformati in sequenze di byte ed impacchettati, secondo un formato compreso sia da client che da server (in particolare, occorre linearizzare i dati strutturati, come array e record).

In generale, **esistono diversi metodi per consentire a due nodi** (sender e receiver) **di scambiare dei dati**: **i dati sono convertiti dal sender in un formato esterno** (concordato tra i due) **e**, quindi, **trasmessi**, mentre **il receiver converte i dati nel proprio formato locale alla ricezione** (in tal caso si parla di external data representation); **i dati sono trasmessi nel formato del sender e accompagnati da un'indicazione sul formato utilizzato**. Alcuni approcci di external data representation/marshalling sono Java Object Serialization, XML (Extensible Markup language) e JSON (JavaScript Object Notation), con questi ultimi due entrambi rappresentazioni dei dati in formato testuale.

In questo modello, **i malfunzionamenti possibili possono accadere o nella rete o nei singoli nodi e possono causare la perdita dei messaggi, di richiesta e/o di risposta, o la caduta del nodo server dopo la ricezione della richiesta ma prima dell'invio della risposta**. Le semantiche possibili per le RCP sono:

- Exactly once, la procedura viene eseguita una ed una sola volta;
- At most once, non è possibile garantire che la procedura sia stata eseguita ma, se è stata eseguita, lo è stata una volta sola;
- At least once, la procedura viene eseguita almeno una volta;
- Zero or more, non è possibile dire se e quante volte la procedura è stata eseguita.

L'implementazione RPC di Sun Microsystems prevede come entità in esecuzione su una macchina server un programma, che contiene una o più procedure invocabili remotamente. Le procedure all'interno di uno stesso programma operano nello stesso ambiente di esecuzione, quindi possono condividere delle variabili; inoltre, è possibile avere attive su un nodo più versioni dello stesso programma. Una tripla di interi (programma, versione e procedura) è sufficiente al cliente per identificar ela procedura da invocare:

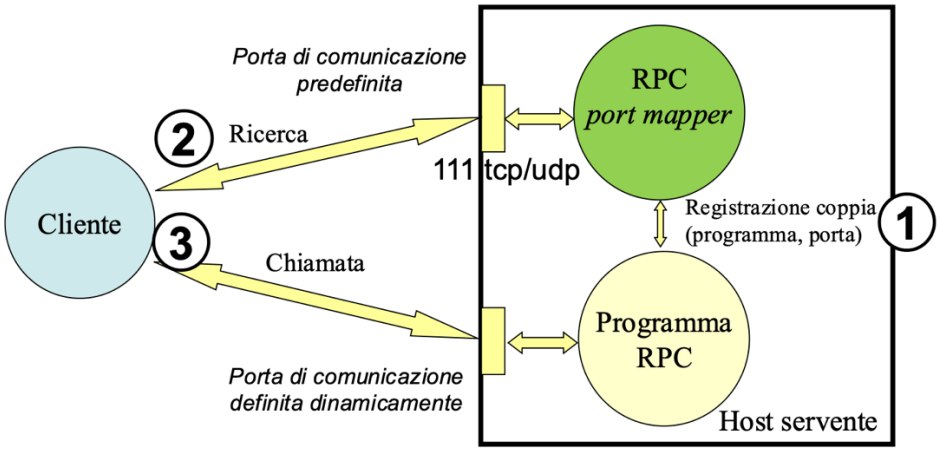


È garantita la mutua esclusione tra le procedure all'interno di uno stesso programma remoto, al più una procedura può essere in esecuzione in un dato istante, anche in presenza di più clienti, mentre per l'esternalizzazione dei dati viene adottato lo standard XDR (External Data Representation). Sun RPC si basa su TCP o UDP:

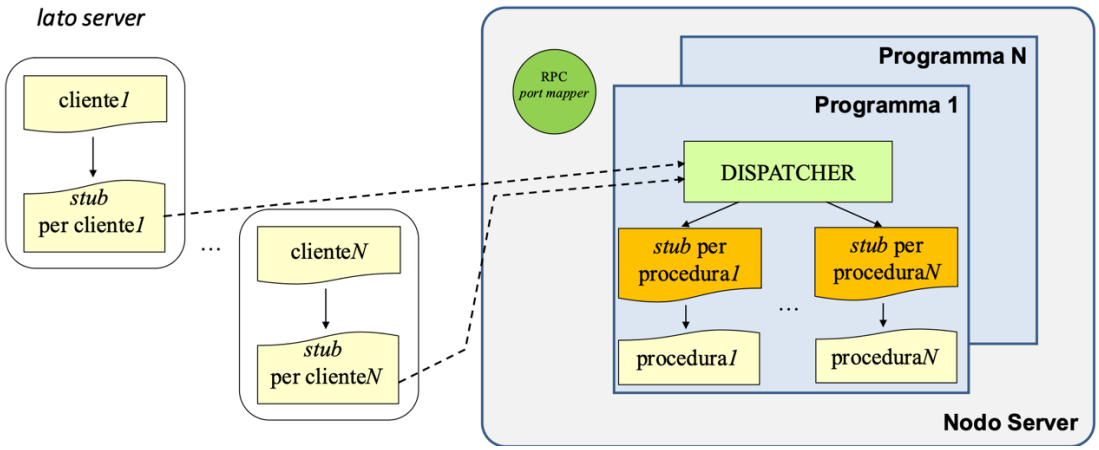
- UDP, se il cliente riceve una risposta la chiamata è stata eseguita almeno una volta (at least once) o se il cliente non riceve una risposta la chiamata è stata eseguita zero o più volte (zero or more);
- TCP, se il cliente riceve una rispsota la chiamata è stata eseguita una volta (exactly once) o se il cliente non riceve una risposta la chiamata è stata eseguita zero o una volta (at most once).

Nel caso si usi UDP, le procedure devono essere idempotenti (ovvero, deve essere garantito che, a parità di parametri di ingresso, il risultato dell'elaborazione sia sempre lo stesso, indipendentemente dal numero di volte in cui la procedura è eseguita).

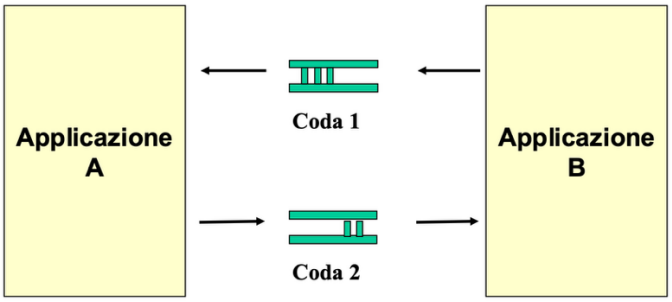
In generale, nella programmazione con socket TCP/UDP, **client e server devono concordare preventivamente il numero di port sul quale il server è in ascolto**, perché entrambi consultano una lista pubblica e prefissata di assegnazione dei port. Sun RPC adotta, invece, un meccanismo di binding dinamico: **il server ottiene dinamicamente un numero di port disponibile, il client non può conoscere il port finché il server non viene attivato**, **sul quale gira un processo RPC port mapper che gestisce una lista di server attivi**; all'attivazione di un programma RPC, **il port mapper registra la coppia** (numero programma, numero port) **e opera su una socket ad un port prefissato** (111), uguale per ogni nodo server, che verrà **contattato dal client prima di eseguire la RCP** (presenta il numero programma e riceve il numero di porto). Il **binding dinamico** consente la trasparenza della locazione menzionata precedentemente.



Quindi, l'algoritmo del port mapper crea una socket al port 111 e accetta indefinitamente richieste di registrazione di un programma RPC (dai server che gestisce) e di ricerca del port di un programma RPC (dai client che vogliono accedere ad uno dei server). Una volta che il client ha trovato il port da utilizzare per la RPC call, effettuerà la chiamata al proprio stub locale, che automaticamente andrà a contattare lo stub opportuno tramite un componente chiamato dispatcher; infatti, sul nodo server, il dispatcher si occuperà di inoltrare i messaggi dei client agli opportuni stub lato server:



I sistemi middleware a scambio messaggi (MOM, Message Oriented Middleware) **si basano sull'astrazione di una coda di messaggi tra processi interoperanti su rete**, che rappresenta una generalizzazione del noto concetto di mailbox, tipico dei sistemi operativi:



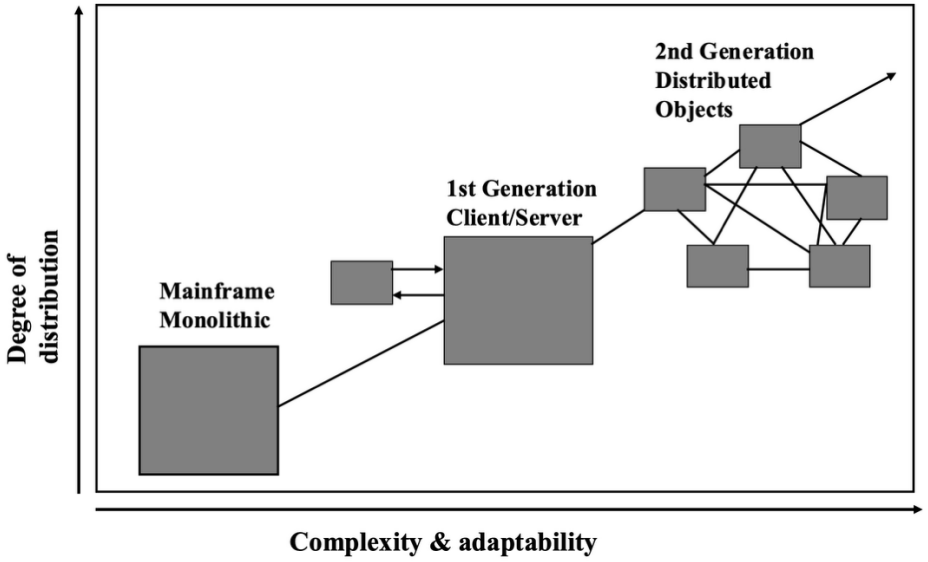
Nel modello MOM la comunicazione non è di tipo client-server ma peer-to-peer, tra pari, e di tipo **produttore-consumatore asincrona** (sebbene alcuni sistemi consentano anche uno scambio di messaggi di tipo sincrono).

I sistemi MOM possono adottare un modello publish-subscribe, in cui i processi produttori pubblicano messaggi differenziati per tipo e i consumatori possono dichiararsi interessati ai messaggi in base al loro tipo. **Le piattaforme MOM sono adatte per applicazioni guidate dagli eventi** (event-driven): quando si verifica un evento, un processo produttore affida al middleware la responsabilità di prendere in consegna un messaggio ed inoltrarlo, o semplicemente notificarne la disponibilità ai processi interessati al suo consumo; inoltre, **molti sistemi MOM realizzano anche meccanismi di persistenza dei messaggi**.

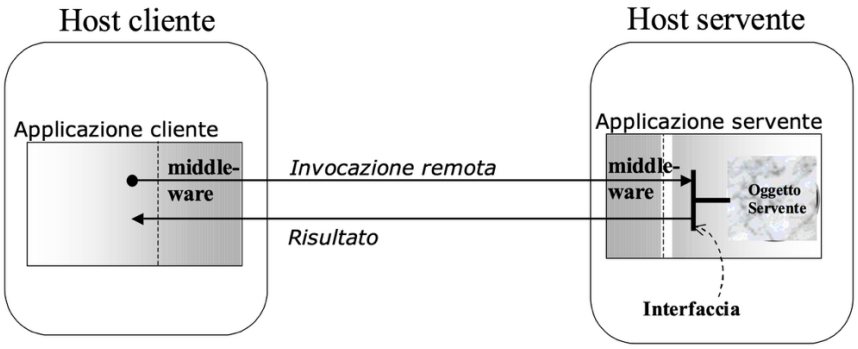
Il modello ad oggetti distribuiti è un'estensione in ambiente distribuito delle tecniche di programmazione orientata agli oggetti; infatti, l'architettura di questo tipo di modelli si basa sulla relazione:

$$DO = OOP + C/S$$

Ovvero, i Distributed Objects sono sistemi che uniscono l'Object Oriented Programming con un'architettura Client-Server. **L'utilizzo di una piattaforma DOM** (Distributed Object Middleware) **permette di invocare un metodo su di un oggetto remoto come se fosse locale**. Si tratta di un modello che viene riguardato come **la seconda generazione di sistemi software distribuiti**:

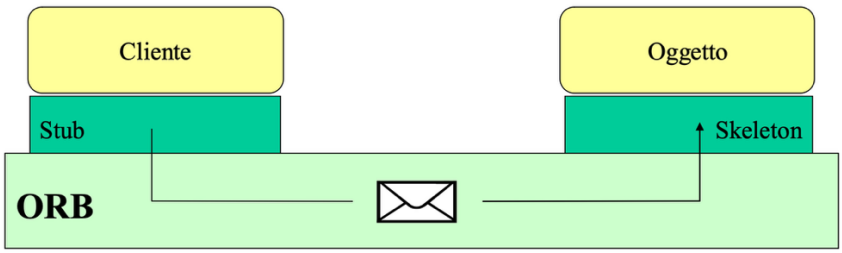


I componenti di un'applicazione distribuita sono oggetti che risiedono su macchine diverse e comunicano mediante invocazione di metodi, con un meccanismo analogo a quello RPC (sono presenti procedure stub e Interface Definition Language, IDL, e relativi compilatori):

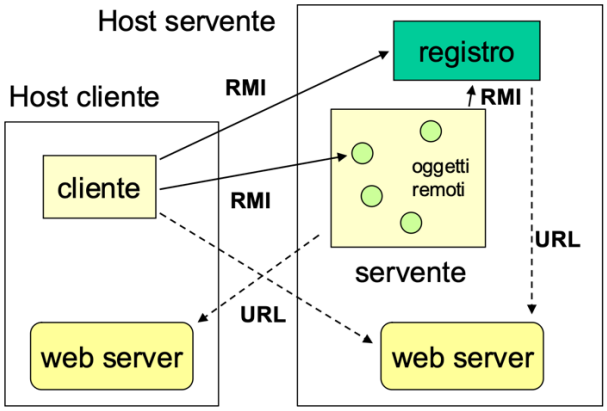


Il programmatore applicativo viene, in questo modo, **sollevato dall'onere di gestione della complessità dovuta alla distribuzione dei componenti su piattaforme eterogenee**, sviluppando software ad oggetti in maniera (il più possibile) simile al caso sequenziale.

Le piattaforme DOM si presentano spesso sotto forma di Object Request Brokers (ORB): una sorta di bus software per la comunicazione tra oggetti su rete; al programmatore, l'ORB si presenta come un insieme di API e sono supportate l'interazione client-oggetto (o meglio, stub-ORB skeleton).



Java RMI è un middleware ad oggetti specifico per la piattaforma Java, un ambiente di elaborazione distribuita di tipo omogeneo (grazie alla JVM). I processi serventi creano oggetti e ne registrano il nome in un catalogo (RMI registry), mentre i client ottengono riferimenti agli oggetti remoti, eventualmente tramite ricerca nel catalogo, e ne invocano i metodi:



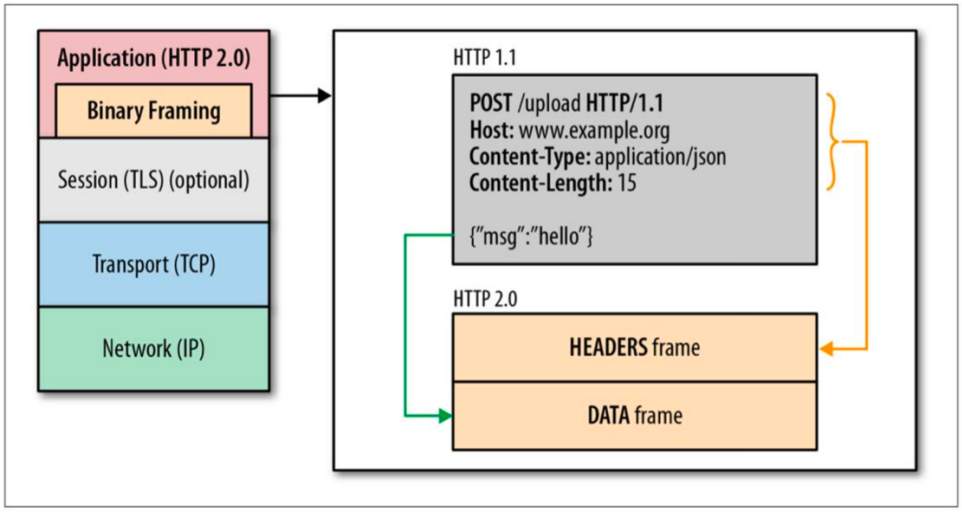
Java RMI fornisce i meccanismi per scambiare oggetti come parametri o come valori di ritorno, trasferire su richiesta il codice di un oggetto e serializzare e trasferire lo stato di un oggetto. Tali meccanismi consentono di realizzare sistemi con codice mobile.

REMOTE PROCEDURE CALL IN PYTHON

gRPC.io è un middleware RPC universale ad alte prestazioni e open source sviluppato da Google (ora un progetto Cloud Native Computing Foundation, CNCF) utilizzato da diverse aziende (come Google, Dropbox, Netflix, …), **che può essere eseguito in qualsiasi ambiente**, framework multilinguaggio e multipiattaforma, i cui principali scenari di utilizzo sono il collegamento di microservizi poliglotti che utilizzano lo stile di comunicazione richiesta-risposta, la connessione di dispositivi mobili e browser ai servizi di backend e la generazione di librerie client efficienti.

gRPC utilizza HTTP/2 per il trasporto (streaming bidirezionale e multiplexing), protocol buffer (protobuf) usato come IDL (per la generazione automatica del codice e perché i contratti rigorosi definiti tramite protobufs prevengono gli errori), ha supporto integrato per l'autenticazione, streaming bidirezionale e controllo del flusso e per la comunicazione bloccante e non bloccante (blocking/synchronous stub, il chiamante attende che il server risponda all'atto di una chiamata RPCC e ritornerà la risposta ricevuta, altrimenti solleva un'eccezione, e Non-blocking/asynchronous stub, il chiamante non attende che il server risponda all'atto di una chiamata RPC, ovvero la risposta dal server avviene in maniera asincrona); inoltre, anche il trasporto dei dati e delle richieste avvengono su HTTP/2, seguendo l'idea di base per la quale è possibile trattare le RPC come riferimenti a oggetti HTTP.

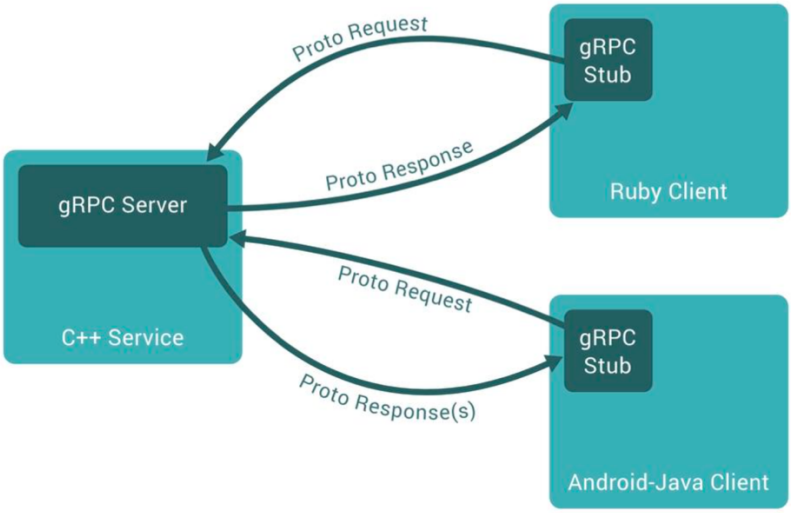
HTTP/2 è un importante revisione di HTTP che fornisce significativi vantaggi in termini di prestazioni rispetto ad HTTP 1.x. Le richieste/risposte HTTP/2 sono suddivise in messaggi di piccole dimensioni e con un frame in formato biario, rendendo la trasmissione dei messaggi efficiente.



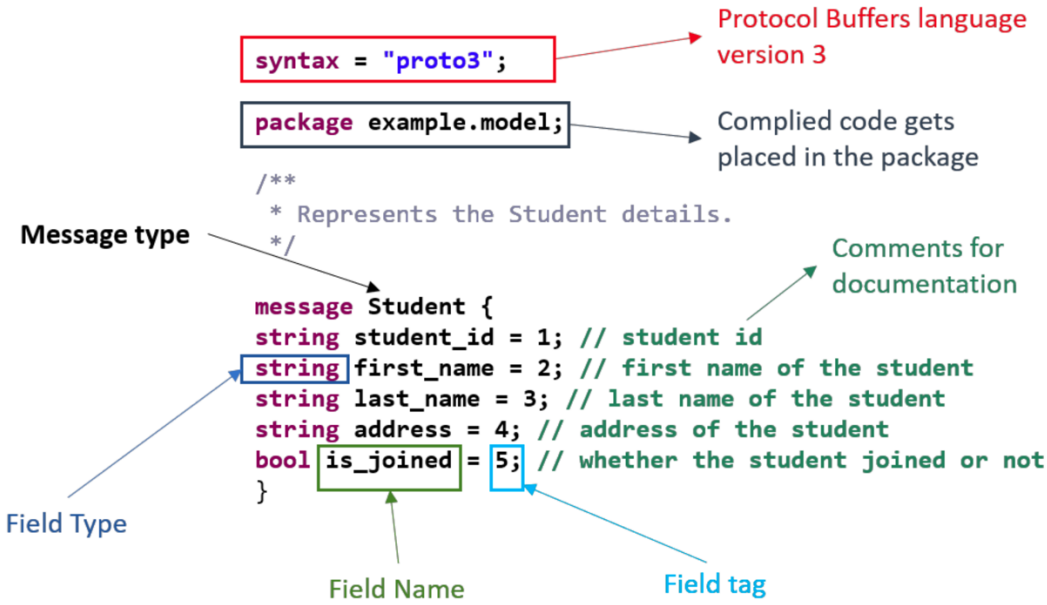
Si distinguono:

- Stream, flusso bidierzionale di byte all'interno di una connessione stabilita, che può trasportare uno o più messaggi;
    - Si ricordi che HTTP/2 fornisce il supporto nativo per lo streaming bidirezionale;
- Message, sequenza completa di messaggi che corrispondono ad una richiesta logica o ad un messaggio di risposta;
- Frame, la più piccola unità di comunicazione in HTTP/2, ciascuna contenente un'intestazione di frame che identifica almeno il flusso a cui il frame appartiene;
- Multiplexing richiesta/risposta (utilizzo di una singola connessione per client), consente un uso efficiente delle connessioni TCP ed evita il blocco della linea di testa a livello HTTP;
- Compressione dell'intestazione HTTP, per ridurre l'overhead del protocollo.

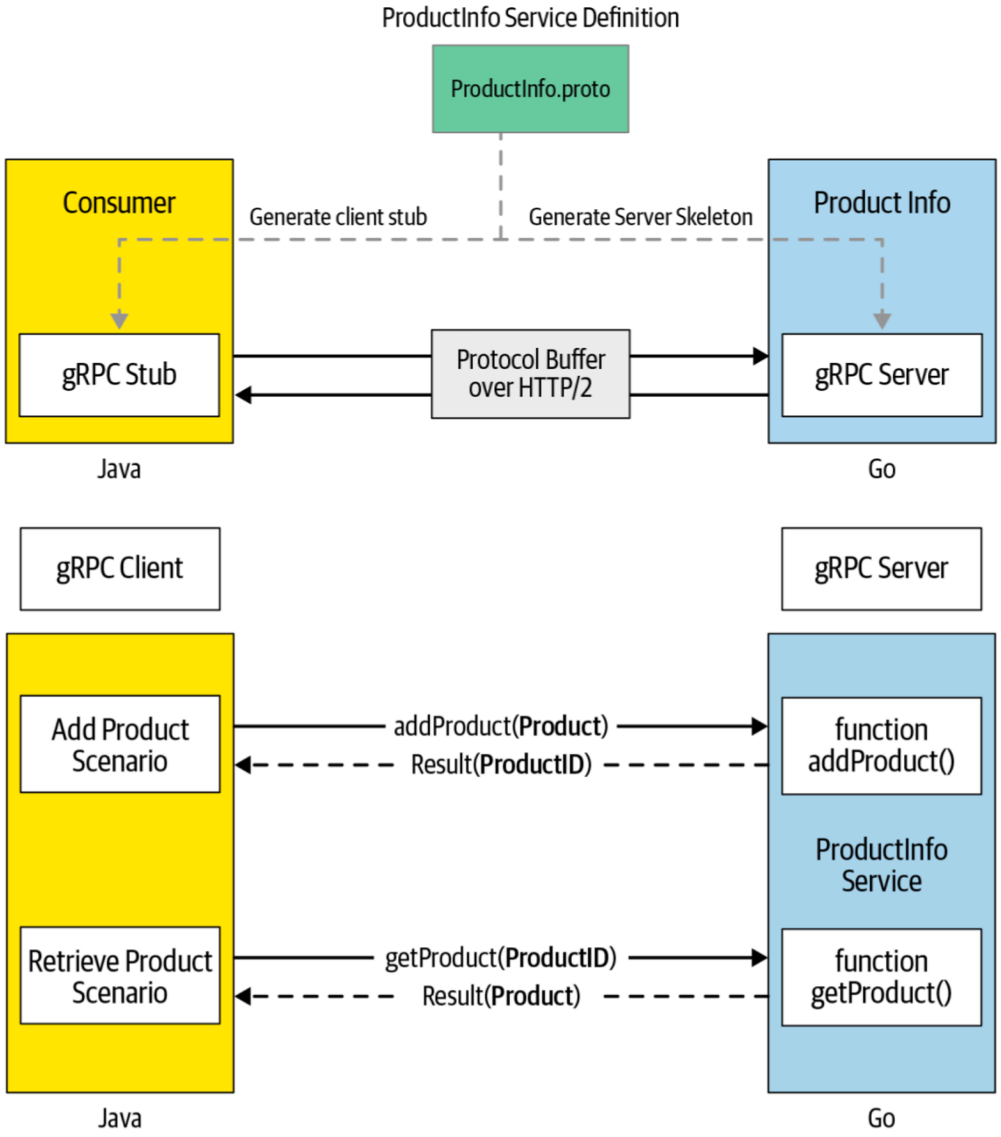
gRPC utilizza i protocol buffer, detti anche protobuf, **come IDL per definire l'interfaccia del servizio** (generazione automatica di stub client e classi server astratte) e come formato di interscambio dei messaggi (i messaggi gRPC sono serializzati utilizzando i protobufs, avendo dei piccoli payload dei messaggi d agestire). I protocol buffer sono basati sul solito modello di Proxy-Skeleton (stub e server).



I protocol buffer sono un meccanismo open source ormai maturo di Google per serializzare i dati strutturati, usano una rappresentazione binaria dei dati e una descrizione fortemente tipizzata; i tipi di dati sono strutturati come messaggi, laddove un messaggio è un piccolo record logico di informazioni contenente una serie di coppie nome-valore (dette campi), i quali hanno numeri univoci utilizzati per identificare i campi nel formato binario del messaggio.



È possibile aggiungere uno specificatore di package (opzionale) a un file `.proto` per evitare conflitti di nome tra i tipi di messaggi di un protocollo; in Python, la direttiva package viene ignorata, poiché i moduli Python sono organizzati in base alla loro posizione nel file system. In ogni caso, è fortemente raccomandato specificare il package per il file `.proto`, perché **altrimenti potrebbe portare a conflitti di denominazione nei descrittori e rendere il proto non portabile per altri linguaggi**. Ad esempio:



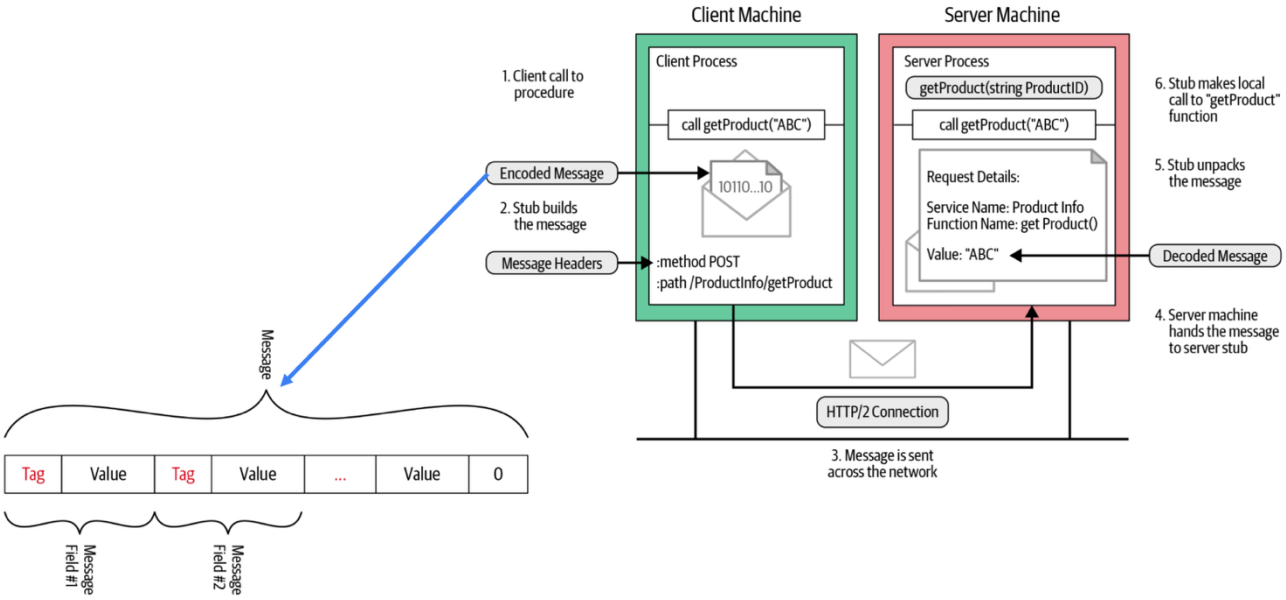

```
// ProductInfo.proto
syntax = “proto3”;
package ecommerce;

service ProductInfo {
     rpc addProduct(Product) returns (ProductID);
     rpc getProduct(ProductID) return (Product);
}

message Product {
     string id = 1;
     string name = 2;
     string description = 3;
}

Message ProductID {
     String value = 1;
}
```

Per utilizzare gRPC, è necessario:

1. Installare (una tantum) i package necessari per usare gRPC e i tool relativi in Python;

```
$ python -m pip install grpcio
$ python -m pip install grpcio-tools
```

2. Definire il servizio (collezione di metodi remoti) e i tipi di messaggi che vengono scambiati tra il client e il servizio nel file `.proto`, utilizzando i protobuf come IDL;
3. Generare il codice stub del server e del client a partire dalla definizione del servizio nel file `.proto`, utilizzando protoc (compilatore di protobuf) per il linguaggio specifico (ogni linguaggio possiede il proprio compilatore protobuf, in Python si usi il modulo `grpc_tool.protoc`);
4. Usare gRPCAPI per il linguaggio scelto (ad esempio, Java o Python) per scrivere successivamente il client e il server.



Per la compilazione del file `.proto`:

```
$ python -m grpc_tools.protoc -I. --python_out=. --pyi_out=. --
grpc_python_out=.NOMESERVIZIO.proto
```

Dove:

- `<NOMESERVIZIO>_pb2.py` contiene le classi di messaggi del buffer di protocollo, che sono usate per serializzare/deserializzare i messaggi, ed è l'output del flag `--python_out`;
- `<NOMESERVIZIO>_pb2_grpc.py` contiene le classi del servizio gRPC, includendo stub per il client e le classi servicer (skeleton) per il server, ed è l'output del flag `--grpc_python_out`.

Un esempio in Python si gRPC:

1. Definizione del servizio (file `helloworld.proto`):

```
syntax = "proto3";

package helloworld;

// La definizione del servizio chiamato Greeter
service Greeter {
  // Sends a greeting
  rpc SayHello (HelloRequest) returns (HelloReply) {}
}

// Il messaggio di richiesta contiene una stringa che è il nome utente
message HelloRequest {
  string name = 1;
}

// Il messaggio di risposta che contiene una stringa di benvenuto
message HelloReply {
  string message = 1;
}
```

2. Compilazione della definizione del servizio per ottenere gli stub:

```
$ python -m grpc_tools.protoc --python_out=. --py_out=. --grpc_python_out=. Helloworld.proto
```

La compilazione genera automaticamente i seguenti file:

a. `helloworld_pd2.py`, contiene codice protobuf per popolare, serializzare e recuperare tipi di messaggi di richiesta e risposta;
b. `helloworld_pb2_grpc.py`, contiene:
    i. l'interfaccia (proxy) utilizzata dai client per invocare (call) i messaggi definiti nel servizio `helloworld`;
    ii. L'interfaccia (skeleton) utilizzata dai server per implementare i metodi definiti nel servizio `helloworld`.

Si noti che, nel caso in cui si riscontrassero problemi con il comando appena riportato, si provi con:

```
$ python -m grpc_tools.protoc -I. --python_out=. --pyi_out=. --grpc_python_out=. ./helloworld.proto
```

Mentre se il file specifica (`helloworld.proto`) è all'interno di una directory, bisogna specificare quest'ultima con il flag `-Idirectory`:

```
$ python -m grpc_tools.protoc -Iprotos -python_out=. --pyi_out=. --grpc_python_out=. ./helloworld.proto
```

3. Compilazione del server:
    a. Implementazione dell'interfaccia del servizio dalla definizione del servizio:
        i. L'interfaccia è ottenuta dal file `helloworld_pb2_grpc` generato tramite il file `.proto` e avrà il nome formattato come `${NOMESERVIZIO}Servicer`;
        ii. `request` contiene i dati (messaggi di richiesta) mandati dal client;

iii. `context` è un'istanza della classe `grpc.ServicerContext` e contiene metadati e controlli legati alla comunicazione RPC (come status code della richiesta RPC);

L'implementazione del metodo `SayHello` restituisce un messaggio (stringa) del tipo `HelloWorld nome_richiesta`.

```
class Greeter(helloworld_pb2_grpc.GreeterServicer):

  def SayHello(self, request, context):

    return helloworld_pb2.HelloReply(message="Hello, %s!" % request.name)
```

b. Creare ed eseguire un server gRPC per accogliere le richieste dei client e inviarle all'implementazione del servizio:

È possibile specificare una serie di opzioni per la socket TCP utilizzata dal server GRPC; ad esempio, se si volesse evitare che più socket possano essere create da più processi con lo stesso `IP:PORT`, è necessario impostare a zero l'opzione `grpc.so_reuseport`.

```
def serve():
                # definisco il porto
                port = "50051"
                server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
                # aggiungo al server l'oggetto istanza del mio servizio Greeter
                helloworld_pb2_grpc.add_GreeterServicer_to_server(Greeter(), server)
                # faccio il bind al porto impostato
                server.add_insecure_port("[::]:" + port)
                # avvio il server
                server.start()
                print("Server started, listening on " + port)
                # attendo che il server termini
                server.wait_for_termination()

if __name__ == "__main__":
                serve()
```

c. Chiamiamo i metodi di servizio sullo stub client:

Si crea e si popola un oggetto di tipo protobuf di richiesta (`HelloRequest`) passando il corpo del messaggio:

```
# creo un canale verso il server RPC
with grpc.insecure_channel("localhost:50051") as channel:

            # creo uno stub (GreeterStub, ovvero ${NOMESERVIZIO}Stub) per invocare tutti i metodi
implementati nel servizio
            stub = helloworld_pb2_grpc.GreeterStub(channel)
            response = stub.SayHello(helloworld_pb2.HelloRequest(name="you"))
```

I channel creati con il modulo grpc sono thread-safe, così come gli stub lato client generati. La parte lato server, multithread, è soggetta alla problematica del GIL in ogni caso, nonostante gRPC utilizzi il package concurrent.

Per aggiornare un servizio bisogna aggiungere un nuovo metodo al file `.proto`, rigenerare il codice gRPC usando `protoc`, aggiornare il codice del server per implementare il nuovo metodo e il codice del client per invocare il nuovo metodo. gRPC supporta quattro tipi di RPC call che possono essere definiti nel file `.proto`:

- **RPC semplice**, il cilent invia una richiesta al server e attende una singola risposta (risposta unaria), come una normale chiamata a funzione:

```
rpc SayHello(HelloRequest) returns (HelloReply) {}
```

- **Streaming lato server**, o response-streaming RPC, il client invai una richiesta al server e ottiene un flusso per leggere una sequenza di messaggi fino al loro esaurimento (in questo caso, gRPC garantisce l'ordinamento dei messaggi all'interno di una singola chiamata RPC):

```
rpc ListFreature(Rectangle) returns (stream Feature) {}
```

- **Streaming lato client**, o request-streaming RPC, il client scrive una sequenza di messaggi e li invia al server, sempre utilizzando uno stream fornito, una volta finiti i quali attende che il server li legga tutti e restituisca la risposta (in questo caso, gRPC garantisce l'ordinamento dei messaggi all'interno di una singola chiamata RPC):

```
rpc RecordRoute(stream Point) returns (RouteSummary) {}
```

- **Bidirectional-streaming RPC**, entrambe le parti inviano una sequenza di messaggi utilizzando un flusso di lettura e scrittura, uno per client e uno per server, indipendenti e per i quali è possibile sia scrivere che leggere nell'ordine che si preferisce (il server potrebbe aspettare di ricevere tutte le richieste prima di iniziare a inviare tutte le risposte, oppure potrebbe intervallare la lettura di una richiesta alla sua risposta), considerando che l'ordine dei messaggi in ogni flusso viene mantenuto:

```
rpc RouteChat(stream RouteNote) returns (stream RouteNote) {}
```

Per creare le funzioni per lo streaming RPC è necessario l'uso dei `Generator`, funzioni che ritornano un iteratore, un oggetto che contiene un certo numero di elementi su cui è possibile iterare:

```
mytuple = ("apple", "banana")      # tupla
myit = iter(mytuple)                          # metodo che permette di ottenere un iterator
print(next(myit))                  # apple
print(next(myit))                  # banana
```

Per creare un Generator è necessario utilizzare il metodo `yield` al posto del metodo `return`, usato per produrre un valore senza fermare la funzione (in particolare, è considerata `Generator` la funzione che contiene `yield`). Ad esempio:

```
def range2iter(n):
    """
    Generator for the squares of numbers 0 to n-1
    Precon: n is an int >= 0
    """

    for x in range(n):
      yield x*x
```

L'utilizzo della return al posto di yield porterebbe la funzione ad interrompersi al primo ciclo. Invece:

```
>>> a = range2iter(3)
>>> a
<generator object>
>>> next(a) 0
>>> next(a) 1
>>> next(a) 4
```

Sono proposti di seguito degli esempi:

- Streaming RPC lato server:
    - Specifica del servizio `helloworld.proto`:

```
syntax = "proto3";

package helloworld;

// La definizione del servizio chiamato Greeter
service Greeter {
  // Sends a greeting
  rpc SayHello_v1 (HelloRequest) returns (stream HelloReply) {}
}

// Il messaggio di richiesta contiene una stringa che è il nome utente
message HelloRequest {
  string name = 1;
}

// Il messaggio di risposta che contiene una stringa di benvenuto
message HelloReply {
  string message = 1;
}
```

    - Server, `Class Greeter` in `helloworld_server.py`:

```
class Greeter(helloworld_pb2_grpc.GreeterServicer):

 # Streaming server
 def SayHello_v1(self, request, context):

   for i in range (0, 5):
    print("[server] SayHello method invoked, returning response...")
    yield helloworld_pb2.HelloReply(message="Hello, " + request.name + "! - " + str(i))
```

- Client, `Run method` in `helloworld_client.py`:

```
def run():

 print("Will try to greet world ...")

 with grpc.insecure_channel("localhost:50051") as channel:

  stub = helloworld_pb2_grpc.GreeterStub(channel)

  # Server streaming
  for response in stub.SayHello_v1(helloworld_pb2.HelloRequest(name="you")):
   print("[CLIENT] SayHello invoked Greeter client received: " + response.message)
```

- Streaming RPC lato client:
  - Specifica del servizio `helloworld.proto`:

```
syntax = "proto3";

package helloworld;

// La definizione del servizio chiamato Greeter
service Greeter {
 // Sends a greeting
 rpc SayHello_v2 (stream HelloRequest) returns (HelloReply) {}
}

// Il messaggio di richiesta contiene una stringa che è il nome utente
message HelloRequest {
 string name = 1;
}

// Il messaggio di risposta che contiene una stringa di benvenuto
message HelloReply {
 string message = 1;
}
```

- Server, `Class Greeter` in `helloworld_server.py`:

```
class Greeter(helloworld_pb2_grpc.GreeterServicer):

  # Streaming client
  def SayHello_v2(self, request_iterator, context):

    names = []
    for request in request_iterator:
      print("[server] SayHello method invoked, with name " + request.name)
      names.append(request.name)

    return helloworld_pb2.HelloReply(message="Hello, " + ' '.join(names) + "!")
```

- Client, `Run method` in `helloworld_client.py`:

```
def generate_requests():

  names = ['Raf', 'Gigi', 'Pippo', 'Pippozzo']

  for name_to_send in names:
    yield helloworld_pb2.HelloRequest(name=name_to_send)

def run():

  print("Will try to greet world ...")

  with grpc.insecure_channel("localhost:50051") as channel:

    stub = helloworld_pb2_grpc.GreeterStub(channel)

    # Client streaming
    response = stub.SayHello_v2(generate_requests())
    print("[CLIENT] SayHello invoked Greeter client received: " + response.message)
```

- Bidirectional-streaming RPC:
  - Specifica del servizio `helloworld.proto`:

```
syntax = "proto3";

package helloworld;

// La definizione del servizio chiamato Greeter
service Greeter {
  // Sends a greeting
  rpc SayHello_v3 (stream HelloRequest) returns (stream HelloReply) {}
}

// Il messaggio di richiesta contiene una stringa che è il nome utente
message HelloRequest {
  string name = 1;
}

// Il messaggio di risposta che contiene una stringa di benvenuto
message HelloReply {
  string message = 1;
}
```

- Server, `Class Greeter` in `helloworld_server.py`:

```
class Greeter(helloworld_pb2_grpc.GreeterServicer):

  # Bi-directional streaming
  def SayHello_v3(self, request_iterator, context):

    for request in request_iterator:
      print("[server] SayHello method invoked, returning response...")
      yield helloworld_pb2.HelloReply(message="Hello, " + request.name + "!")
```

- Client, `Run method` in `helloworld_client.py`:

```
def generate_requests():

  names = ['Raf', 'Gigi', 'Pippo', 'Pippozzo']

  for name_to_send in names:
    yield helloworld_pb2.HelloRequest(name=name_to_send)

def run():

  print("Will try to greet world ...")

  with grpc.insecure_channel("localhost:50051") as channel:

    stub = helloworld_pb2_grpc.GreeterStub(channel)

    # Bi-directional streaming
    for response in stub.SayHello_v3(generate_requests()):
      print("[CLIENT] SayHello invoked Greeter client received: " + response.message)
```

Gli errori più comuni in gRPC sono quello sollevato lato client quanto si sta contattando un server non attivo o irraggiungibile:

```
raise _InactiveRpcError(state)  # pytype: disable=not-instantiable
grpc._channel._InactiveRpcError: <_InactiveRpcError of RPC that terminated with:
                status = StatusCode.UNAVAILABLE
                details = "failed to connect to all addresses; last error: UNKNOWN: ipv4:127.0.0.1:1231: Failed to connect to remote host: Connection
refused"
                debug_error_string = "UNKNOWN:Error received from peer  {created_time:"2025-05-08T11:44:17.986041+02:00", grpc_status:14,
grpc_message:"failed to connect to all addresses; last error: UNKNOWN: ipv4:127.0.0.1:1231: Failed to connect to remote host: Connection refused"}"
```

O quello sollevato lato client quando server-side si sta utilizzando un campo inesistente per un messaggio gRPC:

```
raise _InactiveRpcError(state)  # pytype: disable=not-instantiable
grpc._channel._InactiveRpcError: <_InactiveRpcError of RPC that terminated with:
                status = StatusCode.UNKNOWN
                details = "Exception calling application: Protocol message StringMessage has no "wrongfield" field."
                debug_error_string = "UNKNOWN:Error received from peer  {created_time:"2025-05-19T18:29:25.437936+02:00", grpc_status:2,
grpc_message:"Exception calling application: Protocol message StringMessage has no \"wrongfield\" field."}"
```

O quello client-side quando si utilizza un campo inesistente per un messaggio gRPC:

```
Traceback (most recent call last):
  File "/Users/Idesi/prova_grpc/client.py", line 123, in <module>
    run(port)
  File "/Users/Idesi/prova_grpc/client.py", line 64, in run
    orders.append(order_management_pb2.Order(itemsaa=['A', 'B', 'C'],
ValueError: Protocol message Order has no "itemsaa" field.
```

O quello client-side quando non si utilizza un generator per una RPC di tipo stream lato server:

```
raise self
grpc._channel._MultiThreadedRendezvous: <_MultiThreadedRendezvous of RPC that terminated with:
                status = StatusCode.UNKNOWN
                details = "Exception iterating responses: 'Order' object is not an iterator"
                debug_error_string = "UNKNOWN:Error received from peer  {grpc_message:"Exception iterating responses: \'Order\' object is not an iterat
grpc_status:2, created_time:"2025-05-19T18:33:28.363825+02:00"}"
```

O quello client-side quando non si utilizza un generator per una RPC di tipo stream lato client:

```
grpc._channel._MultiThreadedRendezvous: <_MultiThreadedRendezvous of RPC that terminated with:
                status = StatusCode.UNKNOWN
                details = "Exception iterating requests!"
                debug_error_string = "None"
>
```

Le limitazioni principali di gRPC sono il supporto limitato nel browser (mancanza di supporto completo a HTTP/2, gRPC-Web può fornire il supporto ma con caratteristiche limitate, come la sola RPC semplice e streaming server limitato) e il formato che non è human-readable (protobuf è efficiente nell'invio e nella ricezione ma il suo formato binario non è leggibile all'uomo, gli sviluppatori hanno bisogno di strumenti aggiuntivi, come gRPC a riga di comando, per analizzare i payload, scrivere richieste a mano o eseguire debugging).

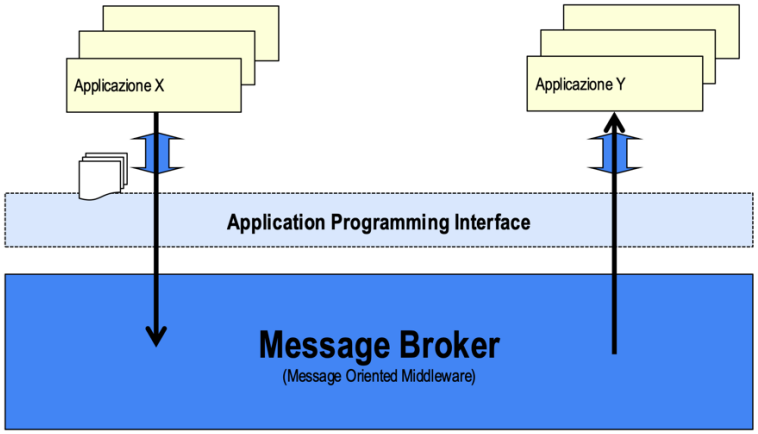
MESSAGE-ORIENTED MIDDLEWARE IN PYTHON

Questo tipo di middleware esiste perché è spesso necessario rimpiazzare quelli basati sul paradigma RPC: le chiamate RPC sono sincrone, portando a potenziali problemi di scalabilità, e le omonime richieste arretrate rallentano l'intero sistema. La soluzione a questo problema potrebbe essere **l'impiego di un meccanismo asincrono**, **incrementando la scalabilità del sistema**. I MOM sono utilizzati per **sviluppare sistemi con high availability e scalability**: laddove c'è necessità di mantenere il sistema funzionante anche dopo il fallimento di uno o più server è possibile migrare da uno all'altro utilizzando i broker.

Per comunicazione indiretta si intende un tipo di comunicazione tra entità di un sistema distribuito tramite un intermediario, ovvero l'assenza di accoppiamento diretto tra sender e receiver; la natura dell'intermediario dipende dallo specifico approccio alla comunicazione indiretta:

- Group communication, astrazione di un gruppo, a cui un messaggio è mandato e, quindi, recapitato ai suoi membri;
- Shared memory, distributed shared memory, tuple space;
- Code di messaggi, astrazione di una coda con meccanismo point-to-point per il quale il sender inserisce il messaggio in coda, il quale è rimosso da un solo receiver;
- Publish-subscribe, il publisher genera messaggi e il subscriber esprime l'interesse per una certa tipologia di messaggi (meccanismo uno-a-molti).

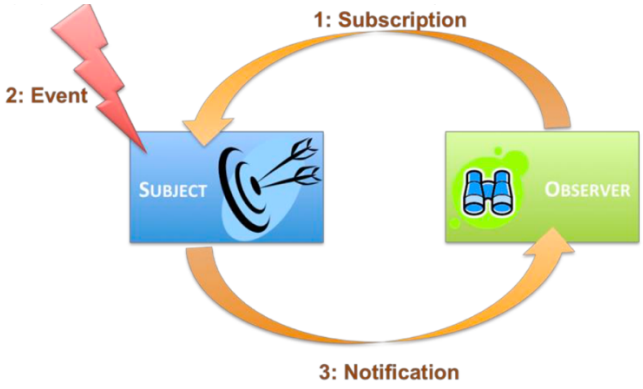
Le soluzioni middleware basate su code o approcci publish-subscribe sono note come Message-Oriented-Middleware (MOM) e, soprattutto facendo riferimento ai secondi, sono noti anche come **distributed event-based systems**. **Il MOM gioca il ruolo di intermediario** (message broker) **nella comunicazione indiretta basata su messaggi**: garantisce lo scambio di messaggi tra applicazioni con tecniche di store-and-forward e solleva il programmatore dai dettagli di basso livello della comunicazione (come, ad esempio, RPC e protocolli di rete) esponendo una API di alto livello (come, ad esempio, le primitive send/receive message).



Un MOM si occupa di:

- Recapitare il messaggio all'applicazione B che può risiedere su una macchina differene rispetto ad A;
- Gestire la comunicazione tramite la rete (il MOM può conservare un messaggio finché la ree diventa disponibile e provvedere ad inoltrarlo successivamente);
- Conservare il messaggio inviato dall'applicazione A finché l'applicazione B non ritorna disponibile, non bloccando la prima in attesa di ricezione (comunicazione asincrona).

I MOM forniscono meccanismi per l'integrazione flessibile e disaccoppiata delle applicazioni distribuite; per disaccoppiamento spaziale si intende la proprietà per la quale il sender non conosce o non ha bisogno di conoscere l'identità del receiver (e viceversa, in modo da rendere possibile lo swap o la migrazione dei partecipanti), mentre per disaccoppiamento temporale si intende la proprietà per la quale il sender e il receiver possono avere cicli di vita differenti (non devono essere necessariamente in esecuzione allo stesso momento per poter comunicare).

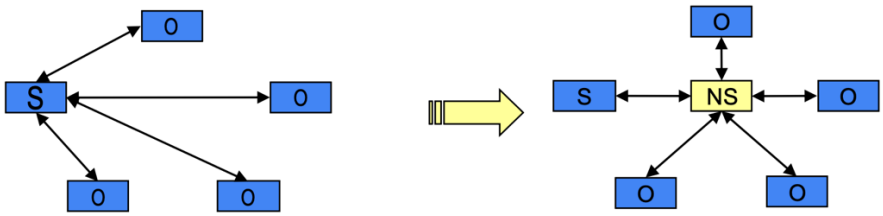


La comunicazione indiretta è fortemente utilizzata per la propagazione (dissemination) **di eventi nei sistemi distribuiti**; un evento è una condizione rilevata da/in un'applicazione e comunicata all'intermediario (ad esempio, il MOM) sotto forma di messaggio, mentre la notifica è l'atto di

informazione di un insieme di applicazioni dell'occorrenza dell'evento. **L'approccio evento-notifica richiama il pattern Observer**, per il quale un observer dichiara interesse agli eventi generati dal/nel subject tramite il concetto di sottoscrizione e per il quale un subject rileva l'occorrenza di un oggetto e notifica gli observer invocandone un'apposita funzione. L'observer riduce l'accoppiamento soggetto-osservatore e supporta la comunicazione uno-a-molti; tale approccio, tuttavia, presenta alcuni inconvenienti:

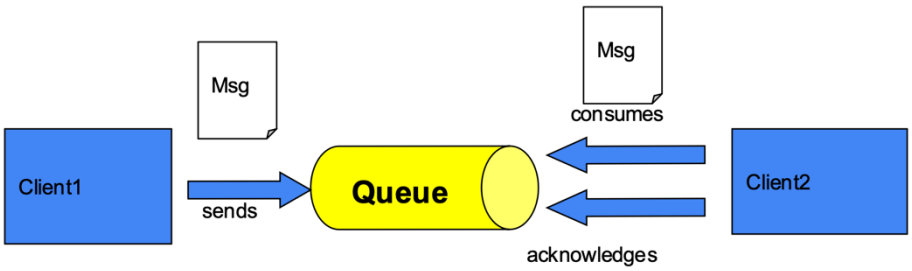
- Il subject deve mantenere una lista di observer e relative sottoscrizioni;
- Il subject è responsabile di notificare gli observer all'occorrenza di un evento di interesse (scarsa scalabilità);
- Gli observer devono conoscere il subject.

Questi inconvenienti possono essere risolti delegando la responsabilità di gestire gli osservatori/sottoscrizioni e di notificarli ad una terza entità, il Notification Service (NS) piuttosto che localizzarla nei soggetti.

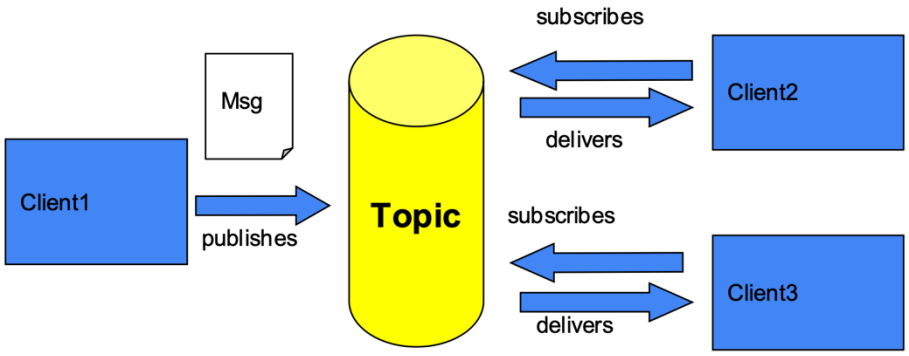


Il publish-subscribe non è l'unico modello per realizzare sistemi event based. Il Notification Service può utilizzare code di messaggi: all'occorrenza di un evento l'applicazione sender inserisce il messaggio in una coda, il quale viene recapitato ai receiver sottoscritti alla coda stessa.

I domini di messaging comuni in MOM sono il Point-to-Point (PTP), che ruota attorno al concetto di coda di messaggi e ad ogni messaggio è associato un solo consumer, **e il Publish-Subscribe**, in cui ogni messaggio è associato ad un topic, il quale è utilizzato per l'invio e la ricezione dei messaggi, considerando che ogni messaggio può avere più consumer. **Il dominio PTP prevede che una coda conservi un determinato messaggio finché esso non scade o non viene consumato dal receiver a cui è destinato**, il quale conferma la corretta ricezione del messaggio tramite segnale di `ACK`.



Il dominio Publish/Subscribe, invece, prevede che i topic conservino un messaggio finché esso non viene rilasciato ai subscriber correnti, considerando che possono essere processati da 0..N subscriber.



Esistono diversi protocolli usati dai broker MOM:

- **AMQP** (Advanced Message Queuing Protocol)
  - Progettato per rimpiazzare middleware di messaggistica proprietari esistenti
  - Protocollo binario wire-level (un modo per portare i dati da un punto all'altro) progettato per l'interoperabilità tra diversi fornitori
  - Ancora molto popolare
- **MQTT** (Message Queuing Telemetry Transport)
  - Protocollo di messaggistica publish-subscribe leggero, standard ISO (ISO/IEC PRF 20922), che usa TCP/IP
  - Progettato per situazioni in cui è richiesto un basso consumo energetico e in cui la larghezza di banda è limitata (scenari fog/edge computing, IoT)
- **STOMP** (Simple [or Streaming] Text Oriented Message Protocol)
  - Semplice protocollo basato su testo
  - Progettato per lavorare con middleware orientati ai messaggi (MOM)
  - Fornisce un formato *wire-level* che consente ai client STOMP di parlare con qualsiasi broker di messaggi che supporti il protocollo

Apache ActiveMQ è un popolare broker di messaggi open source, multiprotocollo basato su Java. Supporta i protocolli standard del settore (AMQP, MQTT, STOMP) e client scritti in diversi linguaggi, come JavaScript, C, C++, Python, .Net, Java, …

STOMP è un protocollo FRAME based che assume l'utilizzo di un 2-way streaming network protocol (come TCP), con client e server che comunicano attraverso dei STOMP frame inviati attraverso lo stream. STOMP fornisce un formato interoperabile che consente di poter comunicare con qualsiasi message broker che lo supporti. Inoltre, fornisce interopreabilità tra differenti linguaggi, piattaforme e broker. stomp.py è un client library Python per accedere a servizi di messaging attraverso il protocollo STOMP e può essere eseguito, oltre che in provider come ActiveMQ, Artemis e RabbitMQ, in modalità standalone, attraverso la command line. La connessione ad un provider avviene come segue:

```
import stomp
conn = stomp.Connection([('127.0.0.1',61613)])
conn.connect()
```

Laddove:

- `Connection()` accetta diversi parametri, principalmente l'`host_and_ports`, una lista di tuple `('IP-ADDRESS', PORT)` dove il message broker è in ascolto e con `IP-ADDRESS` che può anche essere IPv6 e il port 61613 è il port di default utilizzato da ActiveMQ (si consideri che la tupla permette al client di poter effettuare il check delle varie coppia fin quando una socket connection non viene stabilita con successo);

- `connect(username=None, passcode=None, wait=False, headers=None, with_connect_command=False, **keyword_headers)`:
  - `username`, specifica l'username per il login (str);
  - `passcode`, specifica la password dell'utente (str);
  - `wait`, attende che la connessione sia completata prima di ritornare (bool);
  - `headers`, mappa con headers addizionalil da inviare con la connessione (dict);
  - `with_connect_command`, se true usa il comando `CONNECT` invece di `STOMP`;

La disconnessione può essere effettuata con il metodo `disconnect()`. L'invio dei messaggi, invece:

```
conn.send('/queue/test', 'test message')
```

Dopo aver stabilito la connessione è possibile inviare messaggi con il metodo `send` verso una specifica destinazione definita in un provider:

- `send(destination, body, content_type=None, headers=None, **keyword_headers)`:
  - `destination`, destinazione, come una coda o un topic (str);
  - `body`, contenuto del messaggio;
  - `content_type`, MIME type del messaggio (str);
  - `headers`, headers aggiuntivi da inviare con il messaggio (dict);
  - `keyword_headers`, headers aggiuntivi richiesti dal provider.

La ricezione dei messaggi avviene attraverso il setup di un `Listener` e la sottoscrizione ad una destination; i `Listener` sono sottoclassi della `ConnectionListener` (sono fornite diversi sottoclassi `Listener`, come `PrintingListener` che stampa tutte le interazioni tra client e server). Il tutto avviene come segue:

```
conn.set_listener('name', MyListener())
conn.subscribe(destination='queue/test', id=1, ack='auto')
```

Con:

- `set_listener(name, Listener)`:
  - `name`, nome da assegnare al listener, che può essere utilizzato per la successiva rimozione (str);
  - `Listener`, istanza di listener da utilizzare;
- `subscribe(destination, id, ack='auto', headers=None, **keyword_headers)`, permette la sottoscrizione ad una destination:
  - `destination`, topic o queue a cui sottoscriversi (str);
  - `id`, identificativo univoco della sottoscrizione (str);
  - `ack`, modalità di acknowledgment, tra `auto`, `client` o `client-individual` (metodi `ack` e `nack`), (str);
  - `headers`, headers aggiuntivi da inviare con il messaggio (dict);
  - `keyword_headers`, headers aggiuntivi richiesti dal provider.

La classe `Listener` può essere così customizzata:

```
class MyListener(stomp.ConnectionListener):
```

```
    def on_message(self, frame):
        print('received a message "%s"' % frame.body)
```

Abilitano la ricezione asincrona di messaggi; è necessario ridefinire il metodo `on_message`, invocato alla ricezione di ogni messaggio sulla destination a cui fa riferimento:

- `on_message(frame)`:
  - `frame`, messaggio ricevuto (frame STOMP):
    - `frame.body`, corpo del messaggio;
    - `frame.headers`, mappa con gli headers del messaggio;
    - `frame.cmd`, comando STOMP.

Un esempio di comunicazione point-to-point (quindi, con queue) con ActiveMQ è il seguente. Per quanto riguarda il receiver:

```
# receiver.py
import time, stomp

class MyListener(stomp.ConnectionListener):

    def __init__(self, conn):
        self.conn = conn

    def on_message(self, frame):
        print('received a message "%s"' % frame.body)

if __name__ == "__main__":

    conn = stomp.Connection([('127.0.0.1', 61613)])

    conn.set_listener('', MyListener(conn))

    conn.connect(wait=True)
    conn.subscribe(destination='/queue/test', id=1, ack='auto')

    time.sleep(60)

    conn.disconnect()
```

Per quanto riguarda il sender:

```
# sender.py
import stomp

conn = stomp.Connection([('127.0.0.1', 61613)])
conn.connect(wait=True)

conn.send('/queue/test', 'test message')

conn.disconnect()
```

Per quanto riguarda il subscriber:

Un esempio di comunicazione pub-sub (quindi, con topic) con ActiveMQ è il seguente. Per quanto riguarda il subscriber:

```
# receiver.py
import time, stomp

class MyListener(stomp.ConnectionListener):

    def __init__(self, conn):
        self.conn = conn

    def on_message(self, frame):
        print('received a message "%s"' % frame.body)

if __name__ == "__main__":

    conn = stomp.Connection([('127.0.0.1', 61613)])

    conn.set_listener('', MyListener(conn))

    conn.connect(wait=True)
    conn.subscribe(destination='/topic/test', id=1, ack='auto')

    time.sleep(60)

    conn.disconnect()
```

Per quanto riguarda il publisher:

```
# sender.py
import stomp

conn = stomp.Connection([('127.0.0.1', 61613)])
conn.connect(wait=True)

conn.send('/topic/test', 'test message')

conn.disconnect()
```

Il protocollo STOMP mette a disposizione un metodo per trasmettere i messaggi ad un broker all'interno di una transazione, i quali sono mantenuti dal server finché non viene effettuato o il `commit` sulla transazione (in questo caso i messaggi sono effettivamente inoltrati) o l'`abort` (in questo caso i messaggi sono scartati). L'avvio di una transizione può essere effettuato con il metodo `begin`, ritornando un `transaction_id` che va utilizzato nell'invio dei messaggi e con la possibilità di generare il proprio `id` e passarlo al metodo come parametro.

```
# Commit
txid = conn.begin()
conn.send('/queue/test', 'test1', transaction=txid)
conn.send('/queue/test', 'test2', transaction=txid)
conn.send('/queue/test', 'test3', transaction=txid)
conn.commit(txid)

# Abort
txid = conn.begin()
conn.send('/queue/test', 'test4', transaction=txid)
conn.send('/queue/test', 'test5', transaction=txid)
conn.abort(txid)
```

Di default, in ActiveMQ un subscriber riceve i messaggi solo se pubblicati quando esso è attivo: il metodo `subscribe` crea di default un subscriber non durable; ad esempio, nel caso di durable subscriber STOMP è necessario usare gli header custom `client-id` e `activemq.subscriptionName` (che devono essere usati in combinazione). Il campo `client-id` viene impostato con l'hostname se non esplicitamente impostato, mentre la coppia `(client-id, activemq.subscriptionName)` identifica univocamente la sottoscrizione.

```
conn.connect(wait=True, headers = {"client-id":"IDtestsub_durable"})
...

conn.subscribe(destination='/topic/mytesttopic', id=1, ack='auto', persistent=True, headers =
{"activemq.subscriptionName":"DURABLESUB_NAME"})
```

È necessario avviare il provider ActiveMQ prima dell'esecuzione di un applicativo STOMP, digitando `activemq start` (o `activemq` in versioni meno recenti) da prompt, mentre per la terminazione dell'istanza si usa `activemq stop`. ActiveMQ dispone di un'interfaccia di amministrazione web-based accessibile all'indirizzo http://localhost:8161/. Accedendo all'interfaccia web, è possibile gestire sia code che topic (nelle apposite schede).

WEB SERVICES E PYTHON FLASK

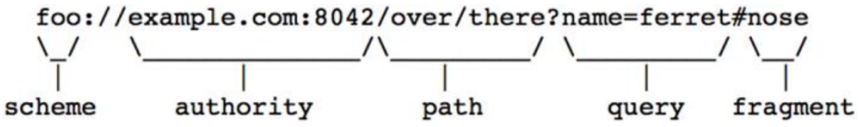
Un web service è un sistema software identificato da un URI (Uniform Resource Identifier) **le cui interfacce pubbliche e i cui vincoli sono definiti in XML e le cui definizioni possono essere trovate da altri sistemi software**; tali sistemi possono, poi, interagire con il web service nei modi definiti dal servizio stesso attraverso messaggi XML trasferiti con appositi protocolli di rete.

Per servizi si intendono moduli software che espongono delle funzionalità invocabili dai client tramite Internet ed accessibili tramite protocolli di rete standard. Un fornitore produce l'implementazione di un servizio e fornisce una descrizione, che può includere, ad esempio, l'interfaccia del servizio stesso; inoltre, un servizio può anche essere riusato o componibile. In linea generale, **la nozione di web service si riferisce alla possibilità di usare richieste HTTP per avviare l'esecuzione di un programma e non va confusa o associata con i web server**.

Con REST (Representational State Transfer) **si intende uno stile architetturale che definisce attributi di qualità architetturale del World Wide Web**, in questo contesto visto come un sistema di **hypermedia aperto, disaccoppiato, massivamente distribuito e decentralizzato**. Rispetto ai meccanismi RPC, **il focus di REST è sulle risorse più che sulle procedure**.

Il web espone i propri dati e le proprie funzionalità tramite risorse, identificate da un URI e considerate come entità indirizzabili tramite web, accessibili e trasferibili tra client e server, e caratterizzate da interfacce uniformi, i client interagiscono con le risorse tramite un insieme fissato di metodi (come `GET`, `POST`, `PUT` e `DELETE` di HTTP). **REST è stateless**: **ciascun ciclo di richiesta-risposta rappresenta un'interazione completa tra server e client**, non esiste il concetto di sessione e non sono mantenuti dati tra un'interazione ed un'altra.

Per URI, Uniform Resource Identifier, si intende uno standard di Internet per l'identificazione delle risorse e può essere utilizzato attraverso template, che specificano in che modo costruire e leggere un determinato URI:



- Template:

http://www.myservice.com/order/{oid}/item/{iid}

- Example URI:

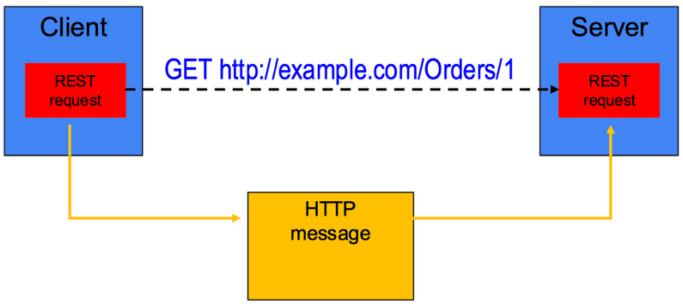
http://www.myservice.com/order/XYZ/item/12345

Lo scenario più comune è costituito dall'implementazione dell'architettura REST sul protocollo HTTP, un protocollo di livello applicativo per il trasferimento di pagine web per il quale ogni pagina e ogni oggetto al suo interno sono identificati da propri URL (Uniform Resource Locator) e per il quale sono previsti una serie di metodi per la gestione delle risorse che consentono di implementare in maniera diretta le tipiche operazioni CRUD (Create, Retrive, Update, Delete).

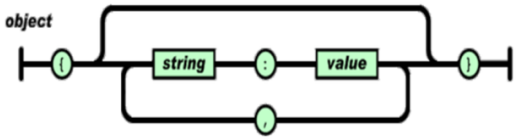
HTTP		SAFE	IDEM POTENT
POST	Create a sub resource	NO	NO
GET	Retrieve the *current state* of the resource	YES	YES
PUT	Initialize or update the state of a resource at the given URI	NO	YES
DELETE	Clear a resource, after the URI is no longer valid	NO	YES

Method	Entity-Body (o Representation) della richiesta	Entity-Body (o Representation) della risposta
GET	Di solito il client invia un entity-body/representation vuoto.	Il sever tipicamente restituisce nell' entity-body la rappresentazione della risorsa.
DELETE	Di solito il client invia un entity-body/representation vuoto.	Il sever può restituire un entity-body vuoto o con un messaggio di stato.
PUT	Di solito il client invia nell'entity-body la rappresentazione proposta per la risorsa.	Il sever può restituire un entity-body vuoto o un messaggio di stato o una copia della reppresentazione della risorsa.
POST	il client invia nell'entity-body la rappresentazione proposta per la risorsa.	Il sever può restituire un entity-body vuoto o un messaggio di stato o una copia della reppresentazione della risorsa.

Una comunicazione RESTful può essere schematizzata come segue:



XML (eXtensible Markup Language) **e JSON** (JavaScript Object Notation) **sono formati tipicamente usati per la rappresentazione dei dati scambiati tra client e servizi** (external data representation), **sono rappresentazioni testuali** (a differenza di soluzioni come la serializzazione Java, che sono binarie). Ad esempio, un oggetto JSON è una serie non ordinata di nomi/valori ed è caratterizzato dalla seguente sintassi:



La differenza tra JSON e XML è puramente sintattica:

XML

```
<empinfo>
  <employees>
    <employee>
      <name>Scott Philip</name>
      <salary>£44k</salary>
      <age>27</age>
    </employee>
    <employee>
      <name>Tim Henn</name>
      <salary>£40k</salary>
      <age>27</age>
    </employee>
    <employee>
      <name>Long yong</name>
      <salary>£40k</salary>
      <age>28</age>
    </employee>
```

JSON

```
{ "empinfo" :
   {
      "employees" : [
      {
        "name" : "Scott Philip",
        "salary" : £44k,
        "age" : 27,

      },
      {
        "name" : "Tim Henn",
        "salary" : £40k,
        "age" : 27,
      },
      {
        "name" : "Long Yong",
        "salary" : £40k,
        "age" : 28,

      }
                ]
   }
}
```

Ciascun web service RPC-style espone un proprio "vocabolario" di metodi (funzioni con nomi differenti), mentre **nel caso REST il "vocabolario" è fissato** (sono i metodi HTTP) e tutti i servizi espongono una stessa interfaccia di base:

RPC-style:	**RESTful:**
`insertOrder(id);`	`POST` *`Order/{id}`*
`getOrder(id);`	`GET` *`Order/{id}`*
`updateOrder(id);`	`PUT` *`Order/{id}`*
`deleteOrder(id);`	`DELETE` *`Order/{id}`*

Per la progettazione REST occorre seguire i seguenti passi principali:

- Identificare le risorse da esporre come servizi;
- Per ogni risorsa:
  - Definire le URI;
  - Esporre un subset adeguato dell'interfaccia uniforme (ossia ragionare su cosa significa fare operazioni di `GET` e `POST` di certe risorse, se e quali operazioni debbano essere consentite, …);

- Progettare le rappresentazioni fornite dal client (al server) o servite dal server (al client):
  - JSON, XML;
  - Messaggi di stato;
  - …;

Per le URI, è meglio preferire nomi a verbi:

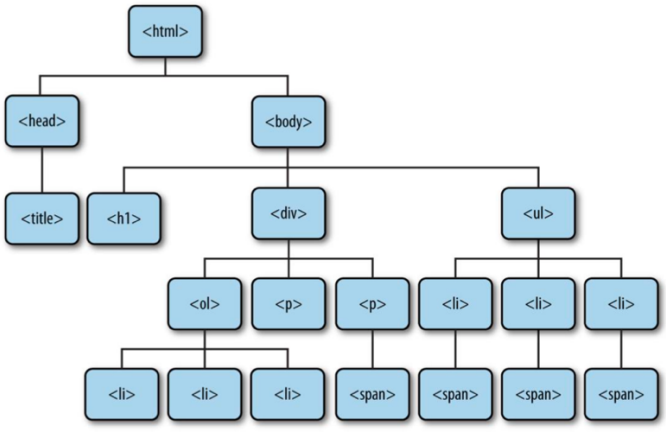
```
DELETE /book/15                      OK
GET /book?isbn=15&action=delete      NO
```

POST può sembrare una via di mezzo tra GET e PUT ma le differenze sono sostanziali:

- GET vs POST: il primo è read-only (idempotente), mentre il secondo è read-write e può modificare lo stato della risorsa;
- POST vs PUT: il primo è tipicamente usato per creare una risorsa figlia rispetto ad una risorsa padre, mentre il secondo è usato per modificare risorse esistenti.

HTML (HyperText Markup Language) **è una tecnologia che permette di specificare la struttura degli elementi visive che compongono una web application**; un documento HTML può essere descritto da una serie di **HTML tags**, ognuno dei quali caratterizzato da un opening (`<name>`) e un closing (`</name>`) **tra i quali è inserito il contenuto**, che può essere sia testo sia altri tag HTML. **I tag definiscono un'architettura gerarchica chiamata DOM** (Document Object Model), **una modalità di rappresentazione degli oggetti che può essere definito attraverso HTML e con il quale si può interagire attraverso scripting language** (Javascript), in particolare i tag definiscono i DOM elements, le entità che vivono all'interno del DOM, e possono essere organizzati, pertanto, come un albero:

```html
<!doctype html>
<html>
  <head>
    <title>Hello World!</title>
  </head>
  <body>
    <h1>Hello World!</h1>

    <div>
      <ol>
        <li>List Item</li>
        <li>List Item</li>
        <li>List Item</li>
      </ol>

      <p>This is a paragraph.</p>

      <p>This is a <span>second</span> paragraph.</p>

    </div>

    <ul>
      <li>List Item <span>1</span></li>
      <li>List Item <span>2</span></li>
      <li>List Item <span>3</span></li>
    </ul>
  </body>
</html>
```



Di seguito è riportata una tabella con tutti i principali tag e il loro impiego:

Tag	Description
`<html>`	*The main container for an HTML document*
`<head>`	*Contains meta-information about the document*
`<body>`	*Contains the content that will be rendered in the browser*
`<header>`	*The header of the page*
`<h1>`	*Most important heading (only one per document)*
`<h2>`	*Second most important heading*
`<h3>`	*Third most important heading*
`<main>`	*The main content area of your document*
`<footer>`	*The footer content of your document*
`<a>`	*Anchor, a link to another document or a clickable element*
`<ul>`	*A list of things where order doesn't matter*
`<ol>`	*A list of things where order matters*
`<li>`	*An element of a list*
`<div>`	*A container for a substructure*

Un web framework è un insieme di tecnologie utili per lo sviluppo di una web app e tipicamente forniscono routes (che mappano gli URL a server, file o funzioni), template (che inseriscono automaticamente server-side data in pagine HTML), meccanismi di autenticazione e autorizzazione, sessioni (che tengono traccia di un utente durante una singola visita) e altro. **Flask è un web application framework WSGI leggero** (detto anche **microframework**), **progettato per consentire la realizzazione rapida e semplice di applicazioni web**, con la possibilità di scalare verso applicazioni complesse, e **il cui core mette a disposizione tutti i servizi di base**, che possono essere estesi attraverso il meccanismo delle estensioni.

Nato come wrapper di Werkzeug e Jinja2, Flask è diventato uno dei web application framework per Python più utilizzati. Wekzeug mette a disposizione le funzionalità di routing, debugging e Web Server Gateway Interface (WSGI, pronunciato whiskey, una specifica che descrive come un web server comunica con le applicazioni web e come tali applicazioni possono essere interconnesse per processare una richiesta), mentre Jinja2 mette a disposizione il supporto ai template, di cui si discuterà a breve. In quanto framework Python, Flask può essere installato (insieme alle sue due dipendenze) tramite l'apposito comando `pip`:

```
pip install Flask
```

Un esempio di Hello World in Flask è il seguente:

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def index():
  return "<h1>Hello, World!</h1>"

@app.route('/user/<name>')
def user(name):
  return '<h1>Hello, %s!</h1>' % name

if __name__ == "__main__":
  app.run(debug=True)
```

Inizializzazione

Routes

Server startup

Tutte le applicazioni Flask devono creare una application instance, avendo come argomento del costruttore Flask il nome del main modulo o del package dell'applicazione. **I client web** (come un browser) **di una web application inviano richieste al web server**, il quale **le instrada verso l'applicazione Flask**; questa, poi, necessita di sapere **quale codice eseguire per ogni URL richiesto**, ovvero di **mantenere il mapping tra gli URL e le funzioni Python**, ovvero le **routes**. Flask permette di definire una route attraverso il decoratore `app.route(URL)`, registrando la funzione definita subito dopo come gestore dell'URL specificato; **la funzione**, in questo contesto, **prende il nome di view function e sarà invocata ad ogni richiesta ricevuta sull'URL specificato** e può **tornare come risposta al client sia del codice HTML che codice in formati più complessi** (come JSON). Sono possibili anche **dynamic route**, cioè **route che contengono porzioni dinamiche** (variabili) **all'interno dell'URL**, **che possono cambiare ad ogni richiesta o essere anche assenti**; quando la view function associata è invocata, **Flask passa la parte dinamica dell'URL come parametro della funzione**. **L'istanza di un'applicazione Flask prevede un metodo run che avvia il web server di sviluppo integrato in Flask**: una volta avviato, il server entra in un loop di attesa di richieste, il quale termina solo forzandone la chiusura da terminale. Di default, il server è su http://127.0.0.1:5000/ ed è raggiungibile solo da localhost, nonostante possa essere configurato con la personalizzazione della chiamata al metodo:

```
app.run(host=None, port=None, debug=None, **options)
```

Ad esempio:

```
app.run(host='0.0.0.0', port=80)
```

Il seguente codice:

```
# hello.py
from flask import Flask

app = Flask(__name__)

@app.route("/")
def index():
  return "<h1>Hello, World!</h1>"

@app.route('/user/<name>')
def user(name):
  return '<h1>Hello, %s!</h1>' % name

if __name__ == "__main__":
  app.run(debug=True)
```

Corrisponde alla seguente esecuzione:

```
user@hostname % python hello.py
 * Serving Flask app 'hello'
 * Debug mode: on
WARNING: This is a development server.
Do not use it in a production deployment.
Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 350-815-848
127.0.0.1 - - [08/May/2024 00:44:44]
    "GET / HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2024 00:46:21]
    "GET /user/pippo HTTP/1.1" 200 -
```

Sulla base di quanto detto finora, **le view function ritornano stringhe in HTML generate direttamente nelle funzioni**, un mix tra business logic e presentation logic che produce codice più difficile da comprendere e da manutenere. Il modo migliore è affidarsi ad un **template engine** per spostare la presentation logic in un template:

- **Template**, file che contiene il testo di una risposta (parte statica), con dei placeholder per la parte dinamica che saranno popolati opportunamente dalla richiesta;
- **Rendering**, processo che sostituisce i placeholder con i valori attuali e ritorna la risposta finale al client.

Flask utilizza il template engine di Jinja2.

Di default, i template devono essere inseriti in un'apposita sottocartella, `./templates`, e sono file HTML (con estensione `.html`). È possibile, poi, effettuare il rendering dei template con la funzione:

```
render_template(template_name_or_ist, **context)
```

Considerando che:

- `template_name_or_list` è il nome del template di cui fare il rendering (se è fornita una lista, il primo file disponibile sarà utilizzato per il rendering);
- `context`, le variabili da mettere a disposizione nel template.

Aggiornando l'esempio precedente, si considerano rispettivamente i template `templates/index.html` e `templates/user.html`:

```
<h1>Hello World!</h1>
<h1>Hello, {{ name }}!</h1>
```

L'applicazione Flask sarà così realizzata:

```
from flask import Flask, render_template
...
@app.route("/index")
def index():
  return render_template('index.html')

@app.route('/user/<name>')
def user(name):
  return render_template('user.html', name=name)
```

Ottenendo lo stesso risultato. Jinja2 riconosce ogni tipo di variabile, comprese tipi complessi come liste, dizionari e oggetti; inoltre, sono offerte strutture di controllo che possono essere utilizzate per alterare il flusso del template, come:

- Costrutti di selezione condizionale

```
{% if user %}
   Hello, {{ user }}!
{% else %}
   Hello, Stranger!
{% endif %}
```

- Cicli for

```
<ul>
   {% for comment in comments %}
      <li>{{ comment }}</li>
   {% endfor %}
</ul>
```

`curl` è un comando da terminale che permette di generare richieste http (e non solo) da terminale; ad esempio:

`curl https://www.google.com/`

Genera un `GET` verso l'URL specificato (metodo utilizzato di default, se i parametri non suggeriscono altro); alcuni parametri utili sono:

- `-X`, permette di specificare il tipo di metodo HTTP da utilizzare per la richiesta;
- `-I`, permette di recuperare solo gli headers della risorsa richiesta, generando una richiesta `HEAD`;
- `-d`, permette di specificare il campo data per una richiesta `POST` (per questo metodo non è necessario il parametro `-X`);
- `--json`, permette di inviare dati in formato JSON in una richiesta `POST`.

Una request object mette a disposizione le informazioni delle richieste HTTP inviate dai client, con Flask che **utilizza i context e gli oggetti request**: i primi permettono di rendere temporaneamente accessibili i secondi come variabili globali e di rendere accessibili le variabili ad un thread senza interferire con gli altri thread, mentre il secondo rende possibile l'accesso ai campi della richiesta HTTP del client.

```
from flask import Flask, request
...

@app.route('/')
def index():
   user_agent = request.headers.get('User-Agent')
   return '<p>Your browser is %s</p>' % user_agent
```

Attraverso l'oggetto request, è possibile accedere al body della richiesta:

- `get_data(cache=True, as_text=False, parse_from_data=False)`, legge i dati provenienti dal client in un unico bytes object, con:
  - `cache`, specifica se i dati devono essere cached;
  - `as_text`, specifica se i dati devono essere decodificati come stringa;

  - `parse_from_data`, specifica se i dati devono essere elaborati (parsed) come form data.
- `get_json(force=False, silent=False, cache=True)`, fa il parsing dei dati JSON presenti nella richiesta se il mimetype è application/json, con:
  - `force`, specifica se ignorare il mimetype ed effettuare sempre parsing JSON;
  - `silent`, silenzia i mimetype e i parsing errors e ritorna `None`;
  - `cache`, memorizza il parsed JSON per successive chiamate.

Ma è anche possibile accedere ai parametri passati attraverso l'URL (la parte dell'URL dopo ?), tramite `request.args`.

```
from flask import Flask, request
...
# test: curl http://127.0.0.1:5000/hello?name=pippo&surname=pippozzo
@app.get('/hello')
def hello():
  params = request.args
  print("name:", params['name']) # pippo
  print("surname:", params['surname']) # pippozzo
  return params
```

```
from flask import Flask, request
...

@app.post('/json')
def json():
  json = request.get_json()
  print(json)
  return json

@app.post('/data')
def data():
  data = request.get_data(as_text=True)
  print(data)
  return data
```

Di default, **le routes rispondono solo a richieste di tipo GET**, sebbene sia possibile **utilizzare l'argomento `methods` del decoratore `route()` per gestire differenti metodi HTTP**:

```
@app.route('/print', methods=['GET', 'POST'])
def print_method():
  if request.method == 'POST':
    return '<h1>This is a POST</h1>'
  else:
    return '<h1>This is a GET</h1>'
```

Test
GET: curl http://127.0.0.1:5000/print
POST: curl -X POST http://127.0.0.1:5000/print

È possibile anche **separare la gestione dei metodi HTTP in differenti funzioni**, dal momento in cui Flask mette a disposizione delle shortcut per decorare tali rotte, ognuna per ogni metodo HTTP.

```
@app.post('/print')
def print_post():
  return '<h1>This is a POST</h1>'

@app.get('/print')
def print_get():
  return '<h1>This is a GET</h1>'
```

Il valore di ritorno di una view function è convertito in automatico in un oggetto `Response`: se il valore di ritorno è una stringa, il corrispettivo oggetto `Response` conterrà la stringa come body, 200 Ok come status code e text/html come mimetype; se **il valore di ritorno è un dizionario o una lista**, invece, **viene chiamato automaticamente il metodo `jsonify()` per generare la risposta**, il quale serializza l'argomento fornito come JSON e ritorna un oggetto `Response` con il mimetype application/json. Flask utilizza, di default, il modulo built-in di Python per gestire il formato JSON.

La logica di conversione utilizzata da Flask è la seguente:

- Se è ritornato un oggetto `Response`, tale oggetto viene direttamente ritornato dalla view function;
- Se il valore di ritorno è una stringa, l'oggetto `Response` conterrà la stringa e valori di default per tutti gli altri parametri (come precedentemente indicato);
- Se viene ritornato un iterator o un generator, che ritornano stringhe o byte, allora viene trattato come una streaming response;
- Se viene ritornato un dizionario o una lista, viene invocato il metodo `jsonify()` per la creazione dell'oggetto `Response`;
- Se viene ritornata una tupla, è possibile includere varie informazioni extra e la tupla dovrà essere nel formato:
    - `(response, status);`
    - `(response, headers);`
    - `(response, status, headers);`
    - Notando che `status` sovrascrive lo status code della risposta e `header` può essere una lista o dizionario con header aggiuntivi;
    - Se non si è in alcuno dei casi indicati, Flask assume che il valore di ritorno sia un'applicazione WSGI valida e la converte in un oggetto `Response`.

Ovviamente, il dizionario o la lista dovranno eventualmente contenere dati che siano JSON serializzabili.

```
@app.route("/me")
def me_api():
  user = {
    "username": "pippo",
    "age": 20,
  }
  return user

@app.route("/users")
def users_api():
  users = get_all_users() #return a list of dictionaries
  return users
```

Nel caso in cui sia necessario ritornare uno status code differente dal 200 Ok (ritorno di default), è possibile includerlo come secondo valore di ritorno (ritornando una tupla):

```
@app.route('/error')
def error():
  return '<h1>Bad Request</h1>', 400
```

Le risposte possono anche includere un terzo argomento che contiene un dizionario di headers che sono aggiunti alla risposta HTTP:

```
@app.route('/errorh')
def errorh():
  headers = {'New header' : 'Flask Application'}
  return '<h1>Bad Request</h1>', 400, headers
```

I possibili status code di HTTP sono:

1XX Information		**4XX Client (Continue)**	
100	Continue	407	Proxy Authentication Required
101	Switching Protocols	408	Request Timeout
102	Processing	409	Conflict
103	Early Hints	410	Gone
		411	Length Required
2XX Success		412	Precondition Failed
200	OK	413	Payload Too Large
201	Created	414	URI Too Large
202	Accepted	415	Unsupported Media Type
203	Non-Authoritative Information	416	Range Not Satisfiable
205	Reset Content	417	Exception Failed
206	Partial Content	418	I'm a teapot
207	Multi-Status (WebDAV)	421	Misdirected Request
208	Already Reported (WebDAV)	422	Unprocessable Entity (WebDAV)
226	IM Used (HTTP Delta Encoding)	423	Locked (WebDAV)
		424	Failed Dependency (WebDAV)
3XX Redirection		425	Too Early
300	Multiple Choices	426	Upgrade Required
301	Moved Permanently	428	Precondition Required
302	Found	429	Too Many Requests
303	See Other	431	Request Header Fields Too Large
304	Not Modified	451	Unavailable for Legal Reasons
305	Use Proxy	499	Client Closed Request
306	Unused		
307	Temporary Redirect	**5XX Server Error Responses**	
308	Permanent Redirect	500	Internal Server Error
		501	Not Implemented
4XX Client Error		502	Bad Gateway
400	Bad Request	503	Service Unavailable
401	Unauthorized	504	Gateway Timeout
402	Payment Required	505	HTTP Version Not Supported
403	Forbidden	507	Insufficient Storage (WebDAV)
404	Not Found	508	Loop Detected (WebDAV)
405	Method Not Allowed	510	Not Extended
406	Not Acceptable	511	Network Authentication Required
Compiled by Ivan Tay.		599	Network Connect Timeout Error

Flask permette ad una view function di costruire un oggetto `Response` all'interno della view function stessa attraverso la funzione `make_response()`, che accetta più argomenti utilizzati per la costruzione del Response object (il quale sarà generato come nel caso di view function che ritorna una tupla di valori); tale modalità è utilizzata quando **si vuole creare esplicitamente il `Response` object nella view function e configurarlo opportunamente**.

```
from flask import Flask, make_response
import time
...
@app.route('/')
def index():
  response = make_response('<h1>This document carries a cookie!</h1>')
  response.set_cookie('Last-access', str(time.time()))
  return response
```

La libreria Python `request` permette di creare facilmente richieste HTTP/1.1 in Python, è basata sulla libreria `urllib3`, che permette il supporto ai meccanismi di Keep-alive e HTTP connection pooling, e supporta tutti i metodi di HTTP. Essendo una libreria Python, si installa attraverso l'apposito comando `pip`:

```
pip install urllib3==1.26.6
pip install request
```

Mentre per l'utilizzo:

```
requests.head(url, **kwargs)
requests.get(url, params=None, **kwargs)
requests.post(url, data=None, json=None, **kwargs)
requests.put(url, data=None, json=None, **kwargs)
requests.delete(url, **kwargs)
```

Considerando che:

- `url`, è l'URL della richiesta;
- `params`, dizionario, lista di tuple o bytes opzionale da inviare nella query string della richiesta;
- `data`, dizionario, lista di tuple, bytes o file-like object opzionale da inviare nel body della richiesta e utilizzabile in `POST` e `PUT`;
- `json`, oggetto JSON serializable opzionale da inviare nel body della richiesta e utilizzabile in `POST`;
- `**kwargs`, keyword arguments opzionali eventualmente necessari nella richiesta.

La libreria `request` permette di utilizzare le **query strings**, cioè passare parametri all'interno dell'URL aggiungendoli attraverso keyword arguments:

```
payload = {'key1': 'value1', 'key2': 'value2'}
r = requests.get('https://httpbin.org/get', params=payload)
print(r.url)
```

⬇

```
https://httpbin.org/get?key2=value2&key1=value1
```

```
payload = {'key1': 'value1', 'key2': ['value2', 'value3']}
r = requests.get('https://httpbin.org/get', params=payload)
print(r.url)
```

⬇

```
https://httpbin.org/get?key1=value1&key2=value2&key2=value3
```

Si noti che se una delle chiavi del dizionario ha un valore nullo, il parametro non sarà aggiunto alla query string dell'URL e che è possibile anche passare una lista come valore.

Il metodo `request.post` permette di popolare il payload della richiesta attraverso l'argomento data o json e, nel caso si utilizzi un dizionario, i dati saranno inseriti nella richiesta nel formato form-encoded:

```
>>> payload = {'key1': 'value1', 'key2': 'value2'}
>>> r = requests.post('https://httpbin.org/post', data=payload)
>>> print(r.text)
{
 ...
 "form": {
   "key2": "value2",
   "key1": "value1"
 },
 ...
}
```

È possibile sia anche prevedere più valori per ogni chiave:

```
>>> payload_tuples = [('key1', 'value1'), ('key1', 'value2')]
>>> r1 = requests.post('https://httpbin.org/post', data=payload_tuples)
>>> payload_dict = {'key1': ['value1', 'value2']}
>>> r2 = requests.post('https://httpbin.org/post', data=payload_dict)
>>> print(r1.text)
{
  ...
 "form": {
  "key1": [
    "value1",
    "value2"
  ]
 },
  ...
}
>>> r1.text == r2.text
True
```

Sia inserire un payload direttamente in formato JSON:

```
url = 'https://api.github.com/some/endpoint'
payload = {'some': 'data'}
r = requests.post(url, json=payload)
```

Il riferimento ottenuto dall'invio di una richiesta è un `Response` object da cui è possibile ottenere informazioni sulla risposta, la libreria `requests` effettua automaticamente la decodifica delle informazioni generate dal server:

```
r = requests.get('https://api.github.com/events')
r.text
```

```
'[{"repository":{"open_issues":0,"url":"https://github.com/...
```

Inoltre, effettua il guessing del Unicode charsets se non specificato nella risposta, sfruttandolo per accedere a `r.text`, mentre `r.encoding` permette di visualizzare l'encoding utilizzato e di impostarne uno nuovo.

Altre operazioni possibili con la libreria `requests` sono:

- `r.content`, mostra il contenuto della risposta in byte;
- `r.text`, mostra il contenuto della risposta in unicode;
- `r.encoding`, mostra l'encoding utilzizato dal server per la risposta;
- `r.status_code`, mostra lo status code della risposta;
- `r.url`, mostra la final URL location della risposta;
- `r.headers`, mostra gli headers della risposta;
- `r.headers['header name']`, mostra uno specifico header.

È previsto dalla libreria anche un decoder JSON built-in che permette di gestire eventuali risposte con contenuti nel formato JSON:

```
r = requests.get('https://api.github.com/events')
r.json()
```

```
[{'repository': {'open_issues': 0, 'url': 'https://github.com/...
```

Se **il decoding JSON fallisce** (contenuto vuoto, la risposta ha un contenuto che non rispetta il formato JSON, …), **la `r.json()` solleva `requests.exceptions.JSONDecodeError`**. Si noti che `r.json()` **decodifica solo il contenuto della risposta**, se la chiamata va a buon fine non è detto che la richiesta abbia avuto successo: **molti server ritornano in risposta un oggetto JSON anche a fronte di fallimenti** (ad esempio, per fornire dettagli sul problema avuto lato server nel caso di un HTTP 500). Per verificare lo stato di una risposta è possibile usare `r.status_code`; tuttavia, esiste anche la funzione **`raise_for_status()`, che solleva `requests.exceptions.HTTPError` se la risposta contiene un HTTP error come status code**.

```
>>> bad_r = requests.get('https://httpbin.org/status/404')
>>> bad_r.status_code

404

>>bad_r.raise_for_status()

Traceback (most recent call last):
  File "requests/models.py", line 832, in raise_for_status
    raise http_error
requests.exceptions.HTTPError: 404 Client Error
```

NO-SQL DATABASE

I **database** sono comunemente utilizzati sulla parte back-end delle web application, dai web server, e **forniscono meccanismi di salvataggio persistente dei dati in memoria**. Memorizzare i dati in un database **riduce il carico sulla memoria centrale del server** e permette di poter **recuperare i dati anche in caso di crash del server stesso**; inoltre, molte richieste inviate ad un server potrebbero richiedere una **query su un database**, un client potrebbe richiedere informazioni che sono memorizzate nel database o sottomettere dei dati al server richiedendone la memorizzazione su database.

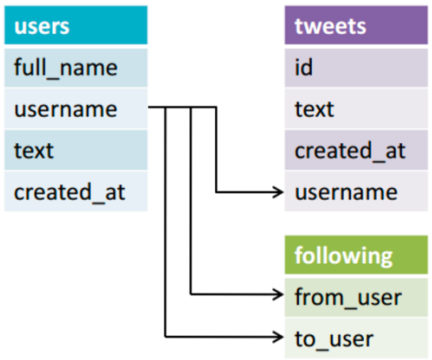
In termini tecnici, **un database è una collection di dati organizzati in maniera tale da poter essere facilmente acceduti, gestiti e aggiornati**. Un **DBMS** (DataBase Management System) **è un software che permette la creazione, definizione e manipolazione di un database**, fornendo un'interfaccia o un tool che permette di effettuare operazioni come la creazione di un database, la memorizzazione dei dati in esso, il loro aggiornamento, la creazione di tabelle, …, il tutto garantendo la sicurezza e la consistenza dei dati; infatti, **un DBMS fornisce anche i meccanismi di protezione e sicurezza del database stesso**, garantendo la consistenza dei dati nel caso di utenti multipli.

Un DBMS è caratterizzato da diverse feature:

- Data stored into Tables;
- Reduced redundancy via normalization;
- Data consistency;
- Support multiple user and concurrent access;
- Query language;
- Security;
- Support for transaction.

I database sono divisi in due principali categorie: database **relazionali** (o Sequence Databases using SQL Languages) e database **non-relazionali** (o Non Sequence Databased using NoSQL Languages). Entrambi i tipi di database possono essere utilizzati da una stessa organizzazione, singolarmente o in maniera combinata (soprattutto nel contesto di Big Data e Data Warehouses), in base alla nature dal dato e della funzionalità necessaria.

Un database relazionale è un database che si basa sul concetto di schema, un template utilizzato per definire la struttura dei dati che saranno memorizzati nel database, possono esserci varie tabelle all'interno di un tabase, ognuna proveniente da uno schema di relazione differente, e per ognuna delle quali saranno associate delle chiavi di accesso, che permettono l'accesso rapido ad una particolare riga/colonna.

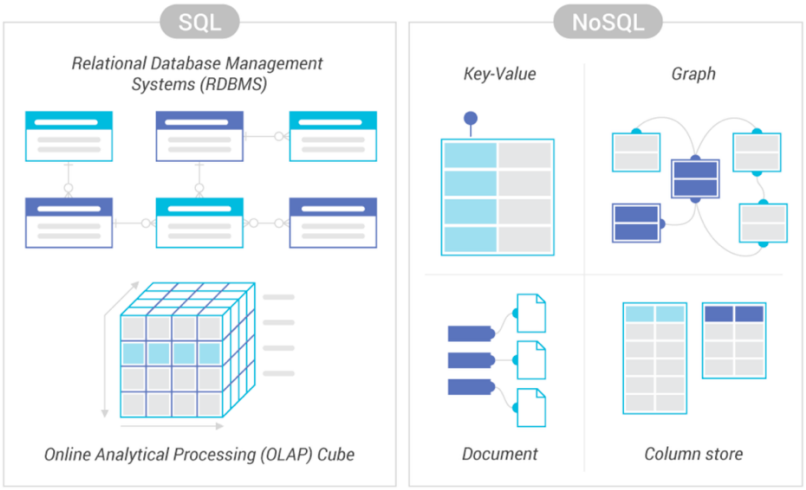


Le tabelle, o **Entità**, **sono in relazione l'una con le altre**, in modo che **una modifica ad una può doversi propagare sulle altre**, e, per individuare univocamente una riga piuttosto che un'altra, sono caratterizzate sempre da uno o più attributi identificativi.

I database relazionali seguono uno schema ben definito, ogni nuova entry deve essere conforme con questo template, in modo da rendere i dati prevedibili e facilmente valutabili, al costo di una **eccessiva rigidità in contesti in cui la produzione di dati è eterogenea e il loro formato non uniformato**. **Un DBMS relazionale** (o RelationalDBSM, o RDBMS) deve necessariamente essere ACID compliant, cioè **deve garantire il rispetto ad ogni istante di tempo delle proprietà ACID** (Atomicity, Consistency, Isolation e Durability). Segue che i database relazionali sono **ben strutturati ma lasciano poco spazio all'occorrenza di errori**; infatti, la presenza di uno schema ben definito e di forti vincoli rende quasi **impossibile il loro utilizzo in scenari Big Data**: gli utenti potrebbero voler scalare il RDBMS su server multipli e più potenti che sono costosi e difficili da gestire, rendendo molto difficile la gestione di tabelle su più dislocazioni fisiche. **I vincoli dello schema ostacolano la migrazione dei dati da e verso RDBMS diversi**, è necessario che siano identici altrimenti il tutto non funzionerà.

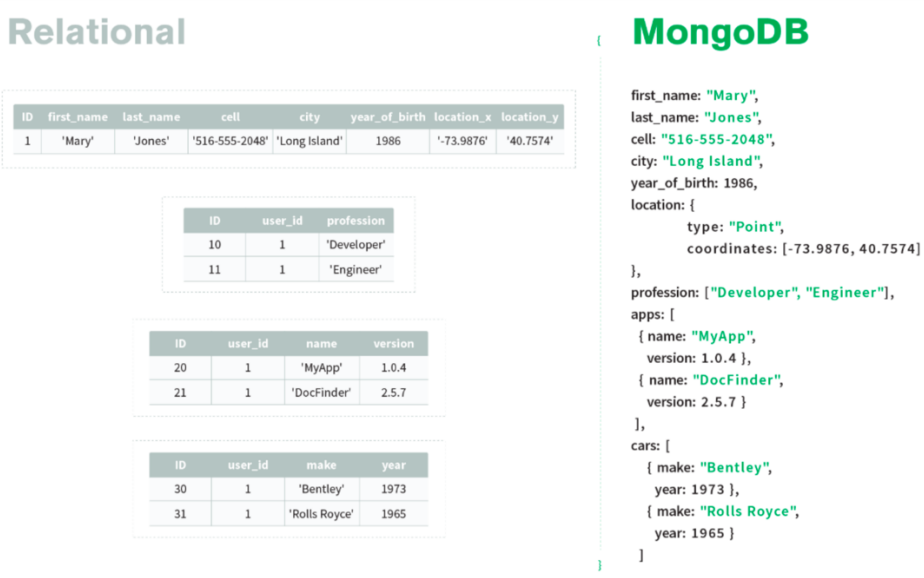
I database non relazionali sono più indulgenti nella loro struttura e forma rispetto ai database relazionali, invece che tabelle con righe e colonne **sono utilizzati collection di differenti categorie**, come utenti e ordini, **caratterizzate da documents**. In una collection, possono esserci documents multipli, che **possono non seguire uno schema o un pattern particolare**; inoltre, **collection differenti non necessariamente devono essere in relazione fra di loro**. A loro volta, i database non relazionali possono essere distinti in:

- Key-value, memorizzano e forniscono dati sottoforma di coppie chiave-valore;
- Documents stores, possono avere un ampio dizionario di tipi e valori (potenzialmente innestati) e i dati sono memorizzati in documenti JSON;
- Search engines, si distinguono dai document stores in quanto permettono di accedere ai dati attraverso semplici text-based searches.



La natura schema-free dei database non relazionali li rende facili da gestire e scalare orizzontalmente e proni ad ospitare grandi quantità di dati; inoltre, **i dati non sono molto complessi e possono essere distribuiti tra differenti nodi per migliorarne l'accessibilità**. Tuttavia, dato che non hanno una struttura o uno schema specifico per la memorizzazione dei dati, **non è possibile fare affidamento sulla presenza di un certo campo**, dal momento in cui non sempre sarà disponibile (o esistente) e non avere relazioni rende molto difficile l'aggiornamento dei dati, dal momento in cui sarà necessario aggiornare ogni singolo dettaglio in maniera separata.

Un **document-oriented database**, o document store, **è un programma progettato per memorizzare, recuperare e gestire document-oriented informations**, noti anche come **semi-structured data**. Uno dei più utilizzati document-oriented database è **MongoDB**, costruito basandosi su **JSON-like documents**: ogni document può essere differente, con un numero variabile di campi, la cui dimensione e contenuto può essere diverso l'uno dall'altro, e non necessitano di uno schema definito in anticipo, **i campi possono essere creati on-the-fly**.



I componenti chiave di MongoDB sono i seguenti:

- _id, campo richiesto in ogni documento e identifica un valore unico nel MongoDB document, impostato come se fosse una chiave primaria per il document stesso;

- Collection, raggruppamento di MongoDB documents, è l'equivalente di una tabella nei DBMS relazionali;
- Cursor, puntatore al risultato di una query, in modo che i client possano iterare per recuperare un risultato;
- Database, contenitore per le collection (nel caso relazionale è un contenitore di tabelle);
- Document, un record in una collection MongoDB;
- Field, una coppia nome-valore in un document, dotato di zero o più campi.

Di seguito sono mostrate alcune operazioni possibili su un database non relazionale con MongoDB:

```
> cards = [{ "rank" : "ace", "suit" : "clubs" }];
[ { "rank" : "ace", "suit" : "clubs" } ]
> ["two", "three", "four", "five"].forEach( function (rank) { cards.push( { "rank":rank, "suit":"clubs" } ) });
> cards
[
        {
                "rank" : "ace",
                "suit" : "clubs"
        },
        {
                "rank" : "two",
                "suit" : "clubs"
        },
        {
                "rank" : "three",
                "suit" : "clubs"
        },
        {
                "rank" : "four",
                "suit" : "clubs"
        },
        {
                "rank" : "five",
                "suit" : "clubs"
        }
]
```

```
> show collections;
startup_log
> show dbs;
admin   0.000GB
config  0.000GB
local   0.000GB
> db.cards
local.cards
> db.cards.save(cards);
BulkWriteResult({
        "writeErrors" : [ ],
        "writeConcernErrors" : [ ],
        "nInserted" : 5,
        "nUpserted" : 0,
        "nMatched" : 0,
        "nModified" : 0,
        "nRemoved" : 0,
        "upserted" : [ ]
})
> show collections
cards
startup_log
> db.cards.find();
{ "_id" : ObjectId("6098f8770542b5c6f887495b"), "rank" : "ace", "suit" : "clubs" }
{ "_id" : ObjectId("6098f8770542b5c6f887495c"), "rank" : "two", "suit" : "clubs" }
{ "_id" : ObjectId("6098f8770542b5c6f887495d"), "rank" : "three", "suit" : "clubs" }
{ "_id" : ObjectId("6098f8770542b5c6f887495e"), "rank" : "four", "suit" : "clubs" }
{ "_id" : ObjectId("6098f8770542b5c6f887495f"), "rank" : "five", "suit" : "clubs" }
```

See databases in MongoDB

Save object to a new collection; Alternative *db.cards.insertMany(cards)*

Show existing collections

What documents are stored in collection **cards**

I metodi di DELETE in MongoDB sono i seguenti:

- Rimozione di alcuni elementi da una collection sulla base di una query:

```
# db.collectionName.remove({QUERY});
```

- Rimozione di tutti gli elementi di una collection:

```
# db.collectionName.remove({});
```

- Rimozione di un database:

```
# use databaseName;
# db.dropDatabase();
```

PyMongo è il driver ufficiale per l'utilizzo di MongoDB in applicativi Python, fornisce tutto il necessario per interagire con un'istanza di MongoDB e si installa con l'apposito comando `pip`:

```
pip install pymongo
```

Il primo step per l'utilizzo è la connessione all'istanza di MongoDB in esecuzione tramite la classe `MongoClient`:

```
from pymongo import MongoClient
client = MongoClient("localhost", 27017)
```

È possibile anche usare una string connection:

```
client = MongoClient("mongodb://localhost:27017/")
```

Ottenuto il client, è possibile accedere ai database gestiti da quell'istanza di MongoDB:

```
db = client['test-datbase']
```

Attraverso il riferimento al database è possibile, poi, accedere alle singole collection che compongono il database:

```
collection = dbname['test-collection']
```

Sulle collection è possibile effettuare le varie operazioni CRUD, considerando che i dati in MongoDB sono rappresentati e memorizzati sottoforma di JSON-style documents; in particolare, in PyMongo sono utilizzati i dizionari per rappresentare i documents:

```
post = {
  "author": "Mike",
  "text": "My first blog post!",
  "tags": ["mongodb", "python", "pymongo"]
}
```

Considerando il dizionario appena riportato, l'inserimento di un solo documento avviene come segue:

```
collection.insert_one(post)
```

Si noti che se il dizionario/documento non presenta un campo _id, tale campo sarà aggiunto e popolato automaticamente. Se si volessero inserire **documenti multipli** (cioè, un array di dizionari), come il seguente, in una collection:

```
collection.insert_many(posts)
```

```
posts = [
  {
    "author": "Mike",
    "text": "Another post!",
    "tags": ["bulk", "insert"],
  },
  {
    "author": "Eliot",
    "title": "MongoDB is fun",
    "text": "and easy too!",
  },
]
```

Per la ricerca di un solo documento in una collection:

```
result = collection.find_one({"author: "Eliot"})
```

Ritornando il primo documento che fa match con il filtro specificato o `None` in caso contrario. Per la ricerca di documenti multipli in una collection, invece:

```
results = collection.find({"author": "Eliot"})
for post in results:
    print("Author: ", post['author'])
```

Ritornando un cursore per iterare sui risultati della query, cioè tutti i documenti che fanno match con il filtro specificato. I filtri possono prevedere anche l'utilizzo di operatori per creare query più complesse:

```
res = collection.find({"items": {"$in" : ["item-A", "item-B"]}})
```

Operator	Meaning	Example	SQL Equivalent
$gt	Greater Than	"score":{"$gt":0}	>
$lt	Less Than	"score":{"$lt":0}	<
$gte	Greater Than or Equal	"score":{"$gte":0}	>=
$lte	Less Than or Equal	"score":{"$lte":0}	<=
$all	Array Must Contain All	"skills":{"$all":["mongodb","python"]}	N/A
$exists	Property Must Exist	"email":{"$exists":True}	N/A
$mod	Modulo X Equals Y	"seconds":{"$mod":[60,0]}	MOD()
$ne	Not Equals	"seconds":{"$ne":60}	!=
$in	In	"skills":{"$in":["C","c++"]}	IN
$nin	Not In	"skills":{"$nin":["php","ruby","perl"]}	NOT IN
$nor	Nor	"$nor":[{"language":"english"},{"country":"usa"}]	N/A
$or	Or	"$or":[{"language":"english"},{"country":"usa"}]	OR
$size	Array Must Be Of Size	"skills":{"$size":3}	N/A

Per la rimozione di un solo documento da una collection:

```
collection.delete_one({"author: "Eliot"})
```

Cancellando il primo documento che fa match con il filtro specificato. Per la rimozione di documenti multipli in una collection:

```
collection.delete_many({"author: "Eliot"})
```

Cancellando uno o più documenti che fanno match con il filtro specificato. Per l'aggiornamento di un solo documento in una collection:

```
collection.update_one({"author: "eliot"}, {"$set": {"author: "Eliot"}})
```

Aggiornando il primo documento che fa match con il filtro (primo parametro), mentre l'aggiornamento da realizzare è indicato dal modificatore (secondo parametro). Per l'aggiornamento di documenti multipli in una collection:

```
collection.update_many({"author: "eliot"}, {"$set": {"author: "Eliot"}})
```

Aggiornando uno o più documenti che fanno match con il filtro (primo parametro), mentre l'aggiornamento da realizzare è indicato dal modificatore (secondo parametro). Esistono, poi, differenti modificatori per le operazioni di aggiornamento:

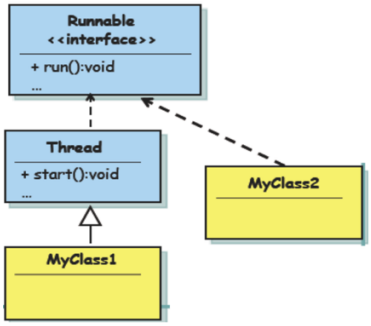
Modifier	Meaning	Example
$inc	Atomic Increment	"$inc":{"score":1}
$set	Set Property Value	"$set":{"username":"niall"}
$unset	Unset (delete) Property	"$unset":{"username":1}
$push	Atomic Array Append (atom)	"$push":{"emails":"foo@example.com"}
$pushAll	Atomic Array Append (list)	"$pushall":{"emails":["foo@example.com","foo2@example.com"]}
$addToSet	Atomic Append-If-Not-Present	"$addToSet":{"emails":"foo@example.com"}
$pop	Atomic Array Tail Remove	"$pop":{"emails":1}
$pull	Atomic Conditional Array Item Removal	"$pull":{"emails":"foo@example.com"}
$pullAll	Atomic Array Multi Item Removal	"$pullAll":{"emails":["foo@example.com", "foo2@example.com"]}
$rename	Atomic Property Rename	"$rename":{"emails":"old_emails"}

PROGRAMMAZIONE AVANZATA IN JAVA

MULTITHREADING IN JAVA

Java supporta la programmazione multithread a livello di linguaggio. Tipicamente, i thread sono **implementati a livello di sistema**, richiedendo un'interfaccia di programmazione dipendente dalla piattaforma su cui girerà il programma; tuttavia, **Java consente di realizzare programmi multithread in maniera standardizzata e indipendente dalla specifica piattaforma**, grazie alla Java Virtual Machine, fornendo primitive per definire attività indipendenti e primitive per la comunicazione e la sincronizzazione tra attività eseguite in maniera concorrente. Java offre due possibili tecniche per creare un thread (a partire da `java.lang`):

- Derivazione dalla classe `Thread`;
- Implementazione dell'interfaccia `Runnable`.



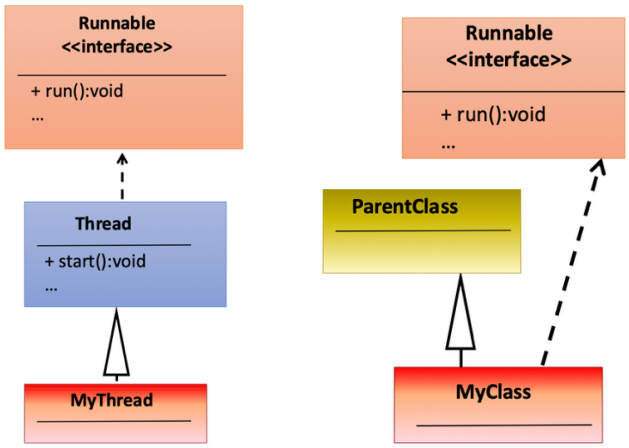
Quando la classe utente deriva da `Thread`, deve ridefinire il metodo `run()` di `Runnable` per poter eseguire quando è invocato il metodo derivato `start()`:

```
public class MyThread extends Thread {
   public void run() {
       codiceThread();
    }
}

Thread t = new MyThread();
t.start();
```

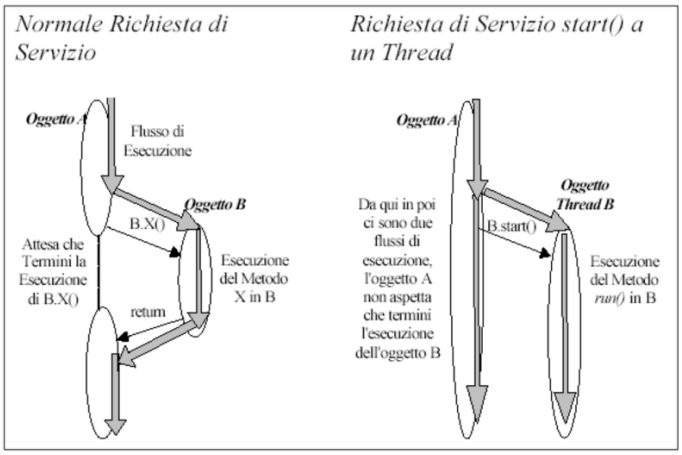
Mentre se implementa `Runnable`, deve solo ridefinire il metodo `run()`, per essere eseguito come thread ha la necessità di essere passato al costruttore di un oggetto istanza della classe `Thread`:

```
public class MyRunnable implements Runnable {
    public void run() {
        codiceThread();
    }
}

Runnable r = new MyRunnable();
Thread t = new Thread(r);
t.start();
```

Java non consente la derivazione multipla e se la classe utente non è già coinvolta in un legame di derivazione è possibile **usare la soluzione con `Thread`**; tuttavia, immaginando una classe utente che è già in un legame di derivazione con un'altra classe, la soluzione proposta nel caso precedente non è più possibile. In questo contesto si inserisce la **duplice soluzione di Java**: **una classe già coinvolta in un legame di derivazione con un'altra classe può comunque implementare un'interfaccia, `Runnable`, la quale permette comunque di creare ed usare un thread**.



È superfluo ribadire l'utilità dell'impiego del multithreading:



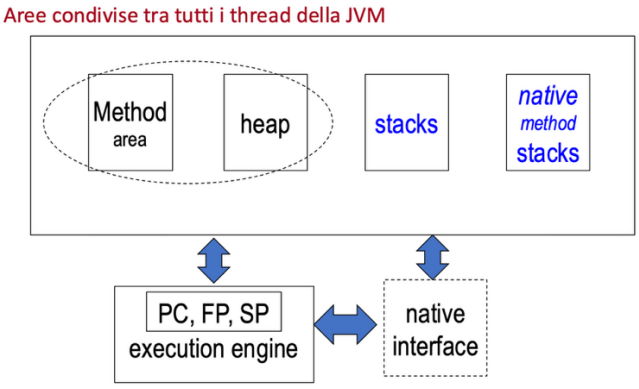
Il costruttore principale della classe `Thread` è il seguente, sebbene non sia l'unica:

```
Thread(ThreadGroup group, Runnable target, String name)
```

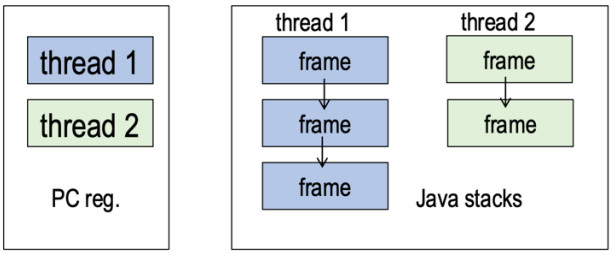
Con:

- `group`, il gruppo a cui appartiene il thread;
- `target`, un oggetto di tipo `Runnable`;
- `name`, il nome del thread.

Threads diversi all'interno della stessa applicazione condividono la maggior parte dello stato; in particolare, **sono condivisi l'ambiente delle classi e l'area heap ma ogni thread ha un proprio stack delle attivazion**i, mentre per quanto riguarda le variabili, **sono condivise le variabili statiche** (classi) **e di istanza** (heap) ma **non quelle locali dei metodi** (stack).



Method area e heap sono condivisi da tutti i thread in esecuzione nella JVM ma quando **un thread viene creato ottiene un proprio PC e una propria stack**, ognuna delle quali composti da frame, a loro volta coincidenti con l'invocazione di un metodo:

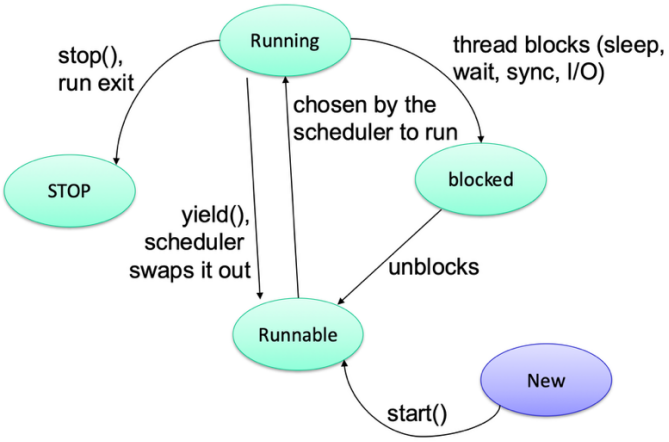


Il cambio contesto tra threads di un programma Java viene effettuato dalla JVM e richiede, tipicamente, l'esecuzione di **meno di 100 istruzioni**. Di seguito, si approfondiscono alcuni metodi della classe `Thread`:

- `start()`, ha come obiettivo quello di allocare un thread nella JVM, determinando a sua volta l'invocazione di `run()`;
- `run()`, definisce il comportamento del thread;
- `sleep(long millisec)`, sospende l'esecuzione del thread per il periodo di tempo specificato;
- `yield()`, causa la sospensione del thread a favore di un altro.

Si consideri che i thread in Java non hanno alcuna funzione di `exit`, verranno deallocati direttamente solo alla fine del metodo `run()`.

Gli **stati di un thread** in Java sono i seguenti:



Stato	Blocked	Interruptible	*Description*
Running			Il thread è *running* sul processore.
Runnable (ready-to-run)			Il thread è in attesa del proprio *turno* per il processore.
Sleeping	SI	SI	Il thread diventerà ready-to-run allo scadere del tempo di *sleep* o a seguito di una interruzione.
Waiting	SI	SI	Il thread diventerà ready-to-run allo scadere di un timeout, dopo una *notify*, o a seguito di una interruzione.
Blocked I/O	SI		Il thread diventerà ready-to-run a seguito di una variazione della condizione di I/O attesa.
Blocked synch	SI		Il thread diventerà ready-to-run quando è acquisito il lock su uno statement synchronized.

Nei sistemi multiprocessore un solo thread alla volta può essere eseguito, ognuno di questi ha una priorità e Java ne mette a disposizione 10, compresi fra `MIN_PRIORITY` (1) e `MAX_PRIORITY` (10), con `NORM_PRIORITY` (5) valore intermedio. Un thread eredita la priorità del thread che lo crea me è possibile cambiarla con il metodo `setPriority(int)`. La specifica Java prevede l'algoritmo "fixed priority scheduling", basato sulle priorità dei thread: **in generale, thread a priorità più alta ottengono più tempo di processore rispetto a quelli a priorità più bassa**.

In ogni momento, se c'è più di un `Runnable` thread in attesa, il sistema sceglie quello a priorità più alta e, se ce ne sono **più con la stessa priorità**, **la scelta viene fatta secondo il criterio First-Come-First-Serve**. Il thread scelto può continuare l'esecuzione finché un thread a più alta priorità diviene `Runnable`, il thread invoca `yield()` o il metodo `run()` termina, il periodo di CPU è terminato (in un sistema operativo time slicing) o passa dallo stato "Running" a "Blocked" (per via di `sleep()`, `sync` o `wait()`, tutte condizioni di I/O).

La JVM utilizza una libreria nativa (POSIX o Win32) o una propria libreria ("green" threads) per fornire l'infrastruttura su cui girano i threads; **la maggior parte delle implementazioni JVM, tuttavia, utilizza i thread del sistema operativo sottostante** e le **priorità** costituiscono una sorta di **suggerimento per lo scheduler** e non devono servire a garantire la correttezza di un programma. La specifica Java per lo scheduling è molto lasca, al fine di ottenere codice portabile non si dovrebbero mai fare troppe assunzioni sul sistema di scheduling sottostante; **una good practice prevede la scrittura di programmi che funzionino bene a prescindere dal livello di priorità dei propri threads** (o addirittura non utilizzarli affatto).

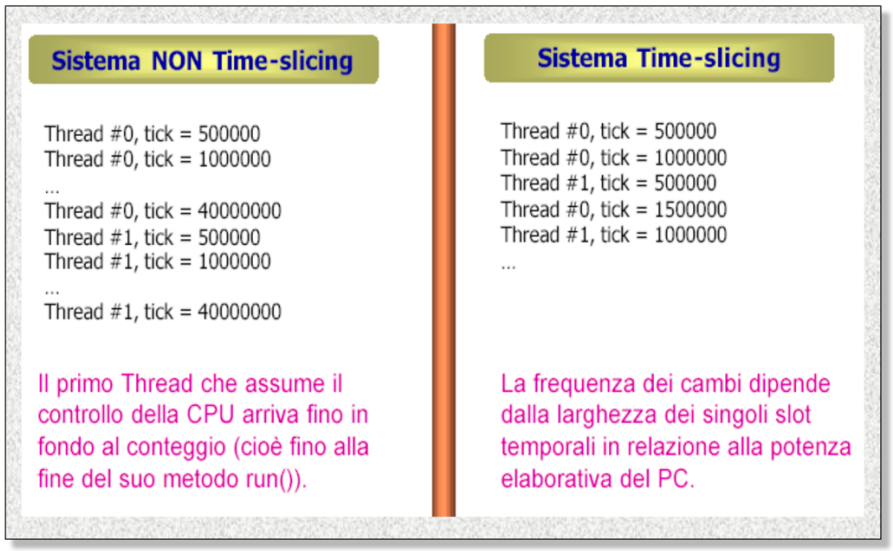
Java Priority	Windows Priority (Java 5)	Windows Priority (Java 6)
1	THREAD_PRIORITY_LOWEST	
2		
3	THREAD_PRIORITY_BELOW_NORMAL	
4		
5	THREAD_PRIORITY_NORMAL	THREAD_PRIORITY_NORMAL
6	THREAD_PRIORITY_ABOVE_NORMAL	
7		THREAD_PRIORITY_ABOVE_NORMAL
8	THREAD_PRIORITY_HIGHEST	
9		THREAD_PRIORITY_HIGHEST
10	THREAD_PRIORITY_TIME_CRITICAL	

Si consideri un esempio di metodo `run` di un thread:

```
public class SelfishRunner extends Thread {
    public int tick = 1;
    public void run() {
        while (tick < 4000000) {
```

```
                tick++;
            }
        }
}
```

Una volta in esecuzione, **tale thread continua fino alla terminazione del ciclo o fino all'arrivo di un altro thread a priorità maggiore**; ciò significa che thread con la stessa priorità di quello scritto potrebbero aspettare a lungo. **Alcuni sistemi limitano i selfish threads con il time slicing**: l'esecuzione di più thread avviene in alternanza solo per uno specifico quanto di tempo (slice); tuttavia, questa pratica si applica quando più thread con identica priorità hanno diritto ad essere eseguiti e non ci sono altri threads a più elevata priorità. **Si consideri che la specifica Java non impone il time slicing**, non si può far affidamento su tale tecnica perché i risultati sarebbero differenti da architettura ad architettura.



Il programmatore ha a disposizione i metodi della classe `Thread` per forzare i threads alla collaborazione, come `yield()`, che permette al sistema di cedere la CPU ad un altro thread eseguibile con la stessa priorità. **Una good practice prevede di assumere, nel caso peggiore, che il sistema sia non time slicing**: se l'elaborazione fosse fortemente CPU-intensive si potrebbe occasionalmente lasciare il controllo (`yield`) o effettuare una `sleep`.

Un'interruzione indica che un thread deve interrompere ciò che sta facendo e fare altro; è compito del programmatore indicare cosa deve fare un thread a seguito di una notifica di interruzione, che avviene invocando il metodo `interrupt()` sull'oggetto `Thread` da interrompere. Alcuni metodi che un thread può invocare durante la sua esecuzione sollevano un'eccezione di tipo `InterruptedException` quando il metodo `interrupt()` viene invocato durante la loro esecuzione; in tal caso, è sufficiente gestire opportunamente tale eccezione.

```
try {

   … metodo che può sollevare Interrupted Exception …

} catch (InterruptedException ex) {

   codice gestione eccezione
}
```

Se, invece, il thread non invoca alcun metodo che potrebbe sollevare eccezioni di tipo `InterruptedException`, allora può periodicamente invocare il metodo `Thread.interrupted()` che ritorna true se sul thread è stato invocato il metodo `interrupt()`. In alcune circostanze ha anche senso sollevare l'eccezione se il metodo restituisce true.

```
for (int i = 0; i < inputs.length; i++) {
    heavyCrunch(inputs[i]);
    if( Thread.interrupted() ) {
            //thread interrotto…
            return;
        }
}
```

Si noti che `Thread.interrupted()` verifica se il thread corrente è stato interrotto, resettando il flag "interrupted status"; invece, il metodo `isInterrupted()` verifica se il thread è stato interrotto ma non modifica il relativo flag. La classe `Thread` fornisce altri metodi per l'interruzione di un thread:

- `suspend()`, sospende un thread;
- `resume()`, riattiva un thread precedentemente sospeso;
- `stop()`, ferma e uccide un thread.

Tuttavia, **questi metodi sono ad oggi deprecati e si consiglia di non utilizzarli**; infatti, il loro uso è rischioso e comporta notevoli complicazioni: un thread può essere interrotto prima di poter rilasciare una risorsa, impedendo così agli altri di potervi accedere e generando un deadlock difficilmente risolvibile. Per questa ragione, si assume che un thread si interrompe e muore solo a conclusione del suo metodo `run()`.

Un metodo della classe `Thread` comunemente adoperato dai programmatori Java è **`sleep()`, che pone in attesa** (o stato dormiente) **un thread per il numero di millisecondi specificati dall'utente**; quando un thread padre genera vari thread figli potrebbe essere necessario attendere la loro conclusione prima di procedere e, per tale scopo, è possibile utilizzare il metodo `join()`.

```
try {
    Thread.sleep(4000);
} catch (InterruptedException ex) {
    System.out.println(ex);
}

Thread[] threads = new Thread[numOfThreads];

for (int i = 0; i < threads.length; i++) {

        threads[i] = new MyThread();
        threads[i].start();

}

for (int i = 0; i < threads.length; i++) {
        try {

           threads[i].join();

        } catch (InterruptedException ignore) {}

}
```

MECCANISMI DI SINCRONIZZAZIONE IN JAVA

Java fornisce il meccanismo di sincronizzazione basato su mutex per l'accesso a sezioni critiche; ogni oggetto ha associato un mutex ma questo non è direttamente acceduto dall'applicazione, bensì tramite l'uso di metodi e blocchi sincronizzati. Quando un thread esegue un metodo/blocco che è stato dichiarato sincronizzato entra in possesso del mutex associato all'istanza, il thread che blocca il mutex (o mutex lock) acquisisce l'accesso esclusivo alla sezione critica ed eventuali thread che vogliono accedere alla sezione critica saranno posti in uno stato di attesa.

Un metodo sincronizzato è definito tale anteponendo la keyword `synchronized` alla firma del metodo:

```
synchronized void Method() {}
```

E l'accesso ad esso è effettuato solo quando il lock all'oggetto associato è stato acquisito.

```
class SharedCounter{

      private int theData;

      public SharedCounter(int initialValue){
            theData=initialValue;
      }

      public synchronized int read(){
            return theData;
      }
}
```

Acquisisce il **lock** sull'oggetto.

Rilascia il **lock** sull'oggetto

I metodi non sincronizzati non richiedono il lock e possono essere eseguiti in ogni istante senza garanzie di mutua esclusione, mentre i metodi sincronizzati garantiscono l'accesso in mutua esclusione ai dati incapsulati in un oggetto solo se si accede ai dati tramite metodi dichiarati syncrhonized. **Anche i metodi statici possono essere dichiarati sincronizzati**, dal momento in cui non sono legati a nessuna istanza ne viene acquisito il mutex associato alla classe; all'invocazione di due metodi statici sincronizzati di una stessa classe da due threads diversi essi verranno eseguiti in sequenza ma se si invocano un metodo statico e uno di istanza, entrambi sincronizzati, di una stessa classe essi verranno eseguiti in concorrenza. Nonostante ciò, **ottenere il lock della classe non influenza i lock di qualsiasi istanza della classe**.

Utilizzando questo approccio, **il programmatore non ha la responsabilità del rilascio del mutex ogni volta che un metodo termina normalmente o per una eccezione**, il tutto viene eseguito automaticamente. Java, inoltre, offre la possibilità di definire sezioni critiche anche quando queste non coincidono con il corpo di un metodo, attraverso i cosiddetti blocchi sincronizzati:

```
public int read(){
      synchronized(this) //oggetto corrente
      {
            return theData;
      }
}
```

La keyword `synchronized` prende come parametro il riferimento ad un oggetto del quale si deve ottenere il lock per continuare.

Tuttavia, **l'utilizzo di blocchi sincronizzati presenta determinate avvertenze**; in primis, c'è una maggiore espressività nell'implementare vincoli di sincronizzazione di un programma ma **l'eccessivo uso di blocchi sincronizzati può rendere il codice disordinato**: i vincoli di sincronizzazione non sono più incapsulati in un singolo posto (come la definizione di un metodo) e non è possibile comprendere i vincoli di sincronizzazione associati ad un oggetto "O" semplicemente guardando "O" (è necessario osservare tutti gli oggetti che accedono ad "O" in un blocco sincronizzato).

Se una classe non ha metodi sincronizzati ma si desidera evitare l'accesso contemporaneo a uno o più metodi, è possibile acquisire il mutex di una determinata istanza racchiudendo le invocazioni dei metodi da sincronizzare in un blocco sincronizzato; questa pratica prende il nome di **sincronizzazione implicita**.

```
private Object obj=new Object();
synchronized ( obj ) {
      comando 1;
      ...
      comando n;
}
```

Come già annunciato, **`syncrhonized` indica che il thread deve acquisire il lock sull'oggetto**; **una classe con metodi synchronized è un monitor in Java**, che ha una sola (ed implicita) variable condition: il thread attivo nel monitor può sospendere la propria esecuzione invocando la primitiva `wait()`, la quale ha l'effetto di rilasciare il monitor ed inserire il thread nel wait set, nel quale rimarrà finché non verrà invocata una `notify()` da un altro thread che è attivo nel monitor.

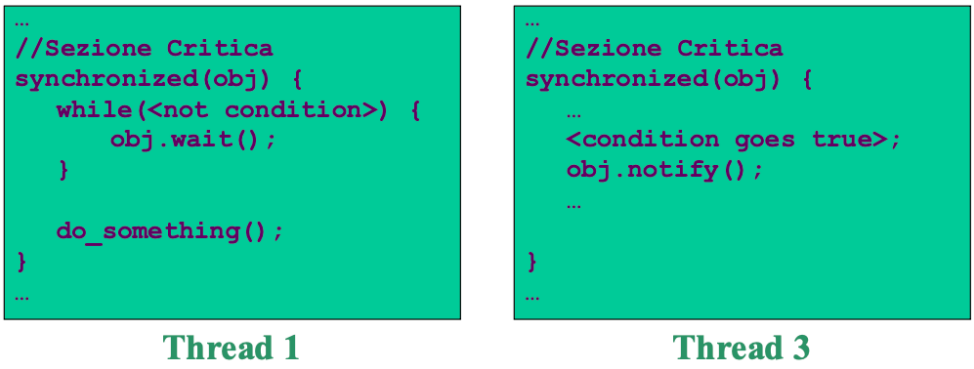
Ad esempio, una ipotetica gestione di un deposito:

```
public synchronized void deposita (){

        while ( disponibilita == 0 ){
                try{
                        System.out.println ( "Deposita: nessuno spazio!" );
                        wait();
                }catch( InterruptedException e ){
                        e.printStackTrace();
                }
        }

        disponibilita=disponibilita-1;
        System.out.println ( "Deposita disp= " + disponibilita );

        notifyAll();

}
```

```
public synchronized void preleva () {

        while ( disponibilita == dimensione ){
                try{
                        System.out.println ( "Preleva: nessun oggetto!" );
                        wait();
                }catch( InterruptedException e ){
                        e.printStackTrace();
                }
        }

        disponibilita=disponibilita+1;
        System.out.println ( "Preleva disp= " + disponibilita );

        notifyAll();

}
```

La soluzione adottata è quella "**signal and continue**" monitor perché, differentemente dalla soluzione di Hoare, **il thread che invoca la `notify()` rimane in possesso del monitor e continua la propria esecuzione all'interno del monitor region**. Al risveglio, un thread non può assumere che la sua condizione sia vera, dal momento in cui altri threads potrebbero essersi risvegliati indipendentemente dalla condizione sulla quale erano in attesa. In definitiva, Java non offre garanzie che un thread risvegliato dopo una wait possa immediatamente acquisire il lock ed è essenziale che un thread verifichi nuovamente la sua condizione appena risvegliato.

Considerando la seguente struttura:



L'esecuzione del thread 1 procede solo se la condizione è verificata, altrimenti il thread si pone in attesa che l'eventuale thread 3 abiliti la condizione e sblocchi con la `notify()`. **Le primitive di cooperazione in questione vengono ereditate dalla classe Object, possono essere invocate solo all'interno di sezioni critiche ma non risolvono il problema di mutua esclusione**. Esse sono:

- Wait:
  - `public final void wait()`, il thread che invoca questo metodo entra nel wait set del monitor, rilasciando il mutex associato all'istanza, e rimane sospeso finché non viene svegliato da un altro thread che invoca una `notify` o una `notifyAll`, oppure viene interrotto con il metodo `interrupt` della classe Thread;
  - `public final void wait(long millis)`, si comporta analogamente al precedente ma se dopo un'attesa corrispondente al numero di millisecondi specificato in `millis` non è stato risvegliato, esso si risveglia;
  - `public final void wait(long millis, int nanos)`, si comporta analogamente al precedente ma permette di specificare l'attesa con una risoluzione temporale al livello di nanosecondi;
- Notify:

  - `public final void notify()`, risveglia un thread nel wait set del monitor ma, poiché il metodo che lo invoca deve aver acquisito il mutex, il thread risvegliato deve attenderne il rilascio per competere per la sua acquisizione come un qualsiasi altro thread;
  - `public final void notifyAll()`, risveglia tutti i threads nel wait set del monitor, i quali competono tutti per l'acquisizione del mutex, se ne esiste uno a priorità più alta esso viene subito eseguito.

I monitor built-in Java sono semplice da utilizzare ma hanno diverse limitazioni, tra cui un'unica wait-queue e l'impossibilità di tentare semplicemente di acquisire un lock. Tecnicamente, però, **è possibile realizzare semafori e/o monitor a partire dai costrutti primitivi Java**. A partire da Java 1.5, sono stati introdotti ulteriori costrutti per la gestione della sincronizzazione; in particolare, tre packages:

- `java.util.concurrent`, fornisce le classi per supportare paradigmi comuni di programmazione concorrente;
- `java.util.concurrent.atomic`, fornisce un supporto per una programmazione lock-free e thread-safe su variabili semplici come atomic integer, atomic boolean, …;
- `java.util.concurrent.locks`, fornisce un framework per diversi algoritmi di locking (come, ad esempio, read-write locks e condition variables).

I vantaggi di questa nuova soluzione sono il riuso (c'è un minor sforzo di programmazione), maggiori prestazioni e affidabilità, package standard e testati e debugging più semplice.

La classe `Semaphore` gestisce un insieme di permessi:

- Ogni `acquire()` eseguito sul semaforo blocca il thread corrente se sul semaforo non c'è almeno un permesso disponibile da acquisire;
- Viceversa, una `release()` aggiunge un permesso al semaforo, potenzialmente sbloccando un thread bloccato in fase di acquisizione;
- `boolean tryAcquire()` acquisisce un permesso sul semaforo, solo se il permesso è disponibile al momento dell'invocazione.

I semafori vengono generalmente utilizzati per limitare il numero di thread che accedono ad una determinata risorsa condivisa; si noti che, sebbene il semaforo incapsuli la sincronizzazione necessaria a restringere l'accesso alla risorsa, la consistenza della risorsa va comunque gestita a parte. **Un semaforo inizializzato ad "1" ed usato in modo da non avere più di "1" permesso disponibile**, **funge da lock per la mutua esclusione** (ed è detto semaforo binario). A differenza di molte implementazioni di Lock (presenti nel package `java.util.concurrent.locks`), il lock effettuato col semaforo binario può essere rilasciato da un qualsiasi altro thread. Il costruttore della classe `Semaphore` ha come parametro il numero di permessi iniziale e un parametro opzionale boolean per imporre l'ordinamento FIFO (`true`) o non garantire alcun ordinamento (`false` o non specificato):

```
Semaphore(int permits)
Semaphore(int permits, boolean fair)
```

```
class Pool {
     private static final MAX_AVAIL = 100;
     private Semaphore available = new Semaphore(MAX_AVAIL, true);

     public Object getItem() throws InterruptedException {
           available.acquire();
           return getNextAvailableItem();
     }

     public void putItem(Object x) {
            if (markAsUnused(x)) available.release();
     }

     protected Object[] items = ...
     protected boolean[] used = new boolean[MAX_AVAIL];

     protected synchronized Object getNextAvailableItem()
       { . . .}

     protected synchronized boolean markAsUnused(Object item)
       { . . . }
}
```

`Lock` è un'interfaccia del package `java.util.concurrent`, **le cui implementazioni offrono un uso più flessibile rispetto ai lock associati ai blocchi o metodi `syncrhonized`**, permettendo l'accesso esclusivo ad una risorsa.

```
Lock l = ...;
  l.lock();
  try {
      // accesso alla risorsa protetta da questo lock
  } finally {
      l.unlock();   //garantire il rilascio della risorsa
  }
```

Le implementazioni di `Lock` offrono funzionalità addizionali oltre a quelle fornite dall'uso dello statement `syncrhonized`:

- Tentativi non bloccanti di acquisizione del lock (`tryLock()`);
- Un tentativo di acquisire il lock che può essere interrotto da timeout (`tryLocking(long, TimeUnit)`);
- Un tentativo di acquisizione del lock che può essere interrotto (`lockInterruptibly()`), con la possibilità di inerrompere il thread in attesa invocando il metodo `interrupt()`;

La classe `ReentrantLock` è un Lock con lo stesso comportamento e semantica dei lock associati ai metodi e blocchi `synchronized`.

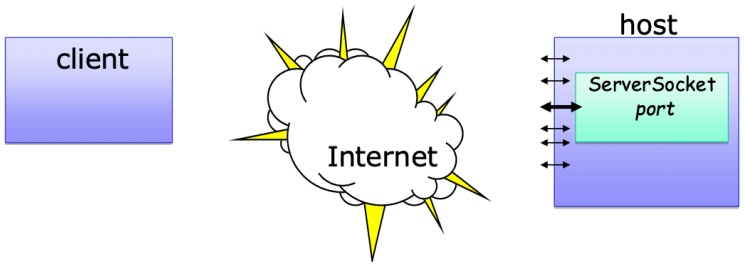
I Lock vengono adoperati per la gestione dell'accesso in mutua esclusione ma offrono meccanismi anche per la cooperazione tra thread attraverso implementazioni dell'interfaccia `Condition`, la quale fornisce un mezzo per un thread di sospendere l'esecuzione fino a quando è notificato da un altro thread che una certa condizione di stato è ora vera; gli oggetti di tipo `Condition`, poi, consentono di implementare le condition variables e sono intrinsecamente legati ad un Lock, richiedendo l'utilizzo del metodo `newCondition()` per essere ottenuti.

```
class BoundedBuffer {
   final Lock lock = new ReentrantLock();
   final Condition notFull  = lock.newCondition();
   final Condition notEmpty = lock.newCondition();

   final Object[] items = new Object[100];
   int putptr, takeptr, count;

   public void put(Object x) throws InterruptedException {
     lock.lock();
     try {
       while (count == items.length)
         notFull.await();
       items[putptr] = x;
       if (++putptr == items.length) putptr = 0;
       ++count;
       notEmpty.signal();
     } finally {
       lock.unlock();
     }
   }
. . .

. . .

   public Object take() throws InterruptedException {
     lock.lock();
     try {
       while (count == 0)
         notEmpty.await();
       Object x = items[takeptr];
       if (++takeptr == items.length) takeptr = 0;
       --count;
       notFull.signal();
       return x;
     } finally {
       lock.unlock();
     }
   }
 }
```

JAVA NETWORKING

Per **comunicare su rete tra applicazioni distribuite**, Java mette a disposizione le seguenti classi: **`Socket`** per la creazione di una connessione di rete, e **`ServerSocket`**, utilizzata da un server, per accettare una richiesta di connessione. **Il processo server che gira sul nodo X si pone in attesa di richieste di connessioni sulla porta Y**; in primis, è creata dal server un'istanza della classe `ServerSocket`, per poi porsi in attesa di richieste di connessione:

```
ServerSocket server = new ServerSocket(port);
Socket client = server.accept();
```



La classe `ServerSocket` ha due costruttori:

- `ServerSocket(int port) throws IOException;`
- `ServerSocket(int port, int backlog) throws IOException.`

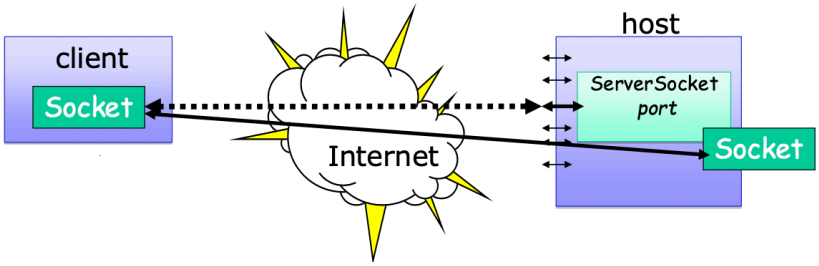
Il parametro `port` indica il port number sull'host locale e può assumere valori interi tra 1 e 65535, anche se quelli tra 1 e 1023 sono riservati, mentre il parametro `backlog` indica il numero massimo di richeste di connessione che possono essere accodate dal sistema operativo (di default sono 50).

Un'istanza della classe `ServerSocket` possiede il seguente metodo:

```
Socket accept() throws IOException
```

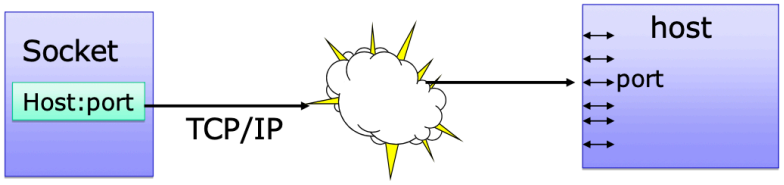
Che comporta l'attesa sul port number del server di una richiesta di connessione (**listening**); alla ricezione di una connessione, l'oggetto `ServerSocket` crea una socket che possa rappresentare la connessione TCP con il client e restituisce al chiamante il puntatore a tale socket. Dal lato client, è possibile creare una `Socket` tramite l'apposito costruttore e può richiedere una connessione:

```
Socket socket = new Socket(address, PORTNUM);
```

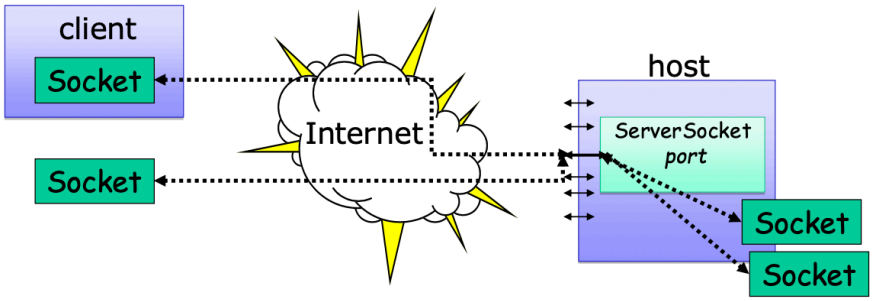


A quel punto, il server esce dal metodo `accept()`, che gli restituisce la socket per la comunicazione con il client. **La socket non fa altro che fornire un'interfaccia per i client TCP**; il tutto può essere svolto utilizzando l'apposito costruttore:

```
Socket conn= new Socket(nomeHost, numport)
```



La classe `ServerSocket`, come detto in precedenza, incapsula le strutture dati per la gestione delle connessioni TCP/IP server-side e, invocando nuovamente il metodo `accept()`, può essere messo in attesa di una nuova connessione client.



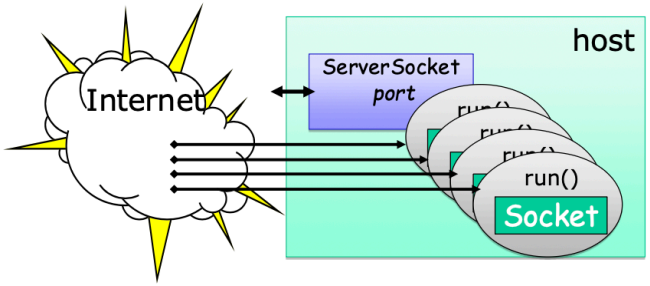
Su una connessione, invocando i metodi `getInputStream()` e `getOutputStream()` di socket, è possibile instaurare il canale di comunicazione tra client e server:

```
InputStream in = conn.getInputStream();
OutputStream out = conn.getOutputStream();
```

In alternativa, è possibile usare anche le classi Reader/Writer:

```
writer = new OutputStreamWriter(out);
reader = new InputStreamReader(in);
```

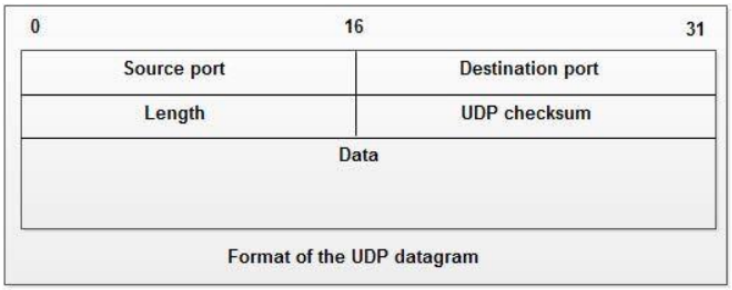
Il multithreading sul server permette di migliorare l'efficienza creando uno o più thread per connessione; in particolare, ci si trova un thread worker che preveda un costruttore che accetti come parametro una socket e che, nel proprio metodo `run()` contiene il codice per il soddisfacimento della richiesta.




```
// Schema generale WORKER

   public class MyWorker extends Thread{
     private Socket socket;
     MyWorker (Socket skt){socket=skt; … }
     public void run(){
       InputStream in=socket.getInputStream();
       OutputStream ot= socket.getOutputStream();
       //elabora richiesta
       …
     }
   }
```

```
// Schema generale Server
   …
   while (…) {
     Socket client= server.accept();
     MyWorker myWorker=new MyWorker(client);
     myWorker.start();
   }
   …
```

Nel package `java.net` sono fornite le classi per la comunicazione basata su UDP; in particolare, `DatagramPacket` (astrazione del pacchetto UDP, il datagramma) e `DatagramSocket` (astrazione del socket UDP). La struttura dati incapsulata in un'istanza di `DatagramPacket` consiste di due parti, address e data:



Format of the UDP datagram

La classe `DatagramSocket` è utilizzata per inviare e ricevere pacchetti, considerando che UDP è connectionless si può utilizzare la stessa socket per inviare pacchetti a destinazioni diverse:

```
DatagramSocket() throws SocketException
DatagramSocket(int port) throws SocketException
```

La trasmissione e la ricezione avvengono, invece, mediante i seguenti metodi:

```
void send(DatagramPacket packet) throws IOException
void receive(DatagramPacket packet) throws IOException
```

Ricezione di un datagramma

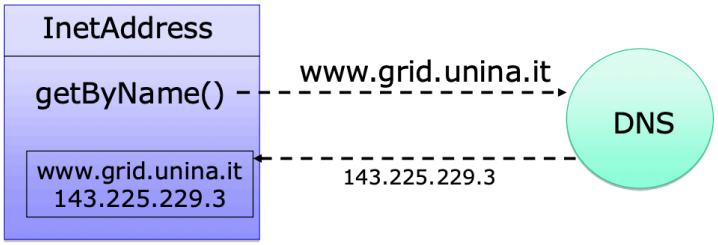
```
DatagramSocket socket= new DatagramSocket(port);
byte buffer[]= new byte[65508];
//65508 e' la max dimensione di un pacchetto UDP
DatagramPacket pkt=new DatagramPacket(buffer, buffer.length);
//Se forniamo una dimensione del buffer troppo piccola
//la dimensione eccedente del pacchetto andrà persa
socket.receive(pkt); //chiamata bloccante
InetAddress fromAddr=pkt.getAddress();
int fromPort= pkt.getPort();
int length=pkt.getLength();
byte[] data= pkt.getData();
```

Invio di un datagramma

```
DatagramSocket socket= new DatagramSocket();
..
//Si costruisce il pacchetto UDP da inviare
DatagramPacket pkt=new DatagramPacket(data, data.length,
   InetAddress.getByName("pippo.pluto.it"), portNum);

//Si invia il pacchetto
socket.send(pkt);
…
```

`buffer` è un array di byte pre-allocato che conterrà il contenuto del pacchetto all'arrivo, mentre `length` è il massimo numero di byte che sono letti nel buffer (dati extra nel pacchetto UDP saranno scartati).

La classe `InetAddress` permette di astrarre l'indirizzo IP tramite i servizi DNS:



Alcuni suoi metodi di utilità sono:

- InetAddress getLocalHost `() throws UnknownHostException`, restituisce l'indirizzo IP della macchina locale;
- `InetAddress getByName(String host) throws UnknownHostException`, restituisce l'indirizzo IP associato ad un dato host name;
- `InetAddress[] getAllByName(String host) throws UnknownHostException`, restituisce un array di indirizzi IP per un dato hostname;
- `byte[] getAddress()`, restituisce l'indirizzo, contenuto nell'oggetto, come sequenza di byte;
- `String getHostName()`, restituisce il nome della macchina su cui sta girando il programma;
- `boolean isMulticastAddress()`, restituisce `true` se l'oggetto incapsula un indirizzo multicast.

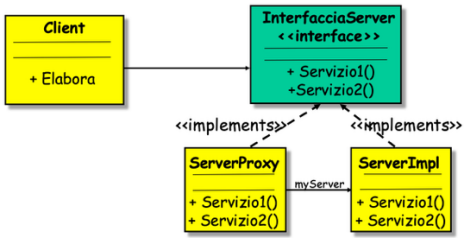
E le eccezioni:

- `UnknownHostException`, l'host non può essere correttamente identificato;
- `SecurityException`, viene sollevato se il `SecurityManager` non permette l'esecuzione di una specifica operazione.

REALIZZAZIONE DI OGGETTI REMOTI E PROXY-SKELETON

In un'applicazione con comunicazione basata su socket, client e server prevedono **meccanismi di connessione, esternalizzazione dei dati, invio-ricezione delle richieste e ricostruzione dei valori ricevuti**; tutto ciò comporta dei potenziali rischi: in primis, l'implementazione dei meccanismi di comunicazione **"distrae" dalla realizzazione delle funzionalità effettive dell'applicazione**, ma poi la sovrapposizione della logica applicativa con quella di comunicazione client-server non è mai una buona pratica. **Il programmatore lato client dovrebbe concentrarsi sulla logica applicativa** (invocando servizi del server semplificati da un'interfaccia) e separare la logica applicativa da tali meccanismi di comunicazione/interazione con il server; analogamente, **il programmatore lato server dovrebbe concentrarsi sulla codifica dei servizi offerti**, separandola dai meccanismi di comunicazione/interazione con il client.

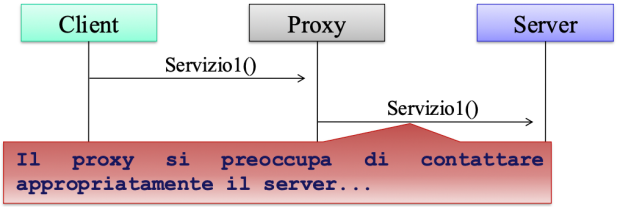
Il **pattern Proxy** introduce nel modello di interazione client-server un Proxy, o surrogato, che si presenta come un'implementazione del servizio remoto locale al client; **i meccanismi di basso livello necessari per l'instaurazione della comunicazione e per l'invio-ricezione dei parametri delle invocazioni tramite TCP o UDP sono implementati e incapsulati all'interno del Proxy**.



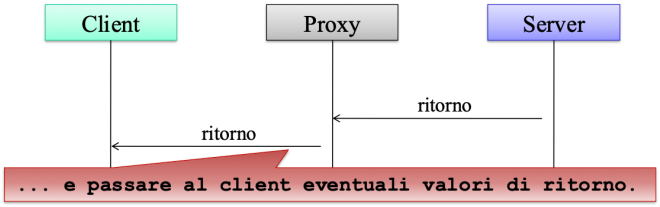
Il servizio fornito è specificato da `InterfacciaServer`, che ha una duplice implementazione:

- `ServerImpl`, implementa effettivamente i servizi chiamati dall'interfaccia;
- `ServerProxy`, si fa carico dell'inoltro delle richieste effettuate dal client verso il lato server.

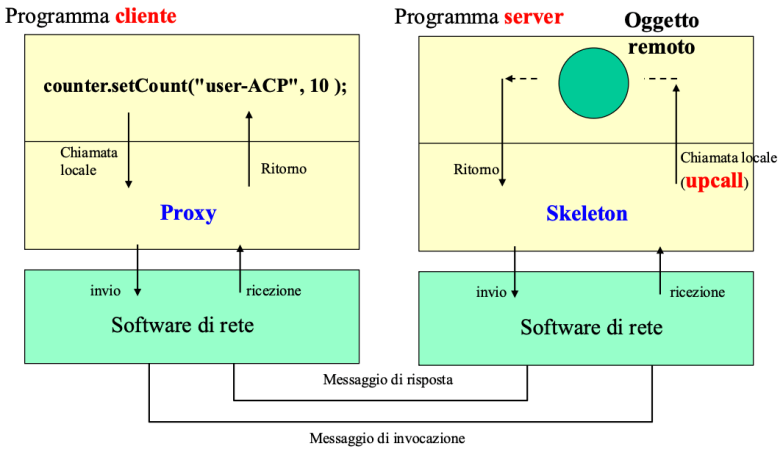
L'associazione direzionale tra `ServerProxy` e `ServerImpl` sintetizza tutti i meccanismi necessari al proxy per riferirsi al server. Il client possiede un riferimento ad un oggetto di tipo `InterfacciaServver` che, in realtà, è un oggetto di tipo `ServerProxy`, il quale viene usato in sostituzione al servizio reale e si fa carico dell'interazione con il lato server.



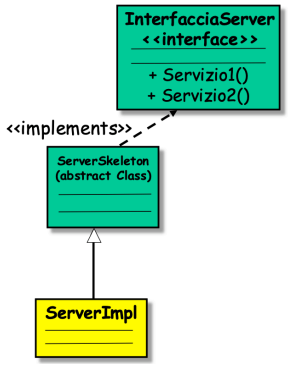
Analogamente per il ritorno dei dati:



Con l'aggiunta del proxy, **il lato client è sollevato dalle problematiche di comunicazione**; in maniera analoga, un proxy lato server, detto **Skeleton**, **si fa carico della comunicazione con il proxy lato client**. Lo Skeleton riceve le richieste di servizio, struttura l'informazione fornita in ingresso tramite socket TCP o UDP, effettua l'up-call al servizio reale, riceve da questi eventuali risultati e invia una risposta al proxy lato client.

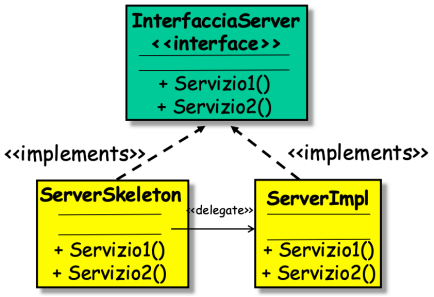


Lo skeleton può essere implementato per ereditarietà: la classe astratta `Skeleton` implementa solo gli opportuni schemi di comunicazione ma lascia senza implementazione i metodi dell'interfaccia, mentre `ServerImpl` è una sottoclasse dello skeleton e fornisce implementazione ai metodi astratti:



Nonostante ciò, **lo skeleton può essere implementato anche per delega**, facendo il modo che la classe `Skeleton` presenti al proprio interno un riferimento ad `InterfacciaServer` (`delegate`) e realizzando così i metodi:

```
void Servizio1() {
    delegate.Servizio1();
}
```



Laddove sono previste eventuali politiche di sincronizzazione atte a disciplinare l'accesso ai servizi remoti, esse vanno implementate nella classe `ServerImpl`, ossia nell'implementazione reale dei servizi dichiarati nell'interfaccia, piuttosto che nei relativi Proxy).

Si noti che **il payload di un messaggio UDP è costituito da un array di byte**, richiedendo di **adattare il nome del metodo** (ed eventuali parametri dell'invocazione) **al pacchetto di richiesta**

UDP; una possibile soluzione può consistere nel **creare una stringa concatenando nome del metodo e singoli parametri e delimitarli con un carattere separatore** (come #). Ad esempio:

```
nomeMetodo#argomento1#argomento2#…#argomentoN
```

La stringa è, quindi, inserita nel pacchetto di richiesta UDP tramite `getBytes()`:

```
DatagramPacket request = new DatagramPacket(
    message.getBytes(),
    message.getBytes().length,
    InetAddress.getLocalHost(),
    9000
);
```

Il ricevente estrae la stringa dal pacchetto UDP ricevuto, individua nome e singoli parametri tramite la classe `StringTokenizer` e ne invoca il metodo `nextToken()` e, quando necessario, converte i parametri ricevuti da `String` al tipo richiesto tramite il metodo `valueOf()` della classe Wrapper associata al tipo richiesto; nel caso di un intero:

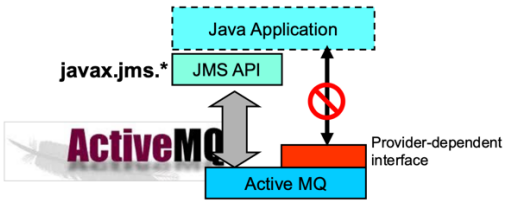
```
int x = Integer.valueOf(messageTokens.nextToken()).intValue();
```

La classe `StringTokenizer` scompone una stringa in pezzi, token, delimitati dal carattere separatore scelto (come #); invocazioni successive del metodo `nextToken()` restituiscono i singoli token. I metodi di utilità delle varie classi Wrapper, poi, si occupano di fare il casting del token verso l'opportuno tipo necessario.

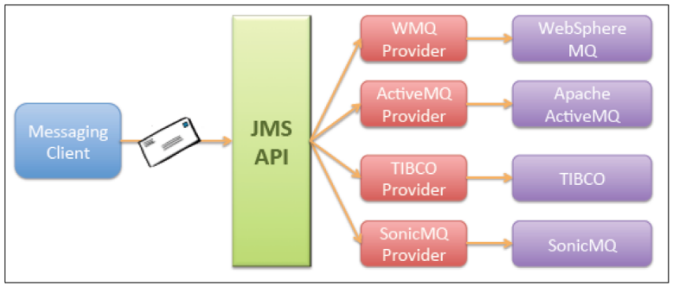
JAVA MESSAGE SERVICE

A partire dalla specifica Java Message Service (JMS) JSR 814, si definisce JMS come un insieme di interfacce, con relativa semantica associata, che definisce le modalità con cui un JMS client accede ai servizi di un enterprise messaging product. In altre parole, **JMS definisce uno standard che consente ai programmi Java di inviare e ricevere messaggi tramite un prodotto MOM**; i programmi scritti con JMS potranno essere eseguiti con un qualsiasi MOM che implementi le interfacce JMS (in altre parole, i prodotti MOM che intendono fornire le funzionalità JMS devono implementare le interfacce definite dallo standard JMS).

Per poter realizzare programmi con JMS non è sufficiente avere la specifica e le API scaricate ma è necessario installare anche un provider che implementi lo standard, come Apache ActiveMQ.



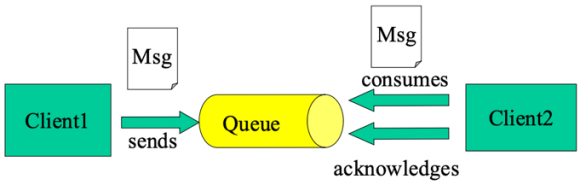
La specifica JMS consente il **disaccoppiamento tra i client** (sender e receiver di messaggi) **e la specifica implementazione definita del provider**:



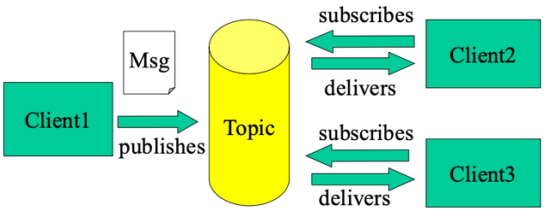
I domini di messaging di JMS sono due:

- **Point to Point** (PTP), ruota intorno al concetto di coda di messaggi e ognuno di essi ha un solo consumer;
- **Publish-Subscribe**, ogni messaggio è associato ad un topic, utilizzato per la ricezione dei messaggi, i quali possono avere più consumer.

Per quanto riguarda il **messaging PTP**, una coda conserva un determinato messaggio finché esso non scade o non viene consumato, mentre il receiver conferma (ack) la corretta ricezione del messaggio:



Per quanto riguarda il **messaging PUB/SUB**, i topic conservano un messaggio finché esso non viene rilasciato ai subscriber correnti, con un messaggio che può essere processato da `0..*` subscriber.



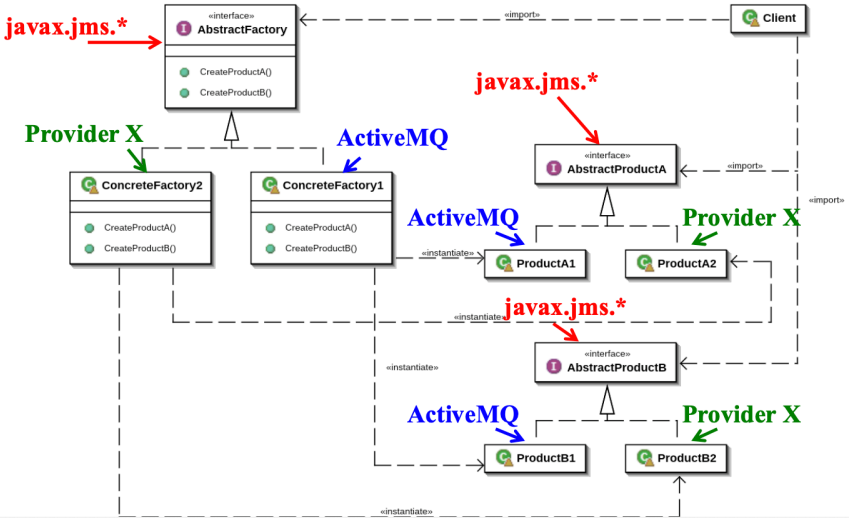
JMS definisce un insieme di interfacce di alto livello che consentono l'accesso ai domini di messaging PT e PUB/SUB; in particolare, `ConnectionFactory` è necessaria per la creazione di una `Connection` (`QueueConnectionFactory` o `TopicConnectionFactory`), mentre `Destination` incapsula la destinazione, ovvero il "dove" i messaggi saranno inviati o "da cui" i messaggi saranno ricevuti (`Coda` o `Topic`). Le istanze di `ConnectionFactory` e `Destination` prendono il nome di administered objects. Altre interfacce sono:

- `Connection`, connessione verso il provider JMS;
- `Session`, contesto single-threaded per inviare o ricevere messaggi;
- `MessageProducer`, utilizzato per inviare messaggi;
- `MessageConsumer`, utilizzato per ricevere messaggi.

A seconda dello specifico dominio di messaging, le interfacce di alto livello finora menzionate saranno specializzate dalle seguenti classi:

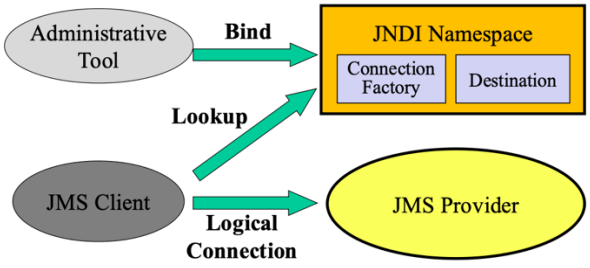
High-level Interface	PTP Domain	Pub/Sub Domain
ConnectionFactory	QueueConnectionFactory	TopicConnectionFactory
Connection	QueueConnection	TopicConnection
Destination	Queue	Topic
Session	QueueSession	TopicSession
MessageProducer	QueueSender	TopicPublisher
MessageConsumer	QueueReceiver, QueueBrowser	TopicSubscriber

Il pattern **Abstract Factory** è utilizzato quando un sistema deve essere indipendente da come i suoi prodotti sono creati o realizzati, quando un sistema deve poter essere configurato con una di tante famiglie di prodotti o quando si vuole fornire una libreria di prodotti e rivelarne l'interfaccia ma non l'implementazione.



Le entità di un sistema JMS sono:

- Client, programmi Java che inviano/ricevono messaggi;
- Provider, realizzazione concreta del sistema di messaging, implementa la specifica JMS, supporta lo scambio messaggi e fornisce funzionalità per la gestione/supervisione del sistema di messaging;
- Messaggi, astrazioni delle informazioni scambiate tra entità che usano JMS;
- Administered Objects, istanze di `ConnectionFactory` e `Destination`:
  - Oggetti JMS pre-configurati che colmano il gap tra le interfacce JMS e la specifica tecnologia del provider;
  - Il provider JMS fornisce i programmi per creare gli administered objects e registrarli in JNDI (Java Naming and Directory Interface).

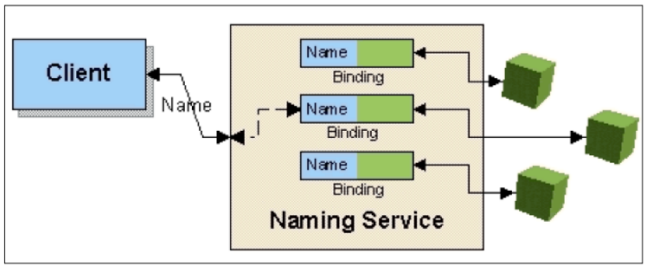


Gli **administered objects** sono recuperati dai client JMS tramite JNDI, attraverso l'operazione di lookup, e ne esistono di due tipi:

- `ConnectionFactory`, creazione di una connection per il messaging;
- `Destination`, dove i messaggi sono inviati o da cui i messaggi sono ricevuti.

Sebbene questi siano **istanze di classi che incapsulano dettagli dipendenti dal provider, essi sono recuperati tramite un meccanismo standard** (JNDI) **e acceduti tramite interfacce standard JMS**. Il programma JMS deve conoscere il nome JNDI e l'interfaccia JMS degli administered objects ma non i dettagli dipendenti dal provider.

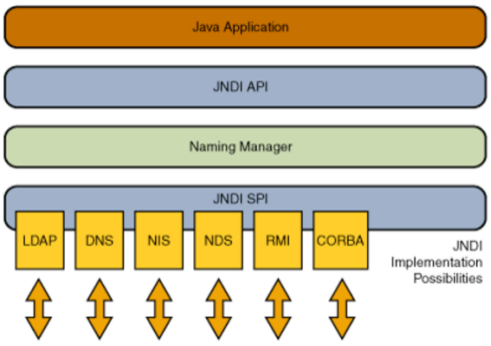
Un naming service mantiene un insieme di binding tra nomi e riferimenti ad oggetti (o direttamente ad oggetti), mentre JNDI è un'interfaccia che supporta le funzionalità comuni di un naming service:



Si definiscono, a tal proposito:

- Name, nome dato all'oggetto registrato;
- Binding, associazione nome-oggetto;
- Reference, puntatore ad un oggetto;
- Context, insieme di associazioni nome-oggetto.

JNDI è indipendente dallo specifico servizio di naming, questo è acceduto tramite plugin chiamati Service Provider:



Un client individua il provider di un servizio JNDI configurando le seguenti proprietà:

- `java.naming.factory.initial`;
- `java.naming.provider.url`.

Si utilizza `Context`, un'interfaccia che specifica i metodi per aggiungere, cancellare e cercare oggetti tramite un servizio JNDI, e rappresenta un insieme di binding, in modo che tutte le operazioni JNDI sono svolte in relazione ad esso. `InitialContext` è un'implementazione di `Context`.

I principali metodi di `Context` sono:

- `void bind(String stringName, Object object)`, il nome non deve essere già associato ad alcun oggetto;
- `void rebind(String stringName, Object object);`
- `Object lookup(String stringName);`
- `void unbind(String stringName);`
- `void rename(String stringName, String newName).`

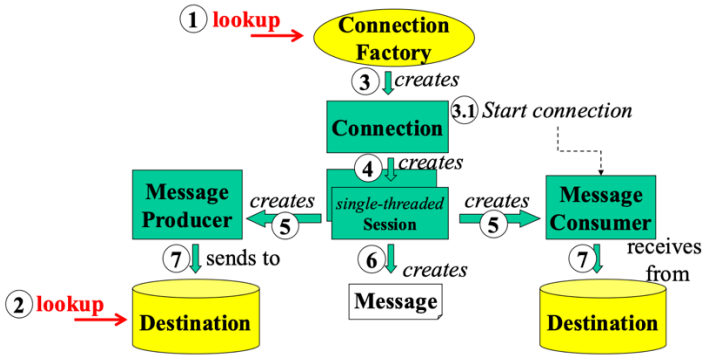
Lo sviluppo di un client JMS prevede i seguenti passaggi:

- Lookup di una `ConnectionFactory` da JNDI;
- Lookup di una `Destination` da JNDI;
- Utilizzo della `ConnectionFactory` per la creazione di una `Connection`:
  - Avviare la `Connection` se si vuole abilitare la ricezione dei messaggi;
- Utilizzo della `Connection` per la creazione di una o più `Session`;
- Utilizzo della `Session` e della `Destination` per la creazione dei `MessageProducer` e `MessageConsumer`;
- Utilizzo di una `Session` per la creazione di un `Message`;
- Invio e ricezione messaggi;
- Cleanup delle risorse.

Prima dell'avvio di un client JMS è necessario avviare il provider, che consente l'attivazione del servizio di naming da cui il client potrà recuperare `ConnectionFactory` e `Destination`.

Il modello di programmazione generico JMS prevede:

- `Connection`, rappresenta una connessione tra il client JMS e il provider;
- `Destination`, incapsula l'identità della destinazione di un messaggio (il "dove" il messaggio sarà inviato);
- `Session`, un contesto single-threaded per l'invio e la ricezione di un messaggio (non sono pensate per l'utilizzo concorrente da parte di più thread);
- `MessageProducer` e `MessageConsumer`, inviano e ricevono i messaggi nel contesto di una `Session`.



Per effettuare il lookup degli administered objects è necessario configurare il client per accedere al servizio JNDI di ActiveMQ:

```
import java.util.Hashtable;
import javax.jms.*;
import javax.naming.*;

Hashtable<String, String> prop = new Hashtable<String, String> ();

prop.put( "java.naming.factory.initial",
            "org.apache.activemq.jndi.ActiveMQInitialContextFactory" );
prop.put( "java.naming.provider.url", "tcp://127.0.0.1:61616" );

//     jndi-queue-name,  physical-queue-name
prop.put( "queue.test", "mytestqueue" );

try{
       Context jndiContext = new InitialContext(prop);
       // omesso //
       // slide successiva …
```

Tramite le operazioni di lookup è possibile, quindi, ottenere gli administered objects:

```
// 1. Lookup administered objects, ConnectionFactory e Destination per queue
QueueConnectionFactory queueConnFactory =
(QueueConnectionFactory)jndiContext.lookup("QueueConnectionFactory");

// 2. Lookup administered objects, Destination per queue
Queue queue = (Queue)jndiContext.lookup("test"); // la mia Destination è Queue
//il prefisso "queue." non fa parte del nome jndi

// 3. Creo la Connection
QueueConnection queueConn = queueConnFactory.createQueueConnection();

// 4. Creo la Session (single-threaded)
QueueSession queueSession = queueConn.createQueueSession(false,Session.AUTO_ACKNOWLEDGE);
```

Per la creazione di un sender e l'invio di messaggi (nel dominio PTP):

```
// 5. Creo il sender (Message Produce)
QueueSender sender = queueSession.createSender(queue);

// 6. Creo il messaggio che userò per l'invio
TextMessage message = queueSession.createTextMessage();

for ( int i =0; i<5; i++){
       message.setText("hello_" + i);

       // 7. Invio il messaggio (send asincrona)
       sender.send( message );
}

message.setText("fine");
sender.send( message );

System.out.println ("I messaggi sono stati inviati!");

// 8. clean up risorse
sender.close();
queueSession.close();
queueConn.close();
```

Il receiver è identico per le fasi di lookup e creazione connessione/sessione, tranne che per l'avvio di una connessione:

```
// 3. Creo la Connection
QueueConnection queueConn = queueConnFactory.createQueueConnection();

// 3.1. Avvio la connection
queueConn.start();  //NOTE: abilita il delivery dei messaggi!

// 4. Creo la Session (single-threaded)
QueueSession queueSession = queueConn.createQueueSession(false,
Session.AUTO_ACKNOWLEDGE);

// next slide ...
```

```
// 5. Creo il receiver (Message Consumer)
QueueReceiver receiver = queueSession.createReceiver(queue);

// 6. Creo il messaggio che userò per la ricezione
TextMessage message;
do{
       System.out.println ("In attesa di messaggi!");
       // 7. Ricezione (bloccante)
       message = (TextMessage)receiver.receive();
       System.out.println ("      + messaggio ricevuto: " +
       message.getText());
}while (message.getText().compareTo("fine") != 0);

// 8. clean up risorse
receiver.close();
queueSession.close();
queueConn.close();
```

Nel dominio PUB/SUB, il publisher:

```
// 1. Lookup administered objects, ConnectionFactory per TOPIC
TopicConnectionFactory connFactory = (TopicConnectionFactory)
jndiContext.lookup("TopicConnectionFactory");

// 2. Lookup administered objects, Destination per TOPIC
Topic topic = (Topic)jndiContext.lookup("test");

// 3. Creo la Connection
TopicConnection topicConn = connFactory.createTopicConnection();

// 4. Creo la Session (single-threaded)
TopicSession topicSession = topicConn.createTopicSession(false, Session.AUTO_ACKNOWLEDGE);

// 5. Creo il publisher (Message Producer)
TopicPublisher pub = topicSession.createPublisher(topic);

// 6. Creo il messaggio che userò per la pubblicazione
TextMessage text = topicSession.createTextMessage();

for ( int i =0; i<5; i++){
         text.setText("hello_" + i);
         // 7. Pubblico il messaggio
         pub.publish( text );
}

text.setText("fine");
pub.publish( text );

System.out.println ("I messaggi sono stati inviati!");

// 8. clean up risorse
pub.close();
topicSession.close();
topicConn.close();
```

Il subscriber:

```
// 1. Lookup administered objects, ConnectionFactory per TOPIC
TopicConnectionFactory connFactory = (TopicConnectionFactory)
jndiContext.lookup("TopicConnectionFactory");

// 2. Lookup administered objects, Destination per TOPIC
Topic topic = (Topic)jndiContext.lookup("test");

// 3. Creo la Connection
TopicConnection topicConn = connFactory.createTopicConnection();

// 3.1 Avvio la Connection (sono un Message Consumer)
topicConn.start();

// 4. Creo la Session (single-threaded)
TopicSession topicSession = topicConn.createTopicSession(false, Session.AUTO_ACKNOWLEDGE);

// 5. Creo il subscriber (Message Consumer)
TopicSubscriber sub = topicSession.createSubscriber(topic);

// 6. Creo il messaggio che userò per la sottoscrizione
TextMessage msg;

do{
         System.out.println ("In attesa di messaggi!");
         // 7. Sottoscrivo (ricevo) il messaggio
         msg = (TextMessage)sub.receive();
         System.out.println ("       + messaggio ricevuto: " + msg.getText());
}while (msg.getText().compareTo("fine") != 0);

// 8. clean up risorse
sub.close();
topicSession.close();
topicConn.close();
```

Per l'avvio di applicazioni JMS, va avviato ActiveMQ:

```
cd /<ACTIVEMQ_PATH>/bin
./activemq start
```

E vanno avviati subscriber (receiver) e publisher (sender):

```
cd /<PROJECT_PATH>/bin
java          -cp          /<ACTIVEMQ_PATH>/activemq-all-X.Y.Z.jar:
package_name.class_name
```

Si noti che `<ACTIVEMQ_PATH>` è il percorso dove è stato estratto il contenuto del file compresso con ActiveMQ e che `X.Y.Z` indica la versione utilizzata. Gli step di avvio dei client vanno ripetuti per ogni entità da avviare (cioè, un terminale per ogni subscriber/receiver e uno per ogni publisher/sender); infine, il file `activemq-all-X.Y.Z.jar` va incluso anche nel classpath del progetto nel text editor come referenced library.

I messaggi JMS possono essere consumati in due modalità:

- Sincrona, un receiver o un subscriber prelevano esplicitamente un messaggio dalla destinazione utilizzando il metodo `receive`, il quale si blocca o finché arriva un messaggio o finché non scade un timeout;
- Asincrona, il client JMS registra un `MessageListener` su un consumer e ogni volta che un messaggio arriva a destinazione il provider JMS invoca il suo metodo `onMessage`.

I messaggi JMS sono ricevuti in modalità sincrona utilizzando uno dei seguenti metodi:

- `receive()`, si blocca finché non c'è un messaggio disponibile;
- `receive(long timeout)`, riceve un messaggio che arriva prima dello scadere del `timeout` specificato;
- `receiveNoWait()`, riceve il messaggio se immediatamente disponibile (è equivalente alla precedente con un `timeout` molto piccolo).

Per la receive asincrona, il listener è un'istanza di una classe che implementa l'interfaccia JMS `MessageListener`, che contiene solo un metodo (`onMessage`), nel quale va scritto il codice che elabora il messaggio al suo arrivo:

```
public class TextMsgListener implements MessageListener {
	public void onMessage ( Message m ){
		try{
			System.out.println ("	+ messaggio ricevuto: " +
				((TextMessage)m).getText());
		}catch ( JMSException e ){
			e.printStackTrace();
		}
	}
}
```

Il client JMS istanzia il listener e lo registra sul receiver:

```
QueueReceiver receiver = queueSession.createReceiver(queue);

TextMsgListener  msglistener = new TextMsgListener ();

receiver.setMessageListener( msglistener );
```

Questo tipo di ricezione è supportata sia dal dominio PTP che dal dominio PUB/SUB. Tutti i messaggi sono strutturati nel seguente modo:

I campi header consentono l'invio e l'identificazione dei messaggi e vi si accede con i metodi `getJMS*` e `setJMS*`; ad esempio:

- `JMSMessageID`, identificativo univoco del messaggio;
- `JMSCorrelationID`, identificativo che collega un messaggio ad un altro (collegare una richiesta ad una risposta, ad esempio);
- `JMSReplyTo`, contiene una `Destination`, definita dal client, verso cui deve essere inviato un eventuale messaggio di risposta;
- `JMSPriority`, contiene la priorità del messaggio (da 0 a 9).

La sezione properties può contenere proprietà specifiche dell'applicazione, proprietà standard JMS o proprietà del vendor JMS e sono coppie `<String, value>`, dove `value` è un tipo built-in Java o una `String`. Una property si gestisce con i metodi `get*Property` e `set*Property` (sebbene per un messaggio ricevuto sia in modalità read-only) e supportano una sintassi SQL-like per selezionare i messaggi. Il metodo `getPropertyNames()` restituisce l'elenco di tutti i nomi delle proprietà.

La sezione body rappresenta il contenuto vero e proprio del messaggio, gestito con i metodi previsti dalla specifica tipologia di body:

Message Type	Contains	Some Methods
TextMessage	String	getText, setText
MapMessage	Set of name/value pairs	setString, setDouble, setLong, getDouble, getString
BytesMessage	Stream of uninterpreted bytes	writeBytes, readBytes
StreamMessage	Stream of primitive values	writeString, writeDouble, writeLong, readString
ObjectMessage	Serializable object	setObject, getObject

Il corretto consumo di un messaggio avviene in tre fasi:

1. Il client riceve il messaggio;
2. Il client processa il messaggio;
3. Il messaggio è acknowledged.

L'acknowledgement (o ack) **può essere effettuato dal client o dal provider JMS, a seconda di come è configurata la sessione**; infatti, la modalità di ack è specificata dal secondo parametro della `createSession`:

```
QueueSession qs = queueConn.createQueueSession(false,
Session.AUTO_ACKNOWLEDGE);
```

- `AUTO_ACKNOWLEDGE`, la sessione conferma la corretta ricezione del messaggio quando il client ritorna correttamente da una receive (o quando il `MessageListener` termina la propria esecuzione);
    - Nel caso di receive sincrona, la ricezione e l'acknowledgement avvengono in una singola fase, seguita poi dal processing;
- `CLIENT_ACKNOWLEDGE`, il cient conferma la ricezione del messaggio invocando il metodo `acknowledge` sul messaggio, la conferma avviene a livello di sessione e l'ack di un messaggio consumato conferma automaticamente tutti i messaggi consumati nella sessione;
- `DUPS_OK_ACKNOWLEDGE`, una forma di acknowledge lasco che, in caso di malfunzionamento del provider JMS, potrebbe generare messaggi duplicati;
    - Ad esempio, la sessione invia un messaggio di acknowledgement ogni N messaggi o ad intervalli di tempo prefissati.

È possibile raggruppare una serie di operazioni JMS (send/receive) **in transazioni**. Quando la sessione è **transacted**, il secondo parametro è ignorato, **una transazione ha significato nel contesto di una singola sessione** ed essa termina quando l'applicazione invoca il metodo `commit()` sulla sessione (se tutto è andato bene) o con un'operazione di `rollback()` se si è verificata qualche anomalia:

```
QueueSession qs = queueConn.createQueueSession(false,0);

try {
    session.commit();
} catch (Exception e) {
    Session.rollback();
}
```

Una transazione è `commit()` quando tutti i messaggi inviati nell'ambito della transazione diventano disponibili per la consegna ad altre applicazioni o quando tutti i messaggi ricevuti nell'ambito della transazione vengono acknowledged in modo che JMS non tenti di consegnarli nuovamente all'applicazione (in una comunicazione PTP, JMS rimuove anche i messaggi ricevuti dalle loro code). Invece, **una transazione è `rollback()` quando tutti i messaggi inviati all'interno della transazione vengono scartati da JMS o quando tutti i messaggi ricevuti all'interno della transazione diventano nuovamente disponibili per l'invio** (in una comunicazione PTP, i messaggi ricevuti vengono rimessi in coda e tornano visibili alle altre applicazioni).

La specifica JMS prevede che sia gestito l'accesso concorrente su oggetti di tipo `Destination`, `ConnectionFactory` e `Connection`; gli oggetti `Session`, `MessageProducer` e `MessageConsumer`, invece, non sono progettati per essere utilizzati in un contesto multithreaded, le sessioni supportano le transazioni, per le quali è complesso implementare un supporto multithread, e **la ricezione asincrona multithread non è supportata**.

JMS consente di specificare **due modalità di delivery**, che indicano se i messaggi debbano essere persi o no a causa di un malfunzionamento del provider JMS; la **modalità `PERSISTENT`** (default) **assicura che i messaggi non siano persi**, ogni messaggio è registrato su memoria stabile, mentre nella **modalità `NON_PERSISTENT`** il provider **non garantisce la memorizzazione del messaggio in caso di malfunzionamenti**. La proprietà di persistenza può essere specificata in due modi:

- Impostando la modalità con il metodo setDeliveryMethod:

```
producer.setDeliveryMethod(DeliveryMode.NON_PERSISTENT);
```

- Specificando la modalità durante l'invio del messaggio:

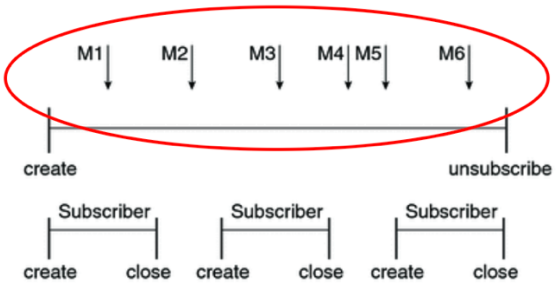
```
producer.send(msg, DeliveryMode.NON_PERSISTENT, 3, 1000);
```

Di default, **un subscriber riceve i messaggi solo se pubblicati quando esso è attivo**, si dice che il metodo `createSubscriber` crea un subscriber non duraturo (non-durable), mentre è il metodo `createDurableSubscriber` che disaccoppia i concetti di sottoscrizione e subscriber; nello specifico, è registrata una sottoscrizione durable sul provider, la quale è specificata da un nome univoco (coppia client ID + nome della sottoscrizione). Se una sottoscrizione non ha subscriber attivi, i messaggi sono conservati dal provider JMS.

```
[… codice omesso …]

TopicConnection topicConn = connFactory.createTopicConnection();

topicConn.setClientID("subdurableID");

topicConn.start();
TopicSession topicSession = topicConn.createTopicSession(false,
Session.AUTO_ACKNOWLEDGE);

TopicSubscriber sub =
topicSession.createDurableSubscriber(topic,"MakeItLastConn");

[… codice omesso …]
```

La coppia (ID, nome_subscription) identifica univocamente la sottoscrizione

Nel caso di una durable subscription, il subscriber può attivarsi e disattivarsi senza perdere messaggi.



Nel caso di una comunicazione tra entità JMS e STOMP bisogna tenere conto dei diversi campi dell'header utilizzati in JMS e in STOMP; quest'ultimo supporta le seguenti proprietà JMS per i messaggi di tipo `SENT`:

STOMP Header	JMS Header	Description
correlation-id	JMSCorrelationID	Good consumers will add this header to any responses they send.
expires	JMSExpiration	Expiration time of the message.
JMSXGroupID	JMSXGroupID	Specifies the Message Groups.
JMSXGroupSeq	JMSXGroupSeq	Optional header that specifies the sequence number in the Message Groups.
persistent	JMSDeliveryMode	Whether or not the message is persistent.
priority	JMSPriority	Priority on the message.
reply-to	JMSReplyTo	Destination you should send replies to.
type	JMSType	Type of the message.

È possibile aggiungere header custom ai comandi STOMP per configurare il protocollo ActiveMQ; ad esempio, è possibile definire durable subscriber in STOMP: gli header custom `client-id` (comando `CONNECT`) e `activemq.subscriptionName` (comando `SUBSCRIBE`) devono essere usati in combinazione (in maniera similare ai durable subscriber JMS). In particolare:

- `client-id` viene impostato con l'hostname se non esplicitamente impostato;
- la coppia `(client-id, activemq.subscriptionName)` identifica univocamente la sottoscrizione.

```
conn.connect(wait=True, headers = {"client-id":"IDtestsub_durable"})
...
conn.subscribe(destination='/topic/mytesttopic', id=1 ack='auto',
      persistent=True,
      headers = {"activemq.subscriptionName":"IDtestsubscription"})
```

Altre estensioni di ActiveMQ per STOMP sono le seguenti:

Verb	Header	Type	Description
CONNECT	client-id	string	Specifies the JMS clientID which is used in combination with the activemq.subcriptionName to denote a durable subscriber.
SUBSCRIBE	activemq.dispatchAsync	boolean	Should messages be dispatched synchronously or asynchronously from the producer thread for non-durable topics in the broker? For fast consumers set this to false. For slow consumers set it to true so that dispatching will not block fast consumers.
SUBSCRIBE	activemq.exclusive	boolean	I would like to be an Exclusive Consumer on the queue.
SUBSCRIBE	activemq.maximumPendingMessageLimit	int	For Slow Consumer Handling on non-durable topics by dropping old messages - we can set a maximum-pending limit, such that once a slow consumer backs up to this high water mark we begin to discard old messages.
SUBSCRIBE	activemq.noLocal	boolean	Specifies whether or not locally sent messages should be ignored for subscriptions. Set to true to filter out locally sent messages.
SUBSCRIBE	activemq.prefetchSize	int	Specifies the maximum number of pending messages that will be dispatched to the client. Once this maximum is reached no more messages are dispatched until the client acknowledges a message. Set to a low value > **1** for fair distribution of messages across consumers when processing messages can be slow. **Note**: if your STOMP client is implemented using a dynamic scripting language like Ruby, say, then this parameter ***must*** be set to 1 as there is no notion of a client-side message size to be sized. STOMP does not support a value of 0.

Verb	Header	Type	Description
SUBSCRIBE	activemq.priority	byte	Sets the priority of the consumer so that dispatching can be weighted in priority order.
SUBSCRIBE	activemq.retroactive	boolean	For non-durable topics make this subscription retroactive.
SUBSCRIBE	activemq.subscriptionName	string	For durable topic subscriptions you must specify the same activemq.client-id on the connection and activemq.subcriptionName on the subscribe prior to v5.7.0. **Note**: the spelling subcriptionName NOT subscriptionName. This is not intuitive, but it is how it is implemented in ActiveMQ Classic 4.x. For the 5.0 release of ActiveMQ Classic, both subcriptionName and subscriptionName will be supported (subcriptionName was removed as of v5.6.0).
SUBSCRIBE	selector	string	Specifies a JMS Selector using SQL 92 syntax as specified in the JMS 1.1 specification. This allows a filter to be applied to each message as part of the subscription.

STOMP è un protocollo molto semplice e non riconosce i messaggi JMS, come `TextMessage` o `BytesMessage`, ma **supporta un header denominato `content-lenght`**; **ActiveMQ utilizza questo header per determinare quale tipo di messaggio creare quando si invia da STOMP a JMS**: se è incluso o meno si invia un `BytesMessage` o un `TextMessage`. La stessa logica può essere seguita anche nel passaggio da JMS a STOMP.

```
# sender.py
import stomp

conn = stomp.Connection([('127.0.0.1', 61613)], auto_content_length=False)
conn.connect(wait=True)

conn.send('/topic/mytesttopic', 'test message')

conn.disconnect()
```

L'inclusione/esclusione dell'header `auto_content_length` specifica il tipo di messaggio da creare quando si invia un messaggio da STOMP a JMS

Inclusion of content-length header	Resulting Message
yes	BytesMessage
no	TextMessage

E' necessario utilizzare il `physical-name` del topic/queue specificato lato JMS

```
//      jndi topic name    physical topic-name
jndiProperties.put( key:"topic.test", value:"mytesttopic" );
```

REMOTE PROCEDURE CALL IN JAVA

grpc.io è un middleware RPC universale ad alte prestazioni e open source che può essere eseguito in qualsiasi ambiente, è un **framework multilinguaggio e multipiattaforma**. I principali suoi scenari di utilizzo sono il collegamento di microservizi poliglotti che utilizzano lo stile di comunicazione richiesta-risposta, la connessione di dispositivi mobile e browser ai servizi di backend e la generazione di librerie client efficienti. È sviluppato da Google e ora è un progetto Cloud Native Computing Foundation (CNCF), è utilizzato da molte aziende e in molti sistemi distribuiti.

Per la generazione delle classi GRPC con grpc.io va eseguita la compilazione con protoc specificando:

- Il percorso di output per la generazione delle calssi per la gestione di stub, canali e messaggi (`--java_out=` e `--grpc-java_out=`);
- Il percorso del plugin protoc da usare per la compilazione.

```
$ protoc --java_out=. --grpc-java_out=. --plugin=protoc-gen-grpc-java=/PATH/TO/protoc-gen-grpc-java-1.9.1-osx-x86_64 nome_file.proto
```

Per alcuni sistemi operativi potrebbero essere sollevate delle eccezioni di sicurezza; pertanto, è **opportuno fare in modo**, attraverso le impostazioni del sistema operativo in utilizzo, **che il plugin sia considerato sicuro**.

Per la creazione di un progetto Java che utilizzi GRPC bisognerà includere le seguenti librerie Java nel classpath, con l'accortezza di utilizzare versioni compatibili tra di loro:

- `failureaccess-1.0`;

- `grpc-api-1.73.0`;
- `grpc-core-1.73.0`;
- `grpc-netty-shaded-1.73.0`;
- `grpc-protobuf-1.73.0`;
- `grpc-protobuf-lite-1.73.0`;
- `grpc-stub-1.73.0`;
- `guava-33.4.8-jre`;
- `perfmark-api-0.26.0`;
- `protobuf-java-4.31.1`.

Dovendo includere più file `.jar`, è possibile usare `-cp /path/folder/jar/files/*` seguito dal separatore.

I concetti di gRPC come channel, stub e messaggi protobuf non differiscono rispetto a quanto detto in precedenza; tuttavia, **Java è più verboso nell'utilizzo di API grpc**, utilizzando costruttori ed eccezioni checked. Nonostante ciò, sia Java che Python supportano chiamate RPC bloccanti (sincrone) e non bloccanti (asincrone).

Per l'implementazione di applicazioni Java che utilizzano GRPC sono necessari:

- Protobuf:
  - È necessario introdurre la sola riga option `java_multiple_files = true;` dopo quella `syntax`, non è obbligatoria ma permette una migliore organizzazione delle calssi generate da protoc;
  - È opportuno impostare appropriatamente il campo `package`, in Java non è ignorato ma viene utilizzato per determinare il percorso di package dove verrà generato il codice, a meno che non venga specificata `option java_package`, e deve corrispondere allo stesso valore di package utilizzato per la generazione di applicazioni Python;
- Classe/i client-side:
  - Creare un channel;
  - Istanziare uno stub che utilizzi tale canale per le chiamate RPC;
  - Invocare le chiamate RPC e attendere le risposte;
- Classe/i server-side:
  - Istanziare un server;
  - Creare un server GRPC e fare il bind con il server;
  - Avviare il server e mettersi in attesa di servire richieste RPC;
  - Gestire le richieste e inviare le risposte;
- Classe/i di implementazione delle funzioni RPC.

Le API client-side e server-side di GRPC in Java sono:

Concetto	Java gRPC API	Note e comparazione con GRPC Python
Import modulo gRPC	import io.grpc.*	Java utilizza importazioni esplicite dal pacchetto io.grpc. Python ha un modulo grpc più semplice e unificato.
Classe Stub	MyServiceGrpc.MyServiceBlockingStub / MyServiceStub	Java genera più tipologie stub (bloccanti, asincroni, *future*). Python fornisce un singolo stub, più semplice ma meno esplicito.
Channel	Grpc.newChannelBuilder(target, InsecureChannelCredentials.create()).build()	Java usa ManagedChannel creato tramite il pattern *builder*. Python chiama semplicemente grpc.insecure_channel(...).
Costruire un messaggio di richiesta	MyRequest.newBuilder().setField(...).build()	Java usa il pattern builder immutabile per i messaggi protobuf. Python usa direttamente gli argomenti delle parole chiave.
Chiamata a un metodo (bloccante)	blockingStub.myMethod(request)	Stessa semantica di stub.MyMethod(request) di Python, ma Java richiede un tipo di stub esplicito.
Chiamata a un metodo (async)	asyncStub.myMethod(request, responseObserver)	Java usa StreamObserver (pattern observer) per callback asincrone. Python usa .future() o asyncio (più pulito per casi semplici).
Gestione della risposta	response.getField()	Java usa metodi getter; Python usa l'accesso diretto agli attributi (response.field).
Gestione degli errori	catch (StatusRuntimeException e)	Java usa eccezioni checked e try-catch strutturati. Python usa try/except grpc.RpcError.
Terminazione e cleanup del canale	channel.shutdownNow().awaitTermination(timeout, unit)	Java impone la chiusura e cleanup manuale del canale. Python: channel.close() è più semplice, ma meno configurabile.

Concetto	Java gRPC API	Note e comparazione con GRPC Python
Istanziare un server	Grpc.newServerBuilderForPort(port, credentials). Le credentials sono di tipo ServerCredentials che viene estesa da InsecureServerCredentials oppure TlsServerCredentials	Java utilizza un modello di costruttore; Python utilizza un modello di istanziazione diretta.
Aggiungere un servente	.addService(new MyServiceImpl())	Java usa una classe che estende *ImplBase, Python usa una funzione helper per registrare il servizio.
Avviare il server	.build().start()	Esplicito in entrambi. Java usa il concatenamento con il builder; Python è più imperativo.
Fermare il server	server.shutdown() oppure server.shutdownNow()	Entrambi supportano uno shutdown graceful; Java consente un controllo più preciso con lo shutdown asincrono.
Implementare un servizio	Estende MyServiceGrpc.MyServiceImplBase e fa override dei metodi	Entrambi seguono principi orientati agli oggetti per definire la logica del servizio.
Gestire risposta unary	Uso di responseObserver.onNext(response) e responseObserver.onCompleted()	Java utilizza callback tramite StreamObserver; Python utilizza lo statement return
Gestire risposta streaming	Uso di chiamete multiple a onNext(), seguite da onCompleted()	Java usa lo stile push con StreamObserver; Python usa i generator per lo streaming server-side
Threading / Execution	Un ExecutorService custom (e.g., utilizzando FixedThreadPool) passato attraverso il metodo .executor(...)	Entrambi consentono il controllo dell'uso dei thread. Java utilizza una configurazione che permette più controllo tramite la classe ExecutorService.
Gestione della terminazione	Registra hook di shutdown alla JVM (Runtime.getRuntime().addShutdownHook(...))	Java ha bisogno di hook specifici per arrestare il server; Python usa tipicamente gestori di segnali o gestori di contesto.
Gestione degli errori	Uso di responseObserver.onError(StatusException)	Entrambi espongono codici di stato gRPC ed eccezioni per propagare gli errori.

Ad esempio:

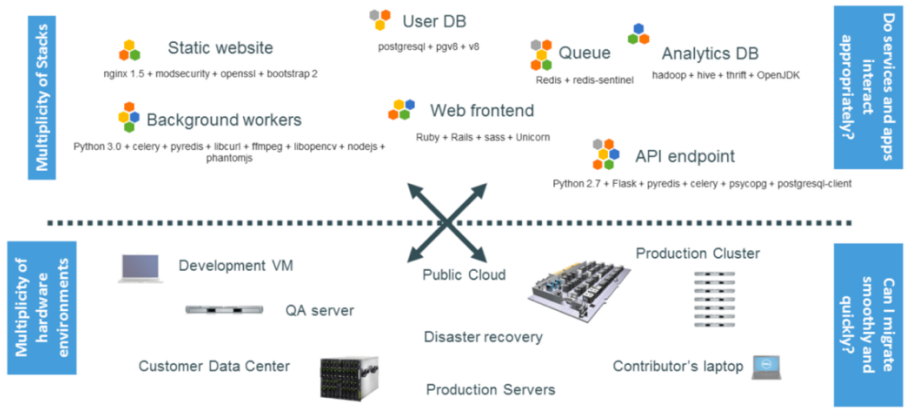
```
public class HelloWorldServer {
  ...
  /**
  * Main launches the server from the command line.
  */
  public static void main(String[] args) throws IOException, InterruptedException {
            final HelloWorldServer server = new HelloWorldServer();
            server.start(); //start routine will await for termination
  }

}
```

```
public class HelloWorldServer {

 private Server server;
 private void start() throws IOException {
  /* The port on which the server should run */
  int port = 0;
  ExecutorService executor = Executors.newFixedThreadPool(2);
  server = Grpc.newServerBuilderForPort(port, InsecureServerCredentials.create())
             .executor(executor)
             .addService(new GreeterImpl())
             .build()
             .start();
  System.out.println("Server started, listening on " + server.getPort());

  . . .

      try {
         server.awaitTermination(); //wait for termination
      } catch (InterruptedException e) {
      // HANDLE. . .
      }
 }
 . . .

public class HelloWorldClient {

 . . .
 /**
 * Greet server. If provided, the first element of {@code args} is the name to use in the
 * greeting. The second argument is the target server.
 */
 public static void main(String[] args) throws Exception {
  String user = "world";
  String target = "localhost:"+args[0];
  ManagedChannel channel = Grpc.newChannelBuilder(target, InsecureChannelCredentials.create()).build();
  try {
    HelloWorldClient client = new HelloWorldClient(channel);
    client.greet(user);
  } finally {
    channel.shutdownNow().awaitTermination(5, TimeUnit.SECONDS);
  }
 }
}
```

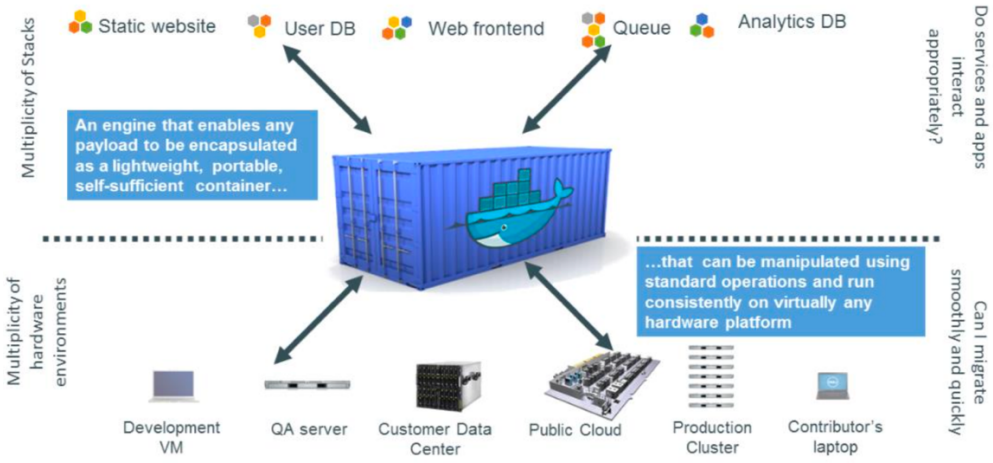
```
public class HelloWorldClient {

 private final GreeterGrpc.GreeterBlockingStub blockingStub;

 /** Construct client for accessing HelloWorld server using the existing channel. */
 public HelloWorldClient(Channel channel) {
  // 'channel' here is a Channel, not a ManagedChannel, so it is not this code's responsibility to
  // shut it down.

  // Passing Channels to code makes code easier to test and makes it easier to reuse Channels.
  blockingStub = GreeterGrpc.newBlockingStub(channel);
 }

 /** Say hello to server. */
 public void greet(String name) {
  System.out.println("Will try to greet " + name + " ...");
  HelloRequest request = HelloRequest.newBuilder().setName(name).build();
  HelloReply response;
  try {
    response = blockingStub.sayHello(request);
  } catch (StatusRuntimeException e) {
    System.out.println("RPC failed: {0} " + e.getStatus());
    return;
  }
  System.out.println("Greeting: " + response.getMessage());
 }
 . . .

 public class GreeterImpl extends GreeterGrpc.GreeterImplBase {

            @Override
            public void sayHello(HelloRequest req, StreamObserver<HelloReply> responseObserver) {
             System.out.println("[SERVER JAVA GRPC] Invoked sayHello");

             HelloReply reply = HelloReply.newBuilder().setMessage("Hello " + req.getName()).build();

             System.out.println("[SERVER JAVA GRPC] Set reply and send...");

             responseObserver.onNext(reply);
             responseObserver.onCompleted();
            }
 }
```

CONTAINER-BASED SERVICE DEPLOYMENT

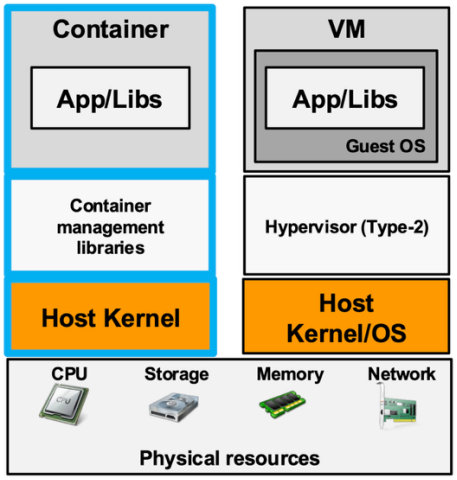
L'uso di macchine virtuali comporta un overhead di prestazioni dovuto alle indirezioni software usate (sistema operativo guest e hypervisor, macchine virtuali innestate, …), al grande ingombro di memoria e storage necessari, al tempo di avvio lento, ai costi di licenza e manutenzione del sistema operativo guest, … La domanda sorge spontanea: **è davvero necessario virtualizzare tutti i dispositivi hardware e un sistema operativo completo?**



In questo contesto intervengono le **soluzioni basate su container**:



Un container non è una macchina virtuale nel senso tradizionale del termine, non c'è emulazione di dispositivi fisici, ma **un tipo di virtualizzazione leggera**, **non c'è nessun overhead dovuto al livello hypervisor e si sfrutta l'astrazione dei processi e le capacità di gestione delle risorse del sistema operativo e del kernel host** (namespace e cgroups).



E **in termini di performance è una soluzione certamente più efficiente**:

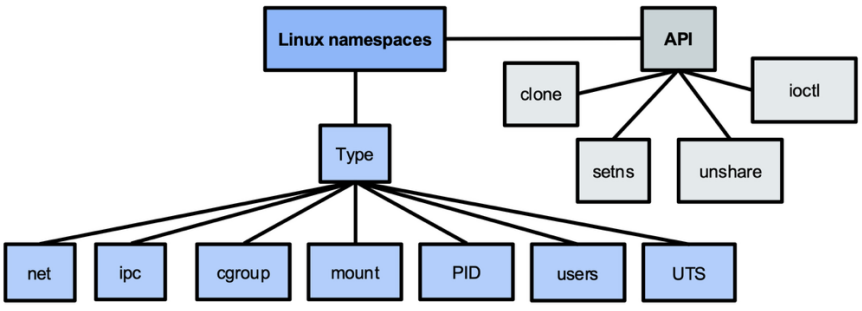
	BARE METAL	VIRTUAL MACHINE	CONTAINER
Boot Time	~5-10 min	~2 min	~2 secs
App Deployment Lifecycle	Deploy in weeks Live for years	Deploy in minutes Live for weeks	Deploy in seconds Live for minutes/hours
Deployment Complexity	**Need to know:** 1. Hardware 2. OS 3. Runtime environment 4. Application code	**Need to know:** 1. OS 2. Runtime environment 3. Application code	**Need to know:** 1. Runtime environment 2. Application code
Investment	Buy/rent dedicated server	Rent a dedicated VM on a shared server	Rent containers and pay for actual runtime
Scaling	Can take months* Should be approved by a panel of experts	Takes hours Should be approved by adminstrators	Takes seconds Policy driven scaling

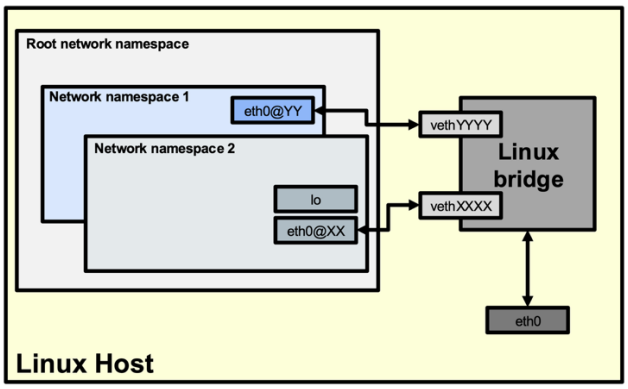
**For on-premises bare metal. Bare metal cloud services can be dynamically provisioned in minutes*

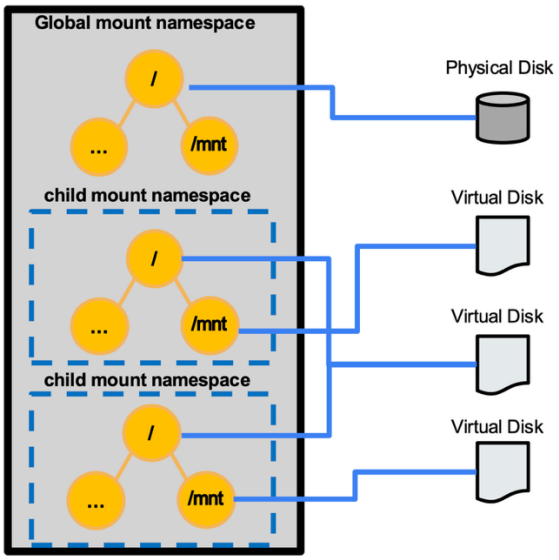
Utilizzando un tipico server fisico, con risorse di calcolo medie, è possibile eseguire circa 10-100 macchine virtuali o 100-1000 container e con questi ultimi è possibile utilizzare pochi MB di disco virtuale, a seconda dell'immagine di partenza.

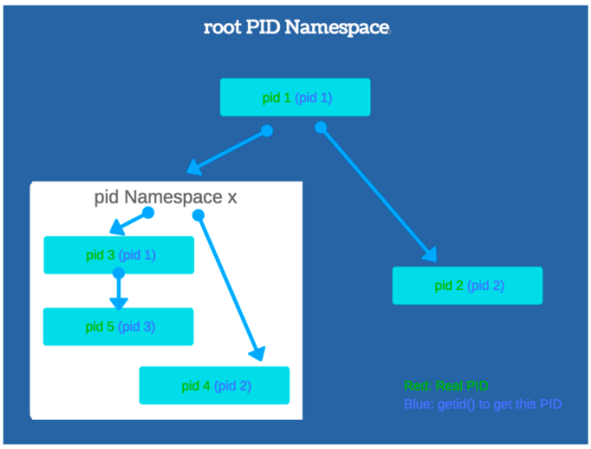
Il namespace, risorsa del kernel host che viene sfruttata dal container, **è un dominio di denominazione per varie risorse**, come `mnt` (mount points, filesystem), `pid` (processi), `net` (network stack), `ipc` (System V IPC), `uts` (hostname) o `user` (identificativi utente). Un namespace di un kernel mette a disposizione le seguenti primitive:

- `clone()`, crea un nuovo processo e un nuovo napespace, con il primo che viene legato al secondo;
- `fork()` e `exit()`, sono modificati in modo da gestire nuovi flag per i namespace;
- `unshare()`, non crea un nuovo processo ma crea un nuovo napespace da legare al processo corrente (aggiunta di recente per la gestione dei namespace e per questioni di sicurezza);
- `setns()`, fa la join di un namespace esistente.







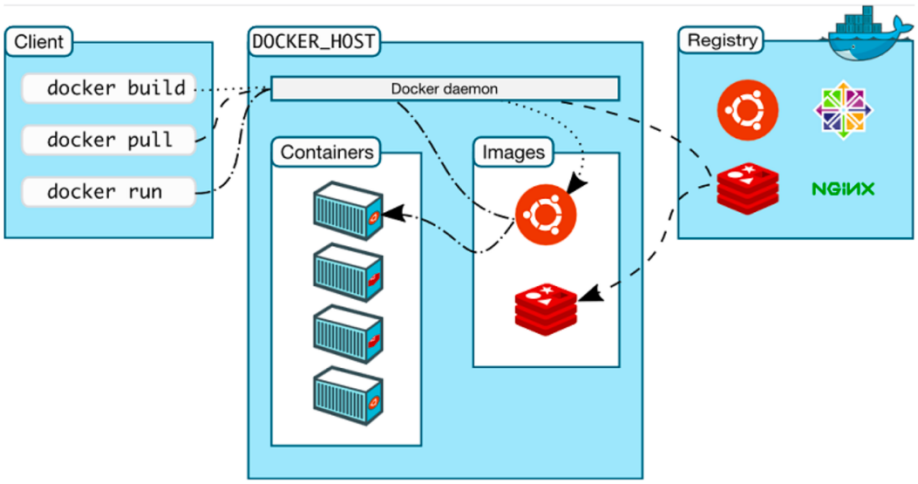


Il sottosistema **Cgroups** (gruppi di controllo) è una **soluzione di gestione delle risorse che fornisce un framework generico per gruppi di processi**, fu progettato e nato da Google all'inizio degli anni 2000 ed è caratterizzato da un controller della memoria (memcg) gestito separatamente (probabilmente il più complesso e articolato); **se i namespace forniscono una soluzione di isolamento delle risorse per processo, i cgroups lo fanno per i gruppi**. I moduli di cgroup non si trovano in un'unica cartella ma sono sparsi nell'albero del kernel in base alla loro funzionalità:

- `memory`, imposta limiti all'uso della memoria da parte dei task di un cgroup e genera report automatici sulle risorse di memoria utilizzate da tali task (`mm/memcontrol.c`);

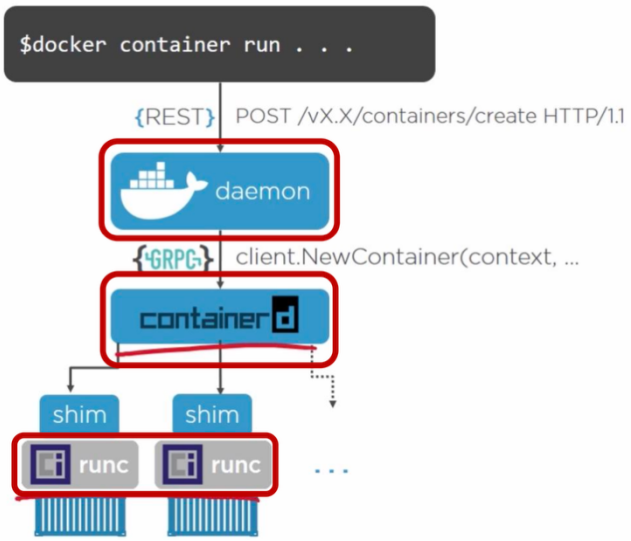
- `cpuset`, assegna le singole CPU (in un sistema multicore) e i nodi di memoria ai task di un `cgroup` (`kernel/cpuset.c`);
- `net_prio`, fornisce un modo per impostare dinamicamente la priorità del traffico di rete per ogni interfaccia di rete (`net/core/netprio_cgroup.c`);
- `devices`, consente o nega l'accesso ai dispositivi da parte dei task di un cgroup (`security/device_cgroup.c`);
- `blkio`, imposta limiti sull'accesso di input/output a e da dispositivi a blocchi, come le unità fisiche (dischi, stato solido, USB, …);
- E molto altro, …

Docker è un motore di container scritto in Go (basato su container Linux) **costruito per eseguire un'intera applicazione in un container** e attualmente gestito da poco meno di 2000 collaboratori su GitHub.

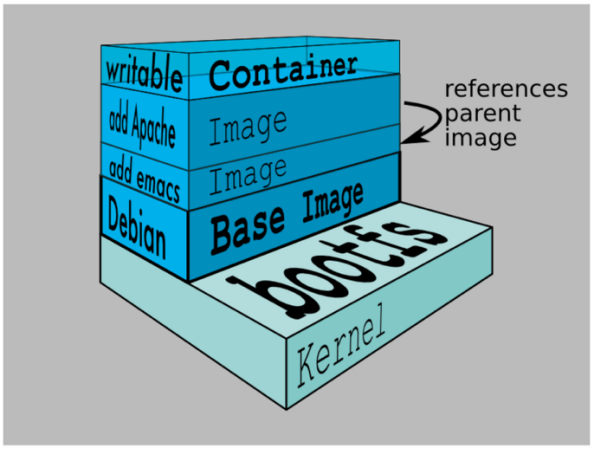


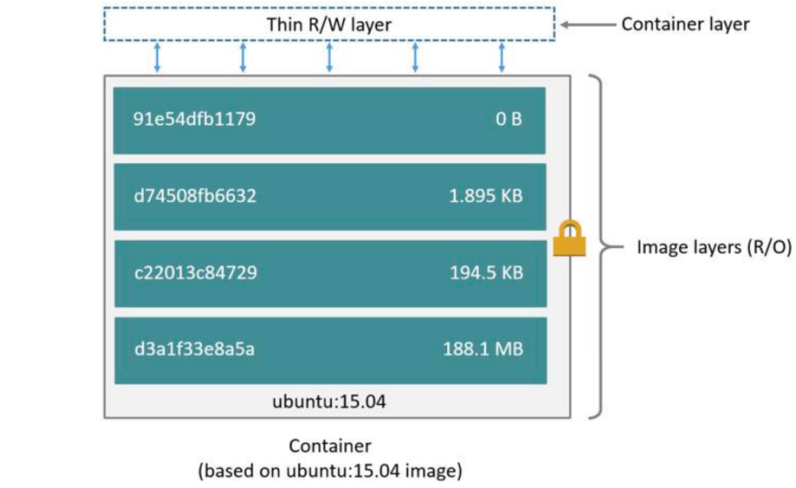
Il **docker engine** è composto da:

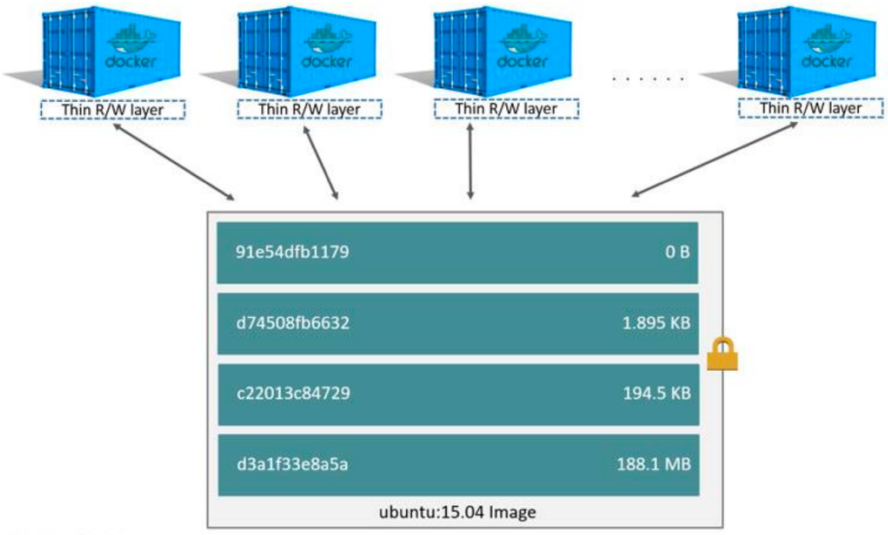
- `daemon`, API REST (ricezione di istruzioni) e altre funzionalità;
- `containerd`, logica di esecuzione (avvio, arresto, pausa, disattivazione, cancellazione di contenitori, …);
- `runc`, un'interfaccia a riga di comando leggera per il runtime conforme a OCI.



Una **docker image non è un disco rigido virtuale, né un filesystem**, **utilizza un Union File System ed è un layer di sola lettura, non ha stato, è fondamentalmente un file compresso con una gerarchia** (di profondità arbitraria).







Un **docker registry** è un **registro contenente immagini docker** (registro locale sullo stesso host, registro di docker hub o registro privato condiviso a livello globale su docker.com), **controlla dove vengono archiviate le immagini e controlla completamente la pipeline di distribuzioni delle**

immagini, integrando l'archiviazione e la distribuzione delle immagini nello specifico flusso di lavoro di sviluppo.

Alcuni comandi di base in docker sono:

- `docker build`, costruisce un'immagine Docker a partire dal Dockerfile;
- `docker run`, avvia un nuovo container Docker da un'immagine;
- `docker ps`, elenca tutti i container Docker in esecuzione;
- `docker stop`, arresta un container in esecuzione;
- `docker rm`, rimuove un container Docker;
- `docker images`, elenca tutte le immaigni Docker attualmente disponibili sul sistema;
- `docker pull`, scarica un'immagine Docker da un registro;
- `docker exec`, esegue un comando in un container in esecuzione.

Alcune delle direttive principali da inserire nel Dockerfile sono le seguenti:

- `FROM`, crea un nuovo layer da aggiungere all'immagine specificata;
- `WORKDIR`, cambia la workind directory di default;
- `COPY`, copia file in directory;
- `RUN`, esegue i comandi specificati creando un nuovo layer al di sopra di quello da cui si sta partendo;
- `ENTRYPOINT`, specifica l'eseguibile di default lanciato all'avvio del container;
- `CMD`, specifica i comandi di default;
- `EXPOSE`, definisce i port ai quali l'app nel container deve mettersi in ascolto.

Ad esempio, per la creazione di un'app Flask containerizzata:

1. Implementazione dell'applicazione Flask:

```
# flask_app.py
from flask import Flask, request
import socket

app = Flask(__name__)

@app.route("/")
def index():

   return "This is an example containerized Flask app served from {} to {}".format(socket.gethostname(), request.remote_addr)


if __name__ == "__main__":
   app.run(host='0.0.0.0', port=5001, debug=True)
```

2. Definizione del Dockerfile:

```
FROM python:3.10-alpine

WORKDIR /app # assumiamo che l'app flask sia all'interno della dir app

COPY requirements.txt /app
RUN  pip3 install -r requirements.txt

COPY flask_app.py /app

ENTRYPOINT ["python3"] # ENTRYPOINT viene trattato come parametro da dare al docker run

CMD ["flask_app.py"] # viene ignorato nel caso in cui passiamo un argomento (oltre a quello specificato in ENTRYPOINT)
tramite docker run
```

3. Build dell'immagine:

```
$ cd /path/to/Dockerfile
$ docker build -t flask_image_hello_world
```

4. Run del container:

```
$ docker run -it --name flask_app_container -p 5001:5001 flask_image_hello_world
```

```
root@dockertest1:~# docker ps -a
CONTAINER ID   IMAGE                   COMMAND            CREATED         STATUS         PORTS                                       NAMES
d0a3f0b406a9   flask_image_hello_world   "python3 app.py"   3 seconds ago   Up 2 seconds   0.0.0.0:5001->5001/tcp, :::5001->5001/tcp   flask_app_container
root@dockertest1:~#
```

Per **fare il deploy** di un'applicazione multi-container bisogna farne la build e il run di tutti i container che costituiscono l'applicazione, eventualmente collegare (manualmente) i container per la comunicazione e fare attenzione alle dipendenze per l'ordine di avvio dei container. **Docker compose ovvia queste limitazioni**, permettendo di specificare un'applicazione multi container definendola in un file di specifica `.yaml`, utilizzare un singolo comando per fare il deploy dell'applicazione, gestire le dipendenze di avvio per i container in maniera programmatica e supportare nativamente l'orchestratore Docker Swarm (che sarà introdotto a breve). Per il Docker compose file, alcuni dei parametri più importanti sono i seguenti:

- `build`, specifica la configurazione (Dockerfile) di compilazione per la creazione di un'immagine del container;
- `context`, è possibile usarlo per definire o un percorso ad una directory che contiene il Dockerfile o un URL ad una repo git;
- `command`, sovrascrive il comando predefinito dichiarato dall'immagine del container (ad esempio dal `CMD` di Dockerfile);
- `ports`, espone i port per il container;
- `volumes`, definisce i percorsi del filesystem dell'host (mount point) accessibili dal container;
- `links`, definisce un collegamento di rete ai container di un altro servizio;
- `image`, specifica l'immagine da cui avviare il container;
- `environment`, definisce le variabili d'ambiente impostate nel container.

```
app:
  build: .
  command: python -u app.py
  ports:
    - "5000:5000"
  volumes:
    - .:/app
  links:
    - db
db:
  image: mongo:latest
  hostname: test_mongodb
  environment:
    - MONGO_INITDB_DATABASE=animal_db
    - MONGO_INITDB_ROOT_USERNAME=root
    - MONGO_INITDB_ROOT_PASSWORD=pass
  volumes:
    - ./init-db.js:/docker-entrypoint-initdb.d/init-db.js:ro
  ports:
    - 27017:27017
```

Per l'installazione (su Linux) di Docker compose:

```
sudo apt-get update
sudo apt-get install docker-compose-plugin
```

Mentre alcuni dei comandi più usati:

- `docker compose up`, build e run di tutti i container (usando -d si attiva la modalità daemonized);
- `docker compose down`, stop e distruzione di tutti i container;
- `docker compose ps`, lista i container legati al progetto compose il cui file di config è nella current directory;
- `docker compose logs`, stampa i log legati al progetto compose il cui file di config è nella current directory.

Per la creazione di un'app Flask containerizzata tramite Docker compose:

1. Implementazione dell'applicazione Flask;
2. Definizione del compose file:

compose.yaml

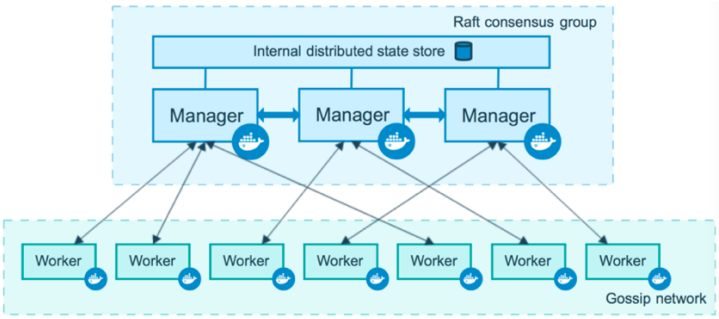
```
services:
 web:
  build:
   context: app
  ports:
   - '5001:5001'
```

3. Avvio del compose e run del container:

$ docker compose -f compose.yaml up -d

```
root@dockertest1:~/ACP/docker_deploy_examples/dockerized_flask_service# docker compose -f compose.yaml up -d
[+] Building 0.0s (0/0)
[+] Running 2/2
 ✔ Network dockerized_flask_service_default  Created
 ✔ Container dockerized_flask_service-web-1  Started
root@dockertest1:~/ACP/docker_deploy_examples/dockerized_flask_service#
```

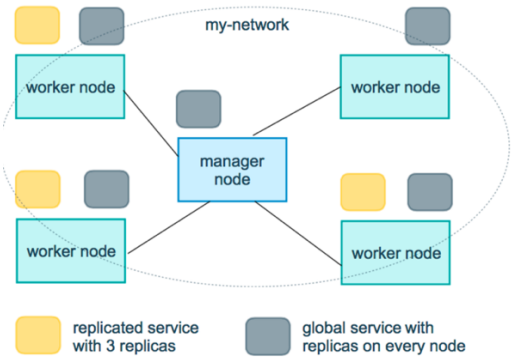
Docker Swarm orchestra un cluster di istanze Docker, **le funzionalità di gestione e orchestrazione del cluster sono incorporate nel docker engine**. Uno **swarm** è costituito da **più host docker che sono esguiti in modalità swarm**: **agiscono come manager** (per gestire la join e la delega dello swarm), **come worker** (che esguono i servizi dello swarm), **o come entrambi**. I nodi manager implementano l'algoritmo di consenso Raft per gestire lo stato globale del cluster, tollerando fino a $(N-1)/2$ fallimenti di un manager (se il manager di uno swarm a singolo manager si guasta, i servizi continuano a funzionare ma è necessario creare un nuovo cluster per recuperare) e richiedendo il quorum di $(N/2)+1$ membri per accordarsi sui valori.



Per un servizio è possibile definire il suo stato ottimale (desiderato) in termini di numero di repliche, risorse di rete e di storage disponibili, porte che il servizio espone al mondo esterno, vincoli di risorse e preferenze di posizionamento, … **Docker lavora per mantenere lo stato desiderato** (se un dato worker diventa indisponibile, docker pianifica le attività di quel nodo su altri nodi).

Il **deploy di un servizio con Docker Swarm** consiste nell'invio di una definizione di servizio a un nodo manager, il quale distribuisce ai nodi worker unità di lavoro chiamate task ed esegue funzioni di orchestrazione e gestione del cluster (necessarie per mantenere lo stato desiderato dello swarm). **I nodi manager**, inoltre, **eleggono un singolo leader per condurre le attività di orchestrazione** e su ogni nodo worker viene eseguito un agente che riferisce sui compiti ad esso assegnati. Un servizio è la definizione dei task da eseguire sui nodi manager o worker e può essere riferito a due modelli:

- **Modello di servizi replicati**, lo swarm manager distribuisce un numero specifico di task di replica tra i nodi in base alla scala imposta nello stato desiderato;
- **Modello di servizi globali**, lo swarm esegue un task del servizio su ogni nodo disponibile nel cluster, quando si aggiunge un nodo allo swarm viene gli viene assegnato un task.



Docker Swarm fornisce meccanismi per controllare la scala e il posizionamento (placement) **dei servizi su nodi diversi**; in particolare, si distinguono:

- Servizio replicato o globale;
- Riserva di memoria o CPU per un servizio;
- Vincoli di placement (il servizio viene eseguito solo su nodi con un'etichetta specifica);
- Preferenze di placement (cerca di posizionare i task sui nodi appropriati in modo uniforme).

Un task implementa un container Docker e i comandi da eseguire all'interno del container, è l'unità atomica di scheduling in Docker Swarm e sono assegnati dai nodi manager ai nodi worker in base al numero di repliche impostato nel parametro di scala nella definizione del servizio. **Una volta che un task è assegnato ad un nodo, non può spostarsi su un altro nodo, può essere solo eseguito o fallire**.

Service update in Docker Swarm permette agli aggiornamenti online della configurazione di rete e dei volumi legati ad un servizio; Docker Swarm avvierà nuovi nodi con la nuova configurazione e arresta automaticamente i vecchi nodi utilizzando la vecchia configurazione ma si occuperà anche della gestione dei rollback e degli aggiornamenti dei servizi.

- **Creazione dello swarm**
  - $ docker swarm init --advertise-addr IP_NODO_MASTER
- **Join allo swarm**
  - $ docker swarm join --token TOKEN_GENERATO IP_NODO_MASTER:2377
  - **P.S.: Utilizzare comando generato dallo swarm init**
- **Creazione/Rimozione del servizio**
  - **docker service**
    - $ docker service create ...
      - E' possibile specificare il grado di replicazione, e altri parametri
    - $ docker service rm ...
  - **docker stack**
    - $ docker stack deploy --compose-file=compose.yaml ...
      - E' possibile specificare un file yaml per le caratteristiche del servizio
    - $ docker stack rm ...
- **Rimozione dello swarm**
  - $ docker swarm leave

Ad esempio, l'orchestrazione di un'app Flask replicata:

1. Creazione dello swarm;
2. Join allo swarm;
3. Build dell'immagine su tutti i nodi dello swarm (come precedentemente riportato);
4. Creazione del servizio (`docker service`):

```
docker service create --replicas 3 --name flask_helloworld_service --publish 5001:5001 flask_image_hello_world
```

```
image flask_image_hello_world:latest could not be accessed on a registry to record
its digest. Each node will access flask_image_hello_world:latest independently,
possibly leading to different nodes running different
versions of the image.

xvymsur4ggtblw0y2sheuub2z
overall progress: 3 out of 3 tasks
1/3: running   [==================================================>]
2/3: running   [==================================================>]
3/3: running   [==================================================>]
verify: Service converged
root@dockertest1:~/ACP/docker_deploy_examples/dockerized_flask_service/app#
```

```
~ while True; do curl http://192.168.100.101:5001/; echo; sleep 1; done
This is an example wsgi app served from 697b6c7a50d3 to 10.0.0.2
This is an example wsgi app served from 1b139379fb74 to 10.0.0.2
This is an example wsgi app served from 8df0ff0e9494 to 10.0.0.2
This is an example wsgi app served from 697b6c7a50d3 to 10.0.0.2
This is an example wsgi app served from 1b139379fb74 to 10.0.0.2
This is an example wsgi app served from 8df0ff0e9494 to 10.0.0.2
This is an example wsgi app served from 697b6c7a50d3 to 10.0.0.2
This is an example wsgi app served from 1b139379fb74 to 10.0.0.2
This is an example wsgi app served from 8df0ff0e9494 to 10.0.0.2
This is an example wsgi app served from 697b6c7a50d3 to 10.0.0.2
This is an example wsgi app served from 1b139379fb74 to 10.0.0.2
```

Le richieste di `GET` verso il nodo manager sono bilanciate tra tutti i nodi che hanno un container in esecuzione per il nostro servizio; ad ogni `GET` corrisponde uno dei tre container in esecuzione, permettendo sia un bilanciamento delle richieste che una disponibilità del servizio. L'availability di queste tre repliche è garantita automaticamente dallo swarm se uno o più nodi falliscono; è possibile, tuttavia, emulare una non disponibilità di un nodo impostando lo stato di `DRAIN` per quel nodo:

```
$ docker node update --availability drain IP_HOST
```

È possibile effettuare il deploy dell'applicazione anche attraverso `docker stack`, che utilizza un file di configurazione `.yaml` e il cui comando `deploy` accetta una descrizione dello stack sotto forma di file Compose:

```
...
deploy:
   replicas: 5
   restart_policy:
     condition: on-failure
     max_attempts: 3
     window: 120s
...
```

Per fare il deploy, poi, basta creare l'immagine del servizio in tutti i nodi del cluster e lanciare il deploy:

```
# // On all worker nodes
# docker build -t /PATH/TO/flask_image_hello_world_DOCKERFILE

# // On master node
# docker stack deploy --compose-file=/PATH/TO/compose_with_stack.yaml dockerized_flask_service_with_stack
Creating network dockerized_flask_service_replicated_default
Creating service dockerized_flask_service_replicated_web
```